

OH MY STACK / READING EDITION

pstack

Build an environment you can trust

Building an environment for development with AI
agents

kaito

Author's English edition

SOURCE EDITION / 2026-10-03
48 ENTRIES • COMPLETE TEXT

About this edition

Author: kaito

Ebook preparation: Oh My Stack

This edition contains all 48 contents entries, including 39 numbered chapters, the preface, part introductions, appendix and afterword. It reproduces the author's English edition from the frozen October 3, 2026 snapshot. First-person statements in the book refer to the author.

Sources and permission

The maintainer reports the author's private verbal permission to publish English and Chinese editions. Inclusion in Oh My Stack does not place the book under the project's MIT license. Third-party quotations, code and images retain their respective attribution.

Reading this ebook

Collapsed code examples are expanded. Original code is unchanged; long lines wrap in the PDF. Diagrams and original images are included. External articles, videos and social posts retain their links and require an internet connection. The associated Simplified Chinese translation has AI review records and awaits human editorial review.

The book discusses pstack 0.15.5. Consult the project's documentation for Oh My Stack runtime behavior.

[Authorization and sources](#) • [Online editions and review records](#)

Typeset by Oh My Stack. Embedded Noto Sans CJK fonts use the SIL Open Font License 1.1. This font license does not apply to the book.

Contents

Preface	6
Part I: The development foundation behind 2,500 PRs a month	11
Chapter 1: 2,500 PRs result from a ready environment, not a target.....	13
Chapter 2: Trust the artifact, not the agent	16
Chapter 3: Make verification a repeatable mechanism	25
Chapter 4: Make the codebase the agent's memory	33
Chapter 5: Build automation on a foundation of trust	40
Part II: What pstack is	46
Chapter 6: What problem pstack solves.....	48
Chapter 7: /poteto-mode, the entry point that routes each request.....	57
Chapter 8: What do Playbooks, Skills, and Principles each handle?	68
Chapter 9: Why pstack loads only what it needs	77
Part III: Moving work forward with Playbooks.....	81
Chapter 10: Investigate and reproduce problems	87
Chapter 11: Fix bugs and improve performance	95
Chapter 12: Rebuild features, structure, and appearance	107
Chapter 13: Write Skills and evaluate them	120
Chapter 14: Open PRs and merge verified changes.....	128
Chapter 15: Delegate long-running work to the agent	139
Chapter 16: Plan many PRs and run them with multiple agents.....	150

Part IV: Principles support the agent's judgment	171
Chapter 17: Core: ten principles that decide how work proceeds	178
Chapter 18: Architecture: 6 principles for boundaries and dependencies.....	208
Chapter 19: Verification: four principles that check real behavior.....	240
Chapter 20: Delegation: two principles for delegated and parallel work	264
Chapter 21: Meta: the principle that improves the system itself	276
Part V: Extend what the work can do with Skills	289
Chapter 22: Understand the codebase and past work.....	299
Chapter 23: Design a Playbook for work that no Playbook fits.....	321
Chapter 24: Run agents in parallel and compare designs and artifacts.....	327
Chapter 25: Extend pstack to your own way of working and to Grok Bot.....	355
Chapter 26: Build verification and keep it current as the app changes	363
Chapter 27: Check what a change could break and where the diff is weak	380
Chapter 28: Record decisions and leave evidence for long agent runs.....	390
Chapter 29: Keep code quality with 16 TypeScript rules	401
Chapter 30: TDD: verify with a failing test before the fix.....	417
Chapter 31: Remove comments and tidy the codebase	427
Chapter 32: Keep writing quality high	441
Chapter 33: Reflect on what the work taught you and apply it.....	461
Chapter 34: Install pstack and choose a model for each role.....	470
Chapter 35: /poteto-mode is pstack's Router	488
Part VI: A practical guide to pstack	506

Chapter 36: Guide 1: start with a verification skill for your own app	508
Chapter 37: Guide 2: share the goal, then align on problem and history	528
Chapter 38: Guide 2: test a design from how the user will use it	541
Chapter 39: Guide 2: revise the design into an executable plan	558
Appendix: README-driven with Grill Driven Development (RGDD)	587
Afterword	607

Preface

kaito • Sources: 日本語 • English • 2026-10-03

This book is also available in Japanese: [日本語版](#).

This book is about **widening the scope of work you can delegate to agents**.

It is no longer hard to get an AI agent to write code. The hard part is building a system that lets you trust what it wrote so that you no longer have to watch over the AI.

When an agent says "I fixed it," you do not know whether it really did until you verify. So a human has to sit beside it, watch the screen, read the diff, and run the app to check. With this setup, however many agents you run side by side, the human who checks their work stays the bottleneck.

This book explains "**how to build an environment that gets you out of that situation**."

Its main subject is [pstack](#).

pstack is a Cursor plugin published by Lauren Tan, known as [poteto](#), an engineer at Cursor and a member of the React core team.

pstack packages the work procedures and judgment standards that poteto uses every day to build high-quality code at Cursor. These procedures and standards draw on poteto's experience at Meta, Netflix, and Cursor, and pstack organizes them as Playbooks, Principles, and Skills.

poteto's aim is not to have AI write large amounts of code. The aim is a way of working in which you can verify quality, so that you can trust several agents to run in parallel. In September 2026, poteto announced shipping 2,500 PRs to production with pstack in the single month of August.

<https://x.com/poteto/status/2102050467505430555>

However, this book is not a how-to for producing more PRs.

One question is at its center.

Under what conditions can an agent keep doing high-quality work without a human watching?

pstack is one concrete answer to that question. It contains mechanisms that verify artifacts, the work procedures of experienced engineers, standards for judgment, knowledge and constraints kept in the codebase, and automation built on top of all of these.

This book explains every Playbook, Principle, and Skill in pstack and breaks the answer down into a form you can take back to your own project.

What "delegating work" to an agent means

Delegating work to an agent does not mean handing over instructions and waiting for the result. In this book, I call it delegating when you have handed the agent the following three things.

1. **What counts as done:** the finish condition is in a form you can run to get a pass or fail result
2. **How to proceed:** steps such as investigation, reproduction, fixing, and checking run at the same quality every time
3. **How to verify:** the agent itself runs the real app or artifact and leaves evidence

If the third is missing, a human still has to act as the checker, even when the other two are in place. For this reason, poteto calls verification the most important foundation. This book starts with verification too.

What this book covers

This book has six parts and 39 chapters.

Part	Topic	Chapters
Part I: The development foundation behind 2,500 PRs a month	What conditions let you delegate work to an agent?	Chapters 1 to 5
Part II: What pstack is	With what components does pstack meet the conditions from Part I?	Chapters 6 to 9
Part III: Moving work forward with Playbooks	pstack's Playbooks in detail	Chapters 10 to 16
Part IV: Principles support the agent's judgment	pstack's Principles in detail	Chapters 17 to 21
Part V: Extend what the work can do with Skills	pstack's Skills in detail	Chapters 22 to 35
Part VI: A practical guide to pstack	Where to start in your own project	Chapters 36 to 39

Part III through Part V cover pstack in concrete detail. They explain all 23 Playbooks, 23 Principles, and 24 Skills in pstack as of September 2026. They do more than list names. For each item, they explain what it is for, when you use it, and how it connects to the other items.

Part VI is the practical part. Its main reference is The Complete Guide to pstack, which poteto posted on X in two parts, [Part 1](#) and [Part 2](#).

Part VI uses the guide to explain how to build a verification skill and how to research, design, and plan with an agent.

How to read this book

On a first read, I recommend that you read the chapters in order. If you first read "why verification comes first" in [Part I](#), you can see more easily which problem each item solves.

If you have a clear goal, you can also read it in one of the following ways.

- **If you want to install pstack and use it right away**, read [Part II](#), then go to [Chapter 34](#) on installation and [Chapter 36](#) on verification skills.

- **If you will not use pstack but want to design your team's development environment**, focus on [Part I](#), [Part IV](#), and [Part VI](#). You can apply the Principles and the practical guide without pstack.
- **If you want to look up a specific Playbook or Skill**, go straight to the right chapter from the chapter list in each part's introduction. I wrote each chapter so that you can read it on its own.

The pstack request this book uses throughout

If you look at pstack's Playbooks, Principles, and Skills one at a time, you do not see how they work together inside a single request. So this book uses the following prompt in [Part II](#) and [Part IV](#).

```
/poteto-mode the export writes duplicate rows when a retry lands mid-run. repro first, then fix and verify.
```

This request comes from pstack's [bundled guide](#), where it appears as an example request.

Its first sentence states the symptom. Its second sentence states what counts as done, which is to reproduce, fix, and check. [Part II](#) and [Part IV](#) use it as a good example, and each part looks at a different side of it:

- [Part II](#) looks at which Playbook the entry point, `/poteto-mode`, picks
- [Part IV](#) looks at which Principles the agent uses for its decisions during the work, and how

I have not run this request on a real project myself. The flow this book shows traces the steps written in pstack's files. It includes no run logs or output.

Reference date and notation rules

pstack changes often. This book covers pstack as of the following point.

Item	Value
Date checked	September 24, 2026
Repository	pstack in cursor/plugins
Version	0.15.5
commit	12d587d (September 23, 2026)
Counts	23 Playbooks, 23 Principles, 24 Skills

If you use a newer version, compare the list in the [README](#) with the lists in this book. Some items may have new names, and there may be new items. Values that change often, such as the default model names, carry the note "default as of 0.15.5."

This book separates the kinds of content it presents as follows.

- **pstack's specification** comes with the name of the relevant file or a link.
- **poteto's views** come with their source, such as the README, The Complete Guide to pstack, or the talk. This book presents them as poteto's own views and does not treat them as facts that hold in every environment.
- **Quotes from the source** are short quotes of the English original, with the source named.
- **My interpretation and experience** carry explicit markers, such as "This book reads it as ...," "I think ...," and "In my case."

Let's begin in [Part I](#) with what lies behind the figure of "2,500 a month."

Part I: The development foundation behind 2,500 PRs a month

kaito • Sources: 日本語 • English • 2026-10-03

Part I covers "**what conditions let you delegate work to an agent.**"

To meet these conditions, you need to put an environment in place before you raise the speed: mechanisms that verify results, and working procedures.

Specifically, there are four conditions, and each one is the precondition, the base, for the next.

1. The agent can run the artifact it made and verify that artifact itself. This book calls that step verification.
2. The way to verify and the working procedure are explicit, so anyone who runs them gets consistent quality.
3. Past decisions and constraints stay in the codebase in a form the next piece of work can refer to.
4. You automate only after these three are in place as the base.

The figure of 2,500 PRs a month is the result of putting these four in place. If you make only the number your target, skip 1 through 3, and start on 4, what grows is not results but **changes nobody can verify**.

What you learn in Part I

After you read Part I, you will know the following.

- What the number 2,500 does not mean
- What to trust, if not the agent

- How to turn verification from a one-time check into a mechanism anyone can run again
- Why the agent's memory belongs in the codebase, not in conversations or notes
- Why automation comes last, and which conditions to meet before you automate

Part I says little about the individual parts of pstack. It covers ideas that hold whether or not you use pstack. From [Part II](#) on, the book ties each part of pstack to these ideas.

Chapters in this part

Chapter	Role
Chapter 1: 2,500 PRs result from a ready environment, not a target	Decides how to treat the number 2,500, and moves the central theme of this book from "the count" to "the conditions for delegating"
Chapter 2: Trust the artifact, not the agent	Makes clear what you trust. Defines verification
Chapter 3: Make verification a repeatable mechanism	Covers three ways to raise trust: direct verification, working patterns, and constraints through structure
Chapter 4: Make the codebase the agent's memory	Covers how to hand the trust you built over to the next agent
Chapter 5: Build automation on a foundation of trust	Covers what you can automate once the conditions in Chapters 2 through 4 are in place

[Chapter 2](#) through [Chapter 5](#) correspond to the four conditions that poteto set out in the talk. Each chapter builds on the one before it, so I recommend that you read them in order.

Chapter 1: 2,500 PRs result from a ready environment, not a target

kaito • Sources: 日本語 • English • 2026-10-03

The 2,500 PRs a month were not a number anyone aimed for. **The number followed from building an environment where you can delegate work to agents with confidence.**

This chapter looks at why you should not make the count a target, and at what the count really stands for.

How this chapter is organized

This chapter is organized as follows.

- Why you should not make the count a target
- What the count stands for
- Summary: shift your view from the count to "the conditions for delegating"

Why you should not make the count a target

There are two reasons.

The first is that when you raise only the count, **what grows is not results but unverified changes.**

As long as a person reads each diff the agent wrote and tries the app to verify it, the ceiling on the count is the amount a person can check.

If you decide in this state to "double the PRs this month," you only double the number of requests you send to the agent. The number of people who check does not grow, so the added requests pile up unverified.

Nobody knows which changes are correct and which are broken until someone verifies them one by one later.

The other reason is that **the count is a number you can move with little effort.**

The same feature is 1 PR if you put it in one large PR, and it is 5 PRs if you split it into five.

If the count is the target, you can make the number look better by splitting PRs into smaller ones, while the content of the changes stays the same.

What the count stands for

I think what the count really stands for is **the amount of checking that moved out of human hands and into the agent's side.**

As the previous section showed, if people verify everything, the count cannot exceed what people can check. Put the other way, if the count far exceeds what people can check, something other than people must be doing most of the checking.

In [Part 1](#) of The Complete Guide to pstack, poteto defines verification as "**the agent being able to check its own work**," and writes that this ability keeps people from becoming the bottleneck.

So what does it take to move checking out of human hands?

It takes the following four things. They were also the four themes of poteto's [talk](#).

1. Trust
2. Ways to raise trust
3. The codebase as memory
4. Automation

The most important point is that these four are not independent tips. Each item is the precondition, the base, for the next one.

In other words, you have to keep this order. First make the artifacts verifiable, then record the knowledge you gain there in the codebase, and then build automation on that base.

The number 2,500 a month is the result of putting these four in place in order. [Chapter 2](#) through [Chapter 5](#) of this book cover the four, one chapter each.

Summary: shift your view from the count to "the conditions for delegating"

- **Why you should not make the count a target.** If you raise the count while the number of people who check stays the same, what grows is unverified changes. The count also moves with the way you split PRs.
- **What the count stands for.** It is the amount of checking that moved out of human hands and into the agent's side. Behind it is a state where all four are in place: trust, ways to raise trust, the codebase as memory, and automation.

The next chapter, [Chapter 2](#), covers the first of the four, "trust." The talk says that what you trust is not the agent but the artifacts the agent makes. The chapter looks at how pstack puts this idea into practice.

Chapter 2: Trust the artifact, not the agent

kaito • Sources: 日本語 • English • 2026-10-03

When you hand work off to an agent, what you should trust is not the agent's report but **the artifact the agent made**.

poteto presented this idea under "trust," the first theme of [the talk](#).

This chapter explains what it means to trust the artifact and how pstack puts that idea into practice.

How this chapter is organized

This chapter is organized as follows.

- Why trust the artifact instead of the report?
- What does verifying an artifact mean, and how does the agent do it?
- What in a reply tells you that the work can be delegated?
- How do you get closer to a state where you can delegate with confidence?
- Summary: move your trust from the report to the artifact

Why trust the artifact instead of the report?

You trust the artifact because **a report does not prove that the work is correct**.

When the agent says "I fixed it" or "it should work," the statement tells you only that the agent judged it so. The report alone does not tell you what the judgment rests on: a passing test, an impression from reading the code, or the result of running it.

To verify that the work is correct, the human ends up reading the diff and trying the app by hand.

If you add more work in this state, you only add more unverified changes. The talk also says that if you increase throughput without verification, **what increases is bugs, not results**. If quality holds only while a human sits beside the agent and watches, you have not delegated the work.

On the other hand, if the agent can verify the artifact, the human no longer needs to believe the report or verify everything again by hand.

pstack provides a verification mechanism for this purpose. In The Complete Guide to pstack [Part 1](#), poteto defines verification as "**the agent being able to verify its own work.**"

What does verifying an artifact mean, and how does the agent do it?

Verifying an artifact means that **the agent itself checks the real thing, in a way that leaves evidence**.

In concrete terms, verification has to meet the following three conditions.

1. **It deals with the real thing.** It checks the running app, the values actually written, and the files actually produced, not whether the build passed or how many tests ran
2. **The agent runs it itself.** The agent operates and observes the app, instead of a human trying the app by hand
3. **Evidence remains.** Something remains that a human can look at and judge later, such as screenshots, videos, traces, and command output

It deals with the real thing

The principle "[Prove It Works](#)" sets the first condition. It opens with this sentence.

Verify every task output by checking the real thing directly. Do not infer from proxies, self-reports, or "it compiles."

This sentence does not forbid builds or tests. What it forbids is treating "it compiles" as evidence that something works. In addition to builds and tests, the agent has to run the real thing and check it.

The description in `SKILL.md` gives these three examples of ways to verify.

1. Run the feature for real
2. Read the actual values
3. Look at the diff

The principle also gives a cached screenshot as an example of an indirect check to avoid. A screenshot looks like an artifact, but if you cannot tell whether it shows the screen after this change, it is weak grounds for verification. From this, I read that when you look at an artifact, you should also check **whether the artifact was made after this change**.

The same idea also applies when the agent hands work off to a subagent. In `/poteto-mode`, the main agent does not pass on a subagent's report as is. It looks at the diff before it replies, so **the reply the human receives has already been checked once against the artifact** (Chapter 8).

The agent runs it itself

The "Bug fix" Playbook (`pplaybooks/bug-fix.md`) supports the second condition. In this Playbook, the agent in charge uses tools that operate the app (the Playbook calls them control Skills). It reproduces the bug itself in the same environment where the bug happened, and verifies the fix in the same environment too (Chapter 11).

pstack itself does not include tools that operate the app. `control-ui`, for browsers and Electron, and `control-cli`, for CLIs and TUIs, live in a separate plugin, `cursor-team-kit`. Chapter 3 and Chapter 36 cover how to build a tool for your own app.

The agent may ask the user to reproduce the bug only under two conditions. The agent must first operate the app as far as its tools go, and it must state a specific reason why its tools cannot reach the target.

For example, suppose login requires two-factor authentication on a physical device partway through, and the tools cannot get past that point. Then the agent asks the user to reproduce the bug, and says how far it got and why it stopped.

I think this limit matters. The limit keeps open the option of asking a human, but it requires a reason. So the agent finds it harder to hand verification back to the human only because verification takes effort.

Evidence remains

The section "[Script the check when you can](#)" of "Prove It Works" supports the third condition.

The strongest proof is a deterministic script that re-runs the same comparison, not a one-time eyeball.

This section recommends that you write a script that can re-run the same comparison and keep its results where a human can see them. The script is what you re-run. The results are what the human reads.

You need to commit the evidence to the repository only in cases that need an audit of the history later, such as a large port or migration. [Chapter 3](#) covers how to turn a check into a script.

I feel this third condition is the one people overlook most. Even if the first and second conditions hold, without evidence only a report that says "I verified it" reaches the human. You are then back where this chapter started.

When you build a verification mechanism, I think you should decide two things: whether the agent can verify the work, and **whether the results of the verification stay somewhere a human can see them.**

What in a reply tells you that the work can be delegated?

When you read a reply, check **whether each "I did X" claim comes with evidence that the agent verified it.**

For example, a reply that says only "I fixed the bug" gives you no way to judge whether it is correct. If it includes the output that showed the bug before the fix and the output that no longer shows it after the fix, the reader can judge for themselves.

pstack requires evidence or a "guess" label for each claim

In pstack, `/poteto-mode (skills/poteto-mode/SKILL.md)`, which you use to start work, has this rule for writing replies.

Every claim carries its evidence or its label in the same sentence. Measured, inferred, or guess.

The agent attaches evidence when it has it, and when it has none, it writes "this is inferred" or "this is a guess." The verification page of the guide bundled with pstack ([docs/guide/06-verify-and-ship.md](#)) uses the same idea to tell good replies from bad ones.

If a check couldn't run, a good reply says "inconclusive", and you should treat a confident reply without evidence as a red flag.

For example, if there was no environment to run the tests and the reply still says "Fixed, no problems," treat that reply as a red flag.

A reply with evidence lets the reader judge correctness

The same page lists the check that fits each kind of change. The table below uses that list to set replies that give only a report beside replies that come with evidence. I made up the middle column for comparison.

Kind of change	Reply with only a report	Reply with evidence
CLI change	"I added the flag"	The output of running the actual command
UI change	"I fixed the display"	A record of operating the changed screen flow in the running app

Kind of change	Reply with only a report	Reply with evidence
Parser or migration	"I fixed the conversion"	The result of feeding saved inputs through again
Performance improvement	"It should be faster now"	A comparison of profiles from before and after the change
Change to saving	"I made it write the value"	The result of reading back the written value

The middle column may not be false, but it gives you nothing to judge correctness with. The right column lets the reader judge for themselves.

"The tests passed" alone does not show that the bug is fixed

A common mistake is to put "the tests passed" in the right column.

Step 4 of the "Bug fix" Playbook, the step that verifies the fix, also says that passing unit tests alone do not show that the bug is fixed. Even if the code works under the conditions the test sets up, you have to reproduce the operation where the reported bug happened and confirm that the bug no longer happens there.

Check whether a test verifies behavior by asking if it passes when functions return `undefined`

The principle "Test Behavior, Not Implementation" lets you check whether a test verifies **the code's behavior**, meaning the result that the people who use the code actually receive.

This principle says that **a test calls the code the same way its users do and compares the result they receive with a concrete expected value**. The following question checks whether a test really looks at that result.

The check: before you keep a test, ask whether it would still pass if every function it imports returned `undefined`. If yes, it observes no behavior and cannot fail for a defect.

For example, a test that checks only whether a function was called still passes when the function returns `undefined`. The principle asks you to rewrite or delete such tests.

If a test passes this check, though, you know only that "the test checks the result the code returns." It does not prove that there are no bugs.

The "Bug fix" Playbook has the reply show the failing and the passing output side by side

Finally, the "Bug fix" Playbook shows what remains in the reply.

The Playbook sets what goes in the reply as follows. "Repro" here is the work that the section "[The agent runs it itself](#)" describes. The agent actually performs the operation that triggers the bug and confirms that the same bug happens.

Reply: what was broken, root cause, fix, how you verified. Paste failing-then-passing repro output verbatim.

The output from when the bug appeared and the output from when it no longer appears sit side by side in the reply.

How do you get closer to a state where you can delegate with confidence?

To get closer to a state where you can delegate with confidence, I think you should **find the checks that are missing and move them into mechanisms one at a time**.

First, look for gaps with the following five items. I turned what this chapter has covered into these five checks.

What to check	If the answer is "no"
Is the finish condition something you can run to get a pass or fail?	Add "what counts as done" to the request
Can the agent run the actual app or artifact itself?	Provide a way to operate the app (Chapter 3)

What to check	If the answer is "no"
Does the result of running it remain as evidence?	Decide on steps that keep screenshots, output, and traces
When the agent could not verify something, does it say so honestly?	Allow "inconclusive," and do not accept success reports without evidence
Do the four checks above keep running when the human steps away?	Replace the parts that do not keep running with mechanisms (Chapters 3 to 5)

The section "State the finish condition up front" on the verification page of the bundled guide (<docs/guide/06-verify-and-ship.md>) covers the first item, the "finish condition."

Put what done means in the first prompt, in whatever words fit:

The example it gives is this request.

```
/poteto-mode add json output to this command. text output stays byte-identical, the json parses, both run against the sample project. show me the evidence.
```

The guide explains that a request written this way **gives the agent three checks it can run and verify, instead of a goal whose completion it can judge only by feel.**

When you write the finish condition, you also decide the second and later check items. If you write "text output stays byte-identical," you need a step that compares the output before and after the change. If you write "both run against the sample project," the agent needs to be able to run that project.

If you cannot write the finish condition as something you can run to get a pass or fail, take that as a sign that you should review whether the specification is settled and whether you have a way to judge pass or fail.

You cannot set up most of the mechanisms in the right column of the table with the request text alone. You need tools that run the app, steps that keep evidence, and mechanisms that keep running when the human steps away. These mechanisms are the theme from [Chapter 3](#) on.

Summary: move your trust from the report to the artifact

- **Why trust the artifact instead of the report?** Because a report does not prove correctness, and if quality holds only while you watch, you have not delegated the work.
- **What does verifying an artifact mean?** The agent itself checks the real thing, in a way that leaves evidence. In pstack, the principle "Prove It Works," `/poteto-mode`, and the "Bug fix" Playbook support these conditions. The one people tend to overlook is that evidence must remain.
- **What should you look at in a reply?** Whether each "I did X" claim comes with evidence that the agent verified it. A passing test is only one input to your judgment, and only when that test verifies the code's behavior. It does not prove that there are no bugs.
- **How do you get closer?** Find the gaps with the five check items, and move them into mechanisms one at a time.

[Chapter 19](#) covers the principles in the Verification group, starting with "Prove It Works," in detail.

Once it is clear what to trust, the next question is how to build more trust. If you verify an artifact once but cannot repeat the same check on the next change, trust does not build up. [Chapter 3](#) explains three ways to raise trust and a way to keep corrected lessons in mechanisms.

Chapter 3: Make verification a repeatable mechanism

kaito • Sources: 日本語 • English • 2026-10-03

To build trust, you need to **keep a one-time check in the repository in a form that gives the same result no matter who runs it or how many times.**

In the second theme of the [talk](#), poteto names three ways to build trust.

1. Verify the artifact directly
2. Teach high-quality ways of working
3. Structure the codebase so that it is easy to understand

This chapter explains how the three ways differ, how pstack implements each one, and how to keep the lessons from corrections in a mechanism.

How this chapter is organized

This chapter covers the following topics.

- Three ways to build trust, and how they differ
- Way 1: turn the way you verify into a tool you can run any number of times
- Way 2: hand over an expert's way of working as steps
- Way 3: make the structure of the codebase itself the instructions
- Keep lessons from corrections in a mechanism, not in a conversation
- Summary: keep a one-time check in a form you can rerun

Three ways to build trust, and how they differ

The three ways differ in **how much they rely on the agent's own attention**.

Here are the three ways, each with an example.

1. **Verify the artifact directly.** Run the app for real. Use the Chrome DevTools Protocol (CDP) to take traces and heap snapshots. A trace records how long each operation took, and a heap snapshot records what is in memory at a point in time. CDP lets a program use the same features as the browser's developer tools.
2. **Teach high-quality ways of working.** Give the agent a pstack Playbook that sets the steps for investigation, reproduction, implementation, and verification.
3. **Restructure the codebase so that it is easy to understand.** Make it possible to tell, from folders, types, and process boundaries, what may be done where, which code runs where, and which imports are allowed.

The talk says that trust does not grow from effort or from the number of conversations. The scope of work you can delegate grows when you keep verification methods, work steps, and structural constraints in a form the agent can refer to.

The difference between the three ways becomes clear when you compare what the agent has to do for each one to work.

- **Way 1 (verification)** works only if the agent runs the tool. A tool that nobody uses checks nothing.
- **Way 2 (steps)** works only if the agent reads the steps and follows them. The agent can miss or misread them.
- **Way 3 (structure)** works even if the agent does not notice it. When types or lint express a boundary, the compiler or the lint step stops a change that crosses the boundary.

So the further you go from way 1 toward way 3, the less quality depends on the agent's attention.

Way 1: turn the way you verify into a tool you can run any number of times

The first way is **to turn the way you verify into a tool that lets anyone make the same observation any number of times.**

The verification the talk asks for is not a passing compile or the agent's report that it "fixed it". The talk treats the app's behavior when you actually run it as the evidence.

In pstack, "Build the Lever" and the verification skill handle it

In pstack, the principle "Build the Lever" and the Skill that builds a verification skill implement this way.

The principle says **not to finish a task by hand, but to build a script or command that a reviewer can read and rerun.** A verification skill is a project-specific Skill that lets the agent start its own app, operate it, and collect evidence.

In pstack, you build a verification skill with `/create-verification-skill` and keep it current with `/maintain-verification-skill`. [Chapter 26](#) and [Chapter 36](#) cover what a verification skill contains and give command examples.

Build the tool after one check by hand, and maintain it afterward

You build a verification tool after you do the first check by hand, and you keep maintaining it after you build it.

"Build the Lever" sets this way of building a tool: **process one target by hand to confirm the steps, turn those steps into a tool, then process the same target with the tool and check that the results match.** For verification, the order is to do the first check by hand, turn that operation into a command, and compare whether the command reproduces the same check.

The principle's file also says that if you write that you followed this principle but the diff has no codemod, script, or similar tool, you did not apply the principle. If a PR says "verified" but **has no rerunnable command or its output attached, the verification tool does not exist yet** ([Chapter 36](#)).

There are two cautions, one for before you build the tool and one for after.

- **Add a way to operate an app that is hard to operate.** [Part 1](#) of The Complete Guide to pstack says that the harder a tech stack is to debug and operate, the harder it is to get value from agents. It recommends that you build your own tools for verification, or even change the tech stack.
- **When verification fails, suspect the tool first.** The principle "[Prove It Works](#)" says, "When verification fails, suspect the method of observation before the system." When the app changes, the tool can go stale and return wrong results.

Way 2: hand over an expert's way of working as steps

The second way is **to give the agent a skilled engineer's pattern of work as steps**. The talk gives pstack's Playbooks as the example of this way.

A Playbook is not the answer itself. It is **a pattern for how to work: "this is how an expert would proceed"**.

Without a pattern, the agent picks a different approach every time. With clear steps, a different agent can work at about the same quality.

An example Playbook

For example, the "Bug fix" Playbook ([playbooks/bug-fix.md](#)) sets these steps in a fixed order: reproduce the bug in the same environment, narrow down the cause with evidence, fix it, confirm the fix in the same environment, and create the PR ([Chapter 11](#)).

In pstack, the entry point `/poteto-mode` picks the Playbook that fits the request and copies its steps unchanged to the top of the TODO list ([README](#)). [Part III](#) covers all 23 Playbooks one by one.

TODO list. The TODO list is the plan, a list of things to do, that the agent makes at the start of a task. Cursor shows it in the chat view. From this list, the user can see

which steps the agent worked through and which steps it skipped. The agent marks a skipped step with skip: <reason>, as the bundled guide [01-setup.md](#) describes.

Way 3: make the structure of the codebase itself the instructions

The third way is **to make the rules visible in folders, types, and process boundaries, so that the structure of the codebase itself is the agent's instructions.**

The goal is a codebase where folders, types, and process boundaries alone tell you three things.

- What may be done where
- Which code runs where
- Which imports are allowed

With a codebase like this, the agent can learn the constraints and rules from the structure without reading any explanation.

In pstack, the Architecture principles handle it

pstack groups its principles by theme, as the [list of principles in the README](#) shows. The "Architecture" group holds the principles for shaping code boundaries, types, and state handling into a structure where mistakes are less likely.

This section takes up two of them: "[Boundary Discipline](#)" and "[Type System Discipline](#)".

The README list summarizes the two principles as follows.

- "Boundary Discipline" ([principle-boundary-discipline](#)): check values that come in from outside, such as CLI arguments, config files, the network, and external APIs, at the point where the system receives them, and convert them to internal types. Internal code trusts those types and does not repeat the checks.
- "Type System Discipline" ([principle-type-system-discipline](#)): make invalid states, such as contradictory ones, impossible to express in the types.

[Chapter 18](#) covers the two principles in detail.

A rule that a machine cannot check cannot become structure

Only rules that types or lint can check mechanically can become structure.

For example, you can record the rule "code in this directory runs only in the browser" in two ways.

1. Write one sentence in the README
2. Put the code in its own directory and use lint to forbid imports from other directories

The first works only if the agent reads it. The second works even if nobody reads it. Put any rule that a machine can check in the second form.

Rules that need judgment of the situation, such as "use understated wording on this screen" or "split this kind of change into small pieces", cannot become types or lint. The only way to keep such rules is in writing. The principle "Encode Lessons in Structure" sets how to write them, as [the section on "Encode Lessons in Structure"](#) explains.

Keep lessons from corrections in a mechanism, not in a conversation

Do not leave the lesson from a correction only in the conversation. **Keep it in one of the tools, steps, or structures that the three ways produce, in the strongest form you can.**

The talk presents this idea right after the three ways.

Keep the lesson from a correction in one of these places.

- The codebase itself
- Automated checks by lint, the compiler, or CI
- Rules, or Bugbot, which is Cursor's automated code review
- A Skill
- The team's style guide

Then the next agent avoids the problem you pointed out once, without getting the same correction in a conversation.

In pstack, "Encode Lessons in Structure" handles it

In pstack, the principle "Encode Lessons in Structure" ([skills/principle-encode-lessons-in-structure/SKILL.md](#)) sets this rule. In the README list, it is the only principle in the "Meta" group. It does not deal with code itself but with the process of building mechanisms, that is, **how to keep lessons in a mechanism**.

The principle says: **when you notice you are writing the same instruction a second time, ask whether it can become a mechanism such as a lint rule or a runtime check. If it can, build the mechanism and delete the written instruction.** For rules that cannot become a mechanism, that is, the "rules that need judgment of the situation" from the section "[A rule that a machine cannot check cannot become structure](#)", make the instruction prominent and add an example of failure.

When you choose a mechanism, choose the strongest one you can, such as types or lint. The agent copies what the surrounding code already does, so a weak mechanism becomes the example for the next code.

If you correct the agent in a conversation and it replies "I'll remember that", no mechanism keeps the lesson. [Chapter 21](#) covers in detail what to do when you write the same instruction a second time, the order of mechanism strength, and how to sort corrections.

Grow review triage criteria as a file too

The criteria for judging review comments can also grow as a file that keeps lessons. The "Babysit" Playbook, which carries a PR to a mergeable state ([playbooks/babysit.md](#)), sorts Bugbot's comments according to a criteria file ([references/bugbot-triage.md](#)). When the Playbook dismisses a comment for a reason the team could reuse, it proposes adding that reason to the shared criteria in a separate PR ([Chapter 14](#)).

This way, what one agent learns in one run goes into a file that the next agent reads.

Summary: keep a one-time check in a form you can rerun

- **The three ways and how they differ.** The three ways the talk names are to verify the artifact directly, to teach high-quality ways of working, and to structure the codebase so that it is easy to understand. They differ in how much they rely on the agent's own attention. A tool works only if someone uses it, steps work only if someone reads them, and structure works even if nobody notices it.
- **Way 1.** As the principle "Build the Lever" says, turn the way you verify into a tool you can rerun. Do the first check by hand, turn it into a command, and when verification fails, first suspect that the tool has gone stale.
- **Way 2.** Hand over a pattern, such as a Playbook, for "how an expert would proceed". In pstack, /poteto-mode picks the Playbook that fits the request and copies its steps to the TODO list.
- **Way 3.** As the Architecture principles say, use checks at boundaries and types to stop mistakes through structure. Rules that need judgment of the situation cannot become structure.
- **Lessons from corrections.** As the principle "Encode Lessons in Structure" says, keep them in the strongest mechanism you can. For rules that can live only in writing, make them prominent and add a failure example. "I'll remember that" keeps nothing.

Chapter 4 takes the direction of "keeping it in the repository" one step further and covers the talk's third theme, "treating the codebase as memory". It explains why knowledge belongs in the repository and how to choose the memory you keep and maintain it over time.

Chapter 4: Make the codebase the agent's memory

kaito • Sources: 日本語 • English • 2026-10-03

The memory that lets the agent carry forward earlier decisions belongs in the **codebase**, not in the model.

poteto presented this idea as the third theme of the [talk](#).

This chapter explains why knowledge belongs in the repository and how to choose the memory you keep and maintain it over time.

How this chapter is organized

This chapter covers the following topics.

- Why put knowledge in the repository rather than in the model
- How kept memory goes stale, and how to maintain it
- What to delete so that the next agent's examples stay clean
- How to keep up the maintenance work
- Summary: choose what memory to keep, and maintain it

Why put knowledge in the repository rather than in the model

You put knowledge in the repository because **the context the agent can handle at once is limited, and you cannot pack all the knowledge it needs into every prompt**.

You cannot include every past decision, known issue, reproduction step, and workaround in every request.

If the knowledge lives in the repository, **the agent can read what is there and continue the work instead of thinking from zero each time.**

Memory here means more than code. The following items also act as memory that passes past decisions to the next agent.

- The Feature Map, a document that records how to use and verify each feature of the app. [The section on how kept memory goes stale](#) covers it in detail.
- Reproduction steps
- Design constraints
- Tests
- Lint rules

poteto writes the same idea in [Part 1](#) of The Complete Guide to pstack.

Personally, I think your codebase is the ultimate form of memory.

poteto explains why. Unlike Markdown notes and the other forms often used for agent memory, **code reflects the decisions the team has actually made, and it is a reliable source for what actually happens and how things work.**

How kept memory goes stale, and how to maintain it

Of the memory you keep, **the kind that needs maintenance is memory like the Feature Map, which goes stale without any signal when the app changes.**

When tests or lint disagree with the app, CI fails, so you notice that they are stale. A Markdown document like the Feature Map, on the other hand, triggers nothing when the app changes.

The verification page of the guide bundled with pstack ([docs/guide/06-verify-and-ship.md](#)) also opens its section on maintenance with this sentence.

Apps change and feature maps rot.

An agent given a stale Feature Map trusts what it says, operates the app accordingly, and spends time looking for screens that no longer exist.

The Feature Map records how to use and verify each feature

The Feature Map is a document that records, for each feature of the app, what it does, how an end user reaches it, how to operate it with the verification skill, and what result to check.

With a Feature Map, the agent does not have to read the whole codebase, because it reads only the files for the features it needs. Part 1 of The Complete Guide to pstack describes the Feature Map as one form of memory that saves context tokens.

In pstack, `/create-verification-skill` creates the Feature Map along with the verification skill (Chapter 3). Chapter 36 covers the file format and what to write.

Keep the Feature Map current with `/maintain-verification-skill`

`/maintain-verification-skill` repeatedly checks that the Feature Map matches the current app. Chapter 36 covers details such as how often to run it.

According to the bundled guide's verification page, `06-verify-and-ship.md`, `/maintain-verification-skill` works in this order.

1. Run subagents in parallel, one per feature, that read only the source
2. Actually run every feature in the map and verify it
3. Return exactly one of these results
 - `clean`. It verified every feature in the map and found nothing to fix.
 - `changed`. It fixed the mismatches and put the verified fixes in one PR. The fixes stay inside the verification skill's directory.
 - `blocked`. It could not continue verifying, and it reports what got in the way.

The same page goes on to say this.

It never edits product code. If the live pass catches a product regression, it reports the regression instead of papering over it in docs.

When the app and the memory disagree, `/maintain-verification-skill` does not rewrite the memory to make them agree.

A mismatch has two possible causes.

- **The app changed.** The memory is out of date, so fix the memory.
- **The app broke.** The app is wrong, so leave the memory alone and report the bug.

If you fix the memory in the second case, **the broken behavior stays in memory as the correct behavior.**

Chapter 26 and Chapter 36 cover how to build and maintain the Feature Map.

What to delete so that the next agent's examples stay clean

Delete **ad hoc workarounds and second ways of doing the same thing.**

Both stay in the code, and the next agent reads them as examples to follow.

A workaround you keep becomes the next agent's example

If you keep a workaround, the next agent copies it, so delete it once you fix the cause.

You should not keep everything as memory. If you keep an ad hoc workaround, **the next agent takes it for a correct implementation and copies it. The copy then produces the next copy, and an undesirable pattern spreads without anyone noticing.**

Two pstack principles also mention this danger.

- "**Encode Lessons in Structure**": as Chapter 3 showed, this principle explains that the agent copies what the surrounding code does, so a weak mechanism becomes the example for the next code.
- "**Fix Root Causes**": this principle says that fixes for symptoms pile up, and it says the following.

If a workaround needs a paragraph-long comment to justify it, the code is wrong.

pstack also guards against leftover workarounds inside its steps. The "Bug fix" Playbook ([playbooks/bug-fix.md](#)) says the following before its steps.

Belt-and-suspenders that "might help" is a hypothesis, not a fix. It does not ship. When evidence refutes a hypothesis, revert what it motivated.

This book reads the rule as a guard that keeps trial changes from staying in the code as workarounds.

Narrow the recommended implementation to one and build a "paved path"

If there are two or more ways to do the same thing, keep the one you recommend, delete the older one, and use lint to block the others.

Remove the technical debt, narrow the recommended implementation to one, and detect anti-patterns mechanically with lint or similar tools. Then **the agent no longer has to choose among several methods every time**.

The talk calls this standard implementation the "**paved path**". It means a state where the agent follows a maintained route instead of looking for its own shortcut every time.

Some pstack principles exist to pave this path. The list of principles in the [README](#) summarizes two of them as follows.

- "Subtract Before You Add" ([principle-subtract-before-you-add](#)): first remove what is unneeded, duplicated checks, and references with nothing behind them, then build on the simpler base.
- "Migrate Callers Then Delete Legacy APIs" ([principle-migrate-callers-then-delete-legacy-apis](#)): if you do not need to keep external compatibility, switch the code that uses the old API to the new API and delete the old API in the same change. Do not keep a compatibility layer, that is, code that you keep only so that code using the old API keeps working.

If you keep a compatibility layer, the codebase has two paths that do the same thing, and the agent treats both as examples. [Chapter 17](#) and [Chapter 18](#) cover each principle in detail.

In this book, I treat the choice of which way becomes the paved path as a human decision. The human's role is to set the direction and take final responsibility ([Chapter 5](#)), and I read the choice of the paved path as part of setting that direction.

How to keep up the maintenance work

You keep up the maintenance work with pstack's parts.

The talk compares the work of keeping this environment in shape to a **gardener's** work. A garden does not stay healthy if you only add plants. You have to keep removing unneeded branches and weeds and keep maintaining it so that the plants you want can grow.

The same is true of a codebase. **You have to keep removing old workarounds and bad implementation examples, in addition to adding knowledge.**

pstack has parts that help with this maintenance.

Maintenance task	pstack part that helps
When you make the same correction twice, turn it into a mechanism	The principle "Encode Lessons in Structure" (Chapter 3 , Chapter 21)
Watch for a growing number of comments that turn off lint or type checks	<code>/no-comments</code> , a Skill that removes unneeded comments. Its step 2, which reviews subagent reports and the diff, asks it to look for lint and TypeScript suppressions that slipped through (Chapter 31)
Replace comments that say "do not delete" or "do not change" with types, tests, or lint	Step 5 of <code>/no-comments</code> , which handles comments that state constraints. The step proposes that the agent replace the comment with the lightest option among a type, a runtime check, a test, and a lint rule in CI
Find two or more ways to do the same thing and narrow them to one	The principles "Subtract Before You Add" and "Migrate Callers Then Delete Legacy APIs" (Chapter 17 , Chapter 18)
Verify that the Feature Map and reproduction steps match the current app	<code>/maintain-verification-skill</code> (Chapter 26)

Maintenance task	pstack part that helps
Keep the steps learned in a long task for next time	<code>/reflect</code> . The README describes it as a Skill that saves the steps of a finished long task as edits to Skills (Chapter 33)

So pstack has Skills that write code, and it also has **parts that remove workarounds, comments that turn off checks, and old APIs, and parts that replace comments that state constraints with types or lint.**

Summary: choose what memory to keep, and maintain it

- **Why put knowledge in the repository.** Context is limited, and you cannot pack the knowledge the agent needs into every prompt. poteto calls the codebase "the ultimate form of memory".
- **How memory goes stale and how to maintain it.** When tests or lint go stale, CI tells you, but Markdown like the Feature Map goes stale without any signal. Check the Feature Map repeatedly with `/maintain-verification-skill`, and when the app is broken, report the bug instead of fixing the memory.
- **What to delete.** Delete ad hoc workarounds and second ways of doing the same thing. Both become examples for the next agent. As the "Bug fix" Playbook says, revert a trial change as soon as evidence refutes it. Narrow the recommended way to one and block the others with lint. A human decides which way to keep.
- **How to keep up the maintenance.** As with a garden, you keep a codebase in shape by removing things all the time, not only by adding them. pstack has parts that help, such as the principle that turns repeated corrections into mechanisms and `/no-comments`, which looks for suppressed checks.

With this chapter, the book has covered what to trust ([Chapter 2](#)), how to build trust ([Chapter 3](#)), and the memory that carries trust forward ([Chapter 4](#)). [Chapter 5](#) covers the talk's last theme, automation. It explains why automation comes last, which work you can start automating, what you can automate, and what remains for humans after automation.

Chapter 5: Build automation on a foundation of trust

kaito • Sources: 日本語 • English • 2026-10-03

Automation **works only once the agent can verify artifacts, you have defined the work procedures, and the codebase keeps past decisions.**

For this reason, poteto put automation last of the four themes in the [talk](#).

This chapter explains, in order, why automation comes last, which work you can automate first, what you can automate once the foundation is in place, and what is left for humans after automation.

How this chapter is organized

This chapter is organized as follows.

- Why automation comes last
- Which work you can automate first
- What you can automate once the foundation is in place
- What is left for humans after automation
- Summary: automation goes last, on top of trust and memory

Why automation comes last

Automation comes last for two reasons.

1. **If you automate without a foundation, what grows is not results but artifacts nobody can check and repeated failures**

2. The agent's ability alone does not make quality stable. You first need an environment where that ability can work

Automation without a foundation multiplies artifacts nobody can check

The talk takes each missing foundation in turn and explains what happens when you automate without it.

Missing foundation	What happens when you automate
You cannot trust the work (Chapter 2 , Chapter 3)	Artifacts whose correctness nobody can check pile up
The codebase does not keep past decisions (Chapter 4)	The same failures repeat again and again
Several ways of implementing the same thing remain, with no order (Chapter 4)	Automation produces undesirable patterns in bulk

In every case, **automation takes a problem the missing foundation already had and multiplies it as is**. So **automation goes in the last stage, after the three foundations are in place**.

A Michelin kitchen needs procedures and tools as well as skilled cooks

The talk explains the second reason with the image of a Michelin kitchen.

The talk describes a system that combines agents, development tools, verification, and automation to keep producing software changes, and calls this system a "software factory." It also compares the same system to a "Michelin kitchen."

A kitchen that serves high-quality food consistently needs more than excellent cooks. It also needs the following systems.

- Cooking procedures
- Tools
- Ingredient management
- Hygiene standards
- Division of roles

- A system for checking quality

Development with agents is the same. **Instead of relying on the agent's ability alone, you have to design an environment where the agent can use that ability safely.**

The verification tools, the work procedures, and the codebase as memory that Chapters 2 to 4 covered make up this environment.

Which work you can automate first

You can automate **work that meets all five of the following conditions.**

Earlier chapters of this book cover each condition.

Condition to set up before automating	Chapter that covers it
The agent can verify artifacts directly	Chapter 2 , Chapter 3 (way 1)
You have defined high-quality work procedures	Chapter 3 (way 2)
The codebase keeps past decisions	Chapter 4
The recommended way to implement things is clear	Chapter 4 (the paved path)
Automated checks can stop undesirable implementations	Chapter 3 (encoding lessons in structure), Chapter 4

What you can automate once the foundation is in place

Once the foundation is in place, **you can automate the work up to the point where the agent receives a bug report from a user and tries to reproduce it. There is also a plan to go as far as the fix once the verification skill and the Feature Map are good enough.**

The talk describes an end state. Once the conditions from "[Which work you can automate first](#)" are in place, a system combines several agents and tools to keep implementing changes, verifying them, and shipping them to production.

Combine the components once the conditions are in place

The talk names the following four components for building this system. Each one is something you combine only after the conditions from the previous section are in place.

- **Grok Bot.** An AI bot app. You can install pstack in it as a plugin.
- **Cloud Agents.** Agents that run in virtual environments on Cursor's cloud infrastructure.
- **automations.** A mechanism that runs a process at a scheduled time or when an event happens.
- **Agent SDK.** A development kit for calling agents from a program.

The Complete Guide to pstack [Part 1](#) explains how to use three of these as follows.

- **Cloud Agents.** Each agent runs on its own real computer, where it can install dependencies, run the app, and record videos and screenshots.
- **Grok Bot.** Instead of doing the work itself, the bot starts Cloud Agents. This division of work keeps the bot free. It also keeps the small back-and-forth of the work, such as reading code and running commands, out of the bot's context window. So **you use the bot as a coordinator that manages and supervises Cloud Agents.**
- **Grok Bot routines and Cursor Automations.** Once you are satisfied with a verification skill, you add it here and have it run on a schedule or when an event happens.

In automatic reproduction of reports, the quality of the foundation shows directly in the result

Automatic reproduction of reports is a clear example of how the quality of the foundation decides how well automation works.

The Complete Guide to pstack [Part 1](#) gives the following use as an example of automation.

1. Have the bot listen to the Slack channel that carries user feedback, or to an internal feedback channel
2. Each time a report arrives, have Cloud Agents try to reproduce it automatically

3. If the verification skill and the Feature Map are good enough, even have the agents go on to the fix automatically

The third step comes with a condition: "if the verification skill and the Feature Map are good enough." From this condition, you can read that **how far automation can go depends on the quality of the tools that operate the app to verify it (Chapter 3) and of the memory that records how to use and verify each feature, the Feature Map (Chapter 4).**

What is left for humans after automation

What is left for humans is **setting the direction and holding final responsibility.**

The agent works inside an environment that provides clear procedures and constraints. If that environment is good enough, a human does not need to watch beside the agent all the time, and the agent keeps working at a steady quality even while the human is away.

The "Autonomy" section of pstack's entry point, `/poteto-mode`, draws the line between the human and the agent in concrete terms. The section says that **the agent goes ahead with reversible work without asking the human, and always waits for the human before an irreversible write, such as a force-push to a shared branch, a deploy, a data deletion, or a message to a customer (Chapter 35).**

Chapter 20 covers the thinking behind this line as the principle "Never Block on the Human."

Individual Playbooks state the same line. The "Babysit" Playbook (`pplaybooks/babysit.md`), which carries a PR to a mergeable state, does not merge on its own. When a human explicitly asks for a merge, the Playbook hands off to the "Shipping" Playbook, which handles merging. The [bundled guide](#) explains that the agent does not merge on its own even when every check passes, because merging is a separate decision.

The talk also looks further ahead. It says that trust in the agent's artifacts grows as a codebase gets closer to one whose correctness can be formally verified. Someday, human code review itself may become an exceptional task instead of routine work.

Summary: automation goes last, on top of trust and memory

- **Why automation comes last.** If you automate without a foundation, you multiply artifacts nobody can check, repeated failures, and undesirable patterns. Also, as the talk's Michelin kitchen comparison shows, the agent's ability is not enough, and you need an environment where that ability can work.
- **Which work you can automate first.** Start with work that meets all five conditions this book laid out: the agent can verify it directly, it has procedures, the codebase keeps past decisions, the recommended implementation is clear, and automated checks can stop undesirable implementations.
- **What you can automate.** You can automate the work up to the reproduction of a bug report from a user. There is also a plan to go as far as the fix once the verification skill and the Feature Map are good enough. You combine Grok Bot, Cloud Agents, automations, and the Agent SDK only after the conditions are in place.
- **What is left for humans.** Humans set the direction and hold final responsibility. / poteto-mode requires the agent to always stop before an irreversible operation.

[Part I](#) followed the talk's four themes to look at the conditions for delegating work, with little mention of pstack's concrete components. The four themes are trust, ways to raise trust, the codebase as memory, and automation. [Part II](#) looks at how pstack meets these conditions with components called Skills, Playbooks, and Principles.

Part II: What pstack is

kaito • Sources: 日本語 • English • 2026-10-03

Part II explains, through pstack's design, "**which parts pstack uses to meet the conditions from Part I.**"

In short, pstack meets these conditions with **a design that has a single entry point and reads the contents only when needed, and only as much as needed.**

As a rule, the only entry point the user has to remember is `/poteto-mode`.

`/poteto-mode` looks at the kind of request and picks one Playbook. The Playbook decides the order of work and calls the Skills each step needs. Meanwhile, the Principles act as the criteria for judgments such as "should I take this step" and "is the work not finished yet."

pstack does not load everything up front. It opens only the one Playbook that fits the request. For Principles, it first reads only the list, and it reads a principle's body when that principle's conditions apply. This design is what lets pstack **follow procedures strictly and still use tokens efficiently.**

What you learn in Part II

After you read Part II, you will know the following.

- The problem pstack tries to solve, and what it was built to achieve
- What happens when you hand a request to `/poteto-mode`, and how to write a good request
- What Playbooks, Skills, and Principles each handle, and how they differ
- The mechanism that "calls up procedures and criteria when needed," and its benefits

Part III through Part V go into detail on every item in pstack. Part II is the map for reading those details. Once you grasp how the parts divide the work, you can read each item without confusion about "which part it belongs to, and which role it plays there."

Chapters in this part

Chapter	Role
Chapter 6: What problem pstack solves	The overall picture and purpose of pstack. What it treats as the problem and what it aims for
Chapter 7: /poteto-mode, the entry point that routes each request	How to use the entry point. How /poteto-mode routes requests to Playbooks, and how to write a request
Chapter 8: What do Playbooks, Skills, and Principles each handle?	The roles of the three parts and how they differ. Where subagents fit
Chapter 9: Why pstack loads only what it needs	The loading design. Why pstack does not make the agent read everything all the time

Chapter 6 covers "what it is for," Chapter 7 covers "how to use it," and Chapter 8 and Chapter 9 cover "how it works inside." In Chapter 8, the duplicate-row request introduced in the Preface returns as an example. The request goes to the "Bug fix" Playbook, and the chapter traces where Skills and Principles come into play. I recommend that you read the chapters in order.

Chapter 6: What problem pstack solves

kaito • Sources: 日本語 • English • 2026-10-03

poteto built pstack as a plugin that answers the problem that "AI writes too much sloppy code". In the words of the [README](#), its concept is "**if you want to go fast, go deep first**". It is a tool for building a high-quality product with less code.

This chapter explains the problem pstack addresses and how pstack answers that problem, what pstack contains and how that maps to the four themes of [Part I](#), and when to use it.

The four themes of [Part I](#)

1. Trust in an artifact through verification ([Chapter 2](#))
2. More trust through verification tools, work procedures, and code structure ([Chapter 3](#))
3. The codebase as a memory that carries past decisions forward ([Chapter 4](#))
4. Automation on top of those ([Chapter 5](#))

How this chapter is organized

The chapter has these sections:

- What problem is pstack trying to solve?
- How does pstack answer that problem?
- What does pstack contain, and what does it leave out?
- How do pstack's parts map to the four themes of the talk?
- What work is pstack for, and how do you use it?

- Summary: pstack answers the sloppy-code problem with "go deep first"

What problem is pstack trying to solve?

The problem pstack tries to solve is that **AI writes too much sloppy code (slop)**.

pstack is a plugin for Cursor that poteto publishes, and its README opens by stating this problem.

there's a growing sense that ai writes too much slop code. i agree. i don't want to ship like a team of twenty slop artists. throughput without quality is not a goal i aspire to. if you want to go fast, go deep first.

The README then calls pstack "my answer," meaning poteto's answer.

How does pstack answer that problem?

pstack's answer is the following three directions:

1. **Write less**
2. **Work deeply with one agent, and parallelize once you trust it to write good, verifiable code**
3. **Assign models by role**

In the README, the three paragraphs that follow the problem statement and start in bold match these three directions:

- **pstack is my answer.** Write less code of higher quality instead of maximizing lines of code.
- **pstack gives you fearless parallelism.** If you work deeply with one agent and trust it to write good, verifiable code, you can parallelize with confidence.
- **cursor gives you the best of all worlds.** Every model has strengths and weaknesses, and pstack can use any model. Many Skills also combine several models to use the strengths of each.

Writing less does not mean cutting evidence

The first direction takes the position that maximizing line count works against the goal. Among the principles, "[Laziness Protocol](#)" matches this direction. This principle tells the agent to choose the smallest change and to prefer deletion ([Chapter 17](#)).

"Write less", however, does not mean cutting the code that provides evidence. The preamble of the "Bug fix" Playbook ([playbooks/bug-fix.md](#)) defines what to cut as follows.

Every shipped line traces to runtime evidence. Belt-and-suspenders that "might help" is a hypothesis, not a fix.

The agent adds no "just in case" changes. It adds only the changes needed to fix a problem it confirmed by running the code.

At the same time, step 5 of the same Playbook, which orders the commits, asks the agent to commit the failing reproduction test before the fix. The agent can skip this commit when, for example, the test is expensive. **The agent cuts unfounded "just in case" measures and changes. It does not cut code that provides evidence, such as a reproduction test that confirms the bug.**

Parallelize only after you trust one agent's artifacts

The second direction says that you can parallelize with confidence only after one agent that works deeply has earned your trust that it writes good code you can verify. In [Part 1](#) of The Complete Guide to pstack, poteto also writes in this order of "trust, then parallelize". In that order, you consider parallelizing only after you have merged a few PRs with the help of a verification skill.

Assign models by role

For the third direction, the README says that the defaults as of 0.15.5 assign models as follows:

- **Delegates that write code** (Feature, Refactoring, Bug fix, Perf, Hillclimb): Grok (xAI's model)
- **The hardest changes, prose, and judgment:** Opus 5.5 (Anthropic's model)

- **Panels:** Opus 5.5, GPT-5.6 Sol, and Grok. A panel gives the same question to several models and compares the answers.

You can change these assignments with `/setup-pstack`. [Chapter 34](#) covers how the assignment works.

The use of several models is also more than a matter of taste. The "[Subagents](#)" section of [/poteto-mode](#) defines a second opinion as follows.

A second opinion is the same prompt against a different model. Agreement is high-signal.

The agent gives the same question to a different model and checks whether the answers agree.

For example, you ask Opus 5.5 and Grok 4.7 separately for their diagnosis of a bug's cause. If both name the same cause, that diagnosis is more trustworthy than either answer alone. **In pstack, splitting work across models is one of the ways to verify.**

What does pstack contain, and what does it leave out?

As of 0.15.5, pstack contains **five kinds of parts**. On the other hand, **some Skills that the Playbook steps use live in other plugins or in Cursor itself and are not part of pstack. pstack also contains no Skill for writing plan documents.**

It contains five kinds of parts

All counts are as of 0.15.5 ([commit 12d587d](#)).

Part	Count	Location	Role
Playbook	23	<code>skills/poteto-mode/playbooks/*.md</code>	A procedure for each kind of work. A reference file inside <code>/poteto-mode</code>

Part	Count	Location	Role
Principle	23	skills/principle-*/SKILL.md	A standard for judgment. A short Skill with one principle per file
Skill	24	skills/*/SKILL.md, except Principles	A capability you invoke. It includes /poteto-mode and /setup-pstack
subagent	2	agents/poteto-agent.md, agents/comment-sicko.md	A definition of a separate agent that the main agent starts
Bundled guide	10 pages	docs/guide/*.md	The guide shipped with pstack. It walks from the first task to overnight runs, with example requests

Three points are easy to misread:

- **A Playbook is not a Skill.** A Playbook is a reference file that /poteto-mode opens to match the request, and you cannot invoke it on its own. In [Part 2](#) of The Complete Guide to pstack, poteto also explains that a Playbook is not a Skill but a reference file inside /poteto-mode. [Chapter 8](#) covers the difference.
- **A Principle is formally a Skill.** The README, however, does not list Principles in its "skills" table but has a separate "principles" table. This book also counts the 24 Skills and the 23 Principles separately.
- **The bundled guide is not the same as The Complete Guide to pstack on X.** This book calls docs/guide/ "the bundled guide" and calls the articles on X The Complete Guide to pstack, Part 1 and Part 2.

Some Skills the steps rely on by name are not bundled

/poteto-mode also refers by name to Skills it does not bundle. Without them, part of a procedure cannot run. The README's "[not shipped here](#)" section lists them.

Skill that /poteto-mode names	Where it actually lives	How /poteto-mode uses it
/deslop (the deslop skill)	The <code>cursor-team-kit</code> plugin	Removes AI-style sloppy code before a commit
<code>control-cli</code>	The <code>cursor-team-kit</code> plugin	Drives a CLI or TUI to verify it
<code>control-ui</code>	The <code>cursor-team-kit</code> plugin	Drives a browser, Electron, or web UI to verify it
/create-skill	Built into Cursor	Conventions for writing a <code>SKILL.md</code>
/autopi lot (formerly /babysit)	Built into Cursor	Checks the status of a PR. pstack uses the " Babysit " Playbook instead (Chapter 14)

The README recommends that you also install `cursor-team-kit` if you want the full set.

`control-cli` and `control-ui` matter most. These two are the app-driving Skills that the "Bug fix" Playbook uses in step 1, where the agent reproduces the bug itself. With only pstack installed, the agent may have no way to drive the app to reproduce the bug and verify the fix.

If you have a verification skill for your own app, such as a `verify-<app>` Skill made with `/create-verification-skill`, that skill is the way to drive the app. If you have neither, first prepare a verification skill with the steps in [Chapter 36](#).

There is no Skill for writing plan documents, because "the best spec is code"

pstack has no dedicated planning Skill because poteto treats code, not plan documents, as the spec. The README's "[why are there no planning skills?](#)" section says that Cursor's plan mode works well with pstack and then states that "the best spec is code" ([Chapter 38](#)).

The same section also says that if you want a plan, /poteto-mode can make one, but not by default. In Part 2 of The Complete Guide to pstack, poteto adds that the missing planning Skill does not mean a lack of planning. It means planning through code. poteto also writes about not running adversarial reviews of abstract plans.

Inside pstack, these three parts do the "planning in code":

- **The "Prototype" Playbook** ([playbooks/prototype.md](#)). It settles design and behavior decisions cheaply with a throwaway prototype. The "Non-negotiables" section of `/poteto-mode` says that **when the agent is about to ask a person for a fact it could observe by running something, it should build a small prototype with this Playbook and decide from the result.**
- `/architect`. A Skill that decides, before implementation, what the caller of a function passes in and gets back, and then designs the types and the module split to fit.
- **The "Multi-phase or multi-PR plan" Playbook** ([playbooks/multi-phase-plan.md](#)). This is where a plan document goes when one is needed. Its first step, however, says to skip the plan when the change fits in one or two files and the direction is clear. Its next step asks the agent to resolve open questions with a prototype before writing the plan document.

The third one, the "Multi-phase or multi-PR plan" Playbook, exists to write plan documents. Even so, it asks for a running prototype before any prose. **The position that code is the spec shows in which Skills exist, and it also shows in the Playbook steps.** [Chapter 16](#) and [Chapter 39](#) cover planning in detail.

How do pstack's parts map to the four themes of the talk?

Each of the four themes of [the talk](#), which [Part I](#) covered one chapter at a time, has matching parts in pstack. This book maps them as follows.

Theme of the talk (chapter in Part I)	Parts in pstack that handle it	Examples
Trust: verify the artifact (Chapter 2)	Verification skills, the principles in the Verification group (Chapter 19)	<code>/create-verification-skill</code> , "Prove It Works"
Raising trust: make verification and procedures reproducible (Chapter 3)	Playbooks, Skills that support the procedures	"Bug fix", <code>/tdd</code> , <code>/show-me-your-work</code>

Theme of the talk (chapter in Part I)	Parts in pstack that handle it	Examples
The codebase as memory: keep decisions and constraints (Chapter 4)	The principles in the Meta and Architecture groups (Chapter 18, Chapter 21), Skills that handle records	"Encode Lessons in Structure", /reflect, /why
Automation: build on the foundation (Chapter 5)	Playbooks for automation	"Autonomous run", "Orchestrate"

pstack keeps Playbooks, Principles, and Skills separate, and calls them together through the entry point `/poteto-mode`. Playbooks hold procedures, Principles hold standards for judgment, and Skills hold capabilities. Chapter 7 through Chapter 9 cover this mechanism.

What work is pstack for, and how do you use it?

Use pstack for **work that needs rigor**. To get started, you need to know **only two commands**: `/setup-pstack` **and** `/poteto-mode`.

It fits work that needs rigor

The README recommends that you use `/poteto-mode` "whenever you're doing anything that requires rigor". Conversely, the `SKILL.md` of `/poteto-mode` says **not to apply it to light exchanges like small talk, which it calls a Casual turn** (Chapter 7).

On that basis, work divides into what fits and what does not:

- **Work that fits.** Bug fixes that need reproduction and confirmation, features that involve design across function or module boundaries, long tasks you want to hand off while you step away, and similar work.
- **Work that does not fit.** A one-line typo fix, a question with a one-word answer, and similar work.

Even for work that fits, pstack's procedures cannot run without a way to run the app and verify it. The "Bug fix" Playbook requires the agent to "reproduce it yourself" and to "verify on the same surface", meaning the same screen or command where the bug

happened. Both requirements assume a way to verify. **So prepare a verification skill first, and then use pstack.**

Summary: pstack answers the sloppy-code problem with "go deep first"

- **The problem it tries to solve.** poteto built pstack as an answer to "sloppy code written by AI". The README says "if you want to go fast, go deep first".
- **How it answers the problem.** It takes three directions. Write less. Work deeply with one agent, and parallelize once you trust it to write good, verifiable code. Assign models by role.
- **What it contains and leaves out.** As of 0.15.5, it contains 23 Playbooks, 23 Principles, 24 Skills, 2 subagents, and a 10-page bundled guide. `/develop`, `control-cli`, and `control-ui` live in `cursor-team-kit`, and `/create-skill` lives in Cursor itself. Without the app-driving Skills in particular, the agent cannot reproduce or verify. There is no planning Skill because pstack takes the position that "the best spec is code", and that position also shows in the steps of the planning Playbook.
- **How it maps to the four themes of the talk.** The four themes are trust, raising trust, the codebase as memory, and automation. Each theme has matching parts.
- **When to use it and how to start.** Use it for work that needs rigor. To get started, you need to know two commands: `/setup-pstack` and `/poteto-mode`.

The next chapter, [Chapter 7](#), looks closely at the entry point, `/poteto-mode`. The chapter explains how to invoke `/poteto-mode` and what happens when you do, the rules that route a request to a Playbook, and how to write a good request.

Chapter 7: /poteto-mode, the entry point that routes each request

kaito • Sources: 日本語 • English • 2026-10-03

/poteto-mode is the entry point to pstack. **It reads the request, picks one of the 23 Playbooks, and follows that Playbook's steps to do the work.**

The user does not have to say which Playbook to use, or which Skills to call in which order. All the user does is **give the goal and the way to verify it, in their own words.**

This chapter explains how to call /poteto-mode, what happens when you call it, the rules that route a request to a Playbook, and how to write a good request.

How this chapter is organized

This chapter has the following sections.

- You call /poteto-mode by putting it at the start of the request
- Calling it makes three things happen
- Fixed rules route the request to a Playbook
- A good request states the goal and the finish condition in your own words
- Summary: start the request with /poteto-mode and state the goal and the check

You call /poteto-mode by putting it at the start of the request

You call /poteto-mode **by putting it at the start of a work request.**

Once you call it, it stays active for later turns in the same conversation even without /poteto-mode, as the section "Once called, it stays active for the rest of the conversation"

explains. If it is tedious to add it at the start of every conversation, you can also pin `/poteto-mode` as a [Custom Mode](#), a Cursor mode that makes the agent use the same Skill on every turn.

The basic usage is to put it at the start of the request

The pstack [README](#) positions `/poteto-mode` as the default entry point for all nontrivial work, and gives examples such as the following.

```
/poteto-mode this pr has a subtle bug where the scroll drifts every 750ms even when idle. repro first, then fix and verify.
```

This request does not say which Playbook or Skill to use. It states only two things. The symptom is that the scroll drifts every 750ms even when idle, and the work the user wants is to reproduce first, then fix, then verify.

`/poteto-mode` picks the Playbook and the Skills from the content of the request, so **the user needs to write only the symptom and the work they want done.**

The reminder, which the section "[In Cursor, you can pin it as a Custom Mode](#)" covers, decides whether `/poteto-mode` applies. The reminder says to apply `/poteto-mode` to a new task only when a Playbook matches or the task needs rigor. It also says not to apply `/poteto-mode` to light requests, such as casual chat, or to questions.

So when you are unsure whether to add `/poteto-mode`, I think it is better to add it. For a light request or a question, the agent answers without following a Playbook's steps.

In Cursor, you can pin it as a Custom Mode

If it is tedious to type `/poteto-mode` for every request, you can pin `/poteto-mode` as a [Custom Mode](#). The Complete Guide to pstack [Part 1](#) shows how.

When `/poteto-mode` autocompletes, press `Opt + Enter` instead of `Enter`. Cursor adds the Skill as a [Custom Mode](#), and on every new turn the agent gets a reminder to use this Skill.

The text of the reminder is in the frontmatter of `/poteto-mode`, as reminder.

```
mode: true
reminder: New task? Playbook match or rigor needed -> apply /poteto-mode. Casual turn or user opts
out -> don't.
```

Because of this one line, a pinned **Custom Mode** does not force a Playbook on every turn. The agent does not follow a Playbook's steps on turns with light requests such as casual chat, on turns with questions, or when the user says to stop the mode.

Once called, it stays active for the rest of the conversation

The README calls `/poteto-mode` a sticky mode. Sticky means that once you send a request with `/poteto-mode`, the mode stays active for later turns in the same conversation even without `/poteto-mode`.

In other words, the user runs `/poteto-mode` once in a conversation and does not need to add `/poteto-mode` explicitly on later turns.

While it is active, `/poteto-mode` follows a Playbook's steps only when the request matches a Playbook or the work needs rigor. For other requests, such as casual chat, it does not follow a Playbook's steps. To stop the mode, tell the agent to stop it.

So when you continue work in the same conversation, the request can be short. The bundled guide's [02-poteto-mode.md](#) lists examples such as these.

```
/poteto-mode do it
```

```
continue
```

```
keep going until done
```

The same page explains that a short request is enough because the mode is sticky and the Playbook contains the steps, which give the work its structure. **The user's words convey the intent, and the Skill takes on the job of following the steps strictly.**

But because `/poteto-mode` is sticky, when you move to a different topic in the same conversation, the agent may treat the new topic as a continuation of the previous task.

The section [Say "new task" when you change topics](#) in this chapter covers how to ask when you change topics.

Calling it makes three things happen

When you call `/poteto-mode`, three things happen. **It copies the Playbook's steps into a TODO list, calls other Skills as the steps require, and replies in a fixed style.**

TODO list. The TODO list is the plan, a list of things to do, that the agent makes at the start of the work. In Cursor, it appears in the chat view. From this list, the user can check which steps the agent worked through and which steps it skipped. The agent marks a skipped step with `skip: <reason>`, as the bundled guide [01-setup.md](#) describes.

The README describes the three as follows.

1. Match the request to a Playbook and open a TODO list. Its first items are the Playbook's steps, copied word for word
2. As the steps proceed, route to the other Skills they need
3. Write the reply to the user with the AI tells removed

1. Copy the Playbook's steps into the TODO list as they are

The central rule of `/poteto-mode` is to copy the steps as they are. The `/poteto-mode` section ["Playbooks"](#) begins with this rule.

Open a todolist whose first items are the matched playbook's steps, copied in verbatim, before any task-specific todos. A step you choose not to do stays in the list with a one-line `skip: <reason>`.

The agent **turns the Playbook's steps into the work list as they are, without summarizing them, and keeps the steps it will not do, with a reason.**

This rule has two effects.

- **The Playbook's order sets the structure of the work.** The agent has less room to summarize or reorder the steps on its own judgment.
- **Skipped steps stay visible.** Steps the agent will not do remain as skip: `<reason>`, so the human can inspect the agent's decisions along the way, not only the result. The bundled guide's `01-setup.md` also says that the TODO list shows what the agent decided not to do.

2. Call other Skills when a step requires them

Each step of a Playbook says which Skill to use and where. The agent does not call `/how` or `/architect` all at once at startup. It calls each one when the steps reach the point that needs it. [Chapter 8](#) and [Chapter 9](#) cover this mechanism.

3. Write the reply in a fixed style

The `/poteto-mode` section "[Writing the reply](#)" sets the style of the reply. Its rules include the following.

- Write short, declarative sentences with one idea each
- Write first who the work is for and what changes
- Attach evidence to each claim, or a label of "measured", "inferred", or "guessed"

Every Playbook ends with a reply in this style, and the "Reply" line at the end of each Playbook says what that Playbook reports. [Chapter 35](#) covers the full set of rules.

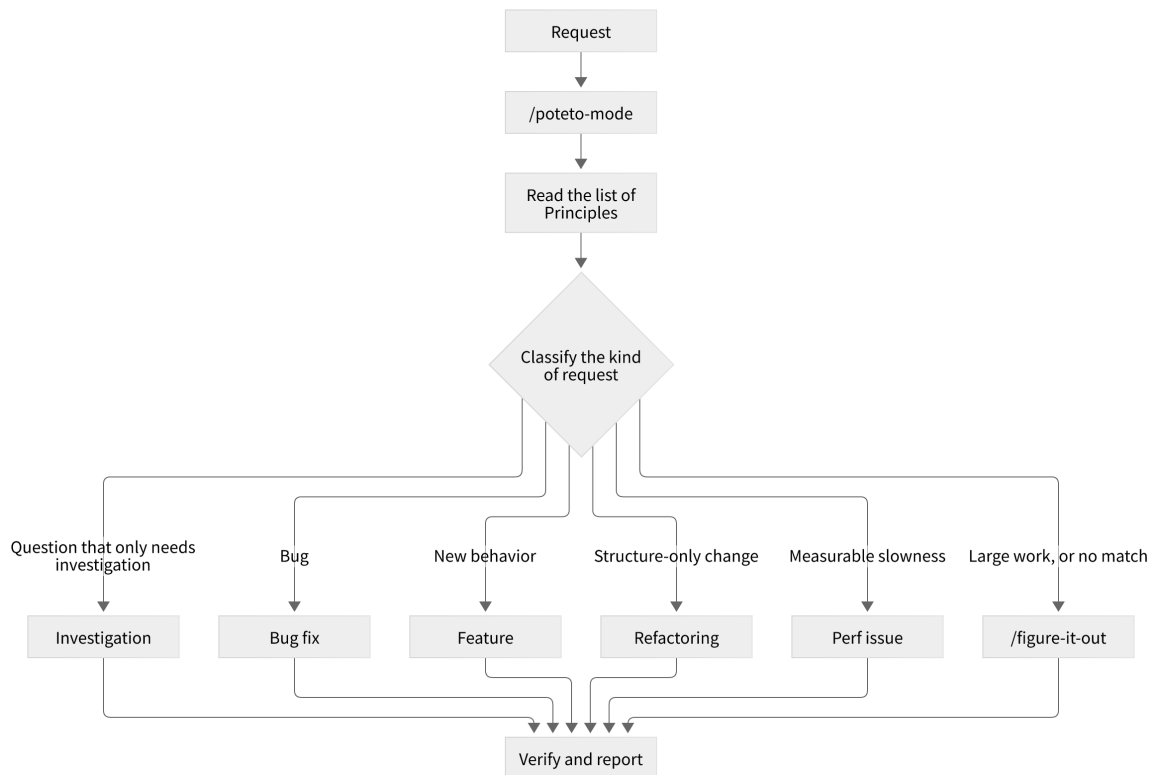
Fixed rules route the request to a Playbook

In the basic routing, **the agent looks at the kind of request, such as a question that only needs investigation, a bug, or new behavior, and picks the one Playbook that fits.**

But **when the work is large, its size decides the route, not the kind of request.** Even a request that matches a Playbook such as "[Feature](#)" goes to `/figure-it-out` or "[Orchestrate](#)".

The common routes are five Playbooks and one exception

To get the overall picture of routing, you need only the five main Playbooks and the one exception for when no Playbook fits ([/figure-it-out](#)). The following diagram redraws the routing diagram in the bundled guide's 02-poteto-mode.md.



[View diagram 1](#)

The diagram shows only the main routes. There are also Playbooks for the following work.

- Continuous improvement of a metric
- Diagnosis of runtime symptoms or of traces already captured
- Prototypes
- Visual matching
- Creating and evaluating Skills
- Long autonomous runs
- Opening a PR

- Watching a PR and shipping it to production
- Running a PR queue automatically
- Coordinating project-scale work
- Handing off and pausing work
- Multi-phase planning
- Cleaning up worktrees

Part III covers all 23 in detail.

In the diagram, a "Read the list of Principles" stage comes before the agent classifies the kind of request. [Chapter 9](#) covers how the agent reads Principles.

Large work goes to /figure-it-out, and project-scale work goes to "Orchestrate"

When the work is large, the agent routes it to /figure-it-out or the "Orchestrate" Playbook, ahead of any individual Playbook. These are two rules in the /poteto-mode section "Playbooks", and both change the route by the size of the work.

The first is /figure-it-out. The rule reads as follows.

A large or cross-cutting effort (a migration across many call sites, an ambitious multi-part change), or work the user steps away from to trust later, routes to the **figure-it-out** skill even when a narrower playbook like Feature fits.

The agent **decides the route by the size of the work, ahead of the wording of the request.**

Work that matches no Playbook also goes to /figure-it-out.

/figure-it-out is a Skill that designs a rigorous, auditable Playbook for that task.

The second is "Orchestrate". It covers project-scale work that lasts for days, where one coordinator agent and many subagents move a large stack of PRs forward.

The difference between the two is that /figure-it-out designs a Playbook for one piece of work, while "Orchestrate" runs a whole effort that lasts for days.

But the description of "Orchestrate" states explicitly that work one agent can finish within the session budget goes to "Autonomous run".

The rules come down to the following.

Nature of the request	Route
Normal-size work that matches one Playbook	The matching Playbook
Large, cross-cutting work, work you step away from and check later, or work that matches no Playbook	/figure-it-out
An ongoing program that spans several days, many PRs, and many subagents	"Orchestrate"
A long task that one agent can finish, such as "keep going until done"	"Autonomous run"

A good request states the goal and the finish condition in your own words

The bundled guide ([docs/guide/](#)) recommends four things for writing a good request.

1. **Write the goal and how to verify it**
2. **Do not list Skills**
3. **Do not make working time the finish condition**
4. **Say "new task" when you change topics**

Write the goal, not the steps

The bundled guide's `02-poteto-mode.md` explains that a request does not need a detailed spec. It is enough to say what is wrong, what you want to achieve, and anything you already know that helps the work.

The guide has the following example of a bug request.

```
/poteto-mode users get two notifications after a retry. repro first, then fix and verify.
```


The guide explains that "repro first" is not politeness but **a real constraint that the Playbook keeps**. `/poteto-mode` does route this request to the "Bug fix" Playbook, and the TODO list starts with the reproduction step.

The table of contents of the bundled guide also gives a one-line summary. If you remember only one thing, remember **to give the goal and the way to verify it in your own words**.

For the verification, a good model is the bundled guide's example from [Chapter 2](#): "text output does not change by a single byte, and the JSON parses." Both conditions are in a form the agent can run and judge as pass or fail by itself.

The verification is not the only thing worth writing in a request. Some example requests in *The Complete Guide to pstack* [Part 2](#) also state the shape of the result the user wants. For example, they ask the agent to show what is known, the data used, and the leading hypotheses separately. They also state when to review, for example "check with me before you proceed."

Do not list Skills

The bundled guide's `02-poteto-mode.md` says that a list of Skills in the request is a pitfall.

Pitfall: don't enumerate skills in your prompt ("use /how, then /architect, then /arena..."). The playbook already sequences them, and a hand-written sequence usually reorders or drops steps the playbook would have kept. Name a skill only when you want to override a specific choice.

For example, if a bug request says "investigate with /how, then design with /architect", the hand-written order may drop the "reproduce it yourself" step that the "Bug fix" Playbook puts first. The guide's table of contents also says at the top that **pstack works best when you stop micromanaging the agent**.

The Playbook treats words like "repro first", from the section "[Write the goal, not the steps](#)", as constraints it keeps. **Write the constraints in the request, and leave the steps for how to proceed to the Playbook**. That is the division of work.

"How many hours to keep going" is not a finish condition

When you step away, write the finish condition before you leave. The bundled guide's 02-poteto-mode.md has the following example.

```
/poteto-mode im stepping away. keep going until the migration check reports zero old callers. log your decisions.
```

The bundled guide's page on overnight runs (07-overnight.md) **says that a length of time as the finish condition is a pitfall.**

"Work on this for 4 hours" gives the agent nothing to verify, and in the morning you have 4 hours of activity instead of a result. [Chapter 15](#) covers overnight runs.

The goal in the example above, "the migration check reports zero old callers", is a check the agent can run and pass or fail by itself. **Instead of a length of time, write a check that runs and returns pass or fail as the finish condition.**

Say "new task" when you change topics

/poteto-mode stays active within a conversation, so a long conversation builds up context from earlier tasks. For this reason, the bundled guide's 02-poteto-mode.md recommends that you say "new task" when you change topics.

```
/poteto-mode new task. figure out why the cache entry survives logout. don't change any code yet.
```

In other words, **"new task" is the signal that makes the agent pick the Playbook again.**

If you change topics without saying "new task", the Playbook in progress may treat the new question as a continuation of the previous work.

Summary: start the request with /poteto-mode and state the goal and the check

- **How to call it.** /poteto-mode is the entry point to pstack. You call it by putting it at the start of a request, and it is the default entry point for all nontrivial work. Once called,

it stays active for the rest of the conversation. In Cursor, you can also pin it as a [Custom Mode](#) with `Opt + Enter`.

- **What happens when you call it.** It picks a Playbook, copies the Playbook's steps word for word into the TODO list, calls Skills as the steps require, and replies in a fixed style. Skipped steps stay as `skip: <reason>`, so you can inspect the work along the way.
- **The routing rules.** In the basic case, it picks one Playbook by the kind of request. Large work goes to `/figure-it-out`, and ongoing project-scale work goes to `"Orchestrate"`.
- **How to write a good request.** Write the goal and how to verify it in your own words, do not list Skills, do not make working time the finish condition, and say `"new task"` when you change topics.

The next chapter, [Chapter 8](#), covers the three parts that work inside `/poteto-mode`: Playbook, Skill, and Principle. It explains how their roles differ, and uses this book's running example request to trace where Skills and Principles act in each step of the `"Bug fix"` Playbook.

Chapter 8: What do Playbooks, Skills, and Principles each handle?

kaito • Sources: 日本語 • English • 2026-10-03

pstack is made of **three kinds of parts: Playbooks, Skills, and Principles**.

A Playbook decides the order of the work. A Skill provides an ability that a step uses. A Principle is a rule that guides a judgment during the work in a specific situation. The agent uses Principles for judgments such as keeping a change small, fixing the root cause, and verifying the real thing before it reports that it is done.

This chapter focuses on how the three parts differ and what each one handles. It then looks at Non-negotiables and the two subagents that connect the parts. Last, it traces where the parts act in the book's example request.

How this chapter is organized

This chapter is organized as follows.

- The three parts divide the work into "order," "ability," and "judgment"
- A Playbook decides the steps and their order for each kind of work
- A Skill is an ability that a step calls
- A Principle is a standard for judgments during the work
- Non-negotiables connect the three parts
- The agent splits work with two subagents
- Trace where Skills and Principles act
- Summary

The three parts divide the work into "order," "ability," and "judgment"

As I said at the start, the three parts cover different areas. **A Playbook handles the pattern of the work, such as "what to do, and in what order."** **A Skill handles an ability, such as "how to do this step."** **A Principle handles a judgment, such as "should I take this move? Am I really not done yet?"**

Inside `/poteto-mode`, the three parts connect as follows.

1. `/poteto-mode` looks at the kind of request and picks one Playbook (the section "Playbooks" of `/poteto-mode`)
2. The Playbook gives the steps of that work and their order, for example "reproduce, find the cause, fix, confirm" for a bug
3. Each step of the Playbook says where to use which Skill. The agent calls `/how`, which explains the current state, and `/arena`, which compares several options, when the work reaches that step, not at startup
4. A Principle is a rule for judgment, separate from the steps. When `/poteto-mode` starts, the agent reads only the list of Principles, which this book calls the index. When the current work meets a Principle's condition, the agent reads that Principle's file in full (the section "Principles" of the same file)

The table below sums up the differences among the three.

Aspect	Playbook	Skill	Principle
What it handles	The order of the work	An ability that a step uses	A standard for judgment
The question it answers	What to do, and in what order	How to do this step	Should I take this move? Am I really not done yet?
Count (0.15.5)	23	24	23
Location	<code>skills/poteto-mode/ playbooks/*.md</code>	<code>skills/<name>/SKILL.md</code>	<code>skills/principle-<name>/ SKILL.md</code>
Form	A reference file inside / <code>poteto-mode</code> , not a Skill	A standalone Skill	A short Skill, one principle per file

Aspect	Playbook	Skill	Principle
How it gets called	/poteto-mode picks the one that fits the request and opens it	When a Playbook step or a Non-negotiables condition applies. You can also call it directly, as with /how	The agent reads the list at startup and reads the body when the condition applies
Does it touch code?	It tells the agent what to do, as steps	It does the work, such as investigating, writing, and verifying	No. It only makes judgments
Examples	"Bug fix," "Feature"	/how, /architect, /tdd	"Prove It Works," "Fix Root Causes"

Chapter 9 covers when the agent loads each part.

A Playbook decides the steps and their order for each kind of work

A Playbook is a **procedure that says "what to do, and in what order" for one kind of work.**

/poteto-mode picks the one that fits the request and opens it.

What it handles: the steps and their order, written in three parts

Playbook files mostly share the same shape.

1. **Preamble.** It gives the basic rules for the whole Playbook.
2. **Numbered steps.** The agent copies this part word for word into the TODO list. Each step says where to use which Skill and which Principles apply.
3. **Reply line.** It sits at the end of each Playbook and says what to include in the reply after the work.

The **TODO list** is the plan, a list of things to do that the agent makes at the start of the work. In Cursor, the TODO list appears in the chat panel. From this list, the user

can check which steps the agent carried out and which steps it skipped. The agent marks a skipped step with skip: <reason> (the bundled guide [01-setup.md](#)).

A Skill is an ability that a step calls

A Skill is a **bundle of abilities or steps for doing one specific job**.

/poteto-mode calls most of them when a step needs them. The user calls the rest directly.

What it handles: one Skill handles one kind of job

Each Skill handles one kind of job. For example, /how and /why handle investigation, /architect handles design, and /arena compares several options.

The exception is /figure-it-out. For work that fits no Playbook, it writes a new Playbook for that work while it runs.

[Part V](#) (Chapters 22 to 35) covers each Skill in detail.

How it gets called: /poteto-mode calls most Skills when a Playbook needs them

/poteto-mode **calls most Skills on the user's behalf when a Playbook needs them**.

The table below groups the 24 Skills by how they get called.

How it gets called	Skill
/poteto-mode calls it based on the step or condition	/how, /why, /architect, /arena, /swarm, /interrogate, /unslop, /no-comments, /technical-writing, /tdd, /figure-it-out, /show-me-your-work
The user calls it	/setup-pstack, /recall, /teach, /bro, /blast-radius, /reflect, /automate-me, /make-bot-ui, /create-verification-skill, /maintain-verification-skill
The agent reads it when it works on TypeScript files	/typescript-best-practices (Chapter 29)
Entry point	/poteto-mode

Most of the Skills the user calls are for milestones in the work: installing pstack (/setup-pstack), resuming work (/recall), and wrapping up after long work (/reflect).

How the user uses them: you can call any Skill by name, and you call the 10 that no Playbook calls

Unlike a Playbook, you can call a Skill on its own. When all you want is to look something up, a direct call is sometimes faster.

```
/how do we cancel runs? do we have an n+1 when we look up every run to cancel?
```

The 10 Skills the user calls are not part of any Playbook's steps.

A Principle is a standard for judgments during the work

A Principle is neither a step nor an ability. It is **a standard for the judgments the agent makes during the work.**

The agent reads only the index at startup, and reads a Principle's body when it applies the Principle.

What it handles: the standard for judgments during the work

A Principle gives **the standard for judgments during the work**, such as whether to take the current move and whether the work is really not done yet. For example, the principle "Laziness Protocol" makes the agent prefer to delete code and make the smallest change when it is tempted to add an abstraction or a layer. The principle "Prove It Works" asks, before the agent declares the work done, whether it verified the result with the real artifact.

Each Principle is its own file, and Playbooks and Skills refer to it by name.

Part IV (Chapters 17 to 21) covers the 23 Principles in detail.

Non-negotiables connect the three parts

Non-negotiables are **a list of conditions of the form "in this situation, use this Skill."** **They connect the parts by applying shared rules across Playbooks.**

A Playbook decides the flow for one kind of work, but some rules apply to every kind of work. The section "Non-negotiables" of `/poteto-mode` sets those rules. The main conditions are as follows.

Condition	What it calls
A non-obvious change or a design decision	<code>/how</code>
Code that crosses function boundaries	<code>/architect</code>
Any writing, including replies	<code>/unslop</code>
Documentation, PR descriptions, or commit messages	<code>/technical-writing</code>
Before a commit	<code>/deslop</code> from the <code>cursor-team-kit</code> plugin
Before a review	<code>/no-comments</code>
A change to a UI, an IDE, or a CLI	The control Skills from the <code>cursor-team-kit</code> plugin
Long work, or work you leave and check later	<code>/show-me-your-work</code>

Non-negotiables **respond to "situations that come up during the work,"** not to "the kind of work."

In both "Bug fix" and "Feature," `/architect` applies when code crosses function boundaries, and `/deslop` applies before a commit.

The agent splits work with two subagents

pstack has **two subagents**: `poteto-agent`, **which works the `/poteto-mode` way,** and **Comment Sicko, which reviews comments.**

poteto-agent **is a subagent that works the** /poteto-mode **way**

poteto-agent (`agents/poteto-agent.md`) is a subagent that reads the /poteto-mode rules before it starts work. When a Playbook step starts a subagent, the step uses poteto-agent by default.

Comment Sicko deletes comments and reports code that needs fixing

Comment Sicko (`agents/comment-sicko.md`) is a subagent that reviews the comments in the code. On its own, it deletes the comments that have no reason to stay. It does not rewrite the app's code. Instead, its report marks code as **MUST KILL** when the code's shape should change so that the code makes sense without a comment. That code includes functions, variables, and similar names. /no-comments takes that report and fixes the code. You do not call Comment Sicko directly. You use it through /no-comments. [Chapter 31](#) covers it in detail.

The main agent is responsible for the subagents' work

When the agent hands work off to a subagent, the main agent stays responsible for it.

/poteto-mode tells the main agent not to pass a subagent's "done" straight to the user. The main agent looks at the diff first and then reports in its own words.

This rule applies the idea "trust the artifact, not the agent" from [Chapter 2](#) to subagents.

Trace where Skills and Principles act

Skills appear where the agent "does work," such as reproducing, investigating, designing, and committing. Principles appear where the agent makes a judgment and "decides" something. Examples of these decisions are how to reproduce, the shape and size of the fix, whether the work is done, and how to order the commits.

The book's example is the following request.

```
/poteto-mode the export writes duplicate rows when a retry lands mid-run. repro first, then fix and verify.
```

/poteto-mode routes this request to the "Bug fix" Playbook, and the TODO list starts with that Playbook's six steps. (Step 6 runs "Opening a PR" to open a PR, so the table below leaves it out.) Here, I trace where each step calls a Skill and where a Principle takes effect.

The table below applies three sets of conditions to this request: the conditions in `playbooks/bug-fix.md`, the Non-negotiables of /poteto-mode, and the Principle index of /poteto-mode. In real work, the agent decides which Principles to use by checking the index conditions during the work.

Each step determines which Skills get called and which Principles take effect

"Bug fix" step	Skill called	Principles whose conditions apply
At startup	(/poteto-mode reads the Principle index)	The whole index. The agent does not read the bodies yet
1. Reproduce it yourself on the same surface	The matching control Skill, or the verification skill	"Fix Root Causes," which says to reproduce first
2. Bisect to find the cause	/how for the target subsystem, /why for the history of the regression	"Fix Root Causes," "Guard the Context Window"
3. Plan the fix	/architect if the fix crosses function boundaries. The implementation goes to a subagent	"Model the Domain," "Make Operations Idempotent," "Laziness Protocol"
4. Verify on the same surface	The same control Skill or verification skill as in step 1, which reproduces the bug	"Prove It Works"
5. Commit the failing reproduction before the fix	/tdd if a cheap local test exists	"Sequence Work into Verifiable Units," "Test Behavior, Not Implementation"
Reply	/unpack, as the section "Writing the reply" requires	Name the Principles that changed a decision

Chapter 11 covers each step in detail, and Chapters 17 through 20 cover the situations where the principles take effect.

The agent does not call a Skill whose condition does not apply

For example, `/show-me-your-work` is a Skill that keeps a record of decisions during long work or work you leave. This request does not say that the user will leave, so the condition does not apply. Long work or work you leave would meet the condition. In that case, however, the rule from [Chapter 7](#) may also route the request to `/figure-it-out`.

Even within the same "Bug fix" Playbook, the Skills that the agent calls and the Principles that take effect change with the content of the request and the situations that come up along the way. This design, in which the agent reads only what meets a condition, is the subject of the next chapter, [Chapter 9](#).

Summary

- **How the three parts differ.** A Playbook handles the order of the work. A Skill handles an ability that a step uses. A Principle handles the judgment of whether to take the current move and whether the work is really not done yet.
- **Playbook.** It has three parts: the preamble, the steps, and the Reply line.
- **Skill.** One Skill handles one kind of job. `/poteto-mode` calls most of them as the steps require. The 10 that the user calls are not part of any Playbook's steps.
- **Principle.** It defines a standard for judgment, such as whether to take the current move and whether the work is done. It is not a step or an ability. [Part IV](#) covers the details.
- **Non-negotiables.** They connect the parts by responding to situations during the work and calling Skills across Playbooks.
- **Subagents.** `poteto-agent` is a subagent that works the `/poteto-mode` way. Comment Sicko deletes comments and reports code that needs fixing. The main agent stays responsible for the work it hands off.
- **The example request.** At each step of "Bug fix," the agent calls control Skills, `/how`, and `/why` for reproduction and investigation, `/architect` for the fix, and `/tdd` for the commit. Principles such as "Fix Root Causes" and "Prove It Works" guide the judgments.

The next chapter, [Chapter 9](#), covers when the agent loads these parts. It explains the mechanism that reads only what is needed, only when it is needed, along with the idea behind it and its strengths and weaknesses.

Chapter 9: Why pstack loads only what it needs

kaito • Sources: 日本語 • English • 2026-10-03

pstack does not load every Playbook, Principle, and Skill up front. It reads only the body of /poteto-mode and the Principle index first.

It loads only the one Playbook that fits the request. When a Principle's conditions apply, it loads that Principle's body. When the Playbook's procedure reaches a step that needs a Skill, it loads that Skill.

[Chapter 7](#) and [Chapter 8](#) showed how pstack calls the parts. This chapter explains the design that splits what pstack loads into four layers, the idea behind it, and the design's strengths and weakness.

How this chapter is organized

This chapter is organized as follows.

- pstack splits what it reads into four layers
- This design draws the same line as the principle "Guard the Context Window"
- This design has three strengths, and checks along the way make up for its weakness
- Summary

pstack splits what it reads into four layers

What pstack reads falls into **four layers: entry point, procedure, judgment, and capability**. The only layer it always reads is the entry point.

Layer	What it reads	When it reads it
Layer 1: entry point	The whole SKILL.md of <code>/poteto-mode</code>	When you call <code>/poteto-mode</code>
Layer 2: procedure	The file of the one Playbook that fits the request	When the agent matches the request against the Playbooks
Layer 3: judgment	The SKILL.md of each Principle whose conditions apply	When the current work matches a condition in the index
Layer 4: capability	The SKILL.md of each Skill that a Playbook step or <code>Non-negotiables</code> calls for	When the Playbook's procedure reaches that step

The Layer 1 SKILL.md holds neither Playbook bodies nor Principle bodies. It holds rules common to all work, such as `Non-negotiables` and how to write replies. It also holds a one-line description and a file location for each Playbook, and the Principle index, which lists each Principle with the conditions for applying it. **Layer 1 is a table of contents that says "what exists and when to use it," plus the common rules.**

Apart from the four layers, there is one small file that always applies: `~/.cursor/rules/pstack-models.mdc`, the model-settings rule that `/setup-pstack` writes. It is a short file with one role per line, so it takes up little context even though it always applies. [Chapter 34](#) covers it in detail.

This design draws the same line as the principle "Guard the Context Window"

pstack's loading design draws **the same line that the principle "Guard the Context Window" asks the agent to draw.**

Keep what you use every time inside, and what you use conditionally outside

This principle states that **the context window is finite, and every token should be worth its cost.** When the context window fills up, the quality of reasoning drops.

The same principle has an item that says to keep references you use every time inside the Skill's file. pstack does the same. It keeps the Principle index and `Non-negotiables`, which it

uses every time, inside SKILL.md. It puts the bodies it uses only when conditions apply in separate files. The line it draws is **what you use every time goes inside, and what you use sometimes goes outside.**

The way it hands work to subagents also saves context

The "Subagents" section of SKILL.md makes it **the default to pass subagents file locations instead of copying context into the handoff** (Chapter 35). Subagents open the files they need themselves, so the main agent does not have to build up a long context and pass it along.

This design has three strengths, and checks along the way make up for its weakness

This design has the following three strengths.

1. **It saves tokens and cost.** Procedures and criteria that the work does not use take up no context, so there is room left before the context window overflows. In Part 2 of The Complete Guide to pstack, poteto also gives token efficiency as the reason pstack loads one Playbook at a time.
2. **The work stays consistent.** The agent reads each part's full body at the moment it needs that part, so the work does not depend on the agent's memory or paraphrase.
3. **There is little to remember.** The user does not memorize 70 names and when to use each. Remembering /poteto-mode is enough.

Weakness: the agent decides which conditions apply

The weakness of this design is that **the agent decides which Principle or Playbook applies.** If the agent decides wrong, it may not read a body it should have read.

In this book, I read the following pstack mechanisms as ways to make up for this weakness with checks along the way.

Mechanism	What the user can check
A skipped step stays in the TODO list as skip: <reason>	Which steps the agent did not do, and why

Mechanism	What the user can check
The reply names each Principle that changed a decision (it may name only Principles whose bodies it read)	Which criteria the agent read, and which choices they shaped
One word that names a Principle is enough to change direction	You can fix a wrong decision during the work

The **TODO list** is the plan, a list of things to do that the agent makes at the start of the work. In Cursor, the TODO list appears in the chat panel. From this list, the user can check which steps the agent carried out and which steps it skipped. The agent marks a skipped step with skip: `<reason>` (the bundled guide `01-setup.md`).

Summary

- **Four layers.** pstack loads four layers, each at the moment the agent needs it. The layers are the entry point, the procedure, the judgment, and the capability. The only thing it always reads is the entry point's `SKILL.md`, which holds the table of contents, the index, and the common rules.
- **The idea behind the design.** Like the principle "Guard the Context Window," it keeps the index it uses every time inside, and the bodies it uses conditionally outside.
- **Strengths and weakness.** The strengths are token efficiency, consistency, and little to remember. The weakness is that the agent decides which conditions apply. The user checks those decisions through the TODO list and the replies.

[Part II](#) has looked at what pstack solves, how the entry point `/poteto-mode` routes requests, how the three parts divide the work, and how pstack loads them. From the next part, [Part III](#), the book goes through the 23 Playbooks one at a time. The first chapter, [Chapter 10](#), covers the Playbook for investigating and diagnosing a problem without changing code.

Part III: Moving work forward with Playbooks

kaito • Sources: 日本語 • English • 2026-10-03

Part III covers all 23 Playbooks in pstack.

You can skip this part. I recommend reading only the chapters you care about.

You can read the source of each Playbook at any time in the [pstack](#) repository. pstack changes often, so it is more reliable to look up the relevant chapter and source file when you need them than to read everything up front.

What Part III shows you

For each Playbook, this part shows you the following.

- When `/poteto-mode` picks the Playbook (for example, it picks "[Bug fix](#)" for a request that says "repro first")
- The steps of each Playbook and what each step asks for (for example, "[Hillclimb](#)" repeats "one change, one measurement, keep or revert")
- How a Playbook differs from similar ones (for example, use "[Runtime forensics](#)" to find only the cause, and use "Bug fix" to fix it)

A Playbook is a written procedure that turns the steps a skilled engineer takes naturally into a fixed form.

For example, the bug fix Playbook lays out the order: reproduce, find the cause, fix, and verify in the same place. With fixed steps, any agent that takes the work proceeds in the same order.

Playbooks are reference files inside `/poteto-mode`, and the user does not call them directly. Even so, knowing what is in them **tells you which words to put in a request**.

This book uses only example requests that appear in pstack's sources: the [bundled guide](#), the [README](#), and poteto's guide on X. Four Playbooks have no example request in the sources: Runtime forensics, Trace forensics, Hillclimb, and Pause safely. For these Playbooks, I give no example request. For Hillclimb, I describe only "what to pass in the request" as the bundled guide states it.

The chapters in this part

Chapter	Role	Playbooks covered
Chapter 10: Investigate and reproduce problems	Investigate and return an answer or a diagnosis. Do not change code	Investigation, Runtime forensics, Trace forensics
Chapter 11: Fix bugs and improve performance	Fix bugs and slowness based on runtime evidence	Bug fix, Perf issue, Hillclimb
Chapter 12: Rebuild features, structure, and appearance	Add features, change structure while keeping behavior, decide by trying, and match visuals	Feature, Refactoring, Prototype, Visual parity
Chapter 13: Write Skills and evaluate them	Write Skills that decide how the agent works, and measure their effect	Authoring or modifying a skill, Eval
Chapter 14: Open PRs and merge verified changes	Open PRs, carry them to merge-ready, and land the verified range in trunk	Opening a PR, Babysit, Shipping
Chapter 15: Delegate long-running work to the agent	Carry one long piece of work forward without stopping, stop and resume it, and clean up after it	Autonomous run, Pause safely, Session pickup, Worktree and simulator cleanup
Chapter 16: Plan many PRs and run them with multiple agents	Plan work that spans many PRs, and run it with several agents	Multi-phase or multi-PR plan, Orchestrate, Autopilot-full, Autopilot-stack

Chapter 10 through Chapter 12 follow the flow of making one change: investigate, fix, and build. Chapter 13 covers the Playbooks that improve how the agent works. Chapter 14 covers the Playbooks that turn changes into PRs and merge the verified ones. Chapter 15 and Chapter 16 cover the Playbooks that run work over long periods and with several agents.

A quick reference to every Playbook

Playbook	Overview	When to use it	Explained in
Investigation	Reads the source code and the change history to answer a question, and returns an explanation or a recommendation with citations	When the user asks a question that reading the code or history can answer, such as "how does X work?" or "why was Y built this way?"	Chapter 10
Runtime forensics	Measures a running process to diagnose the cause of a runtime symptom, and returns the diagnosis without fixing anything	When a symptom such as a memory leak or a CPU that spins while idle is still happening, and the agent can measure the running process	Chapter 10
Trace forensics	Reads a profile or dump handed over after the fact, without rerunning the program, and diagnoses the cause	When all you have are artifacts captured in the user's environment, such as a cprofile, a trace, a spindump, or a heap snapshot	Chapter 10
Bug fix	Reproduces a reported bug, finds the root cause, and fixes it with the smallest change that runtime evidence shows is needed	When the agent fixes a bug where the result the user gets is itself wrong	Chapter 11

Playbook	Overview	When to use it	Explained in
Perf issue	Captures a trace of measured slowness, compares it with a baseline, and improves performance	When the behavior is correct but slow, and the user wants to improve performance with one fix	Chapter 11
Hillclimb	Improves one metric toward a target by repeating "one change, one measurement, keep or revert"	When the user wants to improve one metric repeatedly until it reaches a target, not with a single fix	Chapter 11
Feature	Decides the shape of the data first, then implements new or changed behavior	When the user asks to add a feature or change existing behavior	Chapter 12
Refactoring	Changes the structure of the code by renaming, extracting, removing duplication, moving, and similar edits, while keeping the current behavior	When the user wants to change only the structure of the code without changing behavior	Chapter 12
Prototype	Builds a throwaway prototype and makes a design decision from what it observes	When you want to make a design decision, such as a layout or a behavior, cheaply by building and observing instead of asking a person	Chapter 12
Visual parity	Treats the baseline's look as the spec and matches the UI pixel by pixel until the image diff is zero	When you make two implementations look the same, or migrate the styling system	Chapter 12
Authoring or modifying a skill	Writes a SKILL.md by a fixed procedure and keeps only the instructions that change decisions during the work	When the user or the agent writes a new SKILL.md or fixes one	Chapter 13

Playbook	Overview	When to use it	Explained in
Eval	Measures the effect that a change to a Skill, a structure, or a prompt has on the agent's behavior. It measures blind and before adoption	When the user or the agent wants to confirm that a change to a Skill or a prompt works before adopting it	Chapter 13
Opening a PR	Creates a PR that is ready for review, with small, ordered commits, a Conventional Commits title, and a body in Briefing format	When the agent turns changes into a PR at the end of another Playbook's work	Chapter 14
Babysit	Clears conflicts, review threads, and CI failures, and carries a PR or a stack to merge-ready	When the user asks about the state of a PR, such as "get it green" or "keep an eye on PR X"	Chapter 14
Shipping	Has an agent that did not write the code verify each PR in a stack that passed CI, and merges only the verified range that starts from the root, one PR at a time	When the user asks to land a stack that passed CI in trunk	Chapter 14
Autonomous run	Sets the finish condition first, then carries a long task forward without stopping until the condition is met	When the user hands off a long task to completion, as in "loop until it's done" or "use / loop until X"	Chapter 15
Pause safely	Stops the work at a safe point and leaves a record that an agent that knows nothing can resume from	When the user explicitly says to stop, when the user goes offline, when Cursor restarts, or when conversation compaction is near	Chapter 15

Playbook	Overview	When to use it	Explained in
Session pickup	Reads the previous agent's transcript, Cloud Agents URL, or pushed branch, and resumes from where the finished work ends	When the user asks to resume or take over work that a previous agent left unfinished	Chapter 15
Worktree and simulator cleanup	Deletes merged or abandoned git worktrees and old iOS simulators after confirming that they are not in use, to free disk space	When the user asks what is using the disk, or asks to clean up worktrees or simulators	Chapter 15
Multi-phase or multi-PR plan	Writes no code, and produces a checklist-style plan document that settles even how the agent will verify each PR	When the agent plans work that spans several phases or stacked PRs before it starts that work	Chapter 16
Orchestrate	Hands the management of a project that runs for days, with many PRs and subagents, to one coordinator agent that writes no code	When the user hands one chat a project that cannot finish within the time one agent can run	Chapter 16
Autopilot-full	Gives each PR in a queue of independent PRs to its own owner, and merges each one without the operator's review once it passes the coordinator agent's verification	When the operator hands off a queue of independent PRs and can give the agent permission to merge	Chapter 16
Autopilot-stack	Creates and verifies PRs with the same steps as Autopilot-full, but does not merge them. It hands them to the operator as one stack	When the operator wants to review before merging, when the changes depend on each other, or when the operator cannot give the agent permission to merge	Chapter 16

Chapter 10: Investigate and reproduce problems

kaito • Sources: 日本語 • English • 2026-10-03

This chapter covers the following three Playbooks.

1. Investigation
2. Runtime forensics
3. Trace forensics

All three are Playbooks that investigate and produce an answer, and none of them changes code. **The artifact is an answer or a diagnosis backed by evidence.**

Which Playbook to use depends on the information and evidence available to the investigation. The choice depends on whether reading the source code and history can answer the question, whether you can measure a running process, or whether you are examining a profile or dump that someone already captured.

This chapter explains the rules the three share, then how to use each one.

How this chapter is organized

This chapter is organized as follows.

- The three Playbooks do not change code, and you choose one by the information and evidence available
- "Investigation" answers questions that reading the code or history can answer, with citations
- "Runtime forensics" stops guessing from the source and measures the live process
- "Trace forensics" turns the artifacts it receives into a queryable form instead of rerunning the program

- Summary

The three Playbooks do not change code, and you choose one by the information and evidence available

As the introduction said, **you choose among the three Playbooks by the information and evidence available to the investigation.**

Choose as follows.

- **You have a question that reading the code or history can answer.** Use "Investigation".
- **A bug or slowdown is still happening, and you can measure the running process yourself.** Use "Runtime forensics".
- **All you have is a profile or dump handed to you after the fact.** Use "Trace forensics".

Item	"Investigation"	"Runtime forensics"	"Trace forensics"
Evidence	Source code and history	Measurements taken from the running process	Artifacts handed over after the fact, such as a cprofile, a spindump, or a heap snapshot
Rerunning the program	No	No (it inserts instrumentation into the running process to test hypotheses)	No
Where it goes next	Back to the user, then to "Bug fix" or "Feature"	To "Bug fix" or " Perf issue "	To "Bug fix" or " Perf issue "

Even for the same complaint, "it's slow", the Playbook you choose depends on whether you can measure the live process or have only a file you received. What the agent may do also differs by the kind of evidence.

"Runtime forensics" allows the agent to insert logging or instrumentation code into the running process to verify what is happening. "Trace forensics", on the other hand, only

reads the profile or dump it received, and **forbids rerunning the program to capture it again.**

"Investigation" answers questions that reading the code or history can answer, with citations

Role: answer questions that reading the code or history can settle

"Investigation" is **the Playbook that answers questions that reading the source code or the change history alone can settle**, such as "How does X work?", "Why was Y built this way?", "Is Z really safe?", and "Should we pick X or Y?".

The artifact is an explanation or a recommendation with citations. It does not create a PR, and it does not watch a PR until the PR can merge. That work belongs to the "[Babysit](#)" Playbook ([Chapter 14](#)). If it finds partway through that a code change is needed, it does not start fixing on its own. It hands the matter back to the user.

Steps: four steps that fix even the shape of the output

The Playbook has the following four steps.

1. Pass the question through `/how`, the Skill that explains how code works and where it lives. If the question asks "why", also pass it through `/why`, which investigates the reasons and history behind a design with citations. [Chapter 22](#) covers both Skills
2. Write the throughput checkpoint as the single line "not applicable"
3. Write under the same five headings as `/how`: overview, key concepts, how it works, where it lives, and caveats. For a comparison of options, write a recommendation with a table of trade-offs
4. Run `/unstop`, the Skill that cuts phrasing AI tends to write, on the reply ([Chapter 32](#))

- **TODO list.** The plan, a list of things to do that the agent makes at the start of the work. In Cursor, the TODO list appears in the chat panel. From this list, the user can check which steps the agent carried out and which steps it skipped. The agent marks a skipped step with skip: `<reason>` (the bundled guide [01-setup.md](#)).

- **Throughput checkpoint.** The four items the agent records in the TODO list after it considers how to split the work so that parts of it run in parallel. The four items are the following.

- Steps to finish first
- Independent work that can run in parallel
- Shared state
- The smallest safe split

Every Playbook asks for this checkpoint in non-trivial work with several steps (see [Chapter 12](#) for details).

"Investigation" only investigates and does not change code, so it has no work to divide. It skips the four items and writes the single line "not applicable".

When the user asks "Is it really safe?", the agent **does not adopt the asker's assessment as its own. It writes its own judgment with reasons.** If a premise is wrong, the agent points out the wrong premise too.

How to write the request: write "new task" and "don't change any code yet"

If you write "new task" and "don't change any code yet" in the request, `/poteto-mode` picks a Playbook again and adds the constraint not to change code. The example in the bundled guide ([docs/guide/02-poteto-mode.md](#)) is the following.

```
/poteto-mode new task. figure out why the cache entry survives logout. don't change any code yet.
```

You can also specify the shape of the reply in the request. [Part 2](#) of The Complete Guide to pstack gives the following example.

```
/poteto-mode investigate why background workers periodically fail with timeout errors. give me a breakdown of what we know, what data you used, and your best hypotheses.
```

This request asks for the same distinction as the rule in the section "[Writing the reply](#)" of /poteto-mode. That rule says to **attach either evidence or a label to every claim, in the same sentence**. The label is measured, inferred, or guessed. If the request asks for this distinction too, you can tell which sentences of the reply are confirmed facts and which are guesses.

"Runtime forensics" stops guessing from the source and measures the live process

Role: diagnose runtime symptoms by measuring the live process

"Runtime forensics" is the Playbook that diagnoses runtime symptoms such as memory leaks, a CPU that spins while idle, and rendering glitches **by measuring the running process**, not by guessing from the source.

Reading the source shows the paths that could be the cause. With runtime symptoms, though, the question is which path actually runs. The agent finds that path by measuring the running process.

Steps: capture signals, narrow down to evidence, prove the mechanism, map it to the source

The Playbook has the following four steps.

1. Capture live signals

- Use the control Skill that fits the symptom to capture a CPU profile, a heap snapshot, or a CDP (Chrome DevTools Protocol) trace
- The control Skills do not ship with pstack. They are `control-ui` and `control-cli` in `cursor-team-kit`

2. Narrow down to decisive evidence

- Look for functions on the hot path, the retention chains of leaked objects, and loops that keep spinning with no input
- Have subagents analyze large artifacts. The principle "[Guard the Context Window](#)" asks for that division of work ([Chapter 20](#))

3. Prove the mechanism before believing it

- Confirm the mechanism that the hypothesis says causes the symptom
- To confirm it, insert instrumentation code into the running process, or rewrite the running code directly without a restart, which is called a hotfix

4. Map it to the source

- Trace the cause to the source file, the symbol, and the line

Unless the user asks for a fix, the agent makes none. Once it knows the cause, it hands off to "Bug fix" or "Perf issue".

- **CDP trace.** A log of the work inside the browser, recorded through the Chrome DevTools Protocol. You can use it to examine JavaScript execution, page layout, painting, and similar work.

"Trace forensics" turns the artifacts it receives into a queryable form instead of rerunning the program

Role: stick to reading artifacts that were already captured

"Trace forensics" is the Playbook that diagnoses the cause **by reading artifacts handed over after the fact, without rerunning anything**. Examples are a CPU profile, a spindump, and a heap snapshot that someone captured in the user's environment.

- **spindump.** A macOS diagnostic report that records what an app that stopped responding was doing.

The difference from "Runtime forensics" is that the capture is already done. The artifact is fixed data, so the agent sticks to reading it. So that the Playbook works in any

environment, the Playbook also limits the agent to general-purpose tools such as DevTools and trace parsers.

Steps: load the artifact, reshape it, narrow down to the cause, locate the source code

The Playbook has the following six steps.

1. Identify the format and load it

- Analyze large artifacts with subagents

2. Convert it to a queryable form

- Load the trace or heap snapshot into sqlite so that the agent can query the data before it reads through the data

3. Narrow down to the cause

- Look for the frames that use the most time, the retention chains of leaks, and stalled threads

4. Locate the source code

- Use the symbol information in the artifact to identify which file, which function, and which line of the source code the most expensive frame corresponds to
- Until the agent knows this location, it cannot say which code is the cause. It treats the diagnosis as unfinished until then
- If the location is unknown, resolve the symbol information or state that the artifact contains no symbol information

5. If there is a paired capture, compare them

- If there are two comparable artifacts, such as before and after a fix, take the diff
- Without a comparison, write the finding as "the strongest hypothesis the artifact supports", not as a confirmed cause

6. Return a diagnosis with citations

- Unless the user asks for a fix, make no fix, and hand off to "Bug fix" or "Perf issue"

- **Retention chain.** The chain of references that keeps data from being released from memory.

Summary

- **What they share and how to choose.** All three Playbooks return an answer or a diagnosis without changing code. You choose by one of three questions: can reading the code or history answer it, can you measure a process that is still running, or do you have only captured artifacts?
- **Investigation** answers with `/how` and `/why`, and replies under the five headings or with a recommendation that includes a comparison table. If a code change is needed, it hands the matter back to the user.
- **Runtime forensics** stops guessing from the source, measures the live process, proves the mechanism, and then points to the location in the source.
- **Trace forensics** does not rerun the program. It turns the artifacts it receives into a queryable form and identifies the location of the cause in the source code. If it has no comparable artifact, it writes "the strongest hypothesis".

The next chapter, [Chapter 11](#), looks at the three Playbooks that take a diagnosis and fix the problem: "Bug fix", "Perf issue", and "[Hillclimb](#)". It compares them by what you want to fix.

Chapter 11: Fix bugs and improve performance

kaito • Sources: 日本語 • English • 2026-10-03

This chapter covers three Playbooks.

1. Bug fix
2. Perf issue
3. Hillclimb

All three are Playbooks that fix code. When they fix something, **they base the fix on runtime evidence, which is the output, logs, and measurements that come from running the program.**

Which Playbook to use depends on what you want to fix. You choose by whether the result itself is wrong, whether the behavior is correct but slow, or whether you want to push one number up to a target over many attempts.

This chapter explains the rules the three share and then how to use each one.

How this chapter is organized

This chapter has the following sections.

- The three Playbooks fix code based on runtime evidence, and you choose by what you want to fix
- "Bug fix" reproduces the bug first and then fixes it with the smallest change that evidence shows is needed
- "Perf issue" starts from a measurement before the fix and shows the improvement with a measurement after it

- "Hillclimb" improves one metric by repeating "one change, one measurement, keep or revert"
- Summary

The three Playbooks fix code based on runtime evidence, and you choose by what you want to fix

The three Playbooks share the rule that **no change stays unless runtime evidence backs it**. You choose among them as follows.

- **The result the user gets is itself wrong.** Use "Bug fix".
- **The behavior is correct but slow, and you want to improve performance with one fix.** Use "Perf issue".
- **You want to push one number up to a target over many attempts.** Use "Hillclimb".

Item	"Bug fix"	"Perf issue"	"Hillclimb"
Starting point	Reproduce the bug yourself	Take a baseline trace, which is the reference before the fix	Understand the conditions that affect the result and the mechanism of the target, then decide the metric and the stop condition
Iterations	One fix	One fix	Loop until the stop condition is met
Core of the reply	Repro output that went from failure to success	The baseline, the numbers after the fix, and the diff	The metric's start and end values, and the number of kept and reverted changes

If you want to know only the cause and need no fix, choose "Runtime forensics" or "Trace forensics" from [Chapter 10](#).

In all three, subagents do the implementation, and **the main agent plans the fix and reviews the diff**.

"Bug fix" reproduces the bug first and then fixes it with the smallest change that evidence shows is needed

Role: reproduce the bug and find the root cause

"Bug fix" is a **Playbook** that reproduces the reported bug, finds the root cause, and makes only the smallest change that runtime evidence shows is needed.

A just-in-case change that "might help" is **an unverified hypothesis, not a fix**, so the agent does not make it. If runtime evidence refutes a hypothesis, the agent reverts the changes it made based on that hypothesis.

The [talk](#) says that if you leave an ad hoc workaround in place, the next agent copies it as an example of correct implementation ([Chapter 4](#)). This book reads the rule to revert changes based on refuted hypotheses as a way **to keep such workarounds out of the code, so the next agent does not imitate them**.

Steps: six steps from the reproduction to the PR

The Playbook has the following six steps.

1. Reproduce it yourself

- Use the control Skill that fits where the symptom appears, such as `control-ui` or `control-cli` from [cursor-team-kit](#)
- Ask the user to reproduce it only when there is a concrete reason that the Skill cannot operate that screen or command, and only after you have gone as far as the Skill can
- If the bug does not reproduce right away, keep trying until the bug happens: trigger the bug artificially, narrow to the conditions where it happens more often, or add instrumentation

2. Bisect the cause

- List hypotheses about the cause and narrow them to one. Run first the checks that eliminate the most remaining candidates

- Gather candidate hypotheses with two Skills. `/how` explains the mechanism of the affected subsystem, and `/why` searches the history for how something that used to work broke. [Chapter 22](#) covers both Skills
- If you do not know the program's state, add instrumentation or logs instead of guessing
- Before you move on, verify with runtime evidence the mechanism that the last remaining hypothesis points to

3. Plan the fix

- If the fix crosses function boundaries, use `/architect` first. `/architect` is a Skill that designs types and module structure before the code ([Chapter 24](#))
- Hand the implementation to a subagent with a concretely narrowed scope

4. Verify on the same screen or command where the symptom appeared

- Verify that the original repro steps now succeed
- Do not count it as a pass if you verified on a different screen or command, or if the verification was inconclusive
- A unit test shows the behavior of individual pieces, not that the bug is gone. So unit tests that pass do not by themselves mean that the bug is fixed

5. Commit a test that reproduces the bug before the fix

- Commit the test that fails before the fix first, and put the fix on top of it
- If you can write a test that runs locally and cheaply, follow `/tdd` ([Chapter 30](#)), which writes the failing test first
- You can skip this step if writing the test is costly, if the test depends heavily on an environment that connects several services, or if it is unclear what the test would check

6. Run "Opening a PR", which sets the steps for opening a PR ([Chapter 14](#))

The reply contains the following items.

- What was broken
- The root cause
- The fix

- How the agent verified the fix

The agent also **pastes the repro output that went from failure to success, as is, without summarizing it.**

This book reads the mapping between the steps and the Principles as follows. The principle "[Fix Root Causes](#)" maps to steps 1 to 3, "[Prove It Works](#)" maps to step 4 and the reply, and "[Sequence Work into Verifiable Units](#)" maps to step 5 ([Chapter 19](#)). Of these, the Playbook text itself names only "Sequence Work into Verifiable Units", in step 5.

How to write the request: say "reproduce it first"

Write "reproduce it first" in the request. If you want a test, add the condition "if there is a cheap test path". The example in the bundled guide ([docs/guide/10-recipes-and-pitfalls.md](#)) is the following.

```
/poteto-mode repro the duplicate write first. if there's a cheap test path, /tdd it. then fix and rerun.
```

A run of the real command shows the fix more accurately than a forced test built on fragile mocks. For this reason, step 5 lets you skip the test when the test is costly to write.

"Perf issue" starts from a measurement before the fix and shows the improvement with a measurement after it

Role: measure the slowness and fix it

"Perf issue" is a Playbook that improves measured slowness against a baseline. It starts from a trace of the slowness. A trace is a record of things such as the run time of each operation. It forbids reading the source instead of measuring, and **ties every fix to a measurement.**

Steps: six steps that start with a baseline trace

The Playbook has the following six steps.

1. **Take a baseline trace.** Use the control Skill that fits the symptom
2. **Use /how to study how the code works, and base hypotheses on it.** Do not decide that "it cannot get any faster" before running and measuring it. Form hypotheses from the eight common improvement patterns, called strategy families, which the table below lists
3. **Plan the fix from the trace.** Hand the implementation to a subagent, review the diff, and take a trace after the fix
4. **Analyze and compare the traces before and after the fix.** Take the diff. For example, load the traces into sqlite and query them. Do not count it as a pass if you measured on a different screen or command, or if the measurement was inconclusive
5. **Quote the measurement results in the PR**
6. **Run "Opening a PR"**

The reply includes these four items.

- The baseline numbers
- The numbers after the fix
- The diff
- The artifact paths

The agent cannot write any of these items without measuring, so **the shape of the reply itself keeps the agent from skipping the measurement.**

The eight strategy families are tools that produce hypotheses, not a checklist. The agent may try a family **only when the trace shows signs that the family will work**. The right column of the table lists those signs. The exception is Elimination. A trace shows where the program is slow, but it does not show whether the agent can remove a piece of work. The agent checks that question with /how.

Family	Sign visible in the trace
Elimination	The trace does not show it. Use /how to verify whether the work can be removed

Family	Sign visible in the trace
Divide and conquer	The main cost grows in proportion to the size of the input
Caching	The same computation or fetch repeats with the same input
Indirection	The hot path, which is the path that uses most of the run time, does heavy work such as a scan of every item, and an intermediate structure such as an index or a queue could take over that work
Batching	Each small operation pays a fixed cost such as a network round trip or a query
Redundancy	One slow server or one slow attempt raises the overall latency
Lazy evaluation	Cost goes into results that are not needed yet
Scheduling	The work itself is needed, but it does not have to happen while the user is waiting

How to write the request: give measured numbers, not "it feels slow"

In the request, write measured numbers instead of "it feels slow". The example in the bundled guide ([docs/guide/05-build-and-clean.md](#)) is the following.

```
/poteto-mode startup takes 1.8s on this fixture. trace it, fix the measured cause, show me before and after.
```

If you also want confidence that the improvement is real, ask in the request for the agent to run the verification skill in parallel with `/swarm`. The verification skill is a Skill that operates the app the way the user does and verifies it ([Chapter 3](#)). The example in The Complete Guide to pstack [Part 1](#) is the following.

```
spawn a cloud agent to use /poteto-mode to improve the initial loading time of our app. first use /control-app to take a trace of the status quo, and identify opportunities for improvement. then do a targeted fix and use /control-app + a /swarm to confirm the win
```

`/control-app` is the name of the verification skill built in the guide ([Chapter 36](#)). `/swarm` ([Chapter 24](#)) is a Skill that runs several agents, called Workers, in parallel and merges their results into one report. Because many agents run the verification skill, you can verify the improvement with a large enough sample. The point is to avoid mistaking the variation of a single measurement for an improvement.

"Hillclimb" improves one metric by repeating "one change, one measurement, keep or revert"

Role: improve one metric toward a target in a loop

"Hillclimb" is a Playbook that **improves one metric toward a target continuously, in a loop**. One-time fixes belong to "Bug fix" or "Perf issue".

Its core discipline is "one change, one measurement, keep or revert". The agent does not stack untested changes, and does not claim an improvement from reading the code alone. The principle "[Prove It Works](#)" sets both rules.

Steps: eight steps that loop with one hypothesis at a time

The Playbook has the following eight steps.

1. **Before you choose the metric, understand the conditions that affect the result and the mechanism of the target**
 - Run `/how` on the target and list the realistic conditions that affect the metric's value, such as data volume, history, state, and concurrency
 - Choose a case that reproduces the user's complaint
 - Decide on one metric and a stop condition
 - Make the stop condition a pair of a target and a minimum number of attempts, such as "at least 50% better than the baseline, and at least 10 attempts". The minimum exists so that a lucky early result does not end the loop

2. Build the measurement harness, confirm it can tell the cases apart, and then freeze it

- Verify that the target case reproduces the symptom and that its measurements differ from an easy case where the symptom does not appear
- Once the harness passes this check, freeze the measurement method. A frozen method means that you do not change the measured case, the metric, or the measurement command for the rest of the loop
- After you freeze the method, make one fixed command produce the metric, and take the median of several runs instead of one
- Before you make any change, record the baseline metric and a result in which all regression gates pass. Regression gates are the required tests that must keep passing. The principle "[Build the Lever](#)" asks for this baseline ([Chapter 17](#))

3. Open the decision log

- Create `decision.tsv` with `/show-me-your-work` ([Chapter 28](#)), which keeps a record of decisions
- `decision.tsv` is a table with one row per attempt: the hypothesis, the change, the numbers before and after, and whether the change was kept or reverted
- Read `decision.tsv` again before the next attempt

4. Base each hypothesis on the mechanism understood in step 1

- State a concrete mechanism as the hypothesis, such as "X delays the first paint, so move it out of startup"
- The Playbook does not allow a hypothesis like "try memoizing something"

5. Loop with one hypothesis at a time

- Hand the change to a subagent, as the principle "[Guard the Context Window](#)" asks
- If there are several independent hypotheses, give them to parallel subagents, one per worktree, as the principle "[Separate Before Serializing Shared State](#)" asks
- Measure before and after the change with the harness frozen in step 2
- Keep the change only when the metric improves by more than chance can explain and all regression gates pass
- Otherwise, revert the whole change

- Always write the `decision.tsv` row, whether the change was kept or reverted
- To run unattended, borrow one mechanism from "[Autonomous run](#)" ([Chapter 15](#)), the Playbook that carries a long task to completion without stopping. Borrow only the mechanism that wakes the agent again on a timer or an event. The stop condition stays the one decided in step 1

6. Do not stop as soon as the improvement stalls

- If reverts keep coming, switch the family of hypotheses, that is, the kind of approach. You can also combine near-miss ideas or try bolder ideas before you conclude
- Revert a change that improves the metric but breaks behavior
- Keep a change that simplifies the code while it holds the metric, as the principle "[Laziness Protocol](#)" asks

7. Stop when the stop condition is met, or when the remaining ideas are no longer worth the effort

- Do not loosen the stop condition to call it met
- Do not stop while cheap hypotheses remain
- If you get stuck, report instead of repeating the same attempts

8. Run "Opening a PR"

- Stack the kept commits in the order they landed

What "an improvement larger than chance can explain" means

For example, suppose you measure processing time, and without any code change it varies by a few milliseconds around 100 milliseconds. If it drops to around 80 milliseconds after the change, you can judge that the improvement exceeds the variation. If it changes by only a few milliseconds, it may be measurement noise, and you cannot call it an improvement.

In step 2, the median of several runs, instead of a single run, also reduces the effect of this variation.

- **Harness.** A setup that runs measurements or tests under the same conditions every time

How to write the request: give the metric, the target, and the minimum number of attempts

According to the bundled guide (<docs/guide/05-build-and-clean.md>), **if you give the metric, the target, and the minimum number of attempts**, "Hillclimb" uses a harness with a frozen measurement method and tries one hypothesis at a time in a loop.

Summary

- **The shared discipline and how to choose.** The three Playbooks share the discipline that every change they make has runtime evidence behind it. Choose "Bug fix" for wrong behavior, "Perf issue" to improve slowness with one fix, and "Hillclimb" to improve one metric toward a target over many attempts.
- **Bug fix.** The agent reproduces the bug itself, bisects the hypotheses about the cause down to one, and then fixes it. The reply includes the output that went from failure to success, as is.
- **Perf issue.** The agent starts from a baseline trace. It tries each of the eight improvement patterns, the strategy families, only when the trace shows signs that the pattern will work, and it shows the improvement with the numbers before and after the fix.
- **Hillclimb.** The agent builds a harness with a frozen measurement method and repeats "one change, one measurement, keep or revert" under a stop condition that pairs a target with a minimum number of attempts.

The next chapter, [Chapter 12](#), looks at four Playbooks that "add", "keep", or "try" behavior or "match" appearance: "[Feature](#)", "[Refactoring](#)", "[Prototype](#)", and "[Visual parity](#)". It looks at them from the angle of "what must not change".

Chapter 12: Rebuild features, structure, and appearance

kaito • Sources: 日本語 • English • 2026-10-03

This chapter covers the following four Playbooks.

1. Feature
2. Refactoring
3. Prototype
4. Visual parity

All four Playbooks change code. They differ in how they treat behavior. Behavior is the result a user receives, such as a command's output or what the screen shows.

They treat behavior in these different ways.

- Add behavior or change it
- Fix only the structure, without changing behavior
- Try out behavior to decide a design, then throw the code away
- Match the appearance pixel for pixel

Whichever Playbook you use, **write in the request what must not change**. The bundled guide's page on implementation (<docs/guide/05-build-and-clean.md>) also recommends that a Feature request state the behavior and what must not change.

This chapter explains, in order, how the four differ, how to use each one, and how to write the request.

How this chapter is organized

This chapter is organized as follows.

- Choose among the four Playbooks by whether you add behavior, keep it, try it, or match appearance
- "Feature" decides the shape of the data first, then delegates the implementation and reviews it
- "Refactoring" pins down the current behavior before it moves the structure
- "Prototype" decides a design at low cost with code meant to be thrown away
- "Visual parity" treats the reference appearance, the baseline, as the spec and aims for a zero image diff
- Write "what must not change" in the request
- Summary

Choose among the four Playbooks by whether you add behavior, keep it, try it, or match appearance

The four Playbooks split by **how they treat behavior**. What must not change, what the main agent is responsible for, and how the agent verifies the work all change with that choice.

Item	"Feature"	"Refactoring"	"Prototype"	"Visual parity"
Behavior	Add or change	Do not change	Try out, never ship to production	Do not change the appearance
Responsible for	The design	The contract, meaning the current behavior	The design decision, not the code	Pixel-level identity
Verification	Run it where the change shows up and verify	Prove with the real artifact that behavior matches what it was before the change	Compare the options by observation	A zero image diff

The four also hand work to each other. "Prototype" passes the design option it chose by comparison to "Feature". "Refactoring" splits missing features or real bugs it finds along the way into separate work and ships the structural change first. When a whole design needs rebuilding, "Refactoring" says so explicitly and passes the work to "Feature".

"Feature" decides the shape of the data first, then delegates the implementation and reviews it

"Feature" is a Playbook that first decides the shape of the data a new feature handles, meaning its fields and types, and implements the feature from that shape. **The main agent is responsible for the design** and delegates the code itself to another agent.

There are two key points.

- **Decide the shape of the data before you write code.** The principle "[Model the Domain](#)" ([Chapter 18](#)) explains that structure is cheap when you choose it while you write the code. When you try to recover it later, the work looks like refactoring, and people tend to postpone it.
- **Separate the agent that writes the code from the agent that reviews its diff.** A subagent writes the code, and the main agent reviews that diff (step 4).

Procedure: decide the design, delegate the implementation, and verify before you create the PR

There are eight steps.

1. Run `/how`, the Skill that explains how code works ([Chapter 22](#)), on the affected subsystems
2. Explore options for types and module structure in parallel with `/architect`, the Skill that draws the design before the code ([Chapter 24](#))
3. Write the throughput checkpoint (see the table below)
4. Delegate the implementation to a subagent
5. Verify in the environment where the change shows up: a browser for UI, the CLI for a command

6. **Rebase** into small, ordered commits, as the principle "[Sequence Work into Verifiable Units](#)" asks ([Chapter 19](#))
7. If there is disagreement about the design, run `/interrogate`, where reviewers on several models question the change ([Chapter 27](#)), before you go on
8. Run "[Opening a PR](#)", which sets the procedure for creating a PR ([Chapter 14](#))

rebase is a Git operation that replays commits on top of a different base.

The throughput checkpoint in step 3 is **four items in the TODO list**. The items record what you found when you considered how to split the work so that parts of it run in parallel.

The **TODO list** is the plan, a list of things to do that the agent makes at the start of the work. In Cursor, the TODO list appears in the chat panel. From this list, the user can check which steps the agent carried out and which steps it skipped. The agent marks a skipped step with `skip: <reason>` (the bundled guide [01-setup.md](#)).

Playbooks other than "Feature" also ask for this step in non-obvious work with several steps.

Item	What to write
Steps to finish first	The gates that must pass before work goes parallel
Independent streams of work	Work that causes no conflicts, meaning work whose files and layers do not overlap, can run in parallel
Shared mutable state	The default is to remove the sharing. Split what each stream changes, so that no two streams rewrite the same thing at the same time. The principle " Separate Before Serializing Shared State " sets this default
Smallest safe split	Write the units of work that can be split safely. If one agent alone is the better choice, write why

Even when one of the four items does not apply to the current work, you keep it and write n/a: <reason>. The n/a line shows whether you considered the item and found it unnecessary, or overlooked it.

For the implementation in step 4, the main agent specifies the file paths, the shape of the data, and the success criteria. It then delegates the work to a subagent on the model configured for feature work. You can change that model with /setup-pstack, which [Chapter 6](#) mentions. If the implementation could take several reasonable forms, the agent sends the work through /arena ([Chapter 24](#)). /arena builds several candidates in parallel for the same task and combines their good parts.

This delegation is mandatory. You cannot drop it with skip: <reason>, and the principle "Laziness Protocol", which asks for the smallest change, cannot override it. The goal is not to save lines. The goal is to separate the review.

If the subagent that received the implementation cannot start another subagent, that implementation subagent writes the code. The agent that delegated the work reviews the diff, so implementation and review stay with different agents.

Writing the request: state the behavior and what must not change

The bundled guide's example is the following.

```
/poteto-mode add a --json flag. text output stays byte-identical. verify both forms.
```

"Refactoring" pins down the current behavior before it moves the structure

"Refactoring" is a **Playbook that changes only structure and keeps behavior**, through renames, extractions, removal of duplication, and code moves.

The main agent is responsible for the "contract", meaning the current behavior. The structure may change, but the current behavior does not.

The Playbook also allows a rebuild of the whole design, not only of the structure. In that case, the agent says explicitly that it is a rebuild and passes the work to "Feature".

Procedure: pin down behavior, delete unneeded code before you move things, and verify equivalence

There are eight steps.

1. Pin down the behavior contract
2. Name the missing structure
3. Name the target shape
4. Delete before you add
5. Move in small steps
6. Prove with the real artifact that behavior has not changed
7. Check that the change is worth keeping
8. Rebase into small, ordered commits

The sections below look at each step in turn.

- **Snapshot test.** A test that saves the output to a file once and compares later runs against it to detect changes

1. Pin down the behavior contract

Before you move the structure, write what guarantees the current behavior. Examples are characterization tests, snapshot tests, and a mechanism that compares old and new output. A characterization test records the current output as the correct answer. Type checks and lint that pass do not by themselves pin down behavior.

2. Name the missing structure

This step applies the principle "Model the Domain" ([Chapter 18](#)).

3. Name the target shape

Decide how the modules split, what the data types are, and which functions call which. If the target shape crosses function boundaries, explore it with `/architect`.

4. Delete before you add

Remove unneeded code before you bring in the new structure. For example, fold dead code and functions with only one caller, such as wrappers, into the caller. The principle "[Subtract Before You Add](#)" sets this order.

5. Move in small steps

Make one move at a time, and keep the tests and comparisons from step 1 passing. If you change the shape of an API, move every caller of the API to the new API and delete the old API in the same series of changes. The principle "[Migrate Callers Then Delete Legacy APIs](#)" asks for this migration ([Chapter 18](#)).

6. Prove with the real artifact that behavior has not changed

A compile or build that passes does not by itself prove that behavior has not changed. The principle "[Prove It Works](#)" sets this standard.

7. Check that the change is worth keeping

If the diff does not lower the reader's load, revert it, as the principle "[Minimize Reader Load](#)" asks ([Chapter 17](#)).

8. Rebase into small, ordered commits

Commit in this order: the subtractive change that deletes code, the structural rebuild, then the follow-up cleanup. Then run "Opening a PR".

Writing the request: have the agent record the current output before it moves the structure

The bundled guide's example is the following.

```
/poteto-mode move parsing into one module, zero behavior change. record the current output first and prove it's unchanged after.
```

"record the current output first" maps to step 1, and "prove it's unchanged after" maps to step 6.

"Prototype" decides a design at low cost with code meant to be thrown away

"Prototype" is a Playbook that builds a throwaway prototype and **makes design decisions from what it observes**.

The main agent is responsible for the design decision, not the code. The production implementation happens in "Feature".

Because the code is meant to be thrown away, pstack describes "Prototype" as **the only Playbook that reverses two things**: the "smallest change" of the principle "Laziness Protocol", and the standard for verification.

It puts speed ahead of polish, does not care about code quality, and offers options nobody asked for. **The standard for verification changes from proving that "the change is correct" to "observing what you want to decide"**.

Procedure: make clear what you are deciding, then build prototypes and compare them

There are six steps.

1. Make clear what the prototype decides
2. If the direction is not settled yet, collect reference examples
3. Build in a scratch directory away from the production source
4. Make the alternatives switchable on the same screen for comparison
5. Run each option and compare
6. Present the alternatives, trade-offs, and a recommendation as the comparison result

The sections below look at each step in turn.

1. Make clear what the prototype decides

There are three things to decide: which layout to use, which way of interacting to use, and how densely to pack the screen elements. For choices where a run and an observation show which option is better, decide which behavior, which timing, or which method to use. If there is nothing to decide, do not build a prototype. Pass the work to "Feature".

2. If the direction is not settled yet, collect reference examples

Show the user examples of similar features or screens, and have the user choose the look and interaction before you start building the prototype. If the direction is already settled, skip this step.

3. Build in a scratch directory away from the production source

Do not bring in the production framework, tests, or abstractions. Prepare only the minimal code that answers the question.

4. Make the alternatives switchable on the same screen for comparison

Label each option and switch between them with a button or a key, as the principle "Exhaust the Design Space" asks ([Chapter 17](#)).

5. Run each option and compare

Record screenshots for how a screen looks and measurements for processing time. This material lets you decide the question that step 1 made clear.

6. Present the alternatives, trade-offs, and a recommendation as the comparison result

Explain how the options differ and the pros and cons of adopting each, and state the recommended option. Once the option to adopt is decided, hand that direction to "Feature".

Writing the request: say "build two so we can compare"

The example in pstack's [README](#) is the following.

```
/poteto-mode build two prototypes of the markdown renderer so we can compare. spawn an agent for each.
```

Phrases such as "make a mock" or "try this layout" are also **signals for this Playbook**.

"Visual parity" treats the reference appearance, the baseline, as the spec and aims for a zero image diff

"Visual parity" is a **Playbook that keeps the UI's appearance identical pixel for pixel** when you make two implementations match or migrate a styling system.

It never touches the baseline, and it verifies identity with an image diff.

Pass or fail depends on whether the diff is zero, not on visual inspection. If the baseline itself looks wrong, though, the agent does not fix the baseline. It stops and asks the user, as step 2 says.

Procedure: iterate per component until the image diff is zero

There are five steps.

1. Create the reference, the baseline, before the migration
2. Forbid shortcuts that make the diff pass
3. Move components one at a time
4. Compare each component against the baseline with an image diff
5. Run "Opening a PR" per component, or per safe group of components

The sections below look at each step in turn.

- **worktree**. A Git feature that creates several working directories from one repository, so that you can work on separate branches at the same time

1. Create the reference, the baseline, before the migration

Build a mechanism that automatically takes screenshots of each component before the migration in various states, such as normal, hover, and disabled. These screenshots are the reference that the agent compares the post-migration appearance against.

2. Forbid shortcuts that make the diff pass

Do not change the harness, the mechanism from step 1 that takes and compares the screenshots. Do not tamper with the baseline. Do not rearrange a component's structure to make the diff pass. If the baseline looks wrong, stop and ask the user.

3. Move components one at a time

Replace components one by one with the new implementation or styling method. Work in parallel in separate worktrees, and move the shared base components first.

4. Compare each component against the baseline with an image diff

Iterate with `/loop`, a built-in Cursor command that wakes the agent again and again ([Chapter 15](#)), until the diff is zero.

5. Run "Opening a PR" per component, or per safe group of components

Writing the request: name the baseline with "the second image is correct"

The README's example is the following.

```
/poteto-mode the row spacing is too tall when this flag is on. the second image is correct. reproto
and fix until it matches.
```

"the second image is correct" is the sentence that tells the agent **which appearance is the baseline**. Without it, the agent can only guess which one to match.

Write "what must not change" in the request

Whichever of the four you ask for, **write "what must not change" in the request.**

What the request needs to say	Example wording	Routed to
I want new behavior. Existing output must not change	"text output stays byte-identical"	"Feature"
I want to tidy only the structure	"zero behavior change. record the current output first"	"Refactoring"
I want to decide which design is better	"build two prototypes so we can compare"	"Prototype"
I want the appearance to match exactly	"the second image is correct. fix until it matches"	"Visual parity"

The sentence you write in the request becomes the standard for verifying whether the work is done. For example, if you write "text output stays byte-identical", one finish condition is that the text output is byte-for-byte the same as before the change.

Summary

- **How the four split.** "Feature" is responsible for the design, "Refactoring" for the contract, "Prototype" for the design decision, and "Visual parity" for pixel-level identity.
- **Feature.** It decides the shape of the data, writes the throughput checkpoint, and always delegates the implementation to another agent.
- **Refactoring.** It pins down the current behavior with tests or similar means, deletes unneeded code before it moves structure, and proves with the real artifact that behavior has not changed.
- **Prototype.** It reverses the "smallest change" and the standard for verification. It builds options at low cost, compares them by observation, and hands the chosen direction to "Feature".
- **Visual parity.** It never touches the baseline and iterates per component until the image diff is zero.

- **Writing the request.** When you write "what must not change", that sentence becomes the check for whether the work is done.

The next chapter, [Chapter 13](#), covers "Authoring or modifying a skill", which writes Skills, and "Eval", which verifies the effect of a change.

Chapter 13: Write Skills and evaluate them

kaito • Sources: 日本語 • English • 2026-10-03

This chapter covers the following two Playbooks.

1. Authoring or modifying a skill
2. Eval

Both deal **with Skills, which decide how the agent works**, instead of app code.

You use "Authoring or modifying a skill" when you write or fix a Skill (SKILL.md), and "Eval" when you measure whether a change to that Skill had an effect.

This chapter explains how the two split the work, and how to use each, in order.

How this chapter is organized

This chapter is organized as follows.

- The two Playbooks split the work into writing a Skill and measuring its effect
- "Authoring or modifying a skill" writes a Skill through set steps and keeps only instructions that change decisions during the work
- "Eval" measures the effect of a change on the agent's behavior without letting the agent notice the evaluation
- Summary

The two Playbooks split the work into writing a Skill and measuring its effect

The bundled guide's "Make it yours" page (<docs/guide/09-make-it-yours.md>) asks two things about Skills.

- **Hold text for agents to a higher standard than text for humans.** Even a useless sentence becomes an instruction that some agent will follow in the future.
- **Test Skill edits as experiments.** A Skill edit affects every future session.

"Authoring or modifying a skill" handles the writing, and "Eval" measures the effect.

The steps of Authoring or modifying a skill have no instruction to use several agents. Eval, on the other hand, needs the following three things.

- **Candidates.** These agents work in an environment with the change in place.
- **A judge.** This agent is separate from the candidates and scores the candidates' output.
- **A work environment with the cues removed.** The cues are what would reveal an evaluation, such as directory names and file names.

"Authoring or modifying a skill" writes a Skill through set steps and keeps only instructions that change decisions during the work

"Authoring or modifying a skill" is the Playbook you go through **when you write a new SKILL.md or fix one.**

The bundled guide lists a SKILL.md written freehand as a common mistake, and says that this Playbook puts the Skill through verification and review.

Four steps

The steps are as follows.

1. Use create-skill
2. Verify the Skill

3. Prepare test cases if you change the Skill's structure
4. Open a PR with "Opening a PR" (Chapter 14)

The sections below look at each step in turn.

- **frontmatter** is metadata, such as a name and a description, written between --- lines at the top of a Markdown file

1. Use create-skill

create-skill is the built-in Cursor Skill that writes a SKILL.md.

2. Verify the Skill

Check that the frontmatter has name and description, that the referenced files exist, and that links to other Skills resolve.

Step 2 verifies only things that give a mechanical pass or fail. **"Eval" verifies whether a Skill change changes the agent's behavior.**

3. Prepare test cases if you change the Skill's structure

Skip this step for a subjective change such as tone.

4. Open a PR with "Opening a PR" (Chapter 14)

The key point is the writing discipline "when in doubt, delete"

The key point of this Playbook is the **writing discipline** that follows the steps.

When in doubt, delete. Keep only prose that changes a decision.

The agent that writes a Skill asks of each sentence, "which of the agent's decisions does this sentence change?" and deletes any sentence for which it has no answer.

The main rules of the discipline are as follows.

- **Leave out explanations of why.** Write instructions around "what to do," and explain the reason only when the instruction's meaning or the conditions for applying it are unclear without it.
- **Point to the source instead of copying its contents.** Point to sources that exist in the codebase, such as types, the README, and configuration files. The principle "[Encode Lessons in Structure](#)" asks for these pointers ([Chapter 21](#)).
- **Defer to other Skills.** Do not copy another Skill's contents. Give that Skill's path and let it do the job.
- **Turn repeated work into a Skill.** If you keep doing the same work, propose a new Skill.

If the agent notices a broken Skill partway through its work, it does not stop the work, and it does not quietly work around the problem either. **It fixes only that Skill and opens a PR for it.**

The reason is that reviewers tend to miss a Skill edit mixed into a feature PR, and "Eval" cannot measure its effect.

Verification skills and personal Skills have their own entry points

For some goals, you use a dedicated entry point instead of this Playbook.

Entry point	When to use it	Chapter with details
<code>/create-verification-skill</code>	You want to create a new Skill that verifies how the app behaves	Chapter 26
<code>/maintain-verification-skill</code>	The steps in a verification skill no longer match how the app behaves now	Chapter 26
<code>/automate-me</code>	You want to turn the way you work into a Skill, based on records of past sessions	Chapter 25

Entry point	When to use it	Chapter with details
/reflect	You want to save what you learned in a session into an existing Skill	Chapter 33

How to write the request: describe the Skill you want in plain words

Plain words are enough for the request.

```
/poteto-mode write a skill for verifying database migrations in this repo
```

"Eval" measures the effect of a change on the agent's behavior without letting the agent notice the evaluation

"Eval" is the Playbook that **tests, before you adopt it**, how a change to a Skill, a structure, or a prompt affects the agent's behavior.

The agent that runs "Eval" designs the experiment and summarizes the results. In this experiment, other agents use the changed Skill in real work, and a judge scores how they do.

The experiment starts several agents that solve a task. pstack calls these agents **candidates**.

- **Give each candidate its own working directory.** Put the changed Skill and the project skeleton in each directory. Separate directories keep the candidates from affecting each other's work.
- **Use a different model for each candidate.** For example, run the first on Opus and the second on Grok. Each model has its own biases, and if you test with only one model, its biases mix into the result.
- **Give all of them the same request.** If the requests differ, you cannot tell whether a difference in results comes from the change or from the request.

When the candidates finish, a **judge** that runs on a model different from the candidates scores the output without knowing which candidate produced it.

You can also compare two versions, before and after the change. In that case, one judge scores both versions' output at once, on the same criteria, without knowing which version each came from.

- In a **blind** test, nobody tells the subject that it is under evaluation or which conditions the test compares

Seven steps

The Playbook has the following seven steps.

1. **Frame the experiment.** Write down the change under test and the behavior that counts as success. Build a scoring rubric for the judge with 3 to 6 items, and do not show it to the candidates
2. **Prepare an environment with the evaluation cues removed.** Prepare a working directory for each candidate and put the change under test in it. Also include the context a real task would have, such as the project skeleton and the Skills a candidate would naturally read
3. **Write one natural prompt.** Make it a request the user would plausibly give, and do not reveal what you are measuring
4. **Start N candidates in parallel.** Follow the steps of [/arena](#) and use a different model for each candidate
5. **Start one blind judge.** Use a model family different from the candidates', and do not show model names
6. **Verify from the records, not from self-reports, whether the candidate followed the instructions.** Score whether a candidate followed, in order, the files the Skill pointed to. Base the score on the files that the conversation record shows the candidate opened, and on the shape of its code

7. **The agent that designed the experiment reads every output itself.** It compares them with the judge's verdict, treats any disagreement as a sign of model bias or an ambiguous rubric, and writes up the conclusion

- A **model family** is a group of models released by the same developer as one series. Examples are OpenAI's GPT series and xAI's Grok series

The key point is the blinding rules, placed before the steps

According to the bundled guide, pstack built this Playbook around one failure: "an agent that knows it is being evaluated behaves differently." The name for this failure is the **observer effect**.

The blinding rules fall into three groups, one for each path through which information can leak.

- **Strings the candidates see.** Do not put words such as eval, test, or judge in directory names, file names, or prompts. Use names the user would plausibly choose, and make the prompt look like an ordinary request.
- **What the candidates answer.** Do not have them list which Skills or files they used, and do not tell them other candidates exist.
- **Information the judge sees.** Show the output with labels that do not reveal which model produced it, and hide model names. When the judge compares two versions, one judge scores both in a single pass, on the same scale.

How to write the request: give both versions the same task and keep the candidates blind

The bundled guide's example request compares two versions, before and after a change. It says to give both versions the same task and to keep the candidates blind.

```
/poteto-mode run the eval playbook on this skill change. same task for both variants, candidates stay blind.
```

Summary

- **How the two Playbooks split the work.** "Authoring or modifying a skill" creates and edits Skills, and "Eval" measures their effect.
- **Authoring or modifying a skill.** Write with `create-skill`, verify mechanically, and open a PR. The key point is "When in doubt, delete. Keep only prose that changes a decision." Fix a broken Skill in a separate PR that contains only the fix to that Skill.
- **Eval.** Before you adopt a change, verify its effect on behavior. The candidates do not know that anyone is evaluating them, and the judge does not know the model names. The score relies on the conversation records that show which files each candidate opened, not on the candidates' self-reports.

The next chapter, [Chapter 14](#), looks at three Playbooks: "Opening a PR," which creates a PR, "[Babysit](#)," which carries a PR to a mergeable state, and "[Shipping](#)," which lands only verified changes on trunk, the main branch.

Chapter 14: Open PRs and merge verified changes

kaito • Sources: 日本語 • English • 2026-10-03

This chapter covers three Playbooks:

1. Opening a PR
2. Babysit
3. Shipping

All three handle the path of a change into trunk, the main branch that PRs merge into. Each one owns one of these three stages:

- **Create the PR**
- **Bring the PR to merge-ready**
- **Bring only verified PRs into trunk**

Which Playbook to use depends on the stage the PR is at now. Choose by whether there is no PR yet, whether the PR exists but is not ready to merge, or whether you want to bring a merge-ready PR into trunk.

This chapter explains how the three Playbooks connect, then the steps and key point of each in turn.

How this chapter is organized

The chapter has these sections:

- The three Playbooks connect in the order "create the PR, make it mergeable, and bring it into trunk"
- "Opening a PR" creates PRs that are small, readable, and ready to review

- "Babysit" clears problems from the unmerged frontmost PR first and makes it merge-ready
- "Shipping" brings into trunk only the range of PRs that passed another agent's verification
- Summary

The three Playbooks connect in the order "create the PR, make it mergeable, and bring it into trunk"

The stages named at the start line up in the order "create the PR", "make it green", and "bring it into trunk", and **each one has a fixed end point**. "Green" means that all CI checks pass.

Stage	Playbook	End point
Create	"Opening a PR"	Returns the PR's URL
Make green	"Babysit"	merge-ready
Bring in	"Shipping"	Merges the verified PRs that continue from the root

- **Forge**. In this chapter, "forge" means the service that manages PRs together with the CLI that drives it. The default is the GitHub CLI (`gh`).
- **trunk**. The main branch that PRs merge into. In many repositories it is `main`.
- **Stack**. A chain of PRs linked as parent and child. Only the frontmost PR, the root, targets trunk. The next section shows the shape.
- **Merge frontier**. Among the unmerged PRs in a stack, the PR to merge next.
- **merge-ready**. The state in which the forge judges that all CI checks pass, no review is unresolved, and there are no conflicts.

Each stage also has its own wording for the request. These examples come from the verification page of the bundled guide (<docs/guide/06-verify-and-ship.md>).

```
/poteto-mode babysit this pr. get it green.
```

```
/poteto-mode land the stack.
```

Opening a PR does not start "Babysit".

Ask "get it green" and "Babysit" runs. Ask "land the stack" and "Shipping" runs.

"Opening a PR" creates PRs that are small, readable, and ready to review

"Opening a PR" is **the Playbook that sets the conventions for turning a change into a PR.**

Unlike the Playbooks so far, its steps are not numbered. It groups rules under headings. The main rules are these:

- **Worktree.** The agent works in a git worktree created from main.
- **Commits.** Before you open the PR, use rebase to regroup the work into small, ordered commits. Treat each commit as a "future PR" that could merge on its own.
- **PRs.** Run `/deslop` on the diff before you commit. `/deslop` is a Skill in [cursor-team-kit](#) that removes the extra code AI tends to write. Run `/no-comments`, a Skill that removes unneeded comments, before review. Write the title, description, and commit bodies with `/technical-writing`, which sets the standard for technical writing. Then run `/unslop`, which cuts the phrasing AI tends to write ([Chapter 32](#)).
- **Titles.** Write titles in Conventional Commits format, such as `fix(pstack): retarget opening-a-pr babysit trigger`.
- **Forge.** Use the same forge from creation to merge.
- **Size and stacks.** Prefer five narrow PRs to one large PR, and stack the small PRs.

- **Readiness.** Open the PR ready for review, not as a draft.

- **git worktree.** A Git feature that adds a working directory with a different branch checked out from the same repository. For example, you can keep the main working directory and open a fix branch in a separate directory.

A stack has the following shape. Each child PR targets its parent's branch as its base.

```
main ← trunk
├── PR #1 (base: main) ← only the root PR targets trunk
│   ├── PR #2 (base: PR #1's branch)
│   │   └── PR #3 (base: PR #2's branch)
```

Write the PR body as a "briefing"

The key point of this Playbook is the first sentence of its rules for the body.

The PR body is a briefing, not the lab notebook.

Do not list what you tried or how the work went. **Write only what a reviewer with the diff in hand needs to decide whether to approve the change.** Order the body as ## Why, ## Scope, ## Tradeoffs, ## Blast Radius, and ## Verification, and leave out any section with nothing to say.

Opening a PR does not start Babysit

After the agent opens a PR, it reports the PR's URL and continues with the rest of the work.

A "Babysit" run for each PR interrupts the work every time. Also, if later work rewrites the commits, the CI checks that ran on that PR go to waste, and the checks have to run again on the new commits.

The exception is the owners, the agents that each own one PR, in "Autopilot-full" and "Autopilot-stack", which [Chapter 16](#) covers. Autopilot-full is the Playbook in which agents

take a queue of PRs to merge on their own. Autopilot-stack works the same way but builds one stack instead of merging. The owner's instructions include the "Babysit" loop, so after an owner opens its PR, it keeps going until the PR is merge-ready, or STACK-READY in Autopilot-stack.

How to phrase the request: ask for small, ordered commits and evidence in the description

The example request in the bundled guide asks for small, ordered commits and for evidence in the PR description.

```
/poteto-mode open the pr. small ordered commits, evidence in the description.
```

"Babysit" clears problems from the unmerged frontmost PR first and makes it merge-ready

"Babysit" is the Playbook that clears conflicts, review threads, and CI failures and **brings a PR or a stack to merge-ready**. It stops when it reaches a point that needs a human decision.

Cursor also has a built-in /autopilot Skill that watches PR status. Its former name was babysit. Similar request wording can invoke that Skill. So /poteto-mode states that it **routes every request about PR status to this Playbook**. Here are examples of such requests.

- "Watch this PR"
- "Get it green"
- "Address the Bugbot comments"
- "Check on PR X"

Bugbot is a Cursor feature that reviews PRs automatically. When someone creates or updates a PR, Bugbot reads the diff and posts suspected bugs and security problems as PR review comments.

Steps: clear problems from the frontmost PR first

The main steps are these:

1. **Declare the mode.** Before you check status, choose the mode, which sets how far to go. The default is `drive`. Use `check` for a small PR or a docs-only PR (see the table below for each mode).
2. **Focus only on the frontier.** Fix only the unmerged frontmost PR, and leave the PRs above it alone.
3. **Assign one agent per stack.**
4. **Do not change the stack's structure, which is the parent-child links between PRs.** No rebase, no base retargeting, and no force-push. If the PR needs a rebase, report that need to the branch owner.
5. **Clear conflicts, then review threads, then CI.** Combine all known fixes into one push so the checks rerun only once. Conflicts are the one thing the agent does not resolve itself. It reports which branch needs a rebase and stops.
6. **Trust the forge's judgment.** "ready" means the forge accepts the PR as mergeable, not that all CI checks pass. On GitHub, the bundled script `watch-pr` gets the PR's status.
7. **Classify a CI failure before you rerun it.** Rerun only for flakes or infrastructure problems. A flake is a test that fails now and then for reasons unrelated to the code. Fix with a commit only when the diff's own code caused the failure.
8. **Triage the Bugbot comments.**
9. **Stop where a human decision is needed.**

- **Triage.** The sorting of items to decide whether each needs action and which action to take. Examples of actions are fix, dismiss, and ask.

Mode	What it does	Matching requests
<code>drive</code>	Repeats status checks and fixes until merge-ready	"Get it green", "Make it mergeable"
<code>background</code>	Triages PR status without stopping other work	While a plan is still running

Mode	What it does	Matching requests
threads-only	Answers only review comments	"Address the Bugbot comments"
check	Checks status once and reports	"Check on X", "Is it green?"

Sort Bugbot comments into fix, dismiss, and ask

In step 8, the agent sorts comments from Bugbot and other reviewers into three groups before acting on them. The criteria are in [references/bugbot-triage.md](#).

- **fix.** If the comment may concern correctness, security, data loss, or similar, fix the problem in the earliest-merging PR that contains the problem code, and resolve the thread.
- **dismiss.** If the comment matches a recorded low-risk pattern and the code proves the concern does not apply, reply with a short reason and resolve the thread.
- **ask.** If the finding is of a new kind, of high severity, or ambiguous, ask the user instead of guessing.

The agent first verifies a Bugbot finding against the code. When it cannot tell whether the finding is a false positive, **it asks the user instead of guessing**. A real bug that the agent wrongly dismisses costs more than the time a question takes.

A review comment is a claim to verify, not an instruction to follow as is. So the agent does not change code only to make a finding go away. It fixes the code only when it has verified the problem.

Even at merge-ready, the agent does not merge

The merge rule in step 9 is this:

Babysitting never authorizes merging. Only an explicit request to merge, land, ship, or merge when ready does.

Even when the PR reaches merge-ready, **the agent reports and stops**. When the user asks it to merge, the agent hands off to "Shipping".

When the PR reaches merge-ready, the agent reviews the comment triage from that run. If it finds a dismiss pattern that would help the team, it proposes to add the pattern to `references/bugbot-triage.md` in a separate PR. A dismiss pattern is a recorded low-risk pattern.

The key point is the "frontier only" rule

In "Babysit", **the agent fixes problems in the PR that merges next first.**

It also reads review comments on later PRs and collects what needs doing. But if the fixes for those comments would make the earlier-merging PR rerun its checks, the agent postpones the fixes on the later PRs.

For example, if PR #1's CI is failing and a review comment arrives on PR #3, the agent gets PR #1's CI passing first. A stack can merge only from the bottom up, so the frontier is the only place where merging can move forward right now.

If the agent fixed and pushed each comment on an upper PR as it arrived, the checks would also rerun every time. So the agent holds the comments on upper PRs and fixes them in the same single push as the frontier fix.

How to phrase the request: ask "get it green", and keep the request small when you want only status

When you want the agent to bring the PR to merge-ready, ask it to "get it green". Here is the request from the start of the chapter again.

```
/poteto-mode babysit this pr. get it green.
```

When you want only the status, keep the request small. The agent answers the following request in check mode without starting the loop.

```
/poteto-mode check on pr 123. anything outstanding?
```

"Shipping" brings into trunk only the range of PRs that passed another agent's verification

"Shipping" is the Playbook to use **when the user asks the agent to bring a stack whose CI passed into trunk**, the main branch that PRs merge into. It verifies each PR independently and merges, one at a time, only the verified range that continues from the root. /poteto-mode sums up this Playbook as "Green is not safe."

The key point is the definition of "safe"

Safe means a verdict from an agent that did not write the code. CI green is not a verdict, and an approving bot review is not a verdict.

This definition applies the idea from [Chapter 2](#), "trust the artifact, not the agent", to the check right before merge. **The definition also decides who verifies.**

Steps: verify each PR, find the continuous range, and bring PRs into trunk one at a time

There are six steps.

1. Verify every PR independently
2. Bring in only the continuous verified range that starts at the root
3. Verify that the verdict still describes the same diff
4. Prepare only the frontmost PR, and merge one at a time
5. Recompute after each merge, and watch the status until the frontmost PR merges
6. Stop at the limit

The sections below look at each step in turn.

- `git patch-id`. A Git command that computes an identifier from the contents of a commit's diff. If the diff is the same, the value is the same even when the commit ID changes.

1. Verify every PR independently

Assign one subagent per PR and have it compare the parent branch with the PR's latest commit on the real screen or with real commands. The result is PASS, PASS+NOTES, or FAIL. PASS+NOTES is a pass with notes.

2. Bring in only the continuous verified range that starts at the root

Step 2 decides the "continuous range" as follows.

```
PR #4  PASS           ← verified, but an unverified PR sits below it
PR #3  (unverified) ← the continuous range breaks here
PR #2  PASS
PR #1  PASS           ← frontier
main
```

In other words, **the agent may bring in only PR #1 and #2.**

To merge PR #4, an agent must first verify and merge the unverified PR #3 underneath it. If the agent skipped the verification of #3, unverified changes would get into trunk.

3. Verify that the verdict still describes the same diff

Even if a rebase changed the commit ID, which is the SHA, use `git patch-id` to compare whether the diff's contents are the same. If the contents changed, verify again.

4. Prepare only the frontmost PR, and merge one at a time

To prepare the frontmost PR, rebase it onto the latest trunk if needed, and retarget its base to trunk. Prepare the next PR only after the previous PR has merged.

5. Recompute after each merge, and watch the status until the frontmost PR merges

After each merge, recheck the base, checks, and patch-id of the PR that became the new frontier. Even if auto-merge is queued, do not treat that as evidence that the stack is safe.

6. Stop at the limit

The limit is the top PR of the continuous verified range. In the diagram in step 2, the limit is PR #2. After the agent merges up to that PR, it reports which PRs it brought in and which PR is the next unverified one, then finishes.

How to phrase the request: ask the agent to "land the stack"

Here is the request from the start of the chapter again.

```
/poteto-mode land the stack.
```

Summary

- **How the three Playbooks connect.** "Opening a PR" creates the PR, "Babysit" gets it to a mergeable state, and "Shipping" merges verified PRs into trunk, the main branch that PRs merge into.
- **Opening a PR.** It creates small, ordered commits and PRs with a narrow diff, and keeps the body short as a "briefing". Opening a PR does not start "Babysit".
- **Babysit.** It declares a mode, looks only at the frontier, and brings the PR to merge-ready. It sorts Bugbot comments into fix, dismiss, and ask.
- **Shipping.** It defines "safe" as a verdict from an agent that did not write the code, and merges, one at a time, only the continuous verified range from the root.
- **The difference between Babysit and Shipping.** Of the two, only "Shipping" merges, and only on an explicit request. (The Playbooks in [Chapter 16](#), such as "Autopilot-full", have different rules.)

The next chapter, [Chapter 15](#), looks at four Playbooks that continue one long piece of work while you are away, stop it, resume it, and clean it up.

Chapter 15: Delegate long-running work to the agent

kaito • Sources: 日本語 • English • 2026-10-03

This chapter covers the following four Playbooks.

1. Autonomous run
2. Pause safely
3. Session pickup
4. Worktree and simulator cleanup

All four Playbooks **handle one long piece of work that you delegate to the agent**. Their roles are as follows.

- Carry one task through to the end without stopping
- Stop partway so that another agent can resume
- Clean up unneeded worktrees and other leftovers after the work

- A **worktree** is a git feature that creates several working directories from one repository, so you can check out different branches at the same time.

The bundled guide's page on overnight work (<docs/guide/07-overnight.md>) lists three conditions for work that you delegate while you sleep. The conditions are as follows.

- **A finish condition you can verify**
- **An isolated worktree**
- **A decision record to audit in the morning**

For example, "zero old callers" is a finish condition you can verify, but "work for four hours" does not meet this condition.

This chapter sorts the four Playbooks by role, then explains when to use each one, its steps, and its key point.

How this chapter is organized

This chapter is organized as follows.

- The four Playbooks split one long piece of work into "continue, stop and resume, and clean up"
- "Autonomous run" sets the finish condition first and keeps working without stopping until the condition is met
- "Pause safely" stops the work at a point where an agent that knows nothing can resume
- "Session pickup" trusts the previous agent's record and starts from where the work left off
- "Worktree and simulator cleanup" proves that a worktree is "not in use" before it deletes the worktree
- Summary

The four Playbooks split one long piece of work into "continue, stop and resume, and clean up"

The four Playbooks in this chapter fall into the **three roles** in the following table.

Role	Playbook	What it is responsible for
Carry one task through to the end without stopping	"Autonomous run"	The finish condition
Stop and resume	"Pause safely"	A stop in a state that another agent can resume
Stop and resume	"Session pickup"	Where to resume the previous work

Role	Playbook	What it is responsible for
Clean up	"Worktree and simulator cleanup"	Reclaimed disk space, and a check before deletion that each item is safe to delete

Tools that appear throughout the chapter

The Playbooks in this chapter use the following tools again and again.

- `/loop` is a built-in Cursor command that starts the agent at a fixed interval or on an event, such as the end of a CI run.
- `decisions.tsv` is the decision record that `/show-me-your-work` ([Chapter 28](#)) writes, with one decision per line.

For long work, or work that the agent does while you are away, the agent records its decisions with `/show-me-your-work`. In the morning, you do not reread all of the night's work.

You audit the decisions in this record, `decisions.tsv`.

"Autonomous run" sets the finish condition first and keeps working without stopping until the condition is met

"Autonomous run" is the Playbook that **carries a long task to completion without stopping**, for requests like "loop until it's done" or "use `/loop` until X".

Six steps

The steps are as follows.

1. Set the finish condition
2. Decide how the agent wakes up
3. Make one needed change at a time based on evidence, and verify the result
4. Handle what you find along the way yourself
5. Record the decisions with `/show-me-your-work` every time

6. Stop when the finish condition is met

The sections below look at each step in turn.

1. Set the finish condition

Before the first iteration, write the finish condition in a form such as "the tests pass" or "all N PRs are merged".

2. Decide how the agent wakes up

Set up the wake-up with `/loop`. If there is an event to wait for, such as the end of a CI run or a merge, add a watcher subagent that monitors the event and wakes the agent. Also set up a periodic wake-up at a long interval as a backup. If there is no event to wait for, wake the agent at a fixed interval that matches how often a recheck of the result is worthwhile.

3. Make one needed change at a time based on evidence, and verify the result

Make the smallest change the evidence calls for. After the change, verify whether the work moved closer to the finish condition set at the start. If it did, commit. If it did not, revert the change. The principle "[Sequence Work into Verifiable Units](#)" asks for this cycle ([Chapter 19](#)).

4. Handle what you find along the way yourself

The agent fixes related bugs and flaky checks itself, and puts fixes that stray from the main task in separate PRs. It asks a human only about irreversible operations, decisions that an experiment cannot settle, and real dead ends.

5. Record the decisions with `/show-me-your-work` every time

On every iteration, record one row for what changed and whether the finish condition moved closer.

6. Stop when the finish condition is met

If the condition is not met, try another approach. Do not lower the condition and call the work finished. If the agent cannot make progress at all, it reports the situation.

What the reply includes

The reply includes the following items.

- The finish condition
- The number of loop iterations
- The changes kept in the end, and the changes reverted because they did not move the work closer to the condition
- The final state of the condition

The key point is to make the finish condition a predicate you can verify

A predicate is a condition whose result you can judge as true or false.

As I said at the start of the chapter, **a length of time is not a finish condition**. "Work for four hours" gives nothing to verify, and all that is left in the morning is four hours of activity. The bundled guide's pitfalls page (<docs/guide/10-recipes-and-pitfalls.md>) also lists a vague condition like "make it better" as a failure case. So the agent sets the condition in step 1 and keeps the same condition until it stops in step 6.

"Pause safely" stops the work at a point where an agent that knows nothing can resume

"Pause safely" is the Playbook to use when someone tells the agent to stop, when Cursor is about to restart, or when compaction is near. Compaction is the process that summarizes the part of the conversation that no longer fits into the context. **The Playbook stops the work in a state that even an agent that knows nothing can resume.** It runs only on an explicit instruction, and it does not stop when the user says "keep going" or "don't stop".

Four steps

The Playbook has the following four steps.

1. **Stop at a safe point.** Finish or revert the current smallest step, and start nothing new.
2. **Do not do anything irreversible only to stop.** Do not create a PR or push, unless a PR already exists.
3. **Make the work durable.** Squash the uncommitted edits into one commit that starts with `wip:`.
4. **Write the resume note outside the context.** Write the intent, the progress, the next step, and the caveats. If a `/show-me-your-work` record exists, point to it instead of duplicating it.

The reply includes the following items.

- Where in the loop the agent is
- The information recorded in files and commits, and the state, intent, caveats, and other parts of the work that still live only in the conversation's context
- The commits it made
- The first action on resume

The key point is to move "what exists only in the agent's head" into files

The agent's context disappears at compaction or at the end of a session. In its reply, the agent writes "what is on disk" separately from "what exists only in its head". What is on disk is the information recorded in files and commits. What exists only in its head is the state, intent, and caveats of the work. This separation **shows you what the agent failed to move into files or commits.**

The `wip:` commit saves the half-edited code to disk, and the resume note saves the intent and caveats that cannot go into code. You can read this Playbook as a procedure that applies "the codebase is memory" from [Chapter 4](#) to a single piece of work.

"Session pickup" trusts the previous agent's record and starts from where the work left off

"Session pickup" is the Playbook that takes over work the previous agent left unfinished, from a transcript, a Cloud Agents URL, or a pushed branch. It is the counterpart of "Pause safely".

To take over one specific piece of work, use "Session pickup". **To gather context from several chats, use `/recall` (Chapter 22).**

Five steps

The Playbook has the following five steps.

1. Find the previous record

- Look in a transcript in the current workspace, a Cloud Agents URL, or a pushed branch
- Have a subagent read a long transcript, and keep only the summary in the main thread, which is the conversation that calls the subagents. The principle "[Guard the Context Window](#)" asks for this delegation (Chapter 20)

2. Rebuild the state of the work

- Find out the working branch, the changes already in (verify with `git log` and `git diff` against the base), the remaining TODOs, and the decisions already made

3. Compare what is done with what remains

- Compare what was planned with what is done to find where to resume, and do not repeat finished work

4. Hand the remaining work to the Playbook that fits it

- From here on, the Playbook that fits the work takes over

5. Verify the inherited claims against the real thing

- Do not treat the previous agent's self-report of "it succeeded" as proof. Verify the inherited changes against the real artifact, as the principle "[Prove It Works](#)" asks (Chapter 19)

The reply includes the following items.

- Where the previous agent stopped
- What it took over and what it redid
- The resume point
- The result

The key point is to treat the previous record as trusted input

The agent that resumes uses the previous record as input as it is. It does not derive the record's contents again on its own.

The Playbook asks the agent to use the previous record to tell finished work from remaining work, and to resume from what remains.

For example, if the previous agent reproduced a bug and left the steps, screens, and logs, the next agent does not repeat the same steps. It reads that record and resumes from the investigation of the cause or from the fix.

How to write the request: ask the agent to read the decision log and continue from the finished work

The bundled guide's example request asks the agent to take over a branch, read the decision log to find out what is done, and not redo finished work.

```
/poteto-mode take over this branch. read the decision log, figure out what's done, and continue from there. don't redo finished work.
```

"Worktree and simulator cleanup" proves that a worktree is "not in use" before it deletes the worktree

"Worktree and simulator cleanup" is the Playbook that frees disk space by deleting merged or abandoned git worktrees and old iOS simulators.

Six steps

The steps are as follows.

1. Record and audit
2. The classification is advice, not permission
3. Verify usage before you delete
4. Stop before an irreversible loss
5. Delete what is confirmed
6. Clean up the other places to reclaim, such as simulators

The sections below look at each step in turn.

1. Record and audit

Record the current disk usage with `df -h /`, and run `worktree-audit.sh`. The principle "Build the Lever" asks for this record (Chapter 17).

`worktree-audit.sh` is a read-only script that deletes nothing. It checks the following items.

- The size of each worktree
- The age of each worktree, which is the time since its last update
- Whether the worktree is merged
- Uncommitted edits
- The PR status
- The chat that last touched it

Based on these items, the script proposes a classification.

2. The classification is advice, not permission

The script's classification proposes candidates for deletion. Leave out any worktree that a pinned or active chat is using. A pinned or active chat is a chat with the agent that appears in the sidebar. In the past, the script has marked a worktree that a pinned chat was using as safe. So check the candidates against the list of chats.

3. Verify usage before you delete

For a doubtful worktree, have a subagent read the transcript to verify whether the worktree is in use.

4. Stop before an irreversible loss

For a worktree with uncommitted work, show the user the diff and get a decision.

5. Delete what is confirmed

Delete with `git worktree remove` and `git worktree prune`. The branch stays, so no commits are lost.

6. Clean up the other places to reclaim, such as simulators

Delete old simulators and runtimes, and the build cache if needed.

What the reply includes

The reply includes the following items.

- The free disk space before and after the deletion, and the space gained
- The worktrees it deleted
- What it kept, and the reason for keeping each item

The key point is that the gate before deletion is itself the review

When the agent deletes a worktree or a simulator, the user can lose a working environment or uncommitted changes. So before it deletes anything, the agent confirms that the item is not in use and has no uncommitted changes. This check is the gate that keeps the agent from deleting the wrong thing.

In other Playbooks, code review catches mistakes. This Playbook is the only one that deletes worktrees and simulators on the user's machine with no code review. So **the gate**

in steps 2 to 4, which verifies that no item is in use and that no work is uncommitted, is itself the review.

How to write the request: ask the agent to "prune the worktrees that are safe to prune"

The bundled guide's example request asks what is using the disk, and asks the agent to delete only the worktrees that are safe to delete.

```
/poteto-mode what's eating my disk? prune the worktrees that are safe to prune.
```

Summary

- Of the four Playbooks, "Autonomous run" carries one task to the end, "Pause safely" and "Session pickup" stop the work and resume it, and "Worktree and simulator cleanup" handles the cleanup.
- "Autonomous run" sets a finish condition you can verify and keeps working without stopping until the condition is met.
- "Pause safely" moves what exists only in the agent's head into a wip: commit and a resume note, then stops.
- "Session pickup" trusts the previous record, does not redo work, and verifies the inherited claims against the real artifact.
- "Worktree and simulator cleanup" proves that an item is not in use before it deletes the item. The gate before deletion is itself the review.

The next chapter, [Chapter 16](#), looks at four Playbooks for planning many PRs and running them with several agents.

Chapter 16: Plan many PRs and run them with multiple agents

kaito • Sources: 日本語 • English • 2026-10-03

This chapter covers four Playbooks:

1. Multi-phase or multi-PR plan
2. Orchestrate
3. Autopilot-full
4. Autopilot-stack

All four plan and run work that spans many PRs with multiple agents. "Multi-phase or multi-PR plan" writes the plan document. "Orchestrate" manages work that lasts for days. "Autopilot-full" and "Autopilot-stack" move a queue of PRs forward and verify each one.

The previous chapter, [Chapter 15](#), covered the Playbooks that continue, stop, and resume one long piece of work.

This chapter first sorts out how the four Playbooks divide the work. It then explains when to use each one, its steps, and its key point. It ends by comparing the options for long-running work.

How this chapter is organized

The chapter has these sections:

- The four Playbooks plan and run many PRs, and the central agent writes no code
- "Multi-phase or multi-PR plan" writes no code and produces a checklist plan document that fixes how each PR is verified

- In "Orchestrate", one coordinator agent that writes no code manages many PRs of work that lasts for days
- In "Autopilot-full", an owner per PR takes it to merge, and the coordinator agent only issues verification verdicts
- "Autopilot-stack" builds and verifies PRs the same way as Autopilot-full, but does not merge them and hands them to the operator as one stack
- Pick among the five options for long-running work by who merges and what the unit of work is
- Summary

The four Playbooks plan and run many PRs, and the central agent writes no code

The four Playbooks in this chapter divide responsibility as the following table shows.

Playbook	What it owns
"Multi-phase or multi-PR plan"	The plan. It does not own the code
"Orchestrate"	The program, meaning the whole of work that lasts for days. It does not own code
"Autopilot-full"	The verification verdict. It does not own the PRs
"Autopilot-stack"	The stack. It does not own the merge

Each of the four Playbooks states what it does not own: code, PRs, or the merge.

The coordinator agent writes no code itself. It spends its time on the plan, on briefs for the Workers, and on verdicts. Workers are the agents that run in parallel. "Orchestrate" has one exception. When the local Git operation is fast, the coordinator agent may pull verified changes into a branch and push it.

The coordinator agent is a long-lived chat that talks with the **operator, the human who requests the work and makes the final call**. "Orchestrate" calls it the "coordinator", and "Autopilot-full" and "Autopilot-stack" call it the "root". This chapter uses the name "coordinator agent" for both.

Tools that appear throughout the chapter

The Playbooks in this chapter use these tools again and again:

- `/goal`. A command that sets a goal for the agent. The goal persists across conversation turns until the queued work is done.
- `decisions.tsv`. The decision record that `/show-me-your-work` ([Chapter 28](#)) writes, with one decision per line.
- **state-then-wait**. A rule with two parts. When the operator asks the agent to explain the approach or the plan, the agent gives the explanation and stops. It also starts no work until the operator explicitly says "start".

"Multi-phase or multi-PR plan" writes no code and produces a checklist plan document that fixes how each PR is verified

"Multi-phase or multi-PR plan" is the Playbook that plans work that spans several phases or stacked PRs.

The plan is a checklist. The person doing the work carries out one item at a time and checks it off with evidence that it is done. The operator looks at that evidence to confirm that each item is really done. **An item without evidence does not get checked.**

In [The Complete Guide to pstack, Part 2](#), poteto introduces this Playbook as the way to write an execution plan once you have a design you are satisfied with. [Chapter 39](#) covers planning in practice.

Seven steps

The steps are as follows.

1. Decide whether a plan is needed
2. Settle open questions with a prototype before writing
3. Delegate research to subagents
4. Copy the plan document template and fill in every item
5. Write the plan document to technical-writing standards and cut AI phrasing

6. Check the shape of the plan document
7. Hand off the plan

The sections below look at each step in turn.

1. Decide whether a plan is needed

If the change fits in one or two files and the direction is clear, write no plan. Say that no plan is needed and stop.

2. Settle open questions with a prototype before writing

Settle them with "Prototype" (Chapter 12) and keep the evidence in the plan document's appendix. Ask the operator only about decisions that running something cannot settle. This step follows the principle "Never Block on the Human" (Chapter 20).

3. Delegate research to subagents

Subagents do the research. Each one returns the locations of the relevant files, the code conventions, the commands that run the tests, and the entry points, such as the `main` function or the function that first receives an API request.

A subagent does not paste the full contents of the files it read into its reply. Because the subagent leaves out the file contents, the research itself stays out of the main agent's context. This step follows the principle "Guard the Context Window" (Chapter 20).

4. Copy the plan document template and fill in every item

For each PR, the plan document states the change that PR makes, the files it changes, and how to verify it. One PR holds one change, sized as a unit you can verify.

5. Write the plan document to technical-writing standards and cut AI phrasing

Write it with `/technical-writing`, which sets the standard for technical documents, and run it through `/unstop`, which cuts AI phrasing.

6. Check the shape of the plan document

Run `check-plan.mjs` and fix every error it prints: missing headings, missing verification items, and violations of the writing rules.

7. Hand off the plan

Put the plan's path and the script's output in the reply, then stop. Execution waits for the operator's explicit go-ahead, then starts with the execution Playbook named in the plan: "Autopilot-full", "Autopilot-stack", or "Orchestrate".

What the reply includes

The reply contains these items:

- The plan's path
- The dependencies between PRs
- The PRs that need review
- What the prototype proved
- The output of `check-plan.mjs`

The plan document template and `check-plan.mjs`

The plan document has this structure:

- An opening summary
- How to read the plan
- A checklist for the whole program
- A section per PR
- A closing section
- Appendices for prototype evidence, options not taken, risks, and references

Each PR section has these nine subheadings:

- Dependencies
- Files touched

- Change
- Observable result
- unit verification (unit tests)
- live verification (behavior in the real app)
- perf verification (performance)
- Review gate (whether the operator must check the PR before merge, and how)
- Merge

`check-plan.mjs` checks the plan document for violations in headings, required items, writing style, and similar rules. When it finds a problem, it prints the line number and the reason. This Playbook puts the plan document's rules into a script as well as into prose, which follows the principle "[Encode Lessons in Structure](#)" ([Chapter 21](#)).

The key point is the verification rule that "tests alone are not verification"

This Playbook puts the following sentence at the top of each of the unit, live, and perf verifications as the shared pass condition.

Tests alone are not sufficient verification. A PR is verified only when its unit, live, and perf boxes are all checked.

The plan document gives each PR a unit, a live, and a perf verification field. Each field states the concrete way to check. A PR that affects the screen or interaction also gets a "Review gate", where the operator checks it before merge.

Each field holds the following:

- **Verify, unit.** The test file, the cases to add, and the command to run.
- **Verify, live.** Required. Ten lanes, which are parallel verifiers, drive the real screen or CLI at the PR's latest commit. Lane 1 is the regression lane. It runs the same scenario on both trunk and the PR to catch anything the change broke. Trunk is the main branch that PRs merge into.
- **Verify, perf.** Measure the same metric on both trunk and the PR, and set the number that counts as failure.

- **Review gate.** For a PR that affects the screen or interaction, the operator reviews screenshots and video before merge.

To drive a screen, use `control-ui` for a browser or Electron, and `control-cli` for a CLI or TUI. Both are in [cursor-team-kit](#).

- **TUI.** A text user interface, an interface drawn inside the terminal and operated with keys, such as vim or htop.

How to phrase the request: state the constraints and what you want to see first

The example request in the [README](#) states a constraint, a condition, and a request. The constraint is that nothing internal leaks. The condition is to work in a temporary directory, and the request is to show the dependency graph first.

```
/poteto-mode open source these skills as a plugin. nothing internal leaks, work in a temp dir, show me the dependency graph first.
```

In "Orchestrate", one coordinator agent that writes no code manages many PRs of work that lasts for days

"Orchestrate" is the Playbook that hands one coordinator agent a project that lasts for days, with many PRs and tens to hundreds of subagents. It assumes the operator checks in only twice a day. **The coordinator agent owns no code.** It writes briefs for the Workers, which are the subagents, processes completion notices, and makes decisions.

Seven steps

The steps are as follows.

1. Set the exit condition (Frame)
2. Prepare a place for state (Install the runtime)

3. Do a trial run with one unit (Pilot)
4. Add Workers (Scale)
5. Process completion notices in batches (Drain)
6. Merge verified units in order (Land)
7. Reconcile the records and close (Close)

The sections below look at each step in turn.

1. Set the exit condition (Frame)

Write the exit condition so that the records can decide whether the work is done, for example "all 126 units are merged and each is verified in the ledger". If one agent can finish within the work-time budget you set, switch to "[Autonomous run](#)" ([Chapter 15](#)).

2. Prepare a place for state (Install the runtime)

Create the store with `orch init`. The store is the area that holds work status and decision records. Then open the decision record and write the standing instructions.

3. Do a trial run with one unit (Pilot)

First take one unit of work all the way through: write the brief, implement, verify, add the PR to the stack, record it in the ledger, and merge. Revise the brief and the verification method based on the result, then expand to the rest of the work.

The trial run (Pilot) uses one agent and one unit of work. The coordinator agent fixes problems in the brief, the verification steps, and the unit size during the trial run, before the work expands. **The trial run keeps the same flaw from spreading to many agents and causing rework.**

4. Add Workers (Scale)

Start Workers up to the concurrency limit, and start a new Worker for each one that finishes.

5. Process completion notices in batches (Drain)

Collect completion notices from Workers in the inbox (`inbox/`) and process them together at set points.

6. Merge verified units in order (Land)

Do not save merges for the end. Merge continuously, starting with the first verified unit.

7. Reconcile the records and close (Close)

Check that the table records every started agent in a finished state, such as completed or abandoned. Verify against the real results that the exit condition is met. Find the instruction corrections you had to repeat and the instructions that kept being needed. Write them into `preferences.md` or the brief template so that the next agents get them too.

What the reply includes

The reply reports the following. Items about PRs include links to those PRs.

- Progress toward the exit condition, counted from `units.tsv` and `ledger.tsv`
- Merged PRs
- Abandoned work and the reason
- Items waiting for the operator's decision

The progress numbers come from `units.tsv`, which records the state of each unit, and `ledger.tsv`, which records verification results.

Roles come in three layers

"Orchestrate" uses three layers of roles:

- **Coordinator agent** ("coordinator" in Orchestrate). The main agent. It orchestrates the tasks: the framing, the briefs, inbox processing, and reports to the operator. It writes no code.

- **Sub-coordinator.** A subagent that the coordinator agent adds only when the work exceeds what the coordinator agent can handle. Each track gets one sub-coordinator. A track is a kind of work, such as build or verification.
- **Workers and verifiers.** Workers carry out the work assigned to them, and verifiers verify a Worker's output. Both run in the cloud by default. Verification uses a model from a different family than the Worker's. Only one Worker edits a given worktree or branch.

- **worktree.** A Git feature that creates several working directories from one repository so that you can check out different branches at the same time.

The store, the briefs, and the ledger hold the state

The coordinator agent creates `orchestrate/<project-slug>/` in the store. It writes state into table files with `orch`, a bookkeeping CLI. The main files are these:

File	Contents
<code>preferences.md</code>	Shared instructions every agent must follow. One rule per line. The coordinator agent passes the file to an agent every time the agent starts or resumes
<code>units.tsv</code>	For each unit of work: ID, track (a line of work that runs in parallel), state, branch, PR, head SHA (the ID of the PR's latest commit), and brief location
<code>ledger.tsv</code>	The ledger that records the verification result for each PR
<code>inbox/</code>	The folder that records completion notices from Workers and the locations of their reports
<code>gates.md</code>	Matters that need the operator's decision, recorded as the question, the options, and what happens if no answer comes
<code>decisions.tsv</code>	A record of decisions made during the work and their reasons

The brief is the only explanation a Worker gets

When the coordinator agent gives Workers work, it writes a brief for each unit of work.

Workers cannot ask the coordinator agent questions during the work. **Anything the brief leaves out, the Worker has to fill in by guessing.**

Work that proceeds on guesses tends to produce the wrong result. So the brief has fixed fields to fill in. The main fields are these:

- **GOAL.** The purpose of the work in one sentence, written so that someone who does not know the chat history can carry it out.
- **SCOPE.** The files the Worker may change, the files it must not change, and the branch it uses exclusively.
- **CONTEXT.** The files and PRs to refer to. Workers cannot see other Workers' work, so when the work depends on an earlier result, paste the earlier Worker's full report as is.
- **ACCEPTANCE.** The conditions for counting the work as done, one per line, in a form that can be verified.
- **VERIFY.** The commands or steps to run to verify.
- **TIMEBOX.** A rough upper limit on work time. When the limit comes, the Worker returns its partial result and stops.
- **FORBIDDEN.** What the Worker must not do, for example changes outside the scope or a force-push.
- **REPORT.** The items to return in the report after the work.
- **STANDING.** The contents of `preferences.md`, pasted as is.

If the coordinator agent cannot fill even one field for a unit of work, it starts no

Worker. For example, if you cannot write ACCEPTANCE, you have not decided what counts as done. A Worker started anyway has to guess the finish criteria, and the results vary. A field you cannot fill shows that the scope of the work is not yet decided. Settle the scope first, and then start the Worker.

Still, the length of the brief matches the size of the work. A long brief with every field written in detail for a few-line fix costs more to write and read than the work itself. For small work like that, write only the goal, the scope, the command to verify with, and the report format, in one paragraph.

The ledger records how far each PR was verified

The ledger (`ledger.tsv`) records whether a PR was really verified. It holds one line per PR with a verdict that says how far the PR was verified. There are five verdicts:

Verdict	Meaning
live-ui-verified	A verifier drove the real screen or CLI and verified the behavior
unit-test-verified	Unit tests verified the PR
type-check-only	Only the type check passed
verifier-blocked	Verification could not run, for example because of an environment problem
verifier-failed	Verification found a problem

For example, work that changes the screen or behavior needs more than the type check of `type-check-only`. It needs a stronger verdict, because a type check does not verify real behavior.

`verifier-blocked` is not a pass, so the coordinator agent reruns verification once the environment is fixed. When a PR gets `verifier-failed`, the coordinator agent does not repeat the same verification. It creates a new unit of work to fix the problem.

A passing CI run is one input to a verdict. **A passing CI run alone is not a verdict.** CI runs only the tests that exist and does not verify real behavior.

The ledger records verdicts per pair of PR number and head SHA. Adding a commit to the PR or rebasing it changes the head SHA.

A PR whose head SHA changed may differ from what it was when the verdict was recorded. So **the coordinator agent does not use a verdict recorded at an old head SHA**. When the commit changes, verification runs again, and the ledger records the new verdict under the new head SHA.

Only one stacker restacks

A stack is a chain of PRs linked as parent and child. When a lower PR changes, someone must rebuild the PRs above it on top of the changed PR, as the diagram below shows. The Playbook calls this rebuild a restack.

```
Before the update
main
├── PR #1 (parent: main)
│   └── PR #2 (parent: PR #1)
│       └── PR #3 (parent: PR #2)

Update order after change A is added to PR #1
PR #1 → PR #2 → PR #3

After the update
main
├── PR #1 (change A)
│   └── PR #2 (picks up the updated parent PR #1)
│       └── PR #3 (picks up the updated parent PR #2)
```

"Orchestrate" places exactly one agent per stack to restack it. The Playbook calls this agent the stacker, the stack's manager.

The stacker restacks with the commands (`gt`) of Graphite, a tool that handles stacked PRs together.

- **Graphite.** A tool that creates and manages PRs as a stack with parent-child relationships.

Workers use neither `gt` nor `rebase`. When several agents rebuild the same stack at once, they overwrite each other's changes and break the parent-child links between PRs.

Limiting restacks to one stacker means only one agent ever rewrites the stack, so these conflicts do not happen.

Do not resume an agent only to check on it

To check on progress, the coordinator agent does not resume a waiting agent and ask it. Asking that agent makes it resume its work. Instead, the coordinator agent reads the records that agents leave behind, such as the ledger, `units.tsv`, and pushed branches. It changes nothing in them.

Agents ask the operator to decide only four kinds of things

This Playbook assumes agents work on their own for long periods and the operator checks in only twice a day. If the agents ask about things they could decide themselves, work stops until the answer comes. So agents ask the operator to decide only these four kinds of things:

- Operations that cannot be undone, such as a force-push to a shared branch, a deploy, or a deletion
- Product or taste decisions that trying things cannot answer
- Cases where the shared instructions in `preferences.md` conflict with the real situation
- A dead end that remains even after the plan is rebuilt

Before asking, the agent records the matter in `gates.md`, and it continues other work while it waits for the answer.

- **force-push.** A push that forcibly overwrites the history of a remote branch with the local history.

The key point is three rules about completion, standing instructions, and briefs

This Playbook puts the following three rules at the top, and every later step assumes them.

- **Do not process completion notices as each one arrives. Collect them and process them together.** Dozens of Workers run at once, so if the coordinator agent stopped for

every notice, its own work would not move. Notices collect in the inbox (inbox/), and the coordinator agent processes them together at a good break point (step 5).

- **Always pass the shared instructions, both at start and at resume.** An agent can drop instructions it received earlier when the coordinator agent resumes it. Each dropped instruction makes the operator repeat themselves. So the coordinator agent passes the contents of `preferences.md` as is at every start and every resume.
- **The quality of the brief becomes the quality of the result.** Workers cannot ask questions during the work. So nobody notices a vague brief, and the result comes out wrong.

How to phrase the request: state the exit condition and how often you will check in

The example request in the bundled guide `07-overnight.md` states the exit condition, which is to keep going until every package is converted and merged. It also states that the operator checks in twice a day.

```
/poteto-mode orchestrate the store migration. own it until every package is converted and merged.
i'll check in twice a day.
```

In "Autopilot-full", an owner per PR takes it to merge, and the coordinator agent only issues verification verdicts

"Autopilot-full" is the Playbook that takes a queue of independent PRs all the way to merge without the operator's review. A queue is a line of work that waits to be processed.

Each PR gets one agent assigned to it. The Playbook calls this agent the **owner**. The owner is responsible for its PR from creation to merge.

The coordinator agent does not touch the PRs. It only issues verification verdicts. An owner cannot merge without a passing verdict from the coordinator agent. Owners verify their own work too. But the author's own check leaves gaps, so the pass condition is verification by a different agent.

A passing verdict has conditions. The PR must go through `/swarm` (Chapter 24), which runs several verifiers in parallel. Each of those verifiers is a lane. The lanes must include a live lane, which drives the real screen or CLI to check behavior. The live lane is the minimum requirement, and a verdict without a live lane does not pass. In addition, the owner fixes every finding the lanes report, including findings given only as notes. The Playbook calls a verdict that passes this way a **clean verdict**.

Seven steps

The steps are as follows.

1. Wait for the operator's go-ahead (state-then-wait)
2. Start one owner per PR
3. Run owners in parallel and do not stack PRs
4. Verify each round with `/swarm`
5. When a clean verdict comes, the owner merges the PR itself
6. The coordinator agent inspects the whole run about every 30 minutes
7. When the operator stops the run, stop every owner

The sections below look at each step in turn.

1. Wait for the operator's go-ahead (state-then-wait)

When the operator asks for the plan, explain it and stop. Start only after the operator explicitly says to start, and set the overall goal with `/goal`. For PRs the operator said they will handle themselves, the owner does not merge and leaves them to the operator.

2. Start one owner per PR

An owner is an agent that runs in Cursor's cloud through the Cloud Agents feature. The owner creates the PR, checks behavior on the real thing, triages Bugbot comments, and rebases onto trunk. It works until CI passes with "Babysit" (Chapter 14), and then it merges.

3. Run owners in parallel and do not stack PRs

If the PRs are independent, working on them at the same time causes no conflicts. So the owners do not stack the PRs, and each PR starts from trunk.

4. Verify each round with `/swarm`

A round is one pass in which the owner settles its code and gets it verified. Each push of a fix starts a new round. The verifiers run in parallel on the commit settled at that point. The coordinator agent combines their results into one verdict. A verdict without a live lane does not count as a clean verdict.

5. When a clean verdict comes, the owner merges the PR itself

Right before merging, the owner rebases onto the latest trunk again, then merges and takes the next item from the queue.

6. The coordinator agent inspects the whole run about every 30 minutes

The coordinator agent rereads the Playbook and the `/goal`. If the run has drifted from them, the coordinator agent corrects the drift at once. It also checks for stalled owners and replaces any it finds.

7. When the operator stops the run, stop every owner

The operator's stop reaches every owner at once as the instruction "write nothing from now on".

What the reply includes

The reply contains these items:

- Each PR's owner, state, and verdict
- Merged PRs
- Items waiting for the operator's decision

- The location of the decision record

The key point is to count progress only by side effects

In Autopilot-full, many owners and subagents run at once. Some of them stop on an error and report nothing. If nobody acts on a stalled agent, its PR never moves.

So the coordinator agent does not go by what agents report about themselves. **It counts as progress only the traces that the work left.** The traces are commits, pushes, changes in PR or check state, and reports to the store. The Playbook calls these changes side effects, because they are visible from outside as results of the work.

If an owner shows no side effects after the expected time, the coordinator agent treats the owner as stalled. During the inspection in step 6, the coordinator agent replaces the owner with another agent without waiting for a reply. The replacement keeps a stalled owner from slowing the whole run. "Orchestrate" applies the same idea when it checks commits, PR state, and the ledger instead of resuming a waiting agent only to check progress.

How to phrase the request: state that the items are independent and when they must be merged

The example request in the bundled guide states that the items in the queue are independent of each other and that they should be merged by morning.

```
/poteto-mode full autopilot on this queue. each item is independent. i want them merged by morning.
```

"Autopilot-stack" builds and verifies PRs the same way as Autopilot-full, but does not merge them and hands them to the operator as one stack

"Autopilot-stack" builds and verifies PRs the same way as "Autopilot-full", but **it does not merge**. It puts the PRs that passed verification into one stack and hands that stack to the operator. The operator reviews the stack and merges it themselves.

Use this Playbook when a person wants to review before merge, when the changes depend on each other and must be stacked in order, or when you cannot give the agents merge permission.

Steps: how they differ from Autopilot-full

The steps are almost the same as in "Autopilot-full". The differences are these:

- After verification passes, the owner reports STACK-READY, meaning ready to go on the stack, instead of merge-ready.
- No owner merges. The coordinator agent adds only PRs with a clean verdict, in order, to the single stack. **A PR that has not passed verification does not go on the stack.**
- Only the coordinator agent changes the stack order, meaning which PR sits on top of which. Owners push only to their own branches.
- When trunk moves forward, the coordinator agent rebases the stack again, starting from the frontmost PR. A rebase changes the head SHA, so any PR whose contents changed goes through verification again.
- Finally, the coordinator agent hands the operator the stack, with a verification verdict on each PR. The operator reviews and merges starting from the frontmost PR.

The reply contains these items:

- Links to the frontmost PR of the stack, which targets trunk, and to the topmost PR
- A summary of each PR's verdict
- PRs removed from the stack and the reasons

The key point is the division of work in which only the coordinator agent rewrites the stack order

The stack order is information every owner shares. If several owners rewrite the parent-child links between PRs at the same time, their changes conflict with each other and the stack breaks.

So **only the coordinator agent rewrites the stack order**, and owners only push to their own branches. With one agent doing the rewriting, the order cannot conflict.

This matches how "Orchestrate" gives each branch one Worker and hands restacks to one stacker. You can read both as a division of work that follows the principle "[Separate Before Serializing Shared State](#)" ([Chapter 18](#)).

How to phrase the request: write "stack them, don't ship"

The example request in the bundled guide states that the changes go on a stack, that they are not shipped, and that the operator lands the stack themselves.

```
/poteto-mode autopilot these five changes but stack them, don't ship. i'll land the stack in the morning.
```

Pick among the five options for long-running work by who merges and what the unit of work is

When you hand off long-running work, choose by **what the unit of work is and who merges**. The choices are four Playbooks ("Autonomous run", "Orchestrate", "Autopilot-full", and "Autopilot-stack") and [/figure-it-out](#) ([Chapter 23](#)), which designs an auditable procedure of its own when no Playbook fits.

Aspect	"Autonomous run"	/figure-it-out	"Orchestrate"	"Autopilot-full"	"Autopilot-stack"
Unit of work	One task	One large piece of work	The whole of work that lasts for days	A queue of independent PRs	A queue of changes with order or dependencies
Who merges	Not specified by the Playbook	Depends on the designed procedure	The coordinator agent or the stacker	Each PR's owner	The operator

Aspect	"Autonomous run"	/figure-it-out	"Orchestrate"	"Autopilot-full"	"Autopilot-stack"
When to choose it	The exit condition can be written as one verifiable condition, and one agent can finish	The work is large, spans many places, or no Playbook fits	One agent cannot finish within the time it can run	The PRs are independent and you have merge permission	You want review before merge, the work is interdependent, or you lack permission

Of the two Autopilot Playbooks, choose "Autopilot-full" if the PRs are independent and merge permission is granted. Otherwise, choose "Autopilot-stack".

Summary

- **How the four Playbooks divide the work.** "Multi-phase or multi-PR plan" writes the plan document, "Orchestrate" manages work that lasts for days, "Autopilot-full" has an owner per PR take it to merge, and "Autopilot-stack" hands over one stack without merging. Each one states what the coordinator agent does not own.
- **Multi-phase or multi-PR plan.** It writes a checklist plan document. A PR counts as verified only when unit, live, and perf are all checked.
- **Orchestrate.** It manages work that lasts for days with briefs, the tables in the store, and the ledger. It starts no Worker on a unit whose brief it cannot fill completely, and it records verdicts per latest commit.
- **Autopilot-full.** Each owner takes its PR to merge, and the coordinator agent owns only the verdicts. Progress is counted only by side effects.
- **Autopilot-stack.** It hands over one stack without merging. Only the coordinator agent rewrites the stack order.
- **The five options.** Choose by what the unit of work is and who merges.

Part III has now covered every Playbook pstack provides. Part IV looks at the Principles that guide the agent's decisions during the work, such as which option to adopt and when to finish.

Part IV: Principles support the agent's judgment

kaito • Sources: 日本語 • English • 2026-10-03

Part IV covers all 23 of pstack's Principles.

You can skip this part. I recommend reading only the chapters that interest you.

You can read the source of each Principle in the [pstack](#) repository at any time. pstack changes often. When you need a Principle, look up the matching chapter and the source file at that time. A lookup at the time you need a Principle is more reliable than a full read in advance.

What Part IV covers

For each Principle, Part IV explains three things:

- What each Principle chooses, what it avoids, and why the choice is necessary (for example, "**Prove It Works**" does not count a passing build as done and verifies the written files or the screen directly).
- The situations each Principle applies to and the ones it does not (for example, "**Outcome-Oriented Execution**" applies to a planned migration but not to everyday small fixes with no phase boundaries).
- How to use a Principle's name in a request to change the direction of the agent's work (for example, write `apply prove it works.` to ask the agent to run the real import and show the written records).

A Principle is **a standard for the decisions the agent makes in the middle of its work**. A Playbook sets the order of the work. Inside each step, the agent still has decisions to make, such as "should I add this abstraction?" or "now that it compiles, can I call it

done?" In those moments, the agent uses the Principles to decide whether to make a change and whether it can call the work done yet.

The 23 Principles fall into five groups by the kind of decision they cover. With these groups, a reader can choose a chapter by the kind of decision they are unsure about.

Group	Count	Decisions covered
Core	10	How much to build and when to revisit the design
Architecture	6	Where to put state, validation, and compatibility
Verification	4	What counts as proof
Delegation	2	How to run parallel work and delegation without breaking down
Meta	1	How to improve the system itself

The user does not call a Principle explicitly. **The agent judges whether the work fits a Principle's situation and then reads it.**

`/poteto-mode` is the Skill you use to start work ([Chapter 7](#)). It reads the Principle index at the start of any work with several steps.

The index is a list in the `SKILL.md` of `/poteto-mode`. It gives each Principle's name and the situation it applies to on one line. For example, the line for "Prove It Works" gives the situation "after finishing the work, before declaring it done". When the work fits a Principle's situation, the agent reads that Principle's own `SKILL.md` in full before applying it. In its reply, the agent names the Principles it applied and the decisions they changed.

At the start of the work, the agent reads only the list of Principle names and their situations. It reads a Principle's body only when the work fits that situation, and it uses the body to decide. So the agent does not load the bodies of all 23 Principles every time, and it does not waste context.

The user can also trigger this mechanism directly by **writing a Principle's name in the request**. Suppose the agent misses a situation that applies. One Principle name in the request then makes the agent correct its work to follow the rule that the name points to.

A single Principle name states the request more precisely than a long instruction. The name points to the complete rule written in that Principle's `SKILL.md`, so the user does not need to restate what the Principle says.

For example, suppose the agent is about to declare the work done because the build passed. The bundled guide, which ships with pstack in `docs/guide/` ([Chapter 6](#)), has an example for this case in `docs/guide/08-principles.md`. Write a request like that example:

```
apply prove it works. run the real import flow and show me the written records.
```

The chapters in this part

Chapter	What it covers
Chapter 17: Core: ten principles that decide how work proceeds	How much to build, how to revisit a design, how to build prototypes, and whether to build a tool
Chapter 18: Architecture: 6 principles for boundaries and dependencies	Data structures, validating outside data at the boundary where it enters, such as form input or an API response, types, idempotency, migrating off old APIs, and shared state
Chapter 19: Verification: four principles that check real behavior	Verifying completion with real output, fixing the cause of a bug, moving forward in small verified steps, and testing behavior
Chapter 20: Delegation: two principles for delegated and parallel work	Protecting the agent's context and moving forward without waiting for a human's confirmation
Chapter 21: Meta: the principle that improves the system itself	Keeping lessons in mechanisms, not in prose

This part puts Core first to match the order of pstack's bundled guide.

A quick reference to all Principles

Principle	Summary	When it fires	Chapter
Laziness Protocol	Prefer deleting code and choose the smallest change that solves the problem	When the agent refactors or estimates the size of a diff, or when it wants to add an abstraction or a layer or to pass a new value through many layers	Chapter 17
Subtract Before You Add	Before adding a feature, remove complexity such as unused code and duplicated validation	When the agent decides the order in which to add features or logic, refactor, or rewrite	Chapter 17
Minimize Reader Load	Reduce the layers of indirection a reader must follow and the code state the reader must keep in mind	When the agent reviews code whose flow is hard to follow, or writes or restructures such code	Chapter 17
Foundational Thinking	Before you write logic, decide the data structures and the foundation, such as CI, lint, test infrastructure, and shared types	When the agent picks the central types or data structures before writing logic, decides whether to build the foundation or the feature first, or considers what several concurrent actors share	Chapter 17
Redesign from First Principles	Do not bolt a new requirement onto the existing design. Rebuild the design as if the requirement had been there from the start	When the agent adds a new requirement to the current design	Chapter 17
Attack the Premise	Do not stack more fixes on the same premise. Before writing the next fix, question the premise itself	When two or more fixes that share one premise fail at the same gate, which is a test or check that the fix must pass	Chapter 17

Principle	Summary	When it fires	Chapter
Outcome-Oriented Execution	Prefer a verifiable end state over temporary code that keeps half-migrated code consistent	When the agent carries out a planned rewrite or migration whose phase boundaries are set in advance	Chapter 17
Experience First	Choose the user's experience over implementation convenience. Build fewer features and finish them well	When the agent chooses a trade-off about product, UX, or feature scope, where taking one side means giving up the other	Chapter 17
Exhaust the Design Space	Build prototypes of two or three options with different shapes, compare them side by side, and then decide the design	When the agent builds a UI interaction with no precedent in the codebase, chooses an architecture that several approaches could implement, or judges a product design whose feel depends on intuition more than logic	Chapter 17
Build the Lever	Instead of doing work by hand, build a tool that does the work or a tool that proves the work is correct, such as a codemod, a script, a generator, or a Skill for subagents to follow	When the agent does work that takes more effort than a few obvious fixes, whether editing, migrating, analyzing, or checking	Chapter 17
Model the Domain	Express the domain in a structure such as a state machine, typed objects, a lookup table, or a reducer, not in scattered conditionals	When the agent writes stateful logic or code with many branches, or repeats the same assumption ("this data has this field") across several files	Chapter 18

Principle	Summary	When it fires	Chapter
Boundary Discipline	Gather validation and error handling at the boundary where outside data enters. Inside the boundary, use typed values without checking them again, and keep business logic in pure functions	When the agent builds in input validation or error handling, or writes an adapter, the part that connects a framework to its own code	Chapter 18
Type System Discipline	Use type checking to make impossible states impossible to create at compile time	When the agent designs types, reviews function signatures, or writes code in a statically typed language	Chapter 18
Make Operations Idempotent	Build each state-changing operation so that it ends in the same state no matter how many times it runs or where it restarts	When the agent designs a command, a startup or shutdown procedure, or a loop that repeats work, any of which may stop partway or rerun after a restart or retry	Chapter 18
Migrate Callers Then Delete Legacy APIs	Move the callers to the new API and delete the old API in the same change	When the agent introduces a new internal API while old callers remain	Chapter 18
Separate Before Serializing Shared State	First separate the write targets that several actors share. Serialize writes with a lock or similar only when they cannot be separated	When several concurrent actors might write to the same file, branch, key, or stateful object	Chapter 18
Prove It Works	Verify the result of the work by looking directly at the running feature and the real values, not at a proxy such as a successful build	When the agent finishes the work and is about to declare it done	Chapter 19

Principle	Summary	When it fires	Chapter
Fix Root Causes	Do not add a guard that hides a bug's symptom. Find and fix the root cause that produced the symptom	When the agent debugs a bug	Chapter 19
Sequence Work into Verifiable Units	Split the work into small verifiable units, move to the next unit only after the current unit's checks, such as tests, pass, and order the commits so that they show correctness	When the agent decides the order of commits or PRs, or does work with several stages, such as a bulk fix, a migration, or a series of similar edits	Chapter 19
Test Behavior, Not Implementation	Call the code the same way its users do, and compare the observable result with an expected value written directly in the test	When the agent writes or changes a test, or decides whether to keep one	Chapter 19
Guard the Context Window	Have subagents read large outputs and documents, and keep only summaries in the main thread	When large outputs, long files, or rereading the same file fill up the agent's context, or when the agent plans to split work across several subagents running in parallel	Chapter 20
Never Block on the Human	Do reversible work without waiting for a human's confirmation and show the result. Ask a human only before running an irreversible operation	When the agent wants to ask a human "may I do X?" about reversible work	Chapter 20
Encode Lessons in Structure	Build a fix that keeps coming up into a mechanism such as a lint rule, a metadata flag, a runtime check, or a script, not into a written instruction	When the agent notices it is writing the same instruction a second time, or that the same correction keeps recurring	Chapter 21

Chapter 17: Core: ten principles that decide how work proceeds

kaito • Sources: 日本語 • English • 2026-10-03

This chapter covers the following ten principles.

1. Laziness Protocol
2. Subtract Before You Add
3. Minimize Reader Load
4. Foundational Thinking
5. Redesign from First Principles
6. Attack the Premise
7. Outcome-Oriented Execution
8. Experience First
9. Exhaust the Design Space
10. Build the Lever

These ten belong to the Core group of principles. The bundled guide's page [docs/guide/08-principles.md](#) describes the Core ten as principles that decide "**how much to build and when to revisit the design.**" The agent uses them for decisions such as keeping the diff minimal, cutting complexity before adding a feature, and questioning the premise after two fixes built on it have failed.

A plain list of the ten does not show which principle serves which kind of decision. So this chapter sorts the ten into four groups by what the decision is about. The four groups are as follows.

- **1 to 3.** Principles that reduce code.
- **4 to 7.** Principles that decide where the design sits, meaning its foundation and when to revisit it.

- **8 and 9.** Principles that decide what to aim for, meaning the target shape, based on the user's experience and on several options.
- **10.** The principle that turns manual work into tools.

This chapter explains the ten principles in this order, one at a time, from the following angles.

- **Rule.** What the principle requires.
- **Trigger condition.** The situations where the principle applies.

How this chapter is organized

This chapter is organized as follows.

- Three premises to know before reading the Principles
- A list of the ten principles
- "Laziness Protocol" gets the largest result from the smallest change
- "Subtract Before You Add" cuts complexity before adding a feature
- "Minimize Reader Load" reduces the layers of indirection a reader follows and the code state a reader has to keep in mind
- "Foundational Thinking" settles data structures and foundations before logic
- "Redesign from First Principles" redesigns as if the new requirement had been a premise from the start
- "Attack the Premise" questions the premise when fixes built on it fail the same test twice
- "Outcome-Oriented Execution" puts a verifiable end state ahead of temporary code that keeps mid-migration code consistent
- "Experience First" picks the user's experience over what is convenient to implement
- "Exhaust the Design Space" builds and compares two or three options for an unprecedented design
- "Build the Lever" builds a tool that does the work instead of doing it by hand
- Summary

Three premises to know before reading the Principles

I covered how Principles work in [Chapter 8](#) and [Chapter 9](#). For this chapter, these three points are enough.

1. **A Principle decides whether the current change goes in, or whether the work is not done yet**

A Principle itself is a standard for judgment, not a procedure or a capability. Some Principles, though, such as "Build the Lever" and "[Prove It Works](#)," require you to write a script when they apply.

2. **The trigger conditions are written in the Principle list of `/poteto-mode` and at the top of each Principle**

This book calls the situations where a principle applies its trigger conditions. The trigger conditions in this chapter are my translations of two sources. The first is the "when it applies" part of each item under "[Principles](#)" in `/poteto-mode`. The second is the situations named in the description at the top of each Principle's `SKILL.md`.

3. **The agent uses a principle even if the request does not name it**

When the work matches a trigger condition, the agent reads the principle and applies it on its own. However, **the agent reads the Principle index at the start of multi-step work, and the agent also decides whether a principle applies.** So the agent can miss a situation where a principle applies.

If the agent misses or skips a principle, a human can **name the principle in the request**. The agent then applies that principle and corrects the direction of the work.

Of the ten, this chapter gives example requests for only three: "Subtract Before You Add," "Exhaust the Design Space," and "Build the Lever."

This book limits example requests to the ones that appear in pstack's bundled guide or [README](#). If I wrote example requests for principles whose source has none, I might show a use that pstack does not intend.

A list of the ten principles

Group	Principle	The conclusion in one sentence
Reduce code	"Laziness Protocol"	Get the largest result with the least code and complexity
Reduce code	"Subtract Before You Add"	Cut complexity before adding a feature
Reduce code	"Minimize Reader Load"	Reduce the layers of indirection a reader follows and the code state a reader has to keep in mind
Where the design sits	"Foundational Thinking"	Settle data structures and foundations before logic
Where the design sits	"Redesign from First Principles"	Redesign as if the new requirement had been a premise from the start
Where the design sits	"Attack the Premise"	When fixes built on the same premise fail the same test twice, question the premise
Where the design sits	"Outcome-Oriented Execution"	Put a verifiable end state ahead of temporary code that keeps mid-migration code consistent
What to aim for	"Experience First"	Pick the user's experience over what is convenient to implement
What to aim for	"Exhaust the Design Space"	For an unprecedented design, build and compare two or three options
Turn into tools	"Build the Lever"	Instead of doing the work by hand, build a tool that does the work or a tool that proves the work is correct

"Laziness Protocol" gets the largest result from the smallest change

The principle "Laziness Protocol" requires you to **get the largest result with the least code and complexity**. It judges a solution by one standard: **whether that code would exhaust a human developer who maintains it**. The skill states the standard this way: **"If a human developer would find the code exhausting to maintain, it is a bad solution."**

Rule: delete code, flatten the call hierarchy, and decide each choice in one place

The main rules are as follows.

Look for what you can delete first

When a request asks you to improve something, look for what you can delete before you add code.

Flatten the call hierarchy

If you have to follow four or more files, or four or more layers of calls, to find something in the code, remove the layers in between and flatten the hierarchy. Code that answers a question only after many layers wears out the people who maintain it.

Put each decision in one place

Do not check the same condition in several places. Check it in one place.

For example, check once whether an input field is empty, and use the result in several places, such as "enabling or disabling the button" and "showing a hint." When the condition changes, you fix only that one check. If the same decision lives in many places, the work of keeping the copies in sync never ends.

Keep the diff minimal

Fewer lines beat elaborate boilerplate.

Stop before passing a new value through layers

When a request asks you to pass a new value through types, schemas, pipelines, and other layers, stop and look for a more direct option.

For example, suppose you add a feature that shows a button only to paying members. Instead of adding a new "is a paying member" value to the API types, the conversion step, and the values passed from parent to child, first look for a way to decide directly from user data already available near the button.

Trigger conditions

- When you refactor or estimate the size of a diff
- When you are tempted to add an abstraction or a layer
- When you are tempted to pass a new value across many layers

"Subtract Before You Add" cuts complexity before adding a feature

The principle "Subtract Before You Add" requires that **when you grow a system, you remove complexity first and then build**. Adding a feature to a complex system piles more complexity on top. If you cut complexity first, the code shrinks, the system's essential structure shows, and the next design step usually becomes clear.

Rule: remove complexity before you build

The rules are these:

- Remove complexity before you build, and cut down to the minimum before you polish quality
- Do not add handling for edge cases whose need nobody has confirmed yet. Design for the uses you have observed
- Keep validators and guards to what the spec requires. Do not add them on a guess
- Delete duplicated instructions in the text of prompts and Skills too

Trigger conditions

When you decide the order in which to add features or processing, or when you refactor or rewrite.

Example request: delete the old adapters, then design

The bundled guide gives the following example request for a situation where the agent is about to add a fourth adapter to three existing ones.

use subtract before you add. delete the obsolete adapters first, then design what's left.

An **adapter** is a component that absorbs the differences between external services so you can call them the same way.

For example, the payment services of companies A, B, and C each have a different calling convention. With one adapter per company, the app calls any of them with the same `pay(amount)`.

"Minimize Reader Load" reduces the layers of indirection a reader follows and the code state a reader has to keep in mind

The principle "Minimize Reader Load" defines maintainability as "**the amount of work a reader does to understand the code.**" People read code far more often than they write it.

The principle measures the reader's load on two axes, "layers followed" and "state held."

"Layers followed" is the number of functions or files a reader has to step through between asking a question and reaching the answer.

"State held" is the hidden state and the mutable values a reader has to keep in mind to understand how the code behaves. State held measures how much the reader has to track which value changes where.

The two axes make code hard to read in different ways.

For example, a single file with no deep chain of functions is still hard to follow if it has 50 global variables, which anything can change. The reader struggles to track when and where a value changes. On the other hand, code with few variables is also hard to read if the reader has to go through six layers of adapters, which are layers that handle conversion, to reach the answer.

So the principle asks you to reduce both axes.

"Minimize Reader Load" is the human version of "[Guard the Context Window](#)" in [Chapter 20](#), the principle that keeps large outputs from filling the agent's context window. The agent is not the only one with a working memory, a limit on how much it can hold at once. The humans who read code have the same limit.

Rule: reduce layers and state

Remove layers that only pass things through

If a function only calls another function and is used from only one place, remove it and call the inner function directly from the caller.

For example, if `saveUser()` only calls `db.save()`, call `db.save()` directly.

```
type User = { name: string };
declare const db: { save(user: User): Promise<void> };
declare const user: User;

// Before: saveUser only calls db.save, and this is the only place it is used
function saveUser(user: User) {
  return db.save(user);
}
await saveUser(user);

// After: remove saveUser and call db.save directly
await db.save(user);
```

Remove a layer between the caller and the real work in the same way when no other implementation ever replaces the code behind the layer. The rule includes layers that someone added with future extensibility in mind.

For example, suppose you use only one payment service but put a swappable interface such as `PaymentGateway` in front of it. A layer like that only makes the reader open the layer to check what is inside.

```
declare const aPayClient: { charge(amount: number): Promise<void> };

// Before: a swappable interface sits in front, but the only implementation is the one for company A
interface PaymentGateway {
  pay(amount: number): Promise<void>;
}

class APayPaymentGateway implements PaymentGateway {
  pay(amount: number) {
    return aPayClient.charge(amount);
  }
}

const gateway: PaymentGateway = new APayPaymentGateway();
await gateway.pay(1000);

// After: remove the interface and call company A's payment directly
await aPayClient.charge(1000);
```

Narrow the scope of state

Narrow the range from which a value can change. For example, pick a local variable, which only one function uses, over a global variable, which anything can change.

Also, do not keep a value that you can compute from other values as a separate variable. For example, if you keep a "number of products" variable apart from the product list, you have to update the count every time the list changes. If you add a product to a list of three and forget to update the count, the list has four items but the count still says 3. Count the list when you need the number.

The narrower the scope of state, the less code state the reader has to keep in mind.

Check an invariant once at the boundary and give it a name

An invariant is a condition that must hold the whole time the program runs, such as "the amount is 0 or more." The principle asks you to check an invariant once at the boundary, not at every place that uses the value.

A **boundary** here is a place where data comes in from outside, such as form input or an API response.

For example, the function that receives an amount from an order form checks "is the amount 0 or more" once, and passes the checked value on as a type named `NonNegativeAmount`, which means an amount of 0 or more.

The functions that use the amount, such as one that computes a total and one that creates an invoice, take it as a `NonNegativeAmount` and do not repeat the same check.

The type name tells you that "this amount was checked to be 0 or more at the boundary." With this approach, **the reader learns the condition once by reading one place at the boundary and the name of the type.**

```
// A type for an amount already checked to be 0 or more
type NonNegativeAmount = number & { readonly __brand: "NonNegativeAmount" };

// Boundary: check the amount from the order form once, here
function parseAmount(input: number): NonNegativeAmount {
  if (input < 0) {
    throw new Error("Enter an amount of 0 or more");
  }

  return input as NonNegativeAmount; // We just checked it, so this is the only place we
  assign the type
}

// User side: it takes a NonNegativeAmount, so it does not check "0 or more" again
function calcTotal(price: NonNegativeAmount, quantity: number): number {
  return price * quantity;
}

function createInvoice(amount: NonNegativeAmount): string {
  return `Amount due: ${amount} yen`;
}
```

`calcTotal` and `createInvoice` have no check such as `if (amount < 0)`. If you pass a number that did not go through `parseAmount`, the compiler reports an error, so the user side does not need to check again.

Judge with the 30-second test

The principle uses a "30-second test" to judge whether there are too many layers or too much state. If a new reader cannot answer "where does X come from" and "what can change X" within 30 seconds, cut layers or state.

Trigger conditions

When you review code whose flow is hard to follow, or when you write such code or restructure it.

"Foundational Thinking" settles data structures and foundations before logic

The principle "Foundational Thinking" requires you to **settle the shape of the data before you write logic**.

The principle divides decisions into the following two kinds.

1. Structural decisions
2. Decisions about how to write the code

Structural decisions protect future options. A structural decision here means "deciding the shape of the data and the order of the work," and a future option means "room to choose a different path later."

Decisions about how to write the code protect the code's simplicity. If you choose the right data structure, what to write in the code that follows becomes obvious.

In The Complete Guide to pstack [Part 2](#), poteto describes a division of labor: **in the age of agents, engineers spend their time choosing the architecture and the data**

structures, and the agent fills in the implementation details. This principle puts the choice of data structure at the start of the agent's work.

Rule: settle the data structure, separate concurrent state, and build the foundation before stacking mechanisms on it

Settle the data structure first

Before you write logic, decide the type of the central data. Then list every place that reads or writes that data and how, and pick the shape that fits the most common operation.

For example, for a product list feature, first decide the shape "a product has a name, a price, and a stock count." Only then think about how to write the display or the add-to-cart step. Once the shape of the data is settled, you can write the code that uses it without hesitation.

```
// Decide first: a product has a name, a price, and a stock count
type Product = {
  name: string;
  price: number;
  stock: number;
};

// The shape is settled, so the display and add-to-cart code are clear to write
function formatProduct(product: Product): string {
  return `${product.name} ${product.price} yen (stock: ${product.stock})`;
}

function canAddToCart(product: Product): boolean {
  return product.stock > 0;
}
```

In code, remove duplicated structure before duplicated lines

Instead of pulling every set of similar lines into a shared function, remove duplicated structure, such as the shape of data.

The principle says three similar statements in a row are better left as they are than merged in a hurry. It also prefers code that is obvious on sight to clever code. Clever code

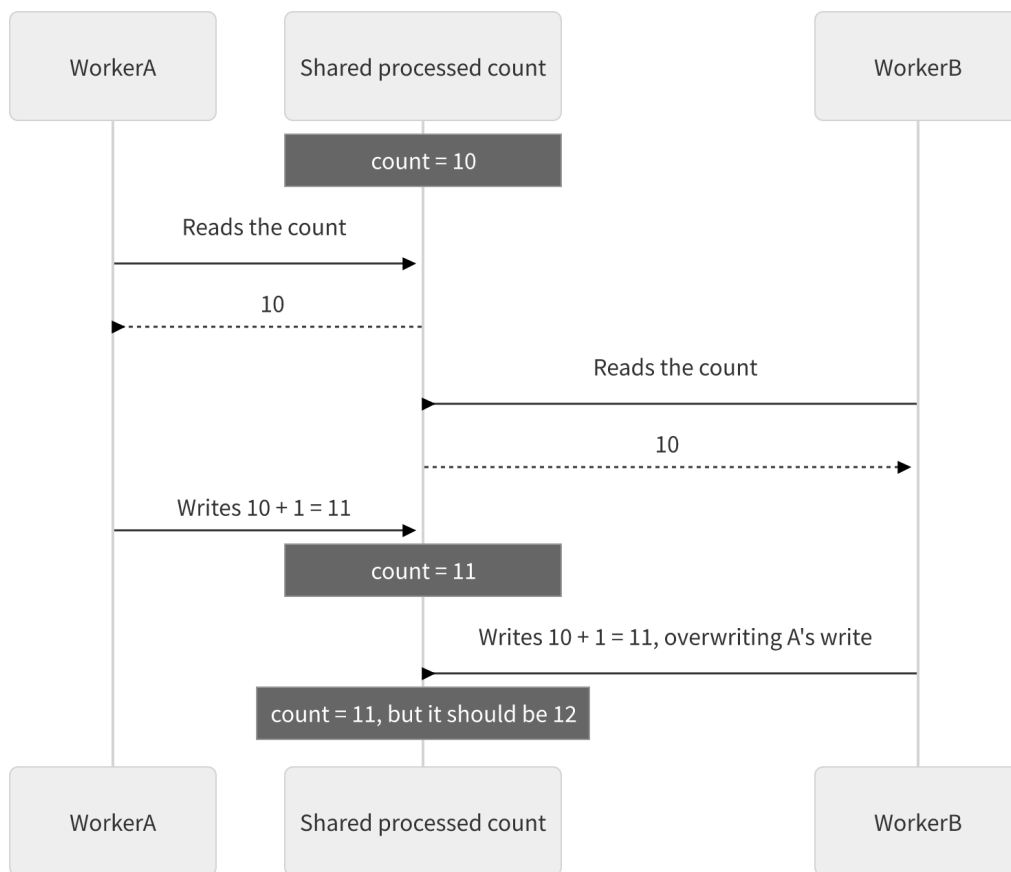
here means code that is short but that a reader cannot understand on the first read. For example, write `n % 2 == 1` to check whether a number is odd, not a bit operation such as `n & 1`.

Premature abstraction and clever code work against the goal of keeping code simple.

Separate values that can be written at the same time instead of sharing them

Here an actor is a Worker or an agent that runs at the same time as others. Before several actors share one value, ask "**what happens if another actor writes the same value at the same time?**" If something goes wrong, do not share the value. Give each actor its own value.

For example, two Workers, A and B, each add 1 to a shared "processed count." If A and B run at nearly the same time while the count is 10, the following happens.



View diagram 1

The two Workers each processed one item, so the count should be 12. But B writes 11 based on the 10 it read before A wrote, so the count ends at 11 and one item is lost.

So give each Worker its own count, and add them up only when you need the total. When each actor has its own value, no other actor can overwrite it in the first place.

```

type Job = { id: string };
declare const store: {
  get(key: string): Promise<number>;
  set(key: string, value: number): Promise<void>;
};
declare function processJob(job: Job): Promise<void>;
declare const jobsForA: Job[];
declare const jobsForB: Job[];

// Before: all Workers share one "processed count" and read it before writing
  
```

```

async function countUp() {
  // A and B read 10 at nearly the same time
  const current = await store.get("processedCount");
  // Both write 11, and one item is lost
  await store.set("processedCount", current + 1);
}

// After: give each Worker its own count, and add them up only when you need the total
async function runWorker(jobs: Job[]): Promise<number> {
  let processedCount = 0; // A value for this Worker only

  for (const job of jobs) {
    await processJob(job);
    processedCount += 1;
  }

  return processedCount;
}

const counts = await Promise.all([runWorker(jobsForA), runWorker(jobsForB)]);
const total = counts[0] + counts[1];

```

Build the foundation first

Ask **"does every later stage benefit from having this?"** If the answer is "yes," build it first. CI, lint, test infrastructure, and shared types are examples of a foundation. The principle calls this kind of foundation **scaffold**.

Do setup before features, and write tests before fixes. Also prefer small commits with a single purpose. When the foundation comes first, all later work can use it.

However, **deleting unused code comes before building the foundation**.

If you build the foundation while unused code is still there, the shared types and tests end up supporting that unneeded code too. If you delete it first, only the parts that are really in use remain visible, and you can build a foundation that fits them. Deleting first is the same idea as ["Subtract Before You Add"](#) in this chapter.

As [Chapter 4](#) showed, the codebase is also the agent's memory. This book reads the priority on deleting unused code as a way to keep wrong context out of the codebase as well.

Put one coherent mechanism in place with each change

With each change, build one coherent mechanism, such as a shared function or component that several places can use. Or make an existing mechanism, such as a shared function or component, easier to use.

When you add a new feature, do not scatter special-case handling across every place that uses it. For example, to add sorting to a product list, do not add sorting logic to every screen that shows the list. Build one component that handles sorting, and have each screen call it.

When a new feature lives in one place, anyone who reads or fixes it only has to look at that one place.

Trigger conditions

- Before you write logic, when you choose the central types or data structures
- When you decide whether to build the foundation or the feature first. The foundation is the set of mechanisms that all later work uses, such as CI, lint, test infrastructure, and shared types
- When you consider what several concurrent **actors** share

An **actor** here is anything that runs at the same time as others and moves work forward.

For example, several Workers that take jobs from a queue and process them in parallel are actors. A Worker here is a program that handles processing. Several agents that work in the same repository at the same time are actors too.

"Redesign from First Principles" redesigns as if the new requirement had been a premise from the start

The principle "Redesign from First Principles" requires you not to bolt a change onto the current design, but to **redesign as if the requirement had existed from the start**. This book reads "Foundational Thinking" as "the principle for building something new" and

"Redesign from First Principles" as "the principle for working a change into an existing design."

Rule: redesign in four steps

1. Read every affected file and understand the current design
2. Ask, "If I had known this new requirement from the start and were writing from scratch, what would I build?"
3. Update every place the change touches to match the new design, down to types, docs, examples, and the sections that record design rationale
4. Think through the whole redesigned picture first, then land the change in small pieces

For example, take the case "add dollar payments to a pricing calculation that only handled Japanese yen." You can think about it like this.

- A bolt-on adds a branch "if it is dollars, calculate it this way" at every place that uses an amount
- A redesign asks, "How would I build this if handling several currencies had been a premise from the start?" and changes the amount into "a pair of a number and a currency"

Finally, to reflect the redesign, update every place that uses an amount, along with the types and the docs, to match that shape.

```
// Bolt-on: keep the amount as a number in yen, and add an "if dollars" branch at every place that
// uses it
function formatPrice(amount: number, isDollar: boolean): string {
  if (isDollar) {
    return `${amount.toFixed(2)}`;
  }

  return `${amount} yen`;
}
```

```
// Redesign: hold the amount as "a pair of a number and a currency," and have every place that uses
// it take this type
type Currency = "JPY" | "USD";
type Money = { amount: number; currency: Currency };
```

```
function formatPrice(price: Money): string {
  if (price.currency === "USD") {
    return `$$${price.amount.toFixed(2)}`;
  }

  return `${price.amount} yen`;
}
```

The principle describes these four steps as the way to **work a change into an existing design while keeping future options open**. A bolt-on leaves the added part and the original design out of step. Every later change then has to work around that mismatch, so fewer designs remain open to choose later.

Trigger conditions

When you work a new requirement into an existing design.

"Attack the Premise" questions the premise when fixes built on it fail the same test twice

The principle "Attack the Premise" says that when two or more fixes built on the same premise fail at the same gate, you **question the premise, not the fix**.

- A **gate** is a test or check that a fix has to pass
- A **premise** is what the person or agent who wrote the fix took for granted as "this is the cause"

For example, suppose the agent thinks "it is slow because there is too much data." Both its first and second fixes reduce the amount of data handled at once, and both fail the same test.

If the amount of data were the cause, reducing it should have made the test pass. The test failed both times, so the slowness may have some other cause. Before writing a third fix, the agent questions the idea "it is slow because there is too much data" itself. **Each**

failure of a fix built on the same premise is evidence about the premise itself, so the agent can question the premise on its own.

"Redesign from First Principles" rebuilds the design around a new requirement. This principle instead **questions a fact that the current design takes for granted as a premise**.

Rule: write down the premise and count the skew before you fix

Write down the premise

Write the premise that every failed fix took for granted in one sentence. The written sentence makes clear what you are questioning.

Take a census before the next fix

A **census** is a breakdown of the skew, counted per actor.

For example, if four Workers split the jobs, count the unprocessed jobs per Worker and build a breakdown such as "Worker 1 has 30, Workers 2 to 4 have 2 each." A census tells you not how large the skew is, but which actor the skew collects on. In this example, that actor is Worker 1.

The agent writes the census as a script that it can rerun any number of times. The rerunnable script follows "[Build the Lever](#)", covered at the end of this chapter.

Read how the skew falls

If the skew collects on the same few actors on every run, something in the configuration, the code, or elsewhere assigns those actors a role that creates the skew. The agent looks for that assignment.

That assignment is the next "why" to ask.

Asking the next "why" is the idea behind "[Fix Root Causes](#)," the principle of asking "why" repeatedly to fix the root cause rather than the symptom ([Chapter 19](#)).

Remove the assignment that creates the skew instead of compensating for the skew afterward

Use one of the following methods so that the same actor does not hold the role every time.

- Rotate the role among the actors
- Randomize the assignment
- Move the role somewhere else

For example, if the configuration always gives the first batch of jobs to Worker 1, change the recipient every time.

A mechanism that compensates afterward leaves the assignment that creates the skew in place and adds extra work on every run. That extra work goes against "Laziness Protocol". Examples of compensating mechanisms are a mechanism where idle Workers take jobs from other Workers, a shared pool where everyone puts jobs, a mechanism that hands jobs over in batches, and periodic redistribution.

Do not start the next fix until the premise and the census exist

The agent does not start the next fix until it has written down the premise and finished the census. If the census shows the work spread evenly across the actors, the premise is not the cause. In that case, the agent looks for the cause somewhere else and keeps the census as evidence.

For example, suppose several Workers process a queue, and the test "the queue empties within the time limit" fails twice.

First, the agent adds the following in each attempt.

- Attempt 1: a mechanism where idle Workers take jobs from other Workers
- Attempt 2: a mechanism that redistributes jobs periodically

Both rest on the same premise: "jobs pile up on some Workers because the size of each job, meaning the time it takes to process, varies."

Next, before writing a third fix, the agent writes a script that counts the unprocessed jobs per Worker from the run logs.

If varying job sizes were the cause, the Worker where jobs pile up should change from run to run.

But if Worker 1 alone has many unprocessed jobs in all three runs, the premise is wrong. The next "why" is "what assigns this role to Worker 1?"

That something could be, for example, a setting that always gives the first batch of jobs to Worker 1.

Trigger conditions

When two or more fixes that share one premise fail at the same gate.

"Outcome-Oriented Execution" puts a verifiable end state ahead of temporary code that keeps mid-migration code consistent

The principle "Outcome-Oriented Execution" says to put **reaching the final target shape (the end state) and proving it correct** ahead of keeping mid-migration code running smoothly. If you try to keep every intermediate stage working, **you produce temporary compatibility code that runs on both the old shape and the new one. That code tends to stay long after the migration and become debt.**

For example, suppose you move a configuration file from an old format to a new one. To read either format at every stage along the way, you write bridge code that converts the old format into the new one.

Once the migration ends, nothing uses the old format and the bridge code should no longer be needed. Yet the bridge code tends to stay, because someone forgets to delete it or worries that some old-format files might remain. An agent that follows this principle moves toward the target shape instead of keeping things consistent along the way with code like that. It then proves correctness at verification boundaries, which are points that you decide in advance for verification.

Rule: decide in advance where things may break along the way, and verify everything at the end

- Allow something to stop working temporarily only when the breakage is planned, limited in scope, and reversible.
- Declare in advance where things may break temporarily. For example, write in the plan, "During the migration, search in the admin screen may not work."
- Even mid-migration, keep checks that catch a problem right away on the parts you are currently changing.
- At the end of the plan, pass both kinds of verification in full: verification without running the code, such as type checks, and verification by running it.

The principle does not allow breakage unconditionally. It requires you to **declare in the plan, in advance, where things may break**.

Trigger conditions

During a planned rewrite or migration whose phase boundaries are set in advance. The agent does not use this principle for ordinary small fixes with no such boundaries.

Phase boundaries are the points between phases when you split a large job into several phases.

For example, to migrate the format of configuration files, split the work into the following phases, and at the end of each phase verify that things work correctly.

1. Make the code able to read the new format
2. Rewrite every configuration file in the new format
3. Delete the code that reads the old format

"Experience First" picks the user's experience over what is convenient to implement

The principle "Experience First" says that **when the approach that is easier for the builder conflicts with the approach that gives the user a better experience, pick the one that gives the better experience.** Three well-finished features beat ten rough ones. Also, a design decision is cheaper to test in throwaway HTML than in production code.

Rule: build fewer features, and build them well

Give every feature, interaction, and option a reason

Be able to explain why each button and each setting option on the screen is needed.

Build fewer features, and build them well

Build three well-finished features instead of ten rough ones.

Build a prototype before you decide

Before writing production code, prototype in throwaway HTML or similar, use the prototype to verify how the design behaves, and only then decide on the design. Finding a mistake at the prototype stage costs less than rebuilding production code.

Get the details right

Get the screen transitions, the alignment of elements, the spacing, the response to interactions, and the error display right.

For example, when a save fails, show what happened instead of showing nothing.

Center on the user's main workflow

Every feature either helps the user's main workflow or stays out of its way.

Explain impact from the user's point of view

The user, in this principle, is anyone who uses what you build. For a screen, that is the person using the app. For a library or an internal API, it is the colleague who imports it. The engineer who maintains the code next is a user too.

For example, instead of "changed the format of the API response," explain what changes for the user, such as "the list screen shows up faster than before."

The principle separates the roles of two principles. "Foundational Thinking" decides the order of the work, and "Experience First" decides the target shape. On that basis, "Experience First" treats foundations such as CI, test infrastructure, and shared types as things that exist to improve the user's experience too.

Trigger conditions

When a trade-off comes up in product, UX, or feature scope, where taking one thing means giving up another.

"Exhaust the Design Space" builds and compares two or three options for an unprecedented design

The principle "Exhaust the Design Space" requires that for an interaction or design decision with no precedent in the codebase, you **try several concrete alternatives before implementing**. Building the wrong thing costs more than trying three options.

Rule: put two or three options of different shapes side by side and compare them

When the right design is not clear, the agent builds two or three competing prototypes of different shapes, compares them, and then decides which one to adopt. Each option is a prototype you can use or a draft. Another name for this rule is "Design it twice."

What matters is that **the options differ fundamentally in shape**. A slight variation on the first option does not count as a second option. Only by comparing options of different shapes do you learn which shape fits the user.

For example, for a sign-up screen, the following two have different shapes, so they are separate options.

Option A: all fields on one page

Sign up

Name

Email address

Password

Sign up

You can see and fill in every field at once.

Option B: one field per screen, asked in order

Sign up 1 / 3

Enter your name

Next

Sign up 2 / 3

Enter your email address

Next

Sign up 3 / 3

Enter your password

Sign up

Each screen asks for one field, so you always know what to do next.

On the other hand, a version of option A with a different button color or position has the same screen shape as option A, so it is not a separate option.

Trigger conditions

- When you build a new UI interaction that has no similar precedent in the codebase yet
- When you choose an architecture that you can build in more than one way
- When you judge a product design whose feel depends on intuition more than on logic

On the other hand, the principle does not apply to the following changes.

- Mechanical implementation with an established method
- Bug fixes or refactoring with a clear target state
- Changes where the constraints leave only one way to build the change

Example request: build two prototypes and compare them

The following example request comes from pstack's README. With this request, the agent enters the "Prototype" Playbook, so it builds prototypes that it expects to throw away.

```
/poteto-mode build two prototypes of the markdown renderer so we can compare. spawn an agent for each.
```

"Build the Lever" builds a tool that does the work instead of doing it by hand

The principle "Build the Lever" says that if a job is not trivial, you do not do it by hand. Instead, you **build a tool that does the job, or a tool that proves the job is correct.**

Building a tool has two payoffs.

The first is throughput. A codemod is a script that rewrites code mechanically. A codemod or another script does the work the same way on every run. So even with 100 target files, the agent does not spend time on them one at a time as it would by hand.

When the agent changes the method partway through, or when more target files appear, it reruns the script, and the script reprocesses every file the same way.

The second is confidence. A reviewer can read the tool's code and rerun it to verify for themselves that the work is correct.

A manual change, by contrast, leaves no record of the steps used to rewrite it. So the only way for a reviewer to re-check its correctness is to redo the same work. For a change that rewrote 100 files by hand, the reviewer also has to look through all 100 one at a time. That review takes time and makes misses likely.

The principle sums up the second payoff this way: 'A deterministic script turns "trust me" into "run this".'

- A **deterministic script** is a script that gives the same result for the same input every time

- **Turning "trust me" into "run this"** describes a change in what the reviewer can do. With manual work, the reviewer can only trust the result. With a script, the reviewer can rerun the script and verify the result themselves

Rule: do the first one by hand to learn the steps, then build the tool

Do the first one by hand, then build the tool

For example, to rename an old function to a new name across 100 files, first rewrite one file by hand to verify the steps, then build a rewriting script.

Next, run the script on that first file and compare the result with your hand-edited version to prove the script is correct. Build the tool so that it is safe to rerun any number of times.

Pick the tool that fits the job

Use a codemod or a script for edits, a generator for files with a lot of repetition, queries over data exported to SQLite for analysis, and scripts you can run any number of times for verification.

Do with a script what a script can do, instead of splitting it across several subagents

If a script can process every unit in one pass, run the script yourself. A script that gives the same result every time is more reliable than subagents that do each unit by hand.

When you hand work out to subagents, write the instructions as a Skill

When you hand work out to subagents, write the instructions everyone reads as a Skill, and put the steps, the way to verify, and the areas that must not be touched in one place.

Put this Skill where the subagents cannot edit it, so they cannot quietly change the agreed steps or verification method.

Build the smallest script, not a framework

Build only the smallest script that does the work or proves it correct, as "Laziness Protocol" requires.

If the work continues across sessions, commit the tool

Keep the tool so the next session can use it too.

The flow from the first manual pass to processing everything with the tool looks like this.

Do the first one by hand and learn the steps

↓

Build the tool (a codemod, a script, or similar)

↓

Rerun the first one with the tool and compare its output with the hand-done version

↓ If they match

Run the tool yourself and process all remaining units

(Do not have subagents do them one at a time by hand)

When the agent applies this principle, tool files appear in the diff. If the agent cites the principle but the diff has no codemod, script, generator, or Skill for delegation, the agent has not applied it.

Two other principles resemble this one in that they also build tools or scripts. To keep them apart, the table shows what each one targets.

Principle	Target
"Build the Lever"	Process the work at hand in volume, in a form a reviewer can verify
"Encode Lessons in Structure" (Chapter 21)	Build instructions that keep coming up into mechanisms such as lint rules or runtime checks, so they become lasting guardrails
"Prove It Works" (Chapter 19)	Before calling the work done, run it and verify

Trigger conditions

Every job that is not trivial, whether it is an edit, a migration, an analysis, or a check.

The agent may skip building a tool only for trivial jobs, such as a fix in a few places that is obviously right on sight.

Also, the agent decides whether to build a tool by whether the job is trivial, not by how many times the same job repeats. A trivial job here is one with few places to change, whose correctness you can see by looking.

A job with many places to change, or with many steps to verify, is not trivial even if it happens only once. So the agent builds a tool even for a one-time job when a tool would let it verify correctness.

Example request: hand off a migration while you are away

pstack's README has the following example request. The README lists it as an example of /figure-it-out. This book reads the request as a situation where a tool helps, because the request hands off a migration that must not change behavior.

```
/poteto-mode i'm stepping away. migrate every caller from the synchronous store to the new async one, keeping behavior identical. i want to trust it was done right when i'm back.
```

Summary

- A Principle is a standard for judging whether the current change goes in or the work is not done yet. A human can correct the direction of the agent's work by naming the principle in the request.
- Three principles reduce code: "Laziness Protocol," "Subtract Before You Add," and "Minimize Reader Load."
- Four principles decide where the design sits. "Foundational Thinking" settles the shape of the data and the foundation first. "Redesign from First Principles" redesigns as if the requirement had existed from the start. "Attack the Premise" questions the premise behind fixes that keep failing the same test. "Outcome-Oriented Execution" declares where things may break and moves toward the end state.
- Two principles decide what to aim for. "Experience First" picks the user's experience, and "Exhaust the Design Space" compares options of different shapes for an unprecedented design.

- "Build the Lever" turns work into a tool you can rerun. If the agent cites the principle but the diff has no tool, the agent has not applied the principle.

The next chapter, [Chapter 18](#), covers the six Architecture principles. Each one decides "where in the code to put data and checks."

Chapter 18: Architecture: 6 principles for boundaries and dependencies

kaito • Sources: 日本語 • English • 2026-10-03

This chapter covers the following six principles.

1. Model the Domain
2. Boundary Discipline
3. Type System Discipline
4. Make Operations Idempotent
5. Migrate Callers Then Delete Legacy APIs
6. Separate Before Serializing Shared State

The six belong to the Architecture group of principles. Each one decides **what shape the agent gives the code, such as how it holds state and where it puts verification**. The bundled guide page [docs/guide/08-principles.md](#) sums up the six as principles that decide "where state, validation, and compatibility live."

With the six principles, the agent works as follows.

- It holds state in a state machine, not in a combination of several booleans.
- It gathers the validation of outside data at the boundary.
- Once it has moved every caller to the new API, it deletes the old API in that same change.

- **State machine.** A way of writing code that fixes in advance the possible states and the moves from one state to another. At any moment, the value is in exactly one of the fixed states.
A traffic light, for example, shows only one of green, yellow, or red, and moves in

the fixed order green, yellow, red. Green and red are never lit at the same time, and the light never jumps from green to red. A state machine expresses rules like these in code.

- **Boundary.** A place where data comes in from outside, such as form input or an API response.

As in [Chapter 17](#), this chapter explains the six principles one at a time in terms of their rules and their triggers. The chapter ends by reading all six as ways to make the structure itself the instructions for the agent.

The example requests in this chapter are also one-line instructions that include the principle name. [Chapter 17](#) set out the premise that the agent reads and applies a principle on its own when the work matches the trigger, even if you leave out the principle name.

Of the six principles, this chapter gives an example request for only one: "Separate Before Serializing Shared State."

This book limits example requests to the ones that appear in pstack's bundled guide or [README](#). If this book made up a request for a principle that has none in the source, it could show a use that pstack does not intend.

How this chapter is organized

This chapter is organized as follows.

- The six principles at a glance
- "Model the Domain" expresses the domain as structure, not as conditionals
- "Boundary Discipline" gathers input validation at the outside boundary, and the inside trusts the types
- "Type System Discipline" uses types to make invalid states impossible to build
- "Make Operations Idempotent" builds operations that give the same result however many times they run

- "Migrate Callers Then Delete Legacy APIs" moves every caller and deletes the old API in the same change
- "Separate Before Serializing Shared State" removes the shared write target first, and a lock is the last resort
- The six Architecture principles make the structure itself the instructions for the agent
- Summary

The six principles at a glance

Principle	One-sentence conclusion
"Model the Domain"	Express the domain as structure, not as scattered conditionals
"Boundary Discipline"	Gather validation and error handling at the boundary where outside data comes in. Code inside the boundary trusts the types
"Type System Discipline"	Use type checking to make impossible states impossible to build at compile time
"Make Operations Idempotent"	Build each operation that changes state so that it ends in the same state, however many times it runs and even when it restarts partway
"Migrate Callers Then Delete Legacy APIs"	Move the callers to the new API and delete the old API in the same change
"Separate Before Serializing Shared State"	Remove the shared write target first. Use a lock to make writes wait their turn only when the lock is truly needed

"Model the Domain" expresses the domain as structure, not as conditionals

The principle "Model the Domain" asks you to express the real domain as data structure, not to scatter it across conditionals in many places.

When states or branches look likely to grow, **the agent picks a structure that represents the domain before it adds a conditional (if/else).**

- **Domain.** The business or subject the software deals with. For an online shop, that means orders, stock, and shipping.
- **Data structure.** The fields that data has and the type of value each field takes, such as `Order` and `OrderFlags` in the example below.
- **Structure that represents the domain.** A data structure with the same shape as the real business rules.

A real order, for example, is in exactly one of these states: "awaiting payment," "awaiting shipment," "shipped," or "canceled."

Here is that order data structure written two ways.

```
// A structure that represents the domain: the state is exactly one of the four values of
status
type Order = {
  address: string;
  status: "unpaid" | "paid" | "shipped" | "canceled";
};

// A structure that does not represent the domain: three booleans
// It also accepts combinations that never happen, such as isShipped: true with isCanceled:
true
type OrderFlags = {
  address: string;
  isPaid: boolean;
  isShipped: boolean;
  isCanceled: boolean;
};
```

Both are data structures, because both fix the fields an order has and the values each field takes.

Only `Order`, which has the same shape as the rules of a real order, is a structure that represents the domain.

Scattered booleans, repeated assumptions, and spreading branches are unneeded complexity

The principle calls the following three things complexity that should not be there.

- Scattered booleans
- The assumption "this data should have this shape," repeated in file after file
- Branches that spread across files

Code with the three kinds of complexity can express impossible states

Take an online shop that handles orders in two files: one that builds shipping labels and one that builds notification emails.

Code with all three kinds of complexity looks like this.

```
// order.ts: holds the order state in three booleans (scattered booleans)
type Order = {
  address: string;
  isPaid: boolean;
  isShipped: boolean;
  isCanceled: boolean;
  trackingNo?: string;
};

// label.ts: builds the shipping label
function labelText(order: any): string {
  // Checks the assumption "an order should have an address" here
  if (!order.address) throw new Error("No address");
  // Branches on the state
  if (order.isCanceled) return "Canceled";
  if (order.isShipped) return "Shipped";
  if (order.isPaid) return "Awaiting shipment";
  return "Awaiting payment";
}

// mail.ts: builds the subject line of the notification email
function mailSubject(order: any): string {
  // Checks the same assumption again, in another file
  if (!order.address) throw new Error("No address");
  // The same branches repeat across files
  if (order.isCanceled) return "Your order has been canceled";
  if (order.isShipped) return "Your item has shipped";
  if (order.isPaid) return "We have received your payment";
}
```

```
    return "We are waiting for your payment";
  }
}
```

This code lets you write a value where `isShipped` and `isCanceled` are both true.

That is an **impossible state**. A real order cannot be "shipped" and "canceled" at the same time.

The code also lets you write a value where `isShipped` is true but the tracking number, `trackingNo`, is missing.

A structure that represents the domain makes impossible states unwritable and cuts branches

When the agent picks a structure that represents the domain, **nobody can write an impossible state, and the code has fewer conditionals**. Structures that represent the domain include state machines and typed objects with fixed fields.

Here is the same order written with a structure that represents the domain.

```
// order.ts: fixes the shape of an order and its possible states in one place
type OrderStatus =
  | { state: "unpaid" }
  | { state: "paid" }
  | { state: "shipped"; trackingNo: string }
  | { state: "canceled" };

type Order = { address: string; status: OrderStatus };

// Puts the text for each state in one table
const LABEL_TEXT: Record<OrderStatus["state"], string> = {
  unpaid: "Awaiting payment",
  paid: "Awaiting shipment",
  shipped: "Shipped",
  canceled: "Canceled",
};

// label.ts: puts the text for each state in a table, trusts the type, and only looks up the table
function labelText(order: Order): string {
  return LABEL_TEXT[order.status.state];
}

// mail.ts: puts the subject for each state in a table, trusts the type, and only looks up the table
const MAIL_SUBJECT: Record<OrderStatus["state"], string> = {
  unpaid: "We are waiting for your payment",
```

```

    paid: "We have received your payment",
    shipped: "Your item has shipped",
    canceled: "Your order has been canceled",
  };

  function mailSubject(order: Order): string {
    return MAIL_SUBJECT[order.status.state];
  }

```

In this code, the three kinds of complexity go away as follows.

- **Scattered booleans.** Instead of three booleans, `OrderStatus` takes exactly one of four states. You cannot write "shipped and canceled." The tracking number exists only in the shipped state, and a shipped order always has one.
- **Repeated assumptions.** The `Order` type states the assumption "an order has an address" once. Each file trusts the type and does not check the address again.
- **Branches across files.** A table lookup decides the text for each state. To add a state, you add one row to the table. If you forget the row, compilation fails.

Pick the structure when you write the code, not later

The agent **picks the structure when it writes the code**. At writing time, picking a structure costs almost nothing.

If the code already uses conditionals, on the other hand, the work to add structure later tends to get delayed.

The work gets delayed because the new structure changes nothing about how the code behaves, so the work counts as refactoring.

Refactoring has no visible result the way a new feature does, so developers and agents alike tend to add features first and leave refactoring for later.

Say the conditionals that decide the shipping fee have spread across three files, and you later replace them with a lookup table. The fee amounts stay the same, and so does the screen. So the replacement gets delayed as "work that can wait," and meanwhile every new feature adds more conditionals.

Rule: each scattered pattern has a structure that replaces it

The principle lists these typical replacements.

Scattered pattern	Structure that replaces it
Booleans that stand for a state, such as <code>isLoading</code> and <code>isError</code> , and checks of "which stage are we in now" all over the code	A state machine, which fixes the possible states and the moves between them
Loose arguments passed to functions, with the same assumption "this data should have this field" repeated in many places	A typed object with fixed fields, such as a <code>User</code> type
The same conditional that splits work by kind, spread across several files	A map or lookup table, which uses the kind as a key to find the matching value or handler, or a discriminated union, a union of types that a kind field tells apart
State rewritten directly, on the spot, wherever it is needed	A reducer, a function that computes the next state from the current state and an action

Use a state machine instead of booleans

When booleans hold the state, you can write a contradictory state where `running` and `failed` are both true, and compilation does not catch it.

When a state machine represents the state, **nobody can write that combination at all.**

```
// Before: contradictory combinations can be expressed
type ExportJobFlags = { running: boolean; failed: boolean; retrying: boolean };

// After: each state fixes which data it can hold
type ExportJob =
  | { state: "queued" }
  | { state: "running"; attempt: number }
  | { state: "failed"; attempt: number; error: string }
  | { state: "done"; rowCount: number };

// Also fix the moves: only a failed job can go back to running as the next attempt
function retry(job: ExportJob): ExportJob {
  if (job.state !== "failed") return job;
  return { state: "running", attempt: job.attempt + 1 };
}
```

Use a typed object instead of loose arguments

Say you pass a member's name, email address, and plan as separate arguments to two functions: one that sends a welcome email and one that creates an invoice. Both functions repeat the same assumption, "a member should have a name, an email address, and a plan," each in its own argument list.

The agent creates one `User` type for a member and changes both functions to take a `User`. After that, to add a field to the member, you change only one place, the `User` type. The `User` type also prevents the mistake of passing the name and the email address in the wrong order.

```
// Before: each function lists the same three arguments
function sendWelcomeMail(name: string, email: string, plan: string) { /* ... */ }
function createInvoice(name: string, email: string, plan: string) { /* ... */ }
```

```
// After: write the member's shape once in the User type, and both functions take it
type User = { name: string; email: string; plan: "free" | "paid" };
```

```
function sendWelcomeMail(user: User) { /* ... */ }
function createInvoice(user: User) { /* ... */ }
```

Use a lookup table or a discriminated union instead of spreading conditionals

Say three files, for the screen, billing, and email, each decide the shipping fee for each kind of order with their own conditionals. The agent creates one lookup table that maps the kind to the fee and changes all three files to look up that table.

After that, for a new kind of order, you do not fix the conditionals in three files. You add one row to the table.

```
type OrderKind = "standard" | "express" | "pickup";
declare const order: { kind: OrderKind };
```

```
// Before: the same if/else sits in three files, for the screen, billing, and email
function shippingFee(kind: OrderKind): number {
  if (kind === "standard") return 500;
  if (kind === "express") return 1200;
  return 0;
}
```



```

}

// After: create a single lookup table from kind to fee, and the three files look it up
const SHIPPING_FEES: Record<OrderKind, number> = {
  standard: 500,
  express: 1200,
  pickup: 0,
};

const fee = SHIPPING_FEES[order.kind];

```

With `Record<OrderKind, number>`, if you add a kind of order to the type `OrderKind` and forget to add its row to the table `SHIPPING_FEES`, compilation fails.

A lookup table fits when each kind maps to one "value."

When each kind **holds different data altogether**, on the other hand, the agent uses a **discriminated union**.

Take three payment methods: "card," "bank transfer," and "cash on delivery." Only a card holds the last four digits of the card number, and only a bank transfer holds the account number to pay into.

If you write these payment methods with a kind field and optional fields, you can also write a value such as "a card with an account number."

With a discriminated union, each value of kind fixes which fields it can hold, so you cannot write such a value.

```

// Before: every payment method can hold every field
type PaymentLoose = { method: "card" | "bank" | "cod"; cardLast4?: string; bankAccount?: string };

// After: each value of kind fixes which fields it can hold
type Payment =
  | { kind: "card"; cardLast4: string }
  | { kind: "bank"; bankAccount: string }
  | { kind: "cod" };

function paymentLabel(p: Payment): string {
  switch (p.kind) {
    case "card":
      return `Card (ending ${p.cardLast4})`;
    case "bank":
      return `Bank transfer (account ${p.bankAccount})`;
    case "cod":

```

```

    return "Cash on delivery";
  }
}

```

Use a reducer instead of rewriting on the spot

Say three places rewrite the contents of a shopping cart directly: the product list screen, the product detail screen, and the cart screen. A **reducer** fits this case.

Here, to find out where and how the cart contents changed, you have to read all three places.

The agent fixes the cart actions, add and remove, as a type and creates one reducer that "computes the next cart from the current cart and an action." The three places stop rewriting the cart directly and only pass actions to the reducer.

After that, **you only need to read the reducer to know how the cart contents change.**

```

type CartItem = { id: string; price: number };
type Cart = { items: CartItem[] };
declare let cart: Cart;
declare const item: CartItem;
declare const id: string;

// Before: each of the three screens rewrites the cart directly
cart.items.push(item);
cart.items = cart.items.filter((i) => i.id !== id);

// After: fix the actions as a type, and compute the next cart only inside the reducer
type CartAction =
  | { type: "add"; item: CartItem }
  | { type: "remove"; id: string };

function cartReducer(cart: Cart, action: CartAction): Cart {
  switch (action.type) {
    case "add":
      return { items: [...cart.items, action.item] };
    case "remove":
      return { items: cart.items.filter((i) => i.id !== action.id) };
  }
}

cart = cartReducer(cart, { type: "add", item });

```

Do not force structure in

The principle does not ask you to force structure in, though. If the current code is clear, sits in one place, and nobody expects it to grow, the agent leaves it as plain code.

The agent also **distrusts a shared function or type that only adds one layer between calls and reduces nothing**. A shared function or type that reduces no branches, no duplicated rules, and no impossible states only adds places the reader must open and check.

Trigger conditions

- When you write logic that holds state
- When there are many branches
- When the same assumption, "this data has this field," repeats across several files

If any of the following happens, it is a sign that the agent skipped this principle.

- For a new feature, the agent added one more branch to an existing chain of conditionals.
- The agent added a second boolean that must always stay in step with an existing boolean.

Say the code has `isLoading` for loading, and you add `isError` for errors. When loading fails, the code must set `isError` to `true` and, at the same time, set `isLoading` back to `false`.

If the code forgets to reset `isLoading`, `isLoading: true` and `isError: true` hold at the same time, and the screen ends up in a contradictory state: "loading, but also an error."

In this way, a second boolean means that every time you change one value, you also have to bring the other into line.

The "Non-negotiables" section of `/poteto-mode` also requires this principle ([Chapter 8](#)).

"Non-negotiables" asks the agent, whatever code it writes, to first make the shape of the data clear and to pick a structure that follows this principle. **Of the 23 principles, "Model the Domain" is the only one that "Non-negotiables" names.**

"Boundary Discipline" gathers input validation at the outside boundary, and the inside trusts the types

The principle "Boundary Discipline" asks you to **put input validation, type narrowing, and error handling at the system boundary**. As examples of boundaries, the principle lists CLI arguments, configuration files, external APIs, and data that arrives over the network.

A value checked at the boundary has a type. So the code inside the boundary trusts that values of that type arrive, uses them, and does not repeat the same check.

The principle gathers checks at the boundary because checks scattered all over the code cause two problems.

- You end up writing the same check in many places.
- Developers and agents alike assume "something must be checking this somewhere." Unless you know which check runs where, you may not notice a path that nothing checks.

The following code is an example of checks scattered all over the code. Only some functions check that the "quantity" from an order form is 1 or more.

```
let stock = 10; // number of items in stock

// Computes the price: checks that the quantity is 1 or more
function calcPrice(quantity: number): number {
  if (quantity < 1) throw new Error("Quantity must be 1 or more");
  return quantity * 500;
}

// Reduces stock: checks nothing, assuming "something checks this somewhere"
function reserveStock(quantity: number) {
  stock = stock - quantity;
}

// Order screen: calls calcPrice first, so it checks before reducing stock
calcPrice(2);
reserveStock(2);

// Another screen: calls only reserveStock directly
reserveStock(-3); // stock goes up from 8 to 11
```

On the order screen path, `calcPrice` does the check, but the path that calls only `reserveStock` has no check at all.

So -3 arrives as is, and the code that should reduce stock increases it.

Here is the same logic written the way the principle says.

```
let stock = 10; // number of items in stock

// Boundary: checks the value from the form here, once
function parseQuantity(input: string): number {
  const quantity = Number(input);
  if (!Number.isInteger(quantity) || quantity < 1) {
    throw new Error("Quantity must be an integer of 1 or more");
  }
  return quantity;
}

// Inside: only checked values arrive, so it writes no checks
function calcPrice(quantity: number): number {
  return quantity * 500;
}

function reserveStock(quantity: number) {
  stock = stock - quantity;
}

// Every screen passes the form value through parseQuantity before using it
const quantity = parseQuantity("-3"); // the error happens here, and nothing goes further
calcPrice(quantity);
reserveStock(quantity);
```

The check now lives in one place, `parseQuantity`, and `calcPrice` no longer needs its own check.

And because every screen passes the form value through `parseQuantity` before using it, a value like -3 never reaches `calcPrice` or `reserveStock`.

The principle also asks you to write business logic as pure functions and to keep the shell thin, so that the shell only calls the pure functions (see "[Extract pure functions that the shell only calls](#)" for details). When the logic is separate from the framework, **you can test the logic alone without running the framework**.

Rule: validate outside data at the boundary

The agent decides where to put validation with the following two questions.

Check only data that came in from outside

The first question is "Is this data crossing the system boundary right now, that is, has it just come in from outside?"

If the data is not crossing the boundary, the check is extra. The boundary code turns the checked value into a typed value and passes it to the inside functions, and the inside functions trust that type.

Say the function that loads a configuration file checks at load time that `batchSize` is a positive number. Then an inside function that uses `batchSize` does not need to check `batchSize > 0` again. Once the boundary has checked the value, every inside function can rely on it.

```
type Config = { batchSize: number };

// Boundary: checks once when loading the configuration file, and returns a Config value
function parseConfig(raw: { batchSize?: unknown }): Config {
  if (typeof raw.batchSize !== "number" || raw.batchSize <= 0) {
    throw new Error("batchSize must be a positive number");
  }
  return { batchSize: raw.batchSize };
}

// Inside: trusts the Config type and does not check batchSize > 0 again
function splitIntoBatches(items: string[], config: Config): string[][] {
  const batches: string[][] = [];
  for (let i = 0; i < items.length; i += config.batchSize) {
    batches.push(items.slice(i, i + config.batchSize));
  }
  return batches;
}
```

Extract pure functions that the shell only calls

The second question is "Can this logic become a pure function that the shell only calls?" If it can, the agent extracts the logic as a pure function.

- **Pure function.** A function that always returns the same value for the same arguments and does not interact with anything outside the program, such as reading or writing the screen or files.
- **Shell.** The part that handles interaction with the outside of the program, such as the screen, files, the network, and frameworks. It takes values from outside, passes them to pure functions, and sends the results back out.

For example, the agent does not write the order total calculation directly inside the button click handler. It extracts the calculation into a function, `calcTotal(items)`, and the button handler only calls it. That way, you can test the total calculation alone without running the screen.

The shell, for its part, has no calculation, so there is almost no room for mistakes in it.

```
type Item = { price: number; quantity: number };
declare const button: HTMLButtonElement;
declare const totalLabel: HTMLLabelElement;
declare function readItemsFromCart(): Item[];

// Pure function: always returns the same total for the same items. Does not touch the screen
function calcTotal(items: Item[]): number {
  return items.reduce((sum, item) => sum + item.price * item.quantity, 0);
}

// Shell: only reads values from the screen, calls the pure function, and writes the result to the screen
button.addEventListener("click", () => {
  const items = readItemsFromCart(); // read from the screen
  const total = calcTotal(items); // leave the calculation to the pure function
  totalLabel.textContent = `${total} yen`; // write to the screen
});
```

Trigger conditions

When you build in input validation or error handling, or when you write an adapter, the part that connects a framework to your own code.

"Type System Discipline" uses types to make invalid states impossible to build

The principle "Type System Discipline" treats the type checker as a proof assistant, a tool that proves "this value always has this shape."

The agent uses the type checker to **stop contradictory states and unhandled variants as compile errors, instead of finding them after running the code**. An unhandled case that the types miss becomes a runtime error that the compiler could have stopped.

- **Variant.** One of the shapes that make up a sum type. In type `Status = "success" | "error" | "pending";`, for example, each value such as `success` and `error` is a variant.
- **Sum type.** A type for a value that takes exactly one of several shapes. In TypeScript, a sum type is a union type. For example, type `FooBar = 'foo' | 'bar'` is either `foo` or `bar`.

Rule: make invalid states unwritable when you define the type

The main patterns are as follows.

Make impossible states unwritable

If a task has the type `{ completed: boolean; completedAt?: Date }`, you can also write a meaningless combination: "completed, but with no completion time."

So the agent uses a sum type such as `{ kind: "open" } | { kind: "done"; at: Date }`. With the sum type, the only shape you can write for "done" is one that always has a time.

A type that makes impossible combinations unwritable from the start is more reliable than a check afterward that a combination is valid.

Mark values that mean different things

The agent makes `UserId` and `OrderId` branded types, so that you cannot swap one for the other even though both hold a string.

Branded type. A primitive, meaning a basic value such as a string or number, with a type marker attached. `UserId` and `OrderId` in the code example below are branded types.

For example, if you pass an order ID to `getUser()`, which needs a user ID, the compiler reports an error. If the two were interchangeable, a mix-up between values that mean different things would still compile.

```
type UserId = string & { readonly __brand: "UserId" };
type OrderId = string & { readonly __brand: "OrderId" };
type User = { id: UserId; name: string };

declare function getUser(id: UserId): Promise<User>;
declare const orderId: OrderId;

// Both hold a string, but passing an OrderId is a compile error
await getUser(orderId);
```

Do not lie to the type system

The agent does not bypass type checking with casts or assertions. Each place that bypasses type checking can crash at runtime.

- **Cast.** A way of writing that forces the compiler to treat a value as another type, for example `value as unknown as User`, which forces a conversion that type checking would not allow.
- **Assertion.** A way of writing that tells the compiler "this value is definitely this type." In TypeScript, you write it with `as`, such as `as User`.

For example, if you write as `User` on a value from an API without checking it, the code compiles.

But if the real response lacks the `name` field, the code reads on as if `name` were there, and it crashes at runtime where it uses `user.name`.

For facts the compiler cannot prove, the agent proves them itself, by checking the actual value or by narrowing the type with a conditional.

Leave the check for unhandled cases to the compiler

The agent writes code so that, when a sum type gets a new variant, every branch that does not handle it fails to compile.

For example, when you add `canceled` to the kinds of task, compilation fails if the function that picks the display text does not handle it.

So nobody has to search for unhandled cases by hand.

```
// Added canceled to the kinds of task
type Task = { kind: "open" } | { kind: "done" } | { kind: "canceled" };

// Before: an unhandled kind falls through to the last return "". It compiles without error
function labelLoose(task: Task): string {
  if (task.kind === "open") return "Open";
  if (task.kind === "done") return "Done";
  return ""; // a canceled task shows up on screen with empty text
}

// After: if any kind is unhandled, compilation fails
function label(task: Task): string {
  switch (task.kind) {
    case "open":
      return "Open";
    case "done":
      return "Done";
    default: {
      // canceled has no case, so compilation fails on this line
      const unhandled: never = task;
      return unhandled;
    }
  }
}
```

The earlier `labelLoose` compiles even if it forgets to handle `canceled`. So you notice the gap only when empty text shows up on screen.

In the later `label`, `never` is a type that means "no value should ever reach here."

If a case handles every kind, no value reaches `default`, so the code compiles.

If the code forgets `canceled`, on the other hand, a `canceled` task reaches `default`, and compilation fails on the line that assigns it to the `never` variable. The place where the error appears is the place to add the `canceled` case.

Strengthen types only where they proved too weak

The principle does not aim to make types as detailed as possible, though. According to the principle, the job of a type is not to describe data in as much detail as possible. It is to **show, without gaps, the cases that each place using the type must handle.**

So the agent strengthens a type only where the type proved too weak. The signs are runtime assertions and throw statements that raise an error because something "cannot happen." A place with such a check is a place where the type lets through a case it could have blocked.

For example, a completed task should always have a time. A place that writes `if (!task.completedAt) throw new Error("cannot happen")` is a place where the type allows a value that is "completed, but with no time."

Trigger conditions

- When you design types
- When you review function signatures, the types of the arguments a function takes and the value it returns
- When you write code in a statically typed language such as TypeScript, which checks types before the code runs

One of the test questions this principle gives treats a type as instructions for the next agent that touches the code.

"If a new variant is added next month, will the compiler tell the next agent where to add a case?" If no, the match isn't exhaustive.

When you write exhaustive branches, the compiler tells the next agent where to make the fix. So nobody needs to write in prose, "when you add a new state, fix these three places too."

/typescript-best-practices is a Skill that shows this principle as concrete TypeScript patterns ([Chapter 29](#)).

"Make Operations Idempotent" builds operations that give the same result however many times they run

The principle "Make Operations Idempotent" asks you to build each operation that changes state so that, however many times it runs and even when it restarts partway, **it converges, which means it settles in the same correct state in the end.**

- **Idempotent.** The property that running the same operation any number of times gives the same result as running it once. An operation that sets a setting to 5, for example, leaves the value at 5 however many times it runs.

The principle asks for this because **you have to build operations on the assumption that they stop partway.**

Commands, procedures such as startup and shutdown, and loops that repeat work often stop partway or run again because of crashes, restarts, and retries.

Suppose an earlier run stopped partway and left data or files that change the result of the next run. Then after every restart, the developer has to investigate why the result is different this time.

If you build for idempotence, the result is the same however many times the operation runs again, so restarts need no investigation.

Rule: check convergence with three questions

The agent checks whether an operation is idempotent with the following three questions.

1. If the same operation runs twice in a row, what is the result?
2. If the earlier run stopped partway, what is the result of the next run? Is it the same whether the run stopped at the start, in the middle, or just before the end?
3. However many times it runs again, does it end in the same state? In other words, does it converge?

If the answer to any one of the three is "it depends on what the earlier run left behind," the operation needs a **reconciliation step**.

Take the second question as an example.

A process writes 1,000 lines to an output file. The next run leaves the earlier file in place and appends to its end, starting again from line 1.

If the earlier run wrote 400 lines and stopped, the file has the 400 lines from that run followed by the 1,000 lines from this run. The earlier 400 lines are the same as the first 400 lines of this run, so the result is a 1,400-line file with duplicate lines. If the earlier run stopped before writing a single line, the file has 1,000 lines.

When the result changes with where the earlier run stopped, you need reconciliation.

- **Reconciliation.** A step that checks what the earlier run left behind and brings things to the correct state, whatever is left.

Say a run that stopped partway left an output file with only 400 of 1,000 lines written. If **the next run first finds that file and deletes it before it starts writing**, the result is the same 1,000-line file every time, wherever the earlier run stopped.

Trigger conditions

When you design commands that may stop partway or get retried, procedures such as startup and shutdown, and loops that repeat work.

"Migrate Callers Then Delete Legacy APIs" moves every caller and deletes the old API in the same change

You use the principle "Migrate Callers Then Delete Legacy APIs" when you have decided that a new internal API is the right design.

The agent leaves no compatibility layer. It moves every caller to the new API and **deletes the old API in that same change**.

If you keep both the old and the new API, there are two ways to call the same thing, cleanup gets delayed, and the code only grows and never shrinks.

Compatibility layer. A layer that keeps the old way of calling an API working.

In the talk (Chapter 4), poteto says you need to narrow the recommended way of implementing something down to one. This book reads this principle as the procedure that brings an API migration back to one way of implementing it.

Rule: find and move the callers, and delete the old API right away

The main rules are as follows.

Do not treat remaining callers as a reason to keep the old API

The agent finds every caller of the old API, moves them to the new API, and deletes the old API right away.

For example, to replace `getUser(id)` with `fetchUser({ id })`, the agent searches for every place that calls `getUser`, rewrites each to `fetchUser`, and deletes `getUser` in the same set of changes.

When old and new ways of calling sit side by side, readers cannot tell which one to use, and cleanup gets delayed.

Make a temporary bridging layer the exception

An adapter, a temporary layer that connects the old way of calling to the new API, is an exception for cases where the agent truly needs one. When the agent adds one, it sets a deadline for removing it. If bridging layers become routine, the code only grows and never shrinks.

For example, suppose the agent keeps the old `getUser(id)` instead of deleting it and cuts its body down to the single line `return fetchUser({ id });`. The old `getUser(id)` is then an adapter.

Align the tests with the new API

The agent rewrites the tests to verify the behavior that the new API promises, and deletes tests that only protect the internal layout of the old implementation.

For example, it keeps a test that checks "does `fetchUser({ id })` return the user with that ID," and deletes a test that only checks "how many times did the old `getUser` call some other function internally."

If tests that protect the old layout remain, the code keeps conforming to the internal layout of an old implementation that should be gone.

Trigger conditions

When you introduce a new internal API while old callers still remain.

As written, this principle does not apply to an external API, which is a public API with external users. The principle rests on the following three premises.

1. No external users use the old API.
2. You can fix, all at once inside the project, every caller that breaks when you delete the old API.
3. The purpose of the new API is to simplify the code or to refactor.

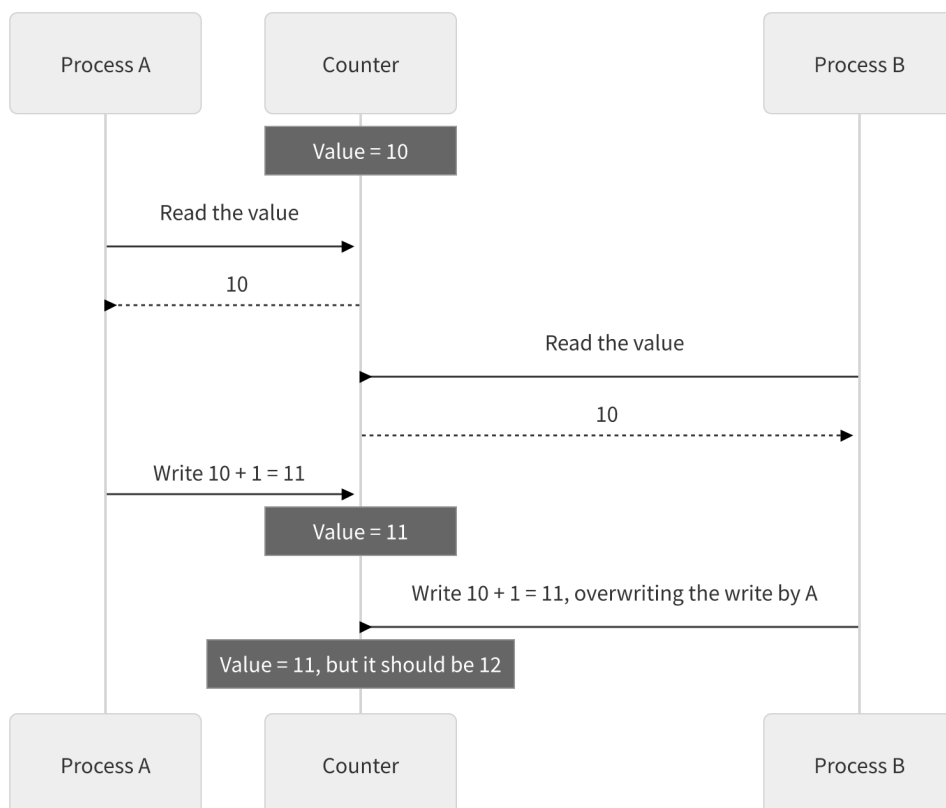
"Separate Before Serializing Shared State" removes the shared write target first, and a lock is the last resort

You use the principle "Separate Before Serializing Shared State" when several actors, such as agents or Workers, may share the same write target.

Before it adds a lock, the agent considers **whether the actors truly need the same write target**. If they do not, it gives each actor its own write target and removes the sharing. The race conditions that concurrent writes cause happen only occasionally and are hard to reproduce, so finding the cause takes a long time.

- **Serializing.** Using a lock or a similar mechanism to make writes happen one at a time, in order.
- **Race condition.** A state where several processes read and write the same data at the same time, so the result depends on the order in which they run.

For example, when two processes, A and B, each add 1 to a counter whose value is 10, the following happens.



View diagram 1

Each of the two processes added 1, so the value should be 12. But B writes 11 based on 10, the value before A wrote, so the value ends up at 11.

A lock does not remove the sharing, so serializing writes is the last resort

This principle **treats serializing writes as the last resort, for when you cannot separate the write targets.**

Serializing is the last resort because a lock does not remove the sharing itself.

If you separate the write targets, no two actors write to the same place, so race conditions cannot happen at all.

With a lock, on the other hand, the write target stays shared. Every actor must take the lock on every write. If even one piece of code writes without taking the lock, race conditions come back.

Also, if an actor stops partway while holding the lock, nothing releases the lock, and the other actors may never be able to write.

So when you think "this needs a lock," the principle asks you not to add one right away, but first to recheck whether you truly cannot separate the write targets.

Even when you cannot separate, enforce the order with a mechanism, not with instructions

Even when you truly cannot separate the write targets, the principle asks you to enforce the order with a mechanism such as a lock, not to rely on instructions to people.

The first paragraph of this principle ends with this sentence.

Instructions and conventions are not concurrency control.

Applied to agents, the sentence means that a person who writes "do not touch the same file" in a request has not built a mechanism that prevents concurrent writes between agents.

Rule: separate write targets before you add a lock

There are three steps.

1. Find the shared places that several actors change

Find the shared places, such as files that several actors read and write and branches that several actors push to.

2. Remove the shared write target

The agent's default response is to remove the shared write target.

First, it considers whether the actors need one authoritative piece of data, or whether each is only writing out its own separate facts.

Then it gives each actor a file, key, or branch that only that actor writes to, and combines the results when someone reads or reports them.

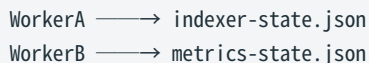
For example, if two Workers each write their own fields to one `state.json`, they share a write target. A split of `state.json` into `indexer-state.json` and `metrics-state.json` ends the sharing.

A diagram of the write targets before and after the split looks like this.

Before the split (the write target is shared)



After the split (each actor has its own write target)



3. Serialize writes only when you cannot separate

Only when the code cannot work correctly without a single write target, add a mechanism that serializes writes, so that they happen one at a time, in order.

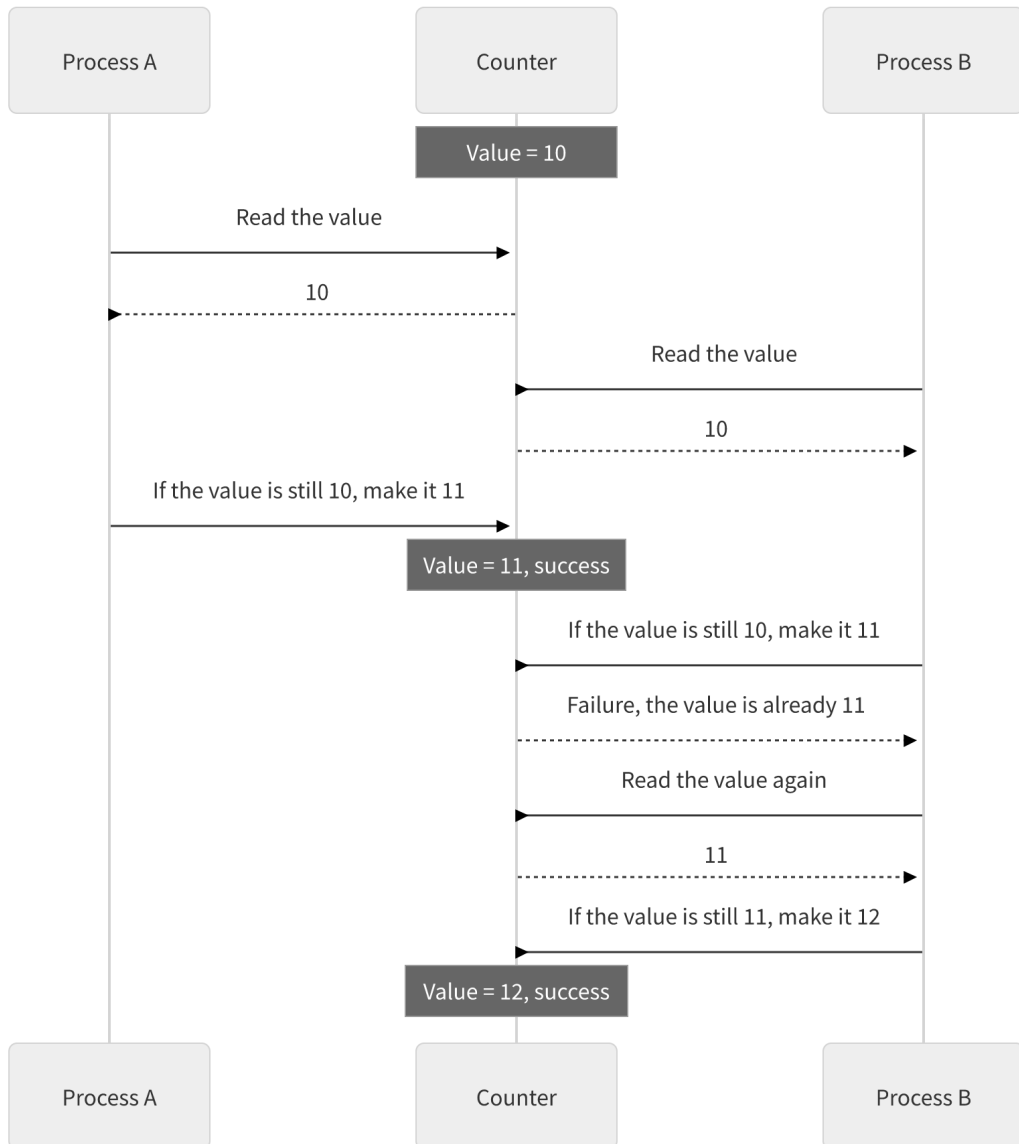
Examples are data that every actor must read and write as one shared value to stay correct, such as the remaining stock count or seat reservations.

The principle names the following four mechanisms.

- **Lock file.** Before writing, an actor creates a file that means "writing in progress," and it deletes the file when it finishes. Other actors wait and do not write while that file exists.
- **Run in stages, in order.** Split the writes into stages, and start each stage only after the one before it finishes. For example, B writes only after A finishes writing.
- **Hand all writes to one actor.** Allow only one actor to write. Other actors ask that actor, "write this value."
- **Atomic compare-and-swap.** Change the value only if it has not changed since you read it, as in "if the value is still 10, make it 11." If it has changed, do not write. Read it

again and retry. The check and the change happen as one atomic operation, which no other process can interrupt partway.

Applied to the earlier counter example, atomic compare-and-swap looks like this.



View diagram 2

The first change by B fails, because A has already set the value to 11. B reads 11 again and retries, so the value ends correctly at 12.

Trigger conditions

When several actors that run concurrently may write to the same file, branch, key, or stateful object.

Example request: give each concurrent attempt its own worktree

The bundled guide page [docs/guide/08-principles.md](#) gives the following one-line request for a case where two attempts that run in parallel are about to write to the same branch.

```
separate before serializing shared state. give each attempt its own worktree, no locks.
```

- **Worktree.** A Git feature that adds another working directory with a different branch checked out from one repository. You can work on several branches at once, each in its own directory.

The six Architecture principles make the structure itself the instructions for the agent

In the talk, poteto says that one way to raise trust in agents is to improve the structure of the codebase itself. The idea is that the agent can tell where it may do what by looking at the lines between folders, types, and processes.

The talk calls this **a design where the structure itself becomes the instructions for the agent.**

Each of the six principles leaves its decision behind as structure, such as types, state machines, and the way files are split. For each principle, the table shows the one question to ask in review and where the decision is left as structure.

Principle	The question to ask in review	Where the decision is left as structure
"Model the Domain"	"Did a second boolean appear?"	State machines, discriminated unions, lookup tables
"Boundary Discipline"	"Is this data crossing the boundary right now, that is, has it just come in from outside?"	A parse function at each boundary, which validates outside data and turns it into a typed value
"Type System Discipline"	"If a new variant is added, will the compiler tell us?"	Sum types, branded types, exhaustiveness checks with never
"Make Operations Idempotent"	"What happens if it runs twice, or stops partway?"	A reconciliation step
"Migrate Callers Then Delete Legacy APIs"	"When does the old API go away?"	The deletion of the old API in the same set of changes that moves the callers
"Separate Before Serializing Shared State"	"Does only one actor write to this write target?"	A file, branch, or worktree for each actor

Summary

- "Model the Domain" gathers scattered branches and booleans into state machines and lookup tables. The agent applies this principle every time it writes code.
- "Boundary Discipline" gathers validation at the boundary, and the code inside the boundary trusts the types.
- "Type System Discipline" makes impossible states impossible to build at compile time.
- "Make Operations Idempotent" uses three questions to ask for operations that converge on the same final state, even through retries and restarts.
- "Migrate Callers Then Delete Legacy APIs" finishes moving the callers and deleting the old API in the same set of changes. This book reads it as a principle that keeps the recommended way of implementing something down to one.

- "Separate Before Serializing Shared State" removes the shared write target itself before serializing with a lock, that is, before making writes happen one at a time, in order.
- This book reads all six as principles that make the structure itself the instructions for the agent.

The next chapter, [Chapter 19](#), covers the four Verification principles, which decide what counts as proof that work is finished.

Chapter 19: Verification: four principles that check real behavior

kaito • Sources: 日本語 • English • 2026-10-03

This chapter covers the following four principles.

1. Prove It Works
2. Fix Root Causes
3. Sequence Work into Verifiable Units
4. Test Behavior, Not Implementation

The four belong to the Verification group, and each one decides **what the agent must verify before it can call the work finished**. The bundled guide page [docs/guide/08-principles.md](#) sums up the four as the principles that decide "what counts as proof".

With the four principles, the agent behaves as follows.

- It does not stop at "the build passed". It opens the file it wrote and checks the lines that are actually in it.
- It does not stop at making the error go away, for example by adding a check that skips the work when a value is null. It finds out why the error happened and fixes that cause.
- It splits the work into small units, runs the tests after each unit, and moves to the next unit only after they pass.
- In tests, it does not check which functions the code called. It passes the code a concrete input and compares the value that actually comes back with an expected value.

As in [Chapter 17](#), this chapter explains the four principles one at a time in terms of their rules and trigger conditions. At the end, it shows how the principles apply to the `duplicate-rows` request.

The duplicate-rows request is the following request from the [Preface](#). In it, the user asks the agent to fix a bug: "when a retry lands mid-run, the export writes duplicate rows".

```
/poteto-mode the export writes duplicate rows when a retry lands mid-run. repro first, then fix and verify.
```

As in [Chapter 17](#), the example requests in this chapter are one-line instructions that name the principle. The premises in [Chapter 17](#) showed that the agent reads and applies a principle on its own when the work matches the trigger condition, even if you do not name the principle.

Of the four principles, this chapter gives an example request only for "Prove It Works".

In this book, I limit example requests to ones that appear in pstack's bundled guide or [README](#). If I wrote an example request for a principle that has none in the source, I could end up showing a use that pstack does not intend.

How this chapter is organized

This chapter is organized as follows.

- A list of the four principles
- "Prove It Works" checks the result of the work by looking at the real thing, not a proxy
- "Fix Root Causes" fixes the cause of a bug, not its symptom
- "Sequence Work into Verifiable Units" splits the work into small units that can be verified
- "Test Behavior, Not Implementation" tests behavior, not implementation
- In the duplicate-rows request, "Fix Root Causes" and "Prove It Works" apply at the steps of the "Bug fix" Playbook
- Summary

A list of the four principles

Principle	The conclusion in one sentence
"Prove It Works"	Check the result of the work by looking at the real thing, not a proxy
"Fix Root Causes"	Do not hide the symptom of a bug, such as the error that surfaced. Fix the root cause that produced the symptom
"Sequence Work into Verifiable Units"	Split the work into small units that can be verified, and move on only after each unit's checks, such as tests, pass
"Test Behavior, Not Implementation"	Call the code the same way its callers do, and compare the observable result with an expected value written directly in the test

A **proxy** is an indirect clue you look at in place of the thing you actually want to verify.

For example, when you decide "the file's modification time is recent, so its contents must have been rewritten correctly" or "the build passed, so the feature must work", the modification time and the successful build are proxies.

"Prove It Works" checks the result of the work by looking at the real thing, not a proxy

The principle "Prove It Works" asks the agent to **check for itself** the running app, the file it wrote, and the value on the screen before it declares the work finished.

The core of this principle is the following sentence at the start of the source. I also quoted it in [Chapter 2](#).

Verify every task output by checking the real thing directly. Do not infer from proxies, self-reports, or "it compiles."

The principle asks for this because **work that nobody verified can look correct and still be wrong.**

To the agent, indirect checks like the ones below look cheaper than looking at the real thing, because none of them require running the feature.

- The file's modification time, as in "the modification time is recent, so the write must have succeeded"
- Whether the output looks new, as in "it looks like this run wrote it, so it must be correct"
- The agent's self-report, such as a report that says "I fixed it"
- A saved screenshot, which is a copy of the screen taken earlier

But **work that builds on a wrong guess costs far more than checking the real thing.**

For example, suppose the agent looks only at the modification time and reports "the write succeeded", while the file actually contains duplicate rows. The user notices the duplicates later and has to go back through the work to find where it broke.

If the agent declares the work finished without checking the real thing, someone has much more work to do later. So the agent must check the real thing directly before it finishes.

Rule: run the feature and read the actual values

To look at the real thing, the agent runs the feature and reads the actual values. The main rules are as follows.

Run the feature to verify it

The agent does not stop at "the build passed" or "the agent said so". It runs the feature and looks at the result, because a successful build shows only that the code compiled.

For example, for a feature where pressing a button adds an item to the cart, the agent presses the button and confirms on the screen that the item is in the cart.

Check directly whether a program is running

The agent does not guess whether a program such as a server is running from other information the program left behind, such as a log file.

For example, the agent does not decide "the log file was updated recently, so the server must be running". It sends a request to the server and confirms that a response comes back. The server may have written the log right before it stopped.

Read the actual value, not a saved copy

The agent reads the actual value, not a cached value or a separate view built from the original value.

For example, to check that the price on the screen is correct, the agent reads the price on the current screen, not a screenshot taken earlier. An earlier screenshot can stay stale after the original value changes.

When a check fails, suspect the check before the system

When a check gives a strange result, the agent asks whether its own way of checking is wrong before it concludes that the system is broken.

For example, if the new value does not appear on the screen, the agent first checks whether it left an old screen open or is looking at a server in a different environment. If the check is wrong, the agent may try to fix a system that works correctly.

Keep the check as a script

According to the principle, the strongest evidence is not a one-time visual check but **a script that can rerun the same comparison any number of times.**

A script makes the same comparison every time it runs. So instead of trusting the agent's word, the reviewer can rerun the same check and see whether the result matches.

When it can, the agent therefore writes a script, runs it, and leaves the output where the reviewer can see it.

For example, a script that compares the contents of the written file with the expected contents looks like this.

```
// check-export.ts: compare the lines of the written file with the expected lines
import { readFileSync } from "node:fs";

const actual = readFileSync("out/export.csv", "utf8").trim().split("\n");
const expected = ["a", "b", "c"];

if (JSON.stringify(actual) !== JSON.stringify(expected)) {
  console.error("NG", { actual, expected });
  process.exit(1);
}
console.log("OK", actual.length);
```

If the written file has the three lines a, b, and c, the script prints OK 3.

If lines are duplicated and the file reads a, b, a, b, c, the script prints NG and both contents, so the reviewer can see for themselves where the two differ.

Usually, though, the agent does not commit the script and its output. It leaves them where the reviewer can see them.

The agent commits them only for large or complex work where someone must be able to audit the trail later, such as a large port or migration.

Trigger conditions

After the work is done, before the agent declares it finished.

Example request: ask for the real output, not the build log

The bundled guide page [docs/guide/10-recipes-and-pitfalls.md](#) lists one common pitfall, which is that the agent reports success only because the build passed. A successful build shows only that the code compiled.

The same page lists the following request as one of the one-line requests that bring a run back when it drifts. It asks for the real output, not the build log, so you can use it when the agent reports success only because the build passed.

apply prove it works. show me the real output, not the build log.

"Fix Root Causes" fixes the cause of a bug, not its symptom

When the agent debugs, the principle "Fix Root Causes" asks it not to stop at the smallest change that makes the error go away. It asks the agent to **trace why the error happened back to the root cause and fix it where the cause is**.

The principle asks for this because workarounds for symptoms pile up.

A workaround leaves the real bug in place, so each time that bug produces another symptom, another workaround appears. The more workarounds there are, the harder it is for a reader to understand how the system behaves.

Also, when a workaround stays in the code, the agent takes it into context as an example of a correct implementation. The agent then produces more wrong copies, and the codebase stops working as memory ([Chapter 4](#)).

Fixing the cause takes longer at first, but it cuts the total time spent on debugging. The benefit also compounds across every later task.

Rule: fix the cause instead of adding a guard that hides the symptom

The main rules are as follows.

Reproduce the bug first

Before it fixes anything, the agent sets up the situation in which the bug happens and confirms that the bug actually occurs.

For example, for a report that says "pressing the save button twice in a row causes an error", the agent first presses it twice and gets the error. If the agent fixes the bug without reproducing it, it cannot verify whether the fix worked.

Ask "why" until you reach the root cause

The agent keeps asking "why" until it reaches the root cause.

Take the save button example from above. The agent asks the following questions.

1. Why does the error happen? Because the save runs twice.
2. Why does the save run twice? Because the button stays clickable after the first press.
3. Why does the button stay clickable? Because nothing disables the button while the save runs.

The third answer is the cause, so the agent changes the code to disable the button while the save runs.

If the agent stopped at the first answer, it would add code that ignores the second save and nothing more. The root cause, a button that stays clickable, would remain.

An agent that stops asking "why" partway treats only the symptom. So it must keep asking "why" until it reaches the root cause.

Do not hide the symptom with a guard

The agent does not add a guard to hide a symptom.

A **guard** is a short check that stops or skips the rest of the work when a value looks likely to cause a problem. An example is `if (user === null) return;` at the top of a function.

For example, if code crashes when a value is null and you add a guard that does nothing when the value is null, the crash goes away. But you still do not know why the value became null.

So the agent does not add the guard. It fixes the reason the value becomes null, because a guard only hides the symptom and the real bug stays.

If you need a long comment to excuse the code, fix the code

If a workaround needs a paragraph of comments to justify it, the code is wrong. The thing to fix is the code, not the comment.

For example, if someone added the following fix for the duplicate-rows bug, the fix is likely a workaround.

```
type Row = { id: string };
declare function writeRow(row: Row): void; // write one row to the file

const writtenIds = new Set<string>();

function writeRowOnce(row: Row) {
  // When a retry happens, a row we already wrote can arrive again.
  // We do not know why it arrives again.
  // So we remember the IDs of written rows and drop any row whose ID we have already seen.
  // This way, no duplicate rows get written.
  if (writtenIds.has(row.id)) return;
  writeRow(row);
  writtenIds.add(row.id);
}
```

This fix stops duplicate rows from being written, but the comment does not answer why the same row arrives again.

Before it writes a comment and code like these, the agent asks why the same row arrives again.

If a fix needs a long comment to justify it, the agent must fix the code instead of writing the comment.

Fix every place that has the same pattern

The agent uses `grep` to search for code with the same pattern as the cause, and fixes every place it finds.

For example, if the cause is that "a button stays clickable after the first press" on one screen, the agent checks whether buttons on other screens use the same code. If the agent fixes only one place, the same bug stays in the others.

When you are stuck, measure instead of guessing

When it is stuck, the agent does not fix things by guessing. It adds logs or reads the actual error messages to see what is happening. When the guess is wrong, a fix based on it adds one more workaround and nothing else.

For a bug like "it stopped working after a restart", the agent suspects stale persistent state, which survives a restart, before it suspects the code. Examples of such state include the following.

- Config files
- Caches
- Lock files
- In-progress state saved to a file

If the program starts working after you delete the file that holds the in-progress state, the cause is the stale state left in that file.

In that case, the agent does not stop at deleting the file. As the fix, it prefers state validation, which is code that checks whether the state read from the file has the shape the current code expects. If the agent only deletes the file, the program breaks again the next time a run leaves stale state behind.

For example, suppose the old code saved the in-progress state in the shape `{ step: 3 }` and the current code expects the shape `{ lastRowId: "b" }`. The state validation looks like this.

```
// The shape of in-progress state that the current code expects
// loadState below rejects the old { step: 3 } shape
type SavedState = { lastRowId: string };

// Check whether the value read from the save file has the SavedState shape
function loadState(raw: unknown): SavedState | null {
  if (
    typeof raw === "object" &&
    raw !== null &&
    "lastRowId" in raw &&
    typeof raw.lastRowId === "string"
  ) {
    return { lastRowId: raw.lastRowId };
  }
}
```

```
// Do not use state in an old shape such as { step: 3 }. Start over
return null;
}
```

If the state that a previous run left behind decides whether the program works, the agent must validate the state it reads.

Trigger conditions

When the agent debugs.

"Sequence Work into Verifiable Units" splits the work into small units that can be verified

The principle "Sequence Work into Verifiable Units" asks the agent to split the work into small units, run checks such as tests after each unit, and **move to the next unit only after every check passes**.

For example, for a rename that spans 50 files, the agent renames in one package, runs that package's tests, and moves to the next package only after they pass.

The principle asks for this because verifying each unit catches a breakage at the unit that caused it.

For example, if the agent renamed in 10 packages at once in the rename above and then a test failed, it would have to find which of the 10 changes caused the failure. By then, it would also have piled more work on top of the broken state.

If the agent verifies one package at a time, a failing test points only at the one package it just changed.

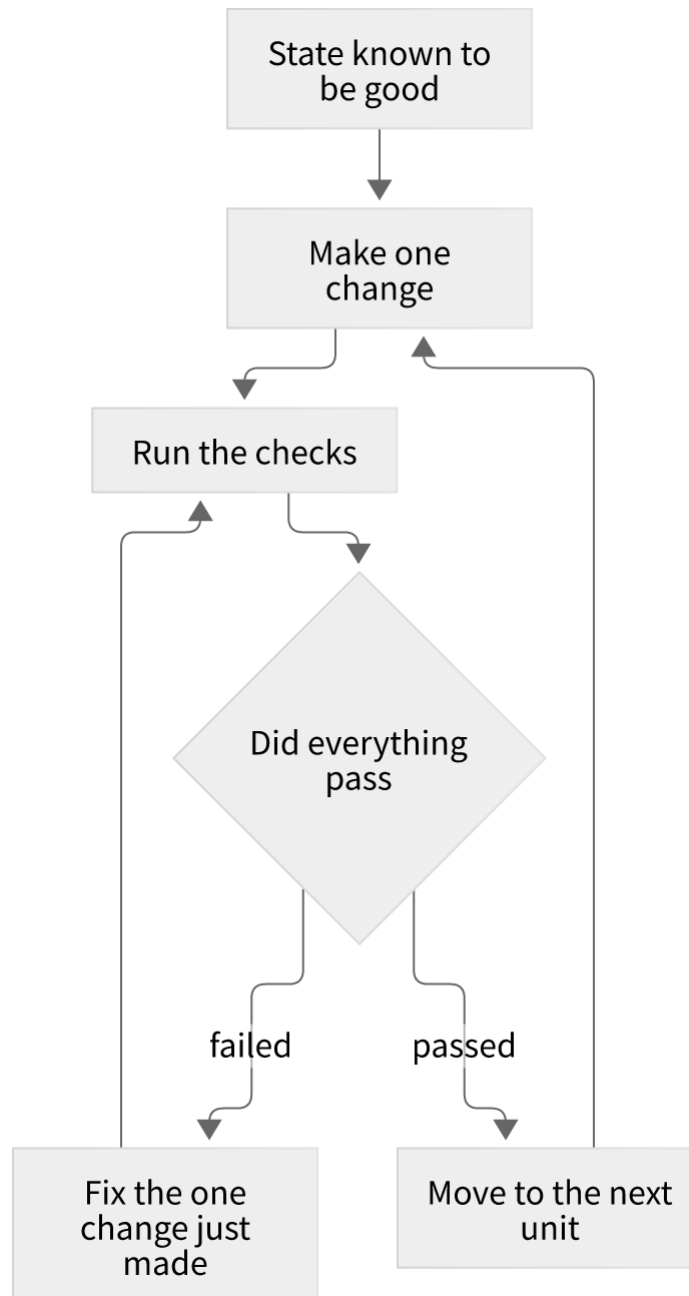
The more changes you verify together, the wider the search for the cause gets. So the agent must verify after each change.

Rule: verify one unit at a time, and order commits so they show correctness

The rules for how the agent proceeds are as follows.

Make one change, then verify once

The agent starts from a state known to be good, makes one change, and runs the checks before it moves on. In the rename example above, one package is one unit.



[View diagram 1](#)

When a check fails, the agent fixes only the one change it just made. So the search for the cause does not widen.

Even when the agent uses a tool that makes edits automatically, such as one built with "Build the Lever" in Chapter 17, it does not skip the per-unit checks. With a tool, the per-unit checks cost almost nothing.

Rebase onto a clean trunk before you start

Before it starts work, the agent rebases its branch onto the latest trunk, with no unrelated changes.

That way, every check result is a comparison against the real baseline, which is the latest trunk.

Suppose the agent works on top of an old trunk or on a branch mixed with other changes. When a check fails, the agent cannot tell whether its own change caused the failure or the failure was already there.

Order commits so they show correctness

The agent orders commits and PRs so that a reviewer can confirm the change is correct by following the commits in order. The basic shape is **"the failing test first, then the fix on top of it"**.

In this order, the reviewer can watch the test fail and then pass.

Commit order (bottom comes first)

```

Fix commit           ← the test passes here
↑
Failing test commit  ← the test fails here
↑
trunk

```

The principle lists three more orderings.

- Put a commit that deletes dead code before a commit that restructures. For example, delete unused functions first, then restructure the remaining files.

- Put a commit that records the current measurement before an improvement commit. For example, record the current processing time before a change that makes the processing faster.
- Put a commit that builds only the base for a feature before the commits that build the feature. For example, before you build the contents of a new screen, add only an empty screen and the route to it.

The agent makes each commit able to land on its own. The whole sequence of commits then reads as a step-by-step explanation that the change is correct.

Trigger conditions

When the agent does multi-step work, such as a bulk fix, a migration, or a series of similar edits, and when it decides the order of commits or PRs.

"Test Behavior, Not Implementation" tests behavior, not implementation

The principle "Test Behavior, Not Implementation" defines a test as something that calls the code the same way its callers do and **compares a result the callers can observe with an expected value written directly in the test.**

- **Behavior** is what the code's callers can see it do or produce, such as the value a function returns or the data it saved.
- **Implementation** is how the code is built inside, such as which functions it calls and how.
- A **literal expected value** is an expected value written as the value itself, not computed by code. In the example below, "hello-world" is a literal expected value.

For example, here are two ways to write a test for `slugify`, a function that turns a string into a URL-friendly form. The test runner is Vitest.

```
// Turn a string into a URL-friendly form
declare function slugify(text: string): string;

// Testing implementation: check whether slugify calls toLowerCase internally
const spy = vi.spyOn(String.prototype, "toLowerCase");
slugify("Hello, World!");
expect(spy).toHaveBeenCalled();

// Testing behavior: call slugify the way callers do and compare the result with a literal
// expected value
expect(slugify("Hello, World!")).toBe("hello-world");
```

The first test passes as long as `slugify` calls `toLowerCase`, even if the result is not `"hello-world"`. The second test fails unless the result is `"hello-world"`.

The principle defines tests this way because a test that checks which functions the code called, or a test that only copies a constant from the code, does not fail when the code has a bug.

Such tests neither call the code the way callers do nor compare an observable result. So they spend CI time and the reviewer's attention and find no bugs.

Also, **a test that only copies a constant fails when someone edits that constant on purpose, so it gets in the way of that edit.**

For example, suppose the setting for "items shown per page" is 20, and a test also writes 20 and checks that the setting is 20.

If you want 30 items per page and change the setting to 30, the page shows 30 items correctly, but the test fails because it expects 20. The only way to make it pass is to change the 20 in the test to 30 too.

A test that only copies a constant's value fails when someone changes the constant correctly, so it gets in the way of correct edits.

Rule: one question, "would it pass if everything returned undefined?", finds tests that check nothing

Before it keeps a test, the agent judges it with one question: "If every function this test imports returned only `undefined`, would this test still pass?"

If it would pass, the test **observes no behavior** and does not fail even when the real code is broken. In that case, the agent rewrites the assertion or deletes the test.

An assertion is the statement in a test that checks whether a result is what you expected. A mock is a fake part that a test puts in place of the real one.

Tests that pass this question mostly take the following five forms.

Using a weak check

These tests have no assertion, or they use only a check that almost anything passes, such as `toBeDefined`, which checks only that the value is not undefined.

For example, if you write `expect(slugify("Hello, World!")).toBeDefined()` for the `slugify` test above, the test passes even if `slugify` returns `"xyz"` instead of `"hello-world"`.

```
// Turn a string into a URL-friendly form
declare function slugify(text: string): string;

// Weak: passes even if slugify returns "xyz"
expect(slugify("Hello, World!")).toBeDefined();

// Strong: fails if it returns anything other than "hello-world"
expect(slugify("Hello, World!")).toBe("hello-world");
```

Looking only at whether something was called

These tests check only that a mock was called, with `toHaveBeenCalled`, or that the result is empty, with `toEqual([])`.

For example, `findUsers` is a function that searches for members by name. If its only test is `expect(findUsers("no-such-name")).toEqual([])`, the test passes even for a broken `findUsers` that always returns an empty array for any name.

```
type User = { name: string };
declare function findUsers(name: string): User[]; // search for members by name

// Looks only at whether the result is an empty array. Passes even for a broken implementation that
```

```
always returns []
expect(findUsers("no-such-name")).toEqual([]);
```

Building the expected value from the code under test

These tests compute the expected value with the very code they test.

For example, suppose `calcTotal`, a function that computes the total price, has a bug that forgets to multiply by the quantity. If you buy two items at 100 yen, `calcTotal` should return 200, but it returns 100.

The "before" test computes the expected value with the same `calcTotal`. So whatever wrong value `calcTotal` returns, the left and right sides are always equal, and the test passes.

```
type Item = { price: number; quantity: number };
// Compute the total price (has a bug that forgets to multiply by the quantity)
declare function calcTotal(items: Item[]): number;

const items = [{ price: 100, quantity: 2 }];

// Before: the expected value is also computed with calcTotal. Both sides are 100, so the test
// passes
expect(calcTotal(items)).toBe(calcTotal(items));

// After: write the expected value directly in the test. 100 is not 200, so the test fails
expect(calcTotal(items)).toBe(200);
```

Only copying a constant

These tests only write a constant, setting, or prompt string from the code a second time in the test and compare the two.

For example, if the code sets the maximum number of tools the agent can use at once to 8, the test only writes the 8 from `LIMITS.maxTools` in the code a second time.

```
// Code: sets the maximum number of tools to 8
const LIMITS = { maxTools: 8 };
```



```
// Test: only writes the 8 from the code a second time
expect(LIMITS.maxTools).toBe(8);
```

Looking only at data the test set up

These tests only read and check data that the test created during setup, and never call the code under test.

```
it("keeps the user name", () => {
  const user = { name: "taro" }; // data the test created itself
  expect(user.name).toBe("taro"); // never calls the code under test
});
```

How to rewrite them

Call the code under test with a concrete input and compare with a literal expected value

When it rewrites a test, the agent calls the code under test with one concrete input and compares the returned value, or an observable change such as saved data, with a literal expected value.

For example, for `slugify` above, it writes `expect(slugify("Hello, World!")).toBe("hello-world")`.

To check that something does not happen, also check that it happens for another input

When the agent wants to check that something does not happen, it also checks, in the same test, that the thing does happen for another input.

For example, in the same test for `findUsers` above, the agent also compares the result for a name that exists, as follows.

```
type User = { name: string };
declare function findUsers(name: string): User[]; // search for members by name

expect(findUsers("no-such-name")).toEqual([]);
expect(findUsers("taro")).toEqual([{ name: "taro" }]);
```

A broken implementation that always returns an empty array fails the second assertion.

For a constant, check the code that reads it instead of copying the value

For a constant, instead of restating the value, the agent checks the code that reads the constant, with one input.

For example, for the "items shown per page" setting above, it rewrites the test as follows.

```
// The setting for items shown per page (currently 20)
declare const PAGE_SIZE: number;
// Split an array into pages. If pageSize is omitted, use PAGE_SIZE
declare function paginate<T>(items: T[], pageSize?: number): T[][];

// Before: only copies the setting. Fails if the setting changes to 30
expect(PAGE_SIZE).toBe(20);

// After: check the code that splits into pages, with one input
expect(paginate(["a", "b", "c"], 2)).toEqual([["a", "b"], ["c"]]);
```

The "after" test fails if the code that splits into pages breaks, but it does not fail when you change the setting to 30.

Delete a test when you cannot find a way to check it

If the agent cannot find a way to check the behavior like this, it deletes the test.

With a mock, check what it received, not that it was called

A mock is fine in itself. But in the test, the agent checks what the mock received, or the state after the call, not that the mock was called.

For example, if the agent mocks the part that sends email, it checks that "the correct recipient and body were passed", not that "send was called". A test that checks only that send was called passes even when the contents passed in are wrong.

```
type Mailer = { send: (mail: { to: string; body: string }) => void };
// Register a member and send a thank-you email with mailer
declare function registerUser(mailer: Mailer, user: { email: string }): Promise<void>;

// Before: Looks only at whether send was called. Passes even if the recipient or body is wrong
it("sends a welcome mail", async () => {
  const mailer = { send: vi.fn() };
  await registerUser(mailer, { email: "taro@example.com" });
  expect(mailer.send).toHaveBeenCalled();
```

```
});

// After: compare the recipient and body the mock received with literal expected values
it("sends a welcome mail", async () => {
  const mailer = { send: vi.fn() };
  await registerUser(mailer, { email: "taro@example.com" });
  expect(mailer.send).toHaveBeenCalledWith({
    to: "taro@example.com",
    body: "Thank you for registering",
  });
});
```

In this test code, if `registerUser` has a bug that sends with an empty recipient, the "before" test still passes. The "after" test fails because the recipient is not `taro@example.com`.

Trigger conditions

When the agent writes a test, changes one, or decides whether to keep one.

Example: write the retry test in terms of behavior

The duplicate-rows request gives an example of a test of the "looking only at whether something was called" kind, and of how to rewrite that test. The duplicate-rows request is the following request from the start of the chapter.

```
/poteto-mode the export writes duplicate rows when a retry lands mid-run. repro first, then fix and verify.
```

In this case, the agent that received the request writes a test for the retry.

The following test passes as long as `exportWithRetry` calls `db.query`, even if it does not write the rows correctly.

```
type Db = { query: (...args: unknown[]) => unknown };
type InMemoryDb = Db & { rows: (table: string) => { source_row_id: string }[] };

// Export the source rows in input, and retry if it fails partway
declare function exportWithRetry(
  db: Db,
  input: string[],
  options?: { failAfterRows?: number },
```

```

): Promise<void>;
// Create an in-memory DB for tests
declare function createInMemoryDb(): InMemoryDb;
// The source rows (three rows: a, b, and c)
declare const input: string[];

// A bad test. It looks only at whether the mock was called
it("writes rows on retry", async () => {
  const db = { query: vi.fn() };
  await exportWithRetry(db, input);
  expect(db.query).toHaveBeenCalled();
});

```

The following test compares a result the callers can observe, the rows that were actually written, with the literal expected value ["a", "b", "c"].

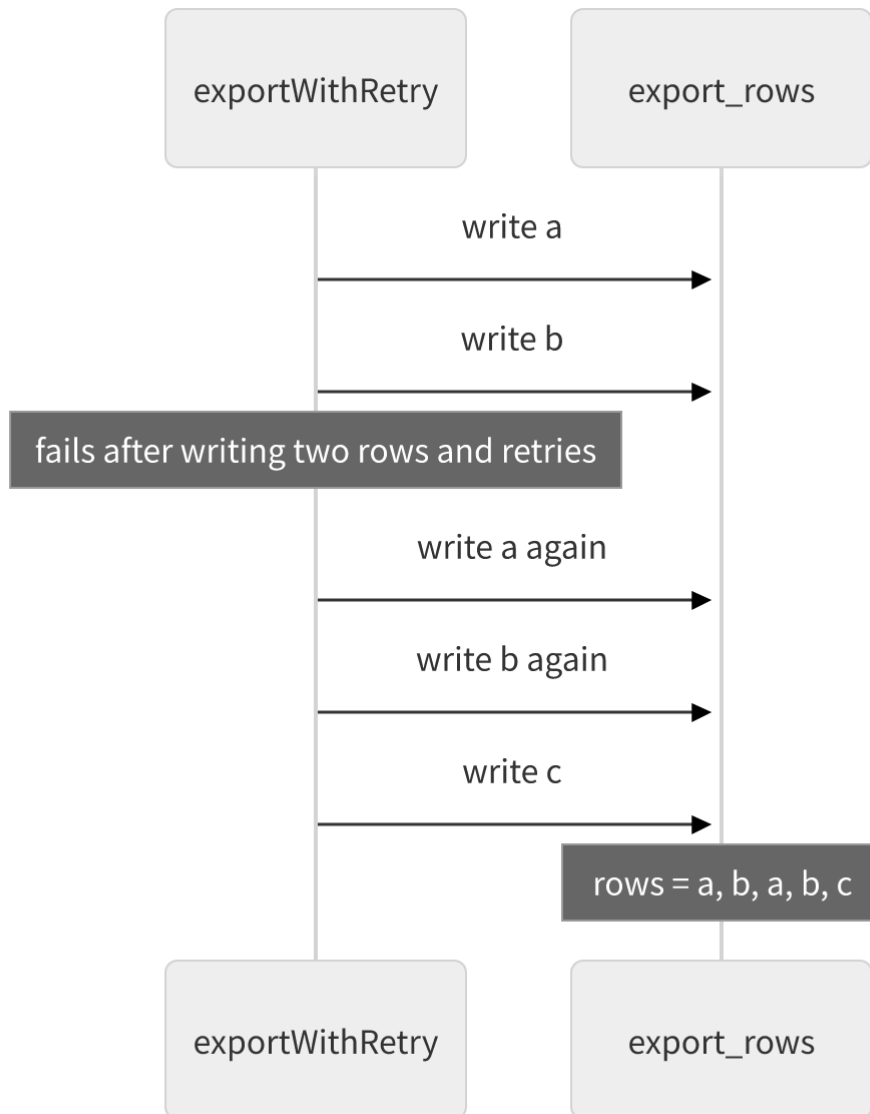
```

// A test that checks behavior
it("writes each source row once when a retry lands mid-run", async () => {
  const db = createInMemoryDb();
  await exportWithRetry(db, input, { failAfterRows: 2 });
  expect(db.rows("export_rows").map((r) => r.source_row_id)).toEqual(["a", "b", "c"]);
});

```

`failAfterRows: 2` makes the export fail once after it writes two rows, so that a retry lands mid-run.

With an implementation whose retry starts again from the first row, the written rows change as follows.



View diagram 2

After the first a and b come a, b, and c, which the retry wrote again from the first row. So the actual rows are ["a", "b", "a", "b", "c"]. These rows do not match the expected value ["a", "b", "c"], so the test fails.

Whether a test can catch a bug depends on what it compares. So the agent must compare an observable result with a literal expected value.

In the duplicate-rows request, "Fix Root Causes" and "Prove It Works" apply at the steps of the "Bug fix" Playbook

Here is the duplicate-rows request from the start of the chapter again.

```
/poteto-mode the export writes duplicate rows when a retry lands mid-run. repro first, then fix and verify.
```

In this book, I read this request as one where two principles mainly apply within the steps of the "Bug fix" Playbook from [Chapter 11](#).

In the request, "repro first" maps to "Fix Root Causes" and "verify" maps to "Prove It Works".

Step in "Bug fix"	The principle used and how it changes the decision
1. Reproduce it yourself	"Fix Root Causes". Set up the condition where a retry lands mid-run, and narrow the condition until the duplicates appear. Write the fix only after you reproduce the bug
2. Narrow the cause with a binary search	"Fix Root Causes". List the candidate causes, halve the suspect range each time, add logs, and check the candidates one by one
3. Plan the fix	"Fix Root Causes". Code that deletes duplicate rows after the write does not fix the reason the duplicates happen, so it treats only the symptom. If you make the write idempotent, check the write's design with the three questions of "Make Operations Idempotent" in Chapter 18
4. Verify with the same environment and steps	"Prove It Works". Check that the original repro passes and look at the rows that were actually written, not only whether the unit tests passed
5. Commit the failing repro first	"Sequence Work into Verifiable Units". Record in the Git history the change from a failing test to a passing one

Step in "Bug fix"	The principle used and how it changes the decision
Reply	"Prove It Works". Write the root cause and the fix, and paste output that shows the repro steps failing before the fix and passing after it

In step 2, the candidate causes the agent checks include, for example, the following.

- The queue hands out the same job twice
- The retry starts over from rows it already wrote, not from the row after the last one written
- The write is not idempotent ([Chapter 18](#))
 - An operation is idempotent when running it any number of times gives the same result as running it once

Summary

- "Prove It Works" does not accept "it compiles" or "the agent said so" as evidence. It looks directly at the real artifact, such as the written file or the running screen.
- "Fix Root Causes" reproduces the bug, asks "why" until it reaches the cause, and fixes the cause instead of hiding the symptom with a guard.
- "Sequence Work into Verifiable Units" confirms that tests and other checks pass for each small unit, and puts the commit with the failing repro before the fix commit.
- "Test Behavior, Not Implementation" uses one question, "would it pass if everything returned undefined?", to find tests that cannot catch any bug.
- In the duplicate-rows request, "Fix Root Causes" applies from the repro of "Bug fix" through planning the fix, and "Prove It Works" applies to verification and the reply.

The next chapter, [Chapter 20](#), covers the two Delegation principles: "[Guard the Context Window](#)" and "[Never Block on the Human](#)".

Chapter 20: Delegation: two principles for delegated and parallel work

kaito • Sources: 日本語 • English • 2026-10-03

This chapter covers the following two principles.

1. Guard the Context Window
2. Never Block on the Human

Both belong to the Delegation group. They decide what the agent does when it hands work to subagents and when it moves on without waiting for a human reply.

The bundled guide's page [docs/guide/08-principles.md](#) sums up the two as principles that **"keep parallel work sane"**.

Both principles let the agent that hands out work keep the parallel work moving. The agent does not fill its context with the large output of subagents, and it does not stop to wait for a human reply.

With the two principles, the agent behaves as follows.

- It has subagents read test logs thousands of lines long or API specifications hundreds of pages long. In the main thread, which is the conversation that calls the subagents, it receives only a few lines of summary, such as "which tests failed, and why".
- It carries out reversible work, such as renaming a function, without asking the human "May I change this?", and then shows the human the result and the reason.
- It checks with the human before it acts only for irreversible operations, such as a force-push to a shared branch.

Also, **the two principles protect different things**. "Guard the Context Window" keeps the agent's context from filling up with raw data. Raw data here means output read in as

is, without a summary, such as test logs thousands of lines long, screenshots, and large documents.

Most of this data never feeds a decision, yet it uses up context in proportion to what the agent reads. So "Guard the Context Window" has the agent protect its context.

"Never Block on the Human" keeps the human's attention from being pulled away every time the agent asks for approval.

The two principles also appear in the steps of Playbooks.

For example, the "Session pickup" Playbook in [Chapter 15](#) has a subagent read long transcripts, which applies "Guard the Context Window". The "Multi-phase or multi-PR plan" Playbook in [Chapter 16](#) does not ask the operator questions that the agent could answer by running a prototype, which applies "Never Block on the Human".

As in [Chapter 17](#), this chapter explains the two principles one at a time through their rules and trigger conditions. At the end, it looks at how the two principles connect to the theme of [the talk](#): a state in which the human does not need to sit next to the agent and watch.

This chapter gives no request examples for its two principles.

In this book, I limit request examples to those that appear in pstack's bundled guide or [README](#). If I made up request examples for principles that have none in the source, I might show uses that pstack does not intend.

How this chapter is organized

This chapter is organized as follows.

- The two principles at a glance
- "Guard the Context Window" has subagents read large output and documents, and leaves a summary in the main thread
- "Never Block on the Human" moves reversible work forward without waiting for the human to confirm

- When the agent's context and the human's attention are protected, the human does not need to sit and watch
- Summary

The two principles at a glance

Principle	One-sentence conclusion	Resource it protects
"Guard the Context Window"	Have subagents read large output and documents, such as logs thousands of lines long or specifications hundreds of pages long, and put only a summary in the main thread	The agent's context
"Never Block on the Human"	Move reversible work, such as writing code or renaming a function, forward without waiting for confirmation, and show the human the result and the reason. Confirm only irreversible operations, such as a force-push or deleting production data, before you act	The human's attention

"Guard the Context Window" has subagents read large output and documents, and leaves a summary in the main thread

The principle "Guard the Context Window" asks the agent to **pick each thing it puts in the main thread by whether it is worth the cost**. The cost here is the number of tokens read into the main thread.

Worth the cost means that, for the number of tokens read into the main thread, the content helps the agent make its next decision.

For example, a few-line summary of which tests failed and why is worth the cost, because the agent can use it as is to decide what to fix next.

On the other hand, a log thousands of lines long, passing tests included, is not worth the cost if the agent reads it into the main thread as is. The agent uses only a tiny part of that log for its decision.

The principle asks for this because **the context window is limited, and within one session, the agent cannot free up space it has already used.**

When the context window fills up, the quality of the agent's reasoning drops. Conversation compaction summarizes the part of the conversation that no longer fits. That summary drops some of what the agent read earlier, and in the end the work itself stops.

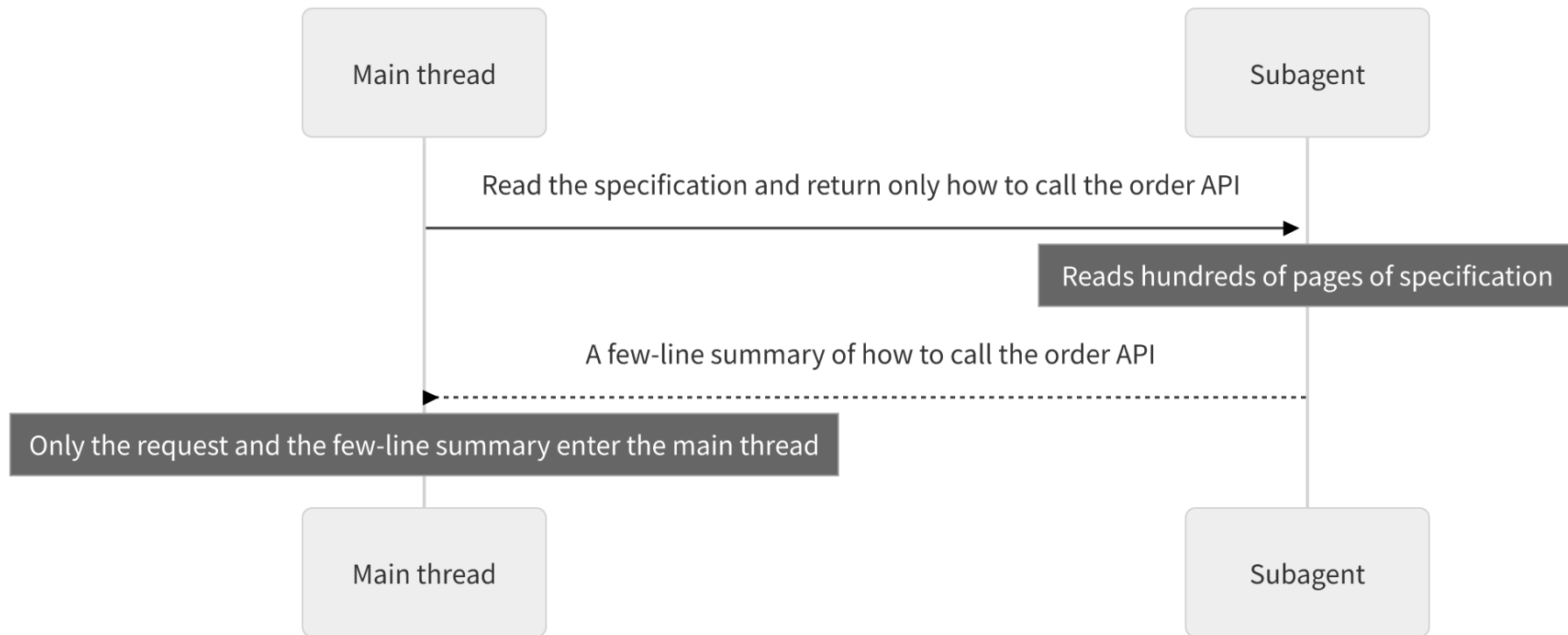
Rules: put only what is worth the cost in the main thread

The principle names the following three methods.

1. Have subagents read large output and documents

The agent has subagents read verbose output, such as test logs thousands of lines long, screenshots, and large documents. It receives only a summary in the main thread, not the raw data. If the agent reads raw data in the main thread, the raw data fills the main thread's context by that amount.

For example, suppose the agent needs to know only how to call the order API from an API specification hundreds of pages long. The agent splits the work as follows.



View diagram 1

The hundreds of pages of specification fill only the subagent's context. The main thread's context shrinks only by the request and the few-line summary.

If the agent read the large document in the main thread, the document would fill the main thread's context by that amount. So the agent needs to have a subagent read the document.

2. Put templates and reference material used every time in the SKILL.md body

When a Skill reads a template or reference material every time it runs, the agent writes that material into the SKILL.md body instead of splitting it into a separate file. A separate file adds the cost of reading that file every time the Skill runs.

For example, when a review Skill writes its report in the same format every time, the files are laid out as follows.

```
Before: the report format used every time is in a separate file
skills/review/SKILL.md          ← the steps, plus one sentence: "write the report in the format
of templates/report.md"
skills/review/templates/report.md ← the report format

After: the report format used every time is in the SKILL.md body
skills/review/SKILL.md          ← the steps and the report format
```

In the before layout, every time the agent uses the Skill, it reads SKILL.md and then also reads templates/report.md. In the after layout, one read of SKILL.md is enough.

So when the location changes how many reads happen each time, the agent needs to put what it uses every time in the SKILL.md body.

3. Cap the scope of work in each phase

The agent splits large work into phases and caps both the number of files one phase handles and the number of turns it may use.

The agent also accounts in advance for the context that the machinery of the work itself uses, such as subagent calls. For example, the request sent to a subagent and the summary that comes back from it both use the main thread's context.

If the agent sets no caps and does not account for what the machinery uses, the context fills up in the middle of a phase. Then, before the phase ends, the quality of the agent's reasoning drops or the work stops.

The test is whether content is worth the cost

The principle tests whether content is worth the cost, not whether the agent reads less. The second method shows the difference.

If the agent puts the template in the `SKILL.md` body, `SKILL.md` gets longer and one read covers more text.

The principle still picks the body because the agent reads a template used every time on every run anyway, even from a separate file. With a separate file, the content read stays the same and the number of reads goes up by one. So the body is the cheaper place for the template.

Trigger conditions

When the context starts to fill up from large output, long files, rereading the same file, and similar causes. It also applies when the agent plans a fan-out, which splits the work across several subagents that run in parallel.

Playbook example: a subagent that explores the codebase may return only four things

The "[Multi-phase or multi-PR plan](#)" Playbook ([Chapter 16](#)) hands the job of exploring the codebase to a subagent before the agent writes the plan document. At that point, it cites this principle and **limits what the subagent returns to the following four things.**

- The locations of the relevant files
- The conventions of the codebase
- The test commands

- The entry points, meaning the functions or files where the feature under study starts its processing, for example the handler that first receives an API request or the function that receives a CLI command

The Playbook also forbids "inlined dumps", in which the subagent pastes the code it read into its reply as is.

When the reply is limited to these four things, only the information used to write the plan document enters the main thread.

"Never Block on the Human" moves reversible work forward without waiting for the human to confirm

For reversible work, the principle "Never Block on the Human" asks the agent to **act without asking permission** and show the human the result and the reason. The human looks at the result and corrects course afterward.

The principle opens with these two sentences.

The human supervises asynchronously. Agents must stay unblocked.

In other words, the human does not stay with the agent while it works. The human checks the results afterward, so the agent must not stop and wait for the human's reply.

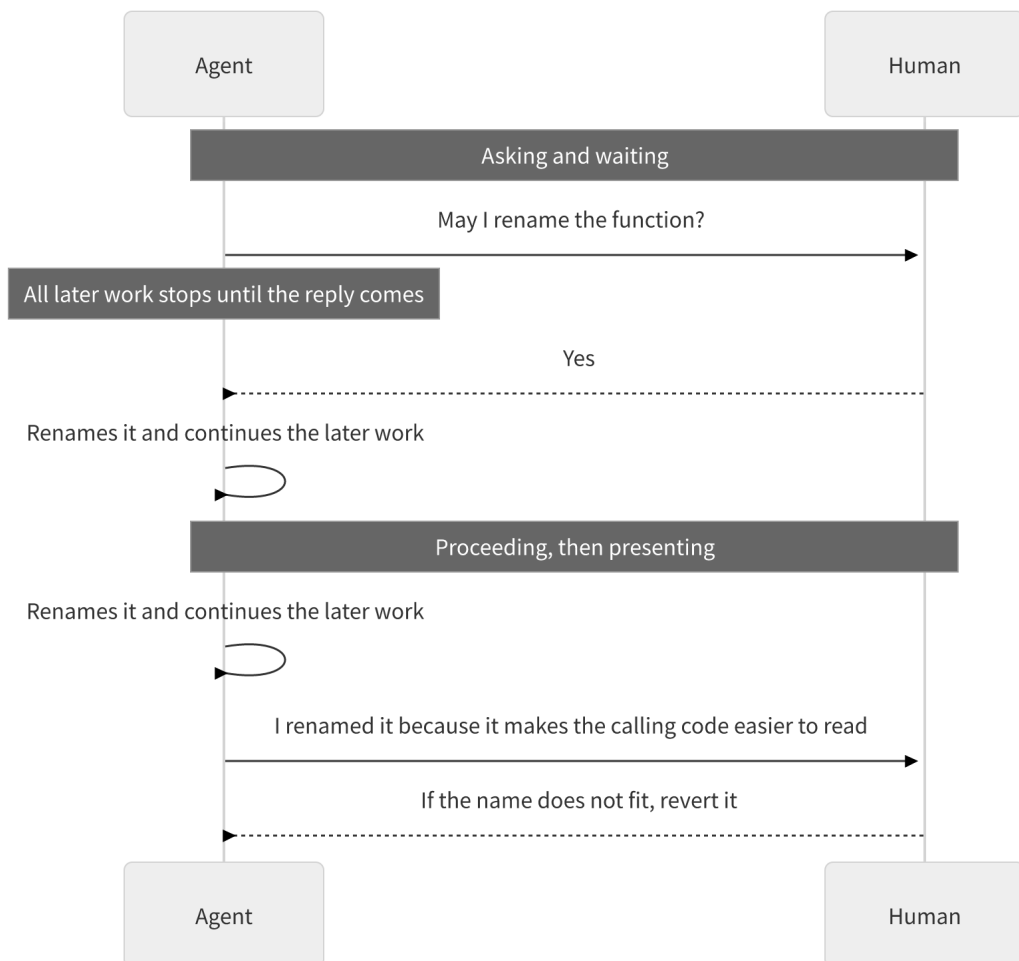
The principle asks for this because **every time the agent stops to wait for the human's permission, the pipeline stops, and the human's reply sets the pace of the whole job.**

Code changes can be reverted and reviewed, so **the cost of a wrong decision is usually smaller than the cost of stopping.** After a wrong decision, all the human needs to do is notice it in review and revert the change. After a stop, nothing downstream moves until the human replies.

Rules: check with the human only for irreversible operations

Instead of asking "May I do X?", the principle asks the agent to do X and then show the result and the reason. The principle calls this "Proceed, then present".

For example, say the agent is unsure whether to rename a function. The diagram below compares asking and waiting with proceeding and then presenting.



[View diagram 2](#)

If the agent asks and waits, the work stops until the human replies. If the agent proceeds and then presents, a name that does not fit costs the human one review reply that says "revert it".

So for reversible work, the agent needs to proceed and then present.

The principle also asks for self-healing, meaning the system fixes itself over time. When the agent notices a problem in the middle of its work, it records the problem without

waiting for the human's instructions and fixes it in the next round of work. If the agent waited for the human's instructions every time it noticed something, the work would stop every time.

For example, say the agent notices in the middle of its work that a different test fails now and then. The agent records the failing test and fixes it in the next round of work. An agent that tells the human about the problem and stops until instructions arrive is not self-healing.

The agent may proceed without waiting for confirmation because the change can be reverted.

A mistake in an irreversible operation cannot be undone, so the agent checks with the human before it acts.

The handling splits into the following three kinds of operation.

Kind	Handling	Examples
Irreversible operation	Needs confirmation	A force-push, deleting production data, sending a message outside
Reversible operation	Proceed without stopping	Writing code, editing notes, splitting tasks
Product direction	The human decides the direction of what to build. Once the direction is set, the implementation proceeds without stopping	Deciding which feature to build

Trigger conditions

When the agent feels like asking "May I do X?" about reversible work.

Two places in /poteto-mode draw the line between operations that stop and operations that proceed

Two sections of /poteto-mode, the Skill you use to start work, spell out which operations make the agent stop and which let it proceed.

- The "[Autonomy](#)" section requires the agent to always stop before irreversible writes such as a force-push to a shared branch, a deploy, deleting data, or a message to a customer. On the other hand, it tells the agent to proceed without asking on reversible work and on outside actions such as posting in the team chat or updating a ticket ([Chapter 35](#)).
- The "[Non-negotiables](#)" section ([Chapter 8](#)) asks that **when the answer is a fact the agent can observe by running something, such as behavior, time taken, screen layout, or output, the agent decides by trying a prototype instead of asking the human**. The agent asks the human only about product and taste decisions that no experiment can settle ([Chapter 38](#)).

The principle "Never Block on the Human" treats sending a message outside as an irreversible operation no matter who receives it, and requires the agent to check with the human before it acts. The first row of the table in "[Rules: check with the human only for irreversible operations](#)" shows this rule. The "Autonomy" section, on the other hand, splits outside actions by recipient. It puts messages to customers on the stop side and posts in the team chat on the proceed side.

In other words, the difference is that the "Autonomy" section draws a finer line.

When the agent's context and the human's attention are protected, the human does not need to sit and watch

One point poteto made in the talk was that if the environment is set up well enough, the human does not need to sit next to the agent and watch all the time.

In this book, I read the point as follows. When the agent follows the two Delegation principles, **the human does not need to sit next to it and watch**. There are two reasons.

- If raw data does not fill the agent's context, the quality of its reasoning is less likely to drop and the work is less likely to stop, even in the middle of a long task. So the human needs to check in less often.
- If the agent moves reversible work forward without asking, the human does not have to answer the agent's approval requests, such as "May I?", on the spot. All the human needs to do is read the result and the reason that the agent showed, at a convenient time, and decide.

Summary

- **Guard the Context Window.** It protects the agent's context. The agent has subagents read large content, such as logs thousands of lines long, and leaves only a summary in the main thread. The test is whether content is worth the cost, not whether the agent reads less, so the agent puts templates used every time in the `SKILL.md` body even if `SKILL.md` gets longer.
- **Never Block on the Human.** It protects the human's attention. The agent proceeds with reversible work, such as writing code, and then shows the result and the reason. It checks with the human only for irreversible operations, such as a force-push or deleting production data, and for product direction, such as which feature to build.
- **Protecting context and attention.** The human does not need to sit and watch. The human needs only to read the result and the reason that the agent showed, and then decide.

The next chapter, [Chapter 21](#), covers the Meta principle "[Encode Lessons in Structure](#)".

Chapter 21: Meta: the principle that improves the system itself

kaito • Sources: 日本語 • English • 2026-10-03

This chapter covers the following principle.

1. Encode Lessons in Structure

This principle asks the agent to build fixes that keep coming back into structure instead of written instructions. A fix that keeps coming back is a fix for the same mistake, made again and again. Structure here means tools, code, metadata, and automation.

When the agent ends up writing the same caution twice, it stops adding that caution as text and **turns it into a lint rule or a runtime check so that a machine enforces it**.

Runtime check means code that verifies, while the program runs, that a rule holds.

For example, code that verifies a date has the form 2026/09/26 right before the program displays it looks like this.

```
// The date string to display on screen
declare const text: string;

// Runtime check: verify the date's form right before displaying it
if (!/^\\d{4}\\/\\d{2}\\/\\d{2}$/.test(text)) {
  throw new Error(`Wrong date format: ${text}`);
}
```

This code notices a date in the wrong form only when the program actually runs this line.

For example, the agent turns the caution "Do not import modules in internal/ from outside it" into a lint rule that checks imports.

The bundled guide's page [docs/guide/08-principles.md](#) sets this principle apart on its own as the meta principle.

The role of this principle differs from that of the other 22 principles.

The other principles decide what the agent does within individual pieces of work, such as fixing the cause instead of the symptom, or moving reversible work forward without waiting for confirmation.

"Encode Lessons in Structure", on the other hand, turns the lessons that the agent finds during work into the structure of the next piece of work, such as a lint rule.

This principle calls **every error, every human correction, and every unexpected result a learning signal**.

When the agent picks up a learning signal, it sends the signal to the right place, such as a lint rule for a fix that repeats. It does not stop at recording the signal. It either fixes the problem now or creates a concrete todo. For details, see ["Feedback loop: three stages that turn a correction into structure"](#).

In [the talk](#), poteto also said to keep the lessons from corrections in the repository in a reusable form, not only in the conversation ([Chapter 3](#), [Chapter 4](#)).

This chapter explains this one principle in this order: the rule, the trigger conditions, and how pstack itself uses it.

This chapter gives no request examples for its principle.

In this book, I limit request examples to those that appear in pstack's bundled guide or [README](#). If I made up request examples for principles that have none in the source, I might show uses that pstack does not intend.

How this chapter is organized

This chapter is organized as follows.

- Rule: turn a repeated correction into the strongest structure and delete the instruction
- Trigger conditions

- pstack itself is built with this principle
- Summary

Rule: turn a repeated correction into the strongest structure and delete the instruction

The principle "Encode Lessons in Structure" asks the agent to enforce repeated cautions with structure instead of written instructions.

When the agent notices it has written the same caution twice, it **builds structure that enforces the caution and deletes the written instruction**.

The principle asks for this because **readers often miss written instructions**.

A written instruction works only when the reader notices the sentence, remembers its content, and follows it.

Corrections that the agent received in the conversation of an earlier session do not carry over to a new session. Even if the agent answers "I'll remember that", that promise does not survive into the next session.

Structure works even when the agent does not remember. So when the agent replaces a written instruction with structure, the agent in the next session follows the same rule too.

Steps: delete the written instruction once it is structure

There are three steps.

1. Ask whether the instruction can become a lint rule, a metadata flag, a runtime check, or a script
2. If it can, turn it into structure and delete the instruction
3. If it cannot, because a machine cannot decide it and it needs judgment, make the instruction prominent and add an example of the failure that happens when nobody follows the instruction

Metadata flag means a value placed in a file's frontmatter.

For example, the `SKILL.md` of each pstack Principle has the following values in the frontmatter at the top of the file.

```
---
name: principle-encode-lessons-in-structure
disable-model-invocation: true
---
```

Of these, `disable-model-invocation: true` is the metadata flag.

This flag stops the agent from picking and running this Skill on its own from the content of the conversation. The value in the frontmatter decides this behavior, so the body does not need the sentence "Do not use this Skill on your own".

Turn an instruction a machine can check into a lint rule and delete the text

The caution from the start of this chapter, "Do not import modules in `internal/` from outside it", falls under step 2.

A rule file is a file, such as `AGENTS.md`, that holds rules for the agent. When a rule file contains the following sentence, the agent adds a lint rule that checks imports and deletes the sentence from the rule file.

`AGENTS.md`

```
Before: written as text in the rule file
- Do not import modules in internal/ from outside internal/
```

`eslint.config.js`

```
// After: add the following config and delete the sentence from the rule file (assumes typescript-
eslint is installed)
import tseslint from "typescript-eslint";

export default [
  {
    files: ["src/**/*.ts"],
    languageOptions: { parser: tseslint.parser },
    // Allow imports between files inside internal/
```

```

    ignores: ["src/internal/**"],
    rules: {
      "no-restricted-imports": ["error", {
        patterns: [{
          group: ["**/internal/**"],
          message: "Do not import modules in internal/ from outside internal/",
        }],
      }],
    },
  },
];

```

With this config, lint reports an error when a file outside `internal/` imports a module in `internal/`. So the rule file no longer needs the same caution.

When a machine can check an instruction, the structure enforces it, and the written instruction is no longer needed.

Make an instruction a machine cannot check prominent and add a failure example

In contrast, "Write error messages so that the user knows what to do next" falls under step 3.

A machine cannot easily judge whether a message is good, so the agent keeps this instruction as text in a prominent place and adds an example of a bad message.

AGENTS.md

```

## Must follow

- **Write error messages so that the user knows what to do next**
  - Failure example: "An error occurred." The user does not know what to do next

```

When a machine cannot check an instruction, the only option is to keep the instruction as text. So the agent needs to make it prominent and add a failure example.

If structure can fix it, use only structure

The principle asks that **if structure can fix a problem, the agent uses only structure.**

If the agent keeps both the structure and the instruction, someone has to manage the same rule in two places. When someone updates only one of them, the two disagree. So the principle asks the agent to delete the written instruction once it becomes structure.

For example, if the agent updates only the lint rule and forgets to update the sentence in the rule file, an agent that reads the rule file writes code that follows the old rule.

Writing the same instruction again and again is a symptom of missing structure

The source calls a written instruction a "symptom".

In this book, I read an instruction written again and again as a symptom of missing structure to enforce the rule.

Under that reading, the principle asks the agent **to build the missing structure, which is the cause, instead of adding more instructions, which are the symptom.**

Order of strength: pick the strongest structure available

When several kinds of structure are available, the agent **picks the strongest one the situation allows.**

From strongest to weakest, the four kinds of structure are as follows.

1. **Unrepresentable state.** Use types so that a state that breaks the rule cannot be written. A rule violation causes a compile error.
2. **Lint or banned APIs that fail CI.** When lint detects code that breaks the rule or an API that is banned, CI fails and the change cannot be merged.
3. **A canonical helper.** Provide a shared function that holds the required way of doing something, and have the code call that function. `formatDate()` later in this chapter is an example.
4. **A runtime check.** Verify the condition while the code runs. Even if code breaks the rule, nobody notices until that part actually runs.

The agent picks the strongest one because **the agent writes code by copying the patterns the surrounding code already uses** ([Chapter 4](#)).

If the agent puts in a weak guard, meaning a low-ranked kind of structure that enforces a rule, that weak guard becomes the template the next agent copies.

For example, suppose that only a runtime check enforces the date display format.

The code for the order list screen formats the date itself and, right before displaying it, verifies that the date has the form 2026/09/26. When the next agent builds the shipment list screen, it copies this code, formats the date itself in the same way, and adds the same runtime check.

```
type Order = { id: string; orderedAt: Date };
type Shipment = { id: string; shippedAt: Date };

// Order list screen: formats the date itself and verifies its form right before displaying it
function orderRow(order: Order): string {
  const d = order.orderedAt;
  const mm = String(d.getMonth() + 1).padStart(2, "0");
  const dd = String(d.getDate()).padStart(2, "0");
  const text = `${d.getFullYear()}/${mm}/${dd}`;

  if (!/^\\d{4}\\\\d{2}\\\\d{2}$/.test(text)) {
    throw new Error(`Wrong date format: ${text}`);
  }

  return `${order.id} ${text}`;
}

// Shipment list screen: the next agent copies the code above and adds the same formatting and the
// same runtime check
function shipmentRow(shipment: Shipment): string {
  const d = shipment.shippedAt;
  const mm = String(d.getMonth() + 1).padStart(2, "0");
  const dd = String(d.getDate()).padStart(2, "0");
  const text = `${d.getFullYear()}/${mm}/${dd}`;

  if (!/^\\d{4}\\\\d{2}\\\\d{2}$/.test(text)) {
    throw new Error(`Wrong date format: ${text}`);
  }

  return `${shipment.id} ${text}`;
}
```

On the other hand, if there is a canonical helper `formatDate()` and lint bans date formatting that bypasses the helper, the pattern the next agent copies is a call to `formatDate()`.

```
type Order = { id: string; orderedAt: Date };
type Shipment = { id: string; shippedAt: Date };
```

```
// date.ts: date formatting happens in one place only, the canonical helper formatDate()
function formatDate(d: Date): string {
  const mm = String(d.getMonth() + 1).padStart(2, "0");
  const dd = String(d.getDate()).padStart(2, "0");
  return `${d.getFullYear()}/${mm}/${dd}`;
}

// Order list screen
function orderRow(order: Order): string {
  return `${order.id} ${formatDate(order.orderedAt)}`;
}

// Shipment list screen: the next agent also copies the code above and calls formatDate()
function shipmentRow(shipment: Shipment): string {
  return `${shipment.id} ${formatDate(shipment.shippedAt)}`;
}
```

```
// Add the following config to eslint.config.js to ban date formatting code outside date.ts (assumes
typescript-eslint is installed)
import tseslint from "typescript-eslint";

export default [
  {
    files: ["src/**/*.ts"],
    languageOptions: { parser: tseslint.parser },
    ignores: ["src/date.ts"],
    rules: {
      "no-restricted-properties": ["error", {
        property: "getFullYear",
        message: "Use formatDate() to format dates",
      }],
    },
  },
];
```

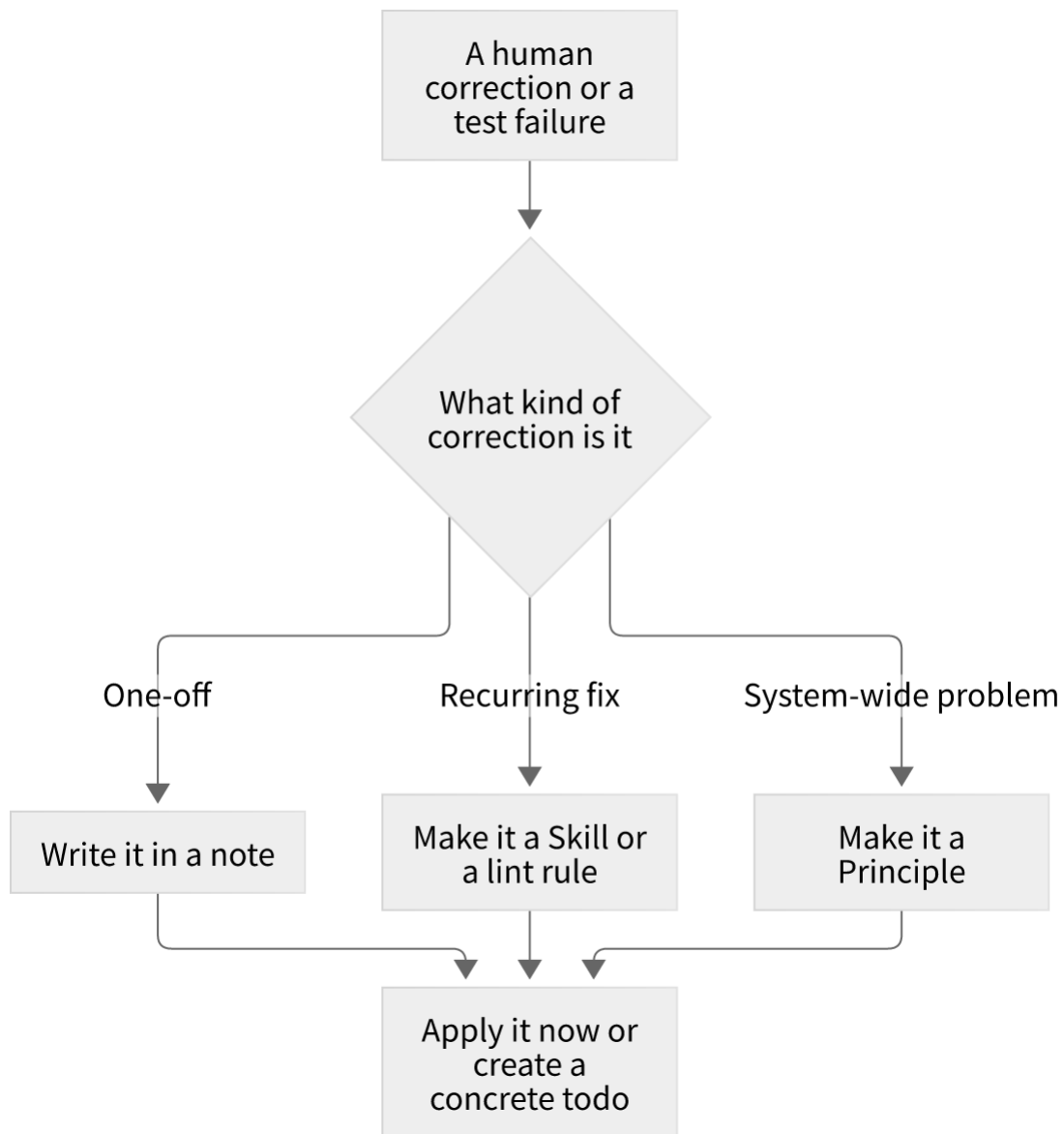
As a result, date formatting gathers in one place, `formatDate()`, and a new screen no longer adds another copy of the same runtime check.

Because the next agent copies the pattern of whatever structure is in place, the agent needs to pick the strongest structure available.

Feedback loop: three stages that turn a correction into structure

The agent turns a correction into structure in the following three stages.

- **Catch the correction.** When a human steps in and fixes the work, or when a test fails, the agent decides whether the correction is a one-off or a recurring pattern.
- **Route to the right layer.** It sends a one-off to a brain note, a recurring fix to a Skill or a lint rule, and a system-wide problem to a Principle.
- **Finish handling the correction.** Instead of only recording it, the agent either applies it now, for example by adding the lint rule, or creates a concrete todo such as "Add a lint rule that bans imports of `internal/`".



[View diagram 1](#)

If the process stops at any of the three stages, the correction does not reach the next piece of work.

The source lists the following three anti-patterns to avoid.

- **Acknowledging without recording.** Even if the agent answers "I'll remember that", those words do not survive into the next session.

- **Recording without routing to the right layer.** Suppose the agent writes the lint rule it should build into a note but never implements the rule. CI then does not fail when the next agent makes the same mistake, and a human has to give the same correction again. So the note is wasted.
- **Fixing one instance without generalizing.** If the agent fixes only the one place that imported `internal/` from outside and adds no lint rule, the next agent writes the same import somewhere else. A fix to one instance leaves in place the pattern that produces the same mistake.

Trigger conditions

When **the agent catches itself writing the same instruction a second time**, or notices that **the same correction keeps coming back**.

This principle differs from two others that also lead to a script

This principle and two others, "[Build the Lever](#)" and "[Prove It Works](#)", can all lead the agent to build a script.

The source of "[Build the Lever](#)" distinguishes the three principles as follows.

- "[Build the Lever](#)" finishes the task at hand faster and in a form that is easier to review
- "[Prove It Works](#)" turns verification itself into a script
- "[Encode Lessons in Structure](#)" turns a repeated instruction into a lasting guardrail

In this book, I sort the three principles by the question the script answers.

- **"How do I finish this task in front of me?"** That is "[Build the Lever](#)" from [Chapter 17](#). An example is a codemod that renames in bulk.
- **"Did this task finish correctly?"** That is "[Prove It Works](#)" from [Chapter 19](#). An example is a script that checks the lines of a written file.
- **"How do I keep the same mistake from ever happening again?"** That is "[Encode Lessons in Structure](#)". An example is a lint rule that bans imports of `internal/`.

pstack itself is built with this principle

pstack also keeps its own lessons in structure, not in text. The main examples follow.

- `/reflect` (**Chapter 33**). `/reflect` gathers lessons from conversations. It does not add a lesson to a Skill's text when lint or a script can enforce the lesson more reliably. Instead, it files such lessons in the team backlog as tasks to implement the lint rule or script.
- `preferences.md`. The "Orchestrate" Playbook (**Chapter 16**) writes standing instructions, such as which model to use or which paths not to touch, in this file, one per line. The Playbook pastes them as is into every subagent when the subagent starts and when it resumes. Instructions tend to drop out at each resume, and every time one drops out, the human has to give it again. When the agent **notices it is repeating the same instruction, it adds that instruction to this file as one line before it continues the work.**

Summary

- **Rule.** When the agent writes the same instruction twice, such as "Do not import modules in `internal/` from outside it", it asks whether the instruction can become structure such as a lint rule or a runtime check. If it can, the agent builds the structure and deletes the written instruction. It picks the strongest structure available. From strongest to weakest, the order is types, lint that fails CI, a canonical helper, and a runtime check. For an instruction that cannot become structure, it makes the instruction prominent and adds a failure example.
- **Trigger conditions.** When the agent writes the same instruction a second time, or notices that the same correction keeps coming back.
- **How pstack itself uses it.** `/reflect` files lessons that can become structure, and the "Orchestrate" Playbook pastes the standing instructions in `preferences.md` into every subagent each time.

In the talk, poteto said that trust grows with the amount of verification methods, work procedures, and constraints left in the repository, not with the amount of conversation. Of these three, "Encode Lessons in Structure" adds to the constraints, such as lint rules and types, with every correction.

This ends [Part IV](#). [Part V](#) covers Skills, the concrete tools that Playbooks and Principles call. The first chapter of that part, [Chapter 22](#), looks at four Skills. Three of them explain how code behaves and why before a change. The fourth resumes work where the last session left off.

Part V: Extend what the work can do with Skills

kaito • Sources: 日本語 • English • 2026-10-03

Part V covers all 24 of pstack's Skills.

You can skip this part. I recommend reading only the chapters you care about.

You can read the source of each Skill at any time in the [pstack](#) repository. When you need one, look up the matching chapter and file.

What Part V tells you

Part V answers the following questions about each Skill.

- What each Skill does and when to use it. For example, `/how` explains what the current code does before you change it.
- What shape of output it returns. For example, `/why` labels each reason it finds with a level that says whether a record backs it or it is a guess.
- Example requests that call it directly, and how to choose between similar Skills. For example, use `/interrogate` to look for weak points inside the diff, and `/blast-radius` to look for what breaks outside the diff.

A Skill packages the steps of a task under a name, so that each call by that name runs the same steps. The tasks include, for example, investigating with `/why`, designing with `/architect`, verifying with `/blast-radius`, and cleaning up with `/unstop`.

`/poteto-mode` calls most of them automatically, as steps of a Playbook.

The user can also call a Skill directly by name. The user calls one directly when they want to change the Skill choice the Playbook made, or when they want a more careful check than the default.

For example, say the diff is small enough that the `/poteto-mode` steps do not call `/blast-radius`, but you do not fully trust the diff. You can call `/blast-radius` directly and verify what that diff could break.

To call a Skill directly, the user needs to know what it does. The chapters of Part V explain what each Skill does.

The chapters in this part

Chapter	Role	Skills covered
Chapter 22: Understand the codebase and past work	Investigate what the current code does and why it has its current shape, and resume where the last session stopped	<code>/how</code> , <code>/why</code> , <code>/teach</code> , <code>/recall</code>
Chapter 23: Design a Playbook for work that no Playbook fits	For work that no Playbook fits, design a procedure that fixes the finish condition and the verification setup first	<code>/figure-it-out</code>
Chapter 24: Run agents in parallel and compare designs and artifacts	Fix the types and module boundaries before implementation, have several agents solve the same problem and compare the results, and split work across several agents	<code>/architect</code> , <code>/arena</code> , <code>/swarm</code>
Chapter 25: Extend pstack to your own way of working and to Grok Bot	Build a mode Skill for one user from that user's chat history, and build a page that starts Grok Bot with a button	<code>/automate-me</code> , <code>/make-bot-ui</code>
Chapter 26: Build verification and keep it current as the app changes	Build a project-specific Skill, called a verification skill, that runs the app along the same path a user takes, and fix it as the app changes	<code>/create-verification-skill</code> , <code>/maintain-verification-skill</code>

Chapter	Role	Skills covered
Chapter 27: Check what a change could break and where the diff is weak	Look for what could break outside the changed files, and have several models look for weak points in the diff	<code>/blast-radius</code> , <code>/interrogate</code>
Chapter 28: Record decisions and leave evidence for long agent runs	Record decisions and their grounds, and have a model from a different family review them	<code>/show-me-your-work</code>
Chapter 29: Keep code quality with 16 TypeScript rules	Make invalid states impossible to write in the types, and validate outside data once, at the place it enters	<code>/typescript-best-practices</code>
Chapter 30: TDD: verify with a failing test before the fix	Before you fix a bug, write a test that fails before the fix	<code>/tdd</code>
Chapter 31: Remove comments and tidy the codebase	Have an agent that did not write the diff review its unneeded comments, and delete them	<code>/no-comments</code>
Chapter 32: Keep writing quality high	Remove AI habits from prose, restate a hard-to-follow reply without jargon, and pick the kind of document before you write	<code>/unslop</code> , <code>/bro</code> , <code>/technical-writing</code>
Chapter 33: Reflect on what the work taught you and apply it	Keep what a long task taught, for example by adding it to an existing Skill	<code>/reflect</code>
Chapter 34: Install pstack and choose a model for each role	Install pstack and set the model pstack uses for each role	<code>/setup-pstack</code>
Chapter 35: <code>/poteto-mode</code> is pstack's Router	Lay out the rules that pick the Playbook that fits a request and load the Skills and Principles it needs	<code>/poteto-mode</code>

This book orders the chapters along the flow of the work. The flow is as follows.

- Understand the current code ([Chapter 22](#))
- Run several agents in parallel and compare designs and artifacts ([Chapter 24](#))
- Implement, and verify that the implementation works ([Chapter 26](#))
- Verify what the change could break ([Chapter 27](#))

- Record decisions ([Chapter 28](#))
- Clean up the code (Chapters 29 to 31)
- Clean up the prose ([Chapter 32](#))
- Keep what you learned ([Chapter 33](#))

[Chapter 23](#) covers the Skill that designs the way of working itself, for work that no Playbook fits. That Skill deals with how the whole task proceeds, not with one step in the flow. So this book treats it apart from the flow.

[Chapter 25](#) collects two Skills that extend pstack to fit the user's habits and to work with Grok Bot. Neither Skill belongs to a particular step in the flow, so this book treats them apart from the flow.

[Chapter 34](#) covers `/setup-pstack`, which sets the models. [Chapter 35](#) covers `/poteto-mode`, which receives the request first.

The pstack [README](#) sums up how to get started as "pick models with `/setup-pstack`, and use `/poteto-mode` for work that requires rigor. That's it." The README also says that `/poteto-mode` uses the other Skills as needed.

`/setup-pstack` is the first Skill you use when you start with pstack, and `/poteto-mode` is the first Skill you use each time you start a task. Neither Skill belongs to a particular step in the flow, so this book covers them in chapters apart from the flow.

Quick reference for all Skills

Skill	Summary	When to use it	Who calls it	Chapter
<code>/how</code>	Explains what the current code does: what triggers the work, which functions it passes through, and how the data flows	When the user or the agent wants to know how a subsystem behaves before a code change, or asks where code should live	<code>/poteto-mode</code> calls it in the "Non-negotiables" rules and in the steps of Playbooks such as "Investigation", "Bug fix", "Feature", and "Refactoring". <code>/teach</code> , <code>/architect</code> , <code>/no-comments</code> , and the user also call it	Chapter 22
<code>/why</code>	Investigates why the code has its current shape from records such as commits, PRs, and tickets, and answers with the reasons that a record backs, kept apart from guesses	When the user or the agent asks about the reason for a design, the history of a regression, or the origin of a threshold	<code>/poteto-mode</code> calls it in the steps of the "Investigation" and "Bug fix" Playbooks. <code>/teach</code> , <code>/recall</code> , <code>/architect</code> , <code>/no-comments</code> , and the user also call it	Chapter 22
<code>/teach</code>	Calls <code>/how</code> and <code>/why</code> , and combines what the code or change is, how it works, and why it was built that way into one plain explanation	When a summary is not enough and the user wants to understand a change or a subsystem in depth	Only when the user calls it by name	Chapter 22
<code>/recall</code>	Gathers the history of recent work from your own chat history and shared records, and returns a short note on how far the work got and what to do next	When the user wants to know where the last session stopped, before they start or resume work	Only when the user calls it by name	Chapter 22

Skill	Summary	When to use it	Who calls it	Chapter
<code>/blast-radius</code>	Looks for what could break outside the changed files, and runs the code to verify the grounds for calling the change safe	When the user wants to know what else a diff could break, for a diff that looks small but that they do not fully trust	Only when the user calls it by name	Chapter 27
<code>/interrogate</code>	Has reviewers on different models look for weak points in the same diff, and the main agent sorts their findings into four groups	When the user or the agent wants several models to look for where a diff breaks, strict code-quality concerns included	<code>/poteto-mode</code> calls it in the "Non-negotiables" rule that applies before a contested design merges, and in the steps of the "Feature" and "Opening a PR" Playbooks. <code>/architect</code> and the user also call it	Chapter 27
<code>/figure-it-out</code>	For work that no Playbook fits, designs a procedure that fixes the finish condition and the verification setup first and keeps a record of decisions as it goes	When the user hands off a migration that spans many call sites or work to continue while they are away, or when no Playbook fits the request	<code>/poteto-mode</code> calls it through the rules in the "Playbooks" section, which send large work, away-from-desk work, and work that no Playbook fits to this Skill. The user also calls it	Chapter 23
<code>/architect</code>	Before the agent writes code that crosses function boundaries, it fixes the types, function signatures, and module boundaries in a skeleton with no function bodies	When the agent is about to write code that crosses function boundaries	<code>/poteto-mode</code> calls it in the "Non-negotiables" rules and in the steps of the "Feature", "Bug fix", "Refactoring", and "Perf issue" Playbooks. <code>/figure-it-out</code> , <code>/no-comments</code> , and the user also call it	Chapter 24

Skill	Summary	When to use it	Who calls it	Chapter
/arena	Has N candidates solve the same problem in parallel, picks the best candidate as the base, and ports the good ideas from the others	When the user or the agent builds an artifact that is costly to change later, such as the format of a cache key	/poteto-mode calls it in the "Non-negotiables" rules and in the steps of the "Feature", "Eval", and "Orchestrate" Playbooks. /architect , /blast-radius , and the user also call it	Chapter 24
/swarm	Runs N Workers in parallel that either split the work or compete, and combines their results into one report	When the agent splits work in parallel for coverage, competition, a gate, or a divided search	/poteto-mode calls it in the "Non-negotiables" rules and in the steps of the "Multi-phase or multi-PR plan" and "Autopilot-full" Playbooks. The user also calls it	Chapter 24
/automate-me	Builds a -mode Skill for one user from that user's chat history and answers to questions	When the user wants the agent to follow a mode Skill that fits their preferences and way of working	Only when the user calls it by name	Chapter 25
/make-bot-ui	Builds a page whose button starts Grok Bot over a webhook, without showing the sender key to the agent	When the user wants to build a page that starts Grok Bot over a webhook, provide the sender key, and expose the page on Tailscale	Only when the user calls it by name	Chapter 25

Skill	Summary	When to use it	Who calls it	Chapter
<code>/create-verification-skill</code>	Generates a project-specific verification skill in <code>.cursor/skills/verify-<app>/</code> that starts and drives the app and keeps evidence	When the user's project has no scripted way to prove the app's behavior again and again	The user calls it. <code>/setup-pstack</code> also calls it in its last step, which offers to generate a verification skill, when the user accepts	Chapter 26
<code>/maintain-verification-skill</code>	Checks whether the verification skill and the Feature Map disagree with the current app, and fixes the drift	When the user has changed the app and wants to fix the drift between the Feature Map and the current app	Only when the user calls it by name	Chapter 26
<code>/show-me-your-work</code>	Records the decisions and grounds of a long task in <code>decisions.tsv</code> , and has a model from a different family review that record	When the agent runs a long task, an autonomous task, or a task the user checks later after stepping away	<code>/poteto-mode</code> calls it in the "Non-negotiables" rules and in the steps of Playbooks such as "Autonomous run", "Orchestrate", "Hillclimb", and "Autopilot-full". <code>/figure-it-out</code> and the user also call it	Chapter 28
<code>/typescript-best-practices</code>	Shows, in sixteen TypeScript rules, how to make invalid states impossible to write in the types	When the agent reads or edits a <code>.ts</code> or <code>.tsx</code> file	Loads automatically when the agent reads or writes a <code>.ts</code> or <code>.tsx</code> file. The user does not need to call it by name	Chapter 29
<code>/tdd</code>	Writes one regression test before the agent fixes a bug, and verifies that the test fails before the fix and passes after it	When the user asks for TDD or a failing test, or when a test that runs locally right away can verify the bug	<code>/poteto-mode</code> calls it in step 5 of the "Bug fix" Playbook. The user also calls it	Chapter 30

Skill	Summary	When to use it	Who calls it	Chapter
/no-comments	Has Comment Sicko, a subagent that did not write the comments, review the comments in the diff, and fixes the findings it accepts	When the agent removes comments from the diff before review	/poteto-mode calls it in the "Non-negotiables" rules and in the steps of the "Opening a PR", "Autopilot-full", " Autopilot-stack ", and "Multi-phase or multi-PR plan" Playbooks. The user also calls it	Chapter 31
/unslop	Uses numbered rules to remove, from any prose, the habits that show an AI wrote it	When the agent writes any prose, replies included	/poteto-mode calls it in the "Non-negotiables" rules and in the steps of the "Investigation", "Opening a PR", and "Multi-phase or multi-PR plan" Playbooks. /technical-writing , /show-me-your-work , /recall , /teach , /blast-radius , /automate-me , and the user also call it	Chapter 32
/bro	Restates the agent's last reply without jargon	When the agent's reply is detailed but the user cannot tell what it says	Only when the user calls it by name	Chapter 32
/technical-writing	Sets the standard for writing technical documents in four layers, from the kind of document down to how to write one sentence	When the agent writes or reviews docs, RFCs, READMEs, PR descriptions, or commit messages	/poteto-mode calls it in the "Non-negotiables" rules and in the steps of the "Opening a PR" and "Multi-phase or multi-PR plan" Playbooks. The user also calls it	Chapter 32

Skill	Summary	When to use it	Who calls it	Chapter
<code>/reflect</code>	Pulls lessons that carry over to the next task out of the conversation, and keeps only the ones the user approves, for example by adding them to an existing Skill	When the user calls <code>/reflect</code> right after a task that taught something	Only when the user calls it by name	Chapter 33
<code>/setup-pstack</code>	Asks for the reasoning budget and the model for each role, and writes the answers to the always-applied rule ~/.cursor/rules/pstack-models.mdc	When the user starts using pstack, or wants to change the model pstack uses for a role	The user. <code>SKILL.md</code> has no <code>disable-model-invocation: true</code> , so the agent can also pick and run it from the content of a request, such as "configure pstack models"	Chapter 34
<code>/poteto-mode</code>	Collects the rules that pick the Playbook that fits a request and have the agent load the Skills and Principles it needs	When the user requests work that needs rigor, such as work where the agent reproduces a bug before it fixes it and verifies that the fix works	Only when the user calls it by name. Once called, it stays in effect for later turns of the same conversation until the user says to stop	Chapter 35

Chapter 22: Understand the codebase and past work

kaito • Sources: 日本語 • English • 2026-10-03

This chapter covers the following four Skills.

1. `/how`
2. `/why`
3. `/teach`
4. `/recall`

All four Skills investigate the current code and past work. **What you want to know decides which Skill you use.**

Choose as follows.

- **How it works now.** Use `/how`, for example to learn which functions a run cancellation passes through and how it changes the database.
- **Why it took that shape.** Use `/why`, for example to learn why the retry limit is five.
- **Both of those, as an explanation a person can follow.** Use `/teach`, for example when you want to understand how this PR changes retries until you are convinced.
- **Where the last session left off.** Use `/recall`, for example to learn how far last week's export work got and what was reverted.

[Chapter 27](#) covers the Skills that look at what a finished change could break and at the weak points of a diff.

This chapter first lists the four Skills. It then explains each one from three angles: when to use it, its procedure, and its output. For the Skills whose source has example requests, I also show those examples.

At the end, I summarize how to choose between similar Skills and a Playbook.

How this chapter is organized

This chapter is organized as follows.

- The four Skills at a glance
- `/how` explains code behavior before a change so that you can trace it in your head
- `/why` answers why code has its current shape, and separates reasons the records back from guesses
- `/teach` explains how code works and why, in plain terms, until a person is convinced
- `/recall` gathers the history of recent work so that you can resume from the last point of failure
- Choose among `/how`, `/why`, `/teach`, `/recall`, and "Session pickup" by the question you hold now
- Summary

The four Skills at a glance

Skill	In one line	Main question
<code>/how</code>	Explains the current code's behavior: what starts the process, which functions it passes through, and how data flows	How does X work? Where should this live?
<code>/why</code>	Investigates why the code took its shape from records such as commits, PRs, and tickets, and answers with a confidence level for each reason	Why is it like this? Why did we pick Y?
<code>/teach</code>	Calls <code>/how</code> and <code>/why</code> and combines their results into one explanation that starts from plain definitions	I really want to understand this
<code>/recall</code>	Rebuilds how far recent work got and what went wrong, from your own chat history and shared records such as PRs, tickets, and production errors	How far did I get? What was going on?

All four are **Skills that run only when someone calls them by name**.

Each of the four `SKILL.md` files has a setting, `disable-model-invocation: true`, that stops the agent from picking the Skill on its own from the conversation. So the four run only when the user calls them by name, or when `/poteto-mode` or another Skill names them in its procedure.

The callers of the four Skills split as follows.

- `/how` **and** `/why`. `/poteto-mode` calls them as part of a Playbook's procedure. It also calls `/how` under the rules in the section "**Non-negotiables**". Other Skills call them in their own procedures too. `/teach`, `/architect`, and `/no-comments` call `/how`, and `/teach`, `/recall`, `/architect`, and `/no-comments` call `/why`. The user can also call them directly by name.
- `/teach` **and** `/recall`. They appear in none of the "Non-negotiables" rules of `/poteto-mode`, no Playbook procedure, and no other Skill's procedure. So they run only when the user calls them by name.

`/how` **explains code behavior before a change so that you can trace it in your head**

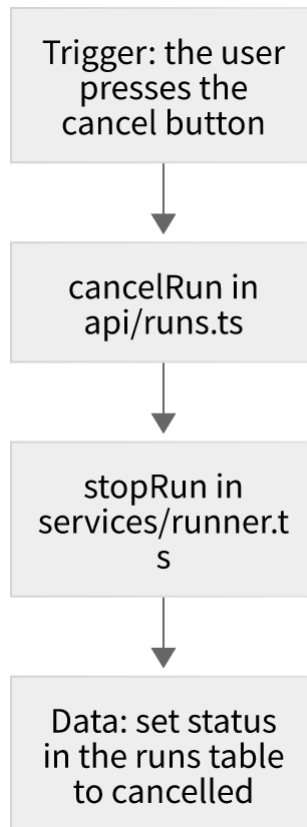
`/how` is a Skill that explains code behavior **at the level where a senior engineer new to the subsystem can replay it in their head**.

Subsystem: the part of an app that handles one coherent feature, such as run cancellation, login, or payments. It often spans several files or modules.

To replay it in your head means that the reader can trace what starts the process, which function in which file it passes through, and where the data flows, without opening the code.

The goal of the explanation is that the reader grasps the flow. So the explanation does not go into line-by-line detail, like comments attached to every line of source code.

For run cancellation, for example, the goal is that you can explain a flow like the one below without opening the code. The file and function names are examples for the explanation.



[View diagram 1](#)

When to use it: explaining code behavior and asking where something belongs

Use it before you change code, when you want to know how a subsystem behaves. For example, you may want to know which functions a run cancellation passes through, in order. Also use it for questions about where code belongs, such as "Where should this live?" or "Is this the right layer?" A layer here is a slice of code split by role, such as the UI, the API, or database access.

When you ask about motive, "Why did we build it this way?", use `/why` instead of `/how`.

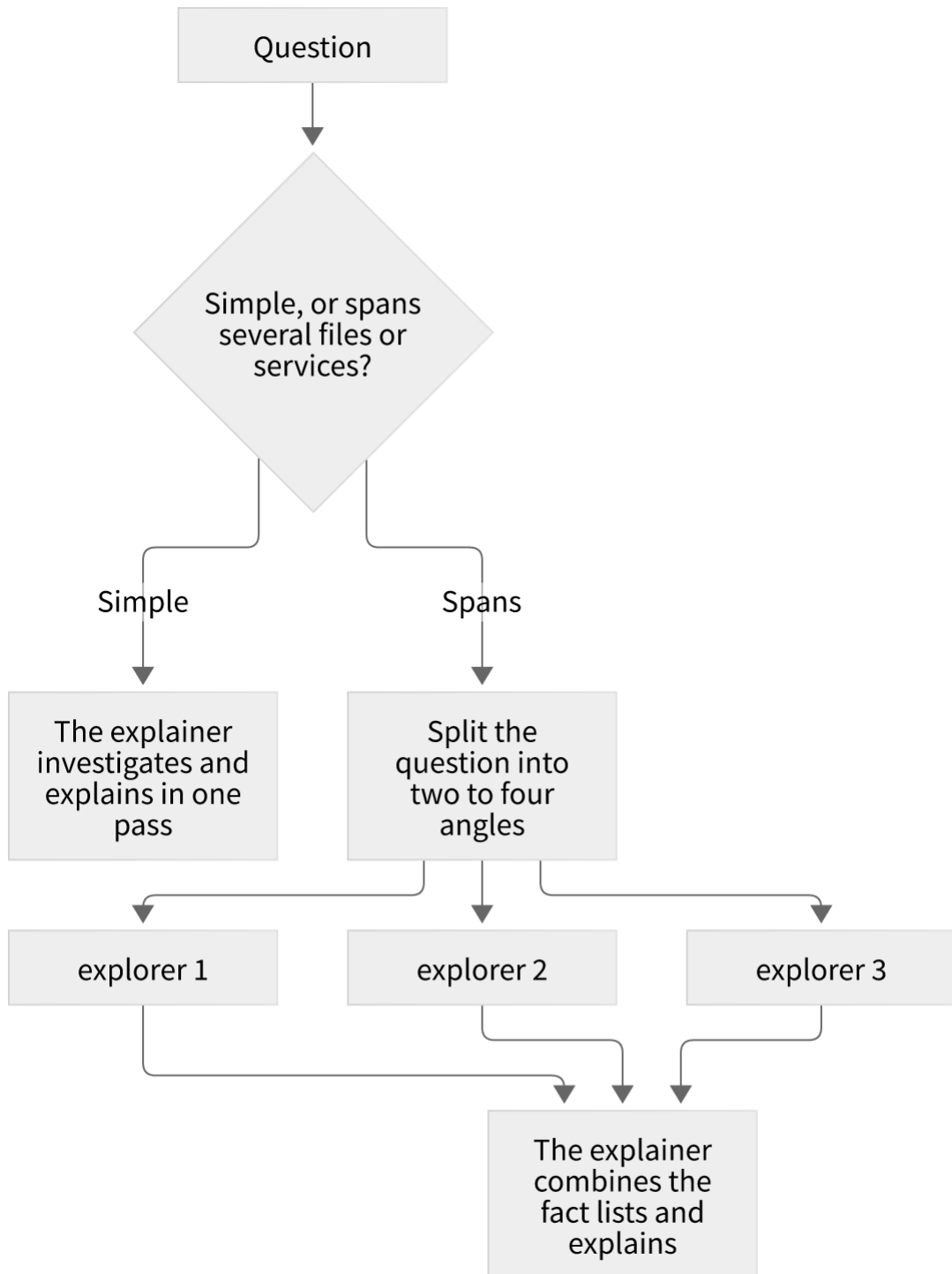
Procedure: separate the role that gathers facts from the role that writes the explanation

If the question is simple, for example a question confined to one module such as "How does function X work?", an **explainer** investigates and explains in one pass.

If the question spans several files or services, for example the structure of a whole subsystem, the agent splits the question into two to four angles and **starts explorers in parallel**. The explainer then combines their results.

An angle is a part of the same subsystem that does not overlap with the other parts when you investigate it separately. For run cancellation, for example, you split it into "accepting the cancellation", "stopping the run", and "updating the database".

If the agent is unsure whether a question is simple or complex, it treats the question as simple and investigates with the explainer alone, without starting explorers.

[View diagram 2](#)

On either path, the explainer writes the explanation the user reads. The explorers gather only the facts that the explainer builds the explanation from.

An explorer returns a list of facts instead of prose: the components it found, the flow of processing, the files it read, the boundaries with other parts, and anything surprising.

The explainer writes the human-readable explanation separately, so an explorer ignores readability and puts depth and accuracy first.

An explorer does not fill unclear points or untraceable spots with invention. It writes down what it could not find out, for example "I could not tell how X connects to Y."

`/how` explains and changes nothing, so every `/how` subagent runs in a read-only mode that does not change files.

Output: write under five headings, from Overview to Gotchas

The goal of the explanation is that a person who does not know the area can start work with confidence after reading it.

So the explainer writes under the following five headings, which move from the big picture to the details and then to where work starts. It leaves out any heading that does not apply. The examples assume someone asked "How does run cancellation work?"

- **Overview.** What the mechanism is, what it does, and why it exists. The reader can read only this part and decide whether to read on. For example, the cancellation mechanism handles everything from the user's action to stopping the run.
- **Key Concepts.** The types and services the reader needs to follow the rest. Examples are the type that represents a run and the service that accepts a cancellation.
- **How It Works.** The longest part. It covers the trigger, the progression, and the data flow. For example, it shows which functions a cancellation request passes through in order and how the request changes the state in the database.
- **Where Things Live.** A short list of which files and directories hold what. It lists only what a person starting work in the area needs to open. For example, it lists `api/runs.ts`, which accepts the cancellation, and `services/runner.ts`, which stops the run.
- **Gotchas.** Points that are hard to notice from reading the code alone, surprising behavior, how the code got its current shape, and easy mistakes. For example, a cancellation does not stop a running process at once. The process stops after it

finishes its current stage. So the process keeps going for a few seconds after the UI shows "Cancelled".

When processing spans several components, the explainer also adds a diagram. The diagram exists only to make the flow clearer. If the text alone gets the flow across, the explainer leaves the diagram out.

Example requests: ask the question you actually have

Many Playbooks call `/how` in their procedures. The main ones are the following.

- "Bug fix", which fixes a defect
- "Feature", which adds a feature
- "Refactoring", which reorganizes structure without a change in behavior

The "Non-negotiables" of `/poteto-mode` also say to use `/how` for a non-obvious change, for a design decision, and when you check "Is this really fine?"

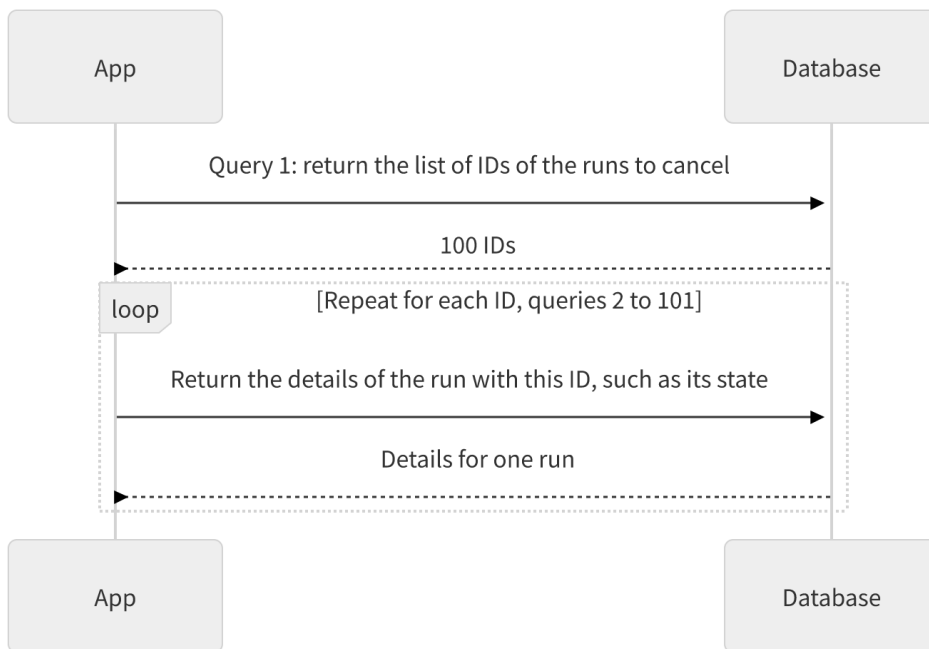
Besides Playbooks, three Skills also call `/how` in their procedures: `/teach`, `/architect`, which decides a design ([Chapter 24](#)), and `/no-comments`, which has another agent review the comments in a diff ([Chapter 31](#)).

When you call it directly, ask the question you actually have. The example in the [README](#) is the following.

```
/how do we cancel runs? do we have an n+1 when we look up every run to cancel?
```

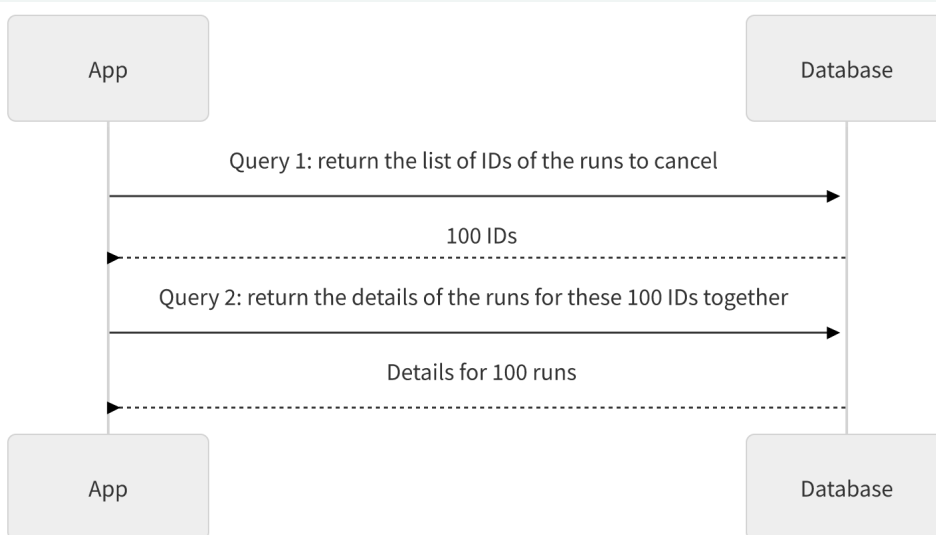
N+1: a way of writing queries where one query fetches a list and then one extra query runs for each item.

For example, when there are 100 runs to cancel, the app queries the database 101 times, as shown below. Each query adds waiting time, so the more items there are, the slower the processing gets.



View diagram 3

To remove the N+1, the app stops querying once per ID and passes all 100 IDs together in the second query. The database returns the details for all 100 at once, so the total is two queries, as shown below.



[View diagram 4](#)

The bundled guide [03-understand.md](#) covers `/how`, `/why`, `/teach`, and `/recall`. The guide warns the user not to skip these Skills on the thought that "the agent will read the code anyway".

The guide gives this warning because **an agent that starts editing without tracing the code's behavior tends to fix only the symptom at the first plausible spot it finds, instead of the root cause**. If you fix only the symptom, the root cause stays and the next defect appears ([Chapter 19](#)).

The same page also says that a call to `/how` up front costs less than the work that the next defect causes.

`/why` answers why code has its current shape, and separates reasons the records back from guesses

`/why` is a Skill that answers why code has its current shape, and **separates the reasons that records back from guesses**.

`/how` answers "How does it work?" `/why` answers "What circumstances and decisions produced that shape?"

If `/why` investigates and finds no reason, it answers "unknown" instead of filling the gap with a guess. **If the answer states a guess with confidence, the user takes the guess as fact and makes decisions based on it.**

When to use it: asking about design reasons, regressions, and where thresholds came from

Use it when you ask about a design reason, such as "Why does it work this way?" or "Why did we pick Y?" Also use it when you investigate a regression, a postmortem, or where a threshold came from, for example why the retry limit is five.

If you are asking about runtime behavior, meaning how it works now, use `/how` instead of `/why`.

Procedure: put one investigator on each place evidence lives, and investigate in parallel

/why splits the places where evidence of a reason is likely to remain into the seven categories in the following table.

Category	Example MCP	The "why" it reveals best
Source control history	git, gh	The reasons at implementation time that someone wrote during review, such as a PR description. Source control is the only place available in every environment, so /why always checks it
Issue and ticket tracking	Linear, Jira	Product or business motives, such as "added at a customer's request"
Long-form documents	Notion, Google Docs	Design reasons written before the code existed, such as a design document that compares option A and option B
Real-time chat	Slack	Discussions that never made it into a document
Infrastructure observability	Datadog	Runtime conditions that led to a timeout or a retry
Error tracking	Sentry	The exceptions that led to defensive code, such as a null check or a retry
Product analytics	BigQuery	The product or data conditions that decided the code's shape, such as where a threshold number came from

The agent first checks which MCP servers are connected to the user's Cursor and assigns each one to the category of evidence it can read.

For example, if the Linear MCP is connected, the agent assigns it to "issue and ticket tracking". If the Slack MCP is connected, the agent assigns that MCP to "real-time chat".

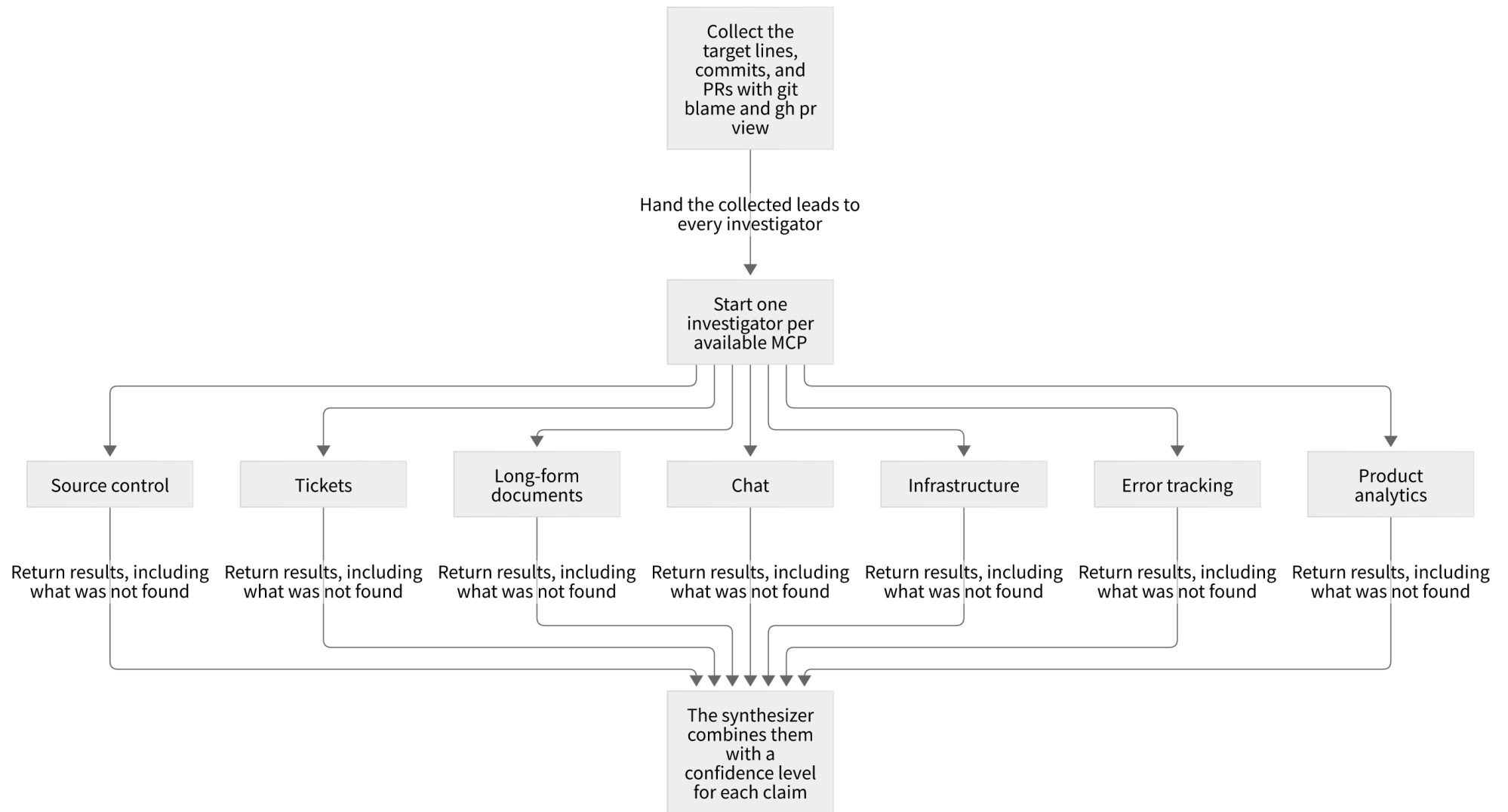
Then the agent starts one **investigator** in parallel for each category that has a connected MCP. /why always investigates source control history, because git and gh can read it without an MCP.

Before it starts the investigators, the agent collects the target lines, commits, and PRs with commands like the following. It hands the collected leads to every investigator, so that each one can start from concrete code.

```
# Show the commit that last changed the target lines
git blame -L <start>,<end> <file>

# Show the PR description and discussion
gh pr view <number> --json
title,body,author,createdAt,mergedAt,labels,closingIssuesReferences,comments,reviews
```

A **synthesizer** combines the investigators' results. The flow is as follows.



View diagram 5

The investigators search in parallel. If one finds nothing, it returns the result "not found" to the synthesizer.

The synthesizer also records, with reasons, the categories it did not check and the categories where the search found nothing. **"I searched and found nothing" is one result of the investigation. Without that record, the user cannot tell whether a category went unchecked or was checked and held nothing.**

Output: sort every claim into five confidence levels, from Direct to Unknown

From code, you can read what it does but not why it exists. The reasons live only in incomplete records, such as commits, PRs, tickets, and chat.

So `/why` sorts every claim into the following five levels by how certain it is.

The examples assume an investigation into why the retry limit is five.

Level	Meaning	Example
Direct	A record exists where the author stated the reason explicitly	A code comment says "Stop at five, because the upstream API rejects the sixth retry and later ones"
Supported	No explicit statement, but several pieces of indirect evidence agree	The PRs and tickets from the same week as the PR that set the limit to five all mention the same outage
Inferred	A reasonable reading of the context, with no explicit backing	The limit is five elsewhere in the codebase too, so it probably follows a team convention
Speculative	Thin evidence, and other explanations hold up about as well	It may match the SLA, but the value five does not appear in the SLA document
Unknown	The investigation found nothing. The entry states what the agent searched and where	Searched the tickets for "retry" and read six PRs that changed this file, but found no reason

The synthesizer **uses assertions such as "therefore X" only for Direct and Supported claims**. Readers take the assertive form as a claim of cause and effect that has evidence.

For example, a Direct claim asserts "This limit exists because the upstream API rejects the sixth retry and later ones" and attaches the source.

An Inferred claim, on the other hand, uses wording that shows it is an inference, such as "This limit probably follows a team convention."

Even when the user puts a hypothesis into the question, such as "I think it's for performance", `/why` does not accept the hypothesis as is. It treats the hypothesis as one candidate and checks the evidence on its own.

In [Part 2](#) of The Complete Guide to pstack, poteto writes that **when you ask an agent for something and do not state your own hypothesis first, you do not steer the agent in the wrong direction**.

If the user plans to change the code after this question, `/why` **turns the findings into Preserve, Change, Avoid, and Risk constraints, so that they can go into the change plan**.

For example, if you investigated the reason for the retry limit because you plan to change it, the constraints look like this.

```
Preserve: Do not exceed the number of retries the upstream API rejects (a record gives this as the
reason for the limit)
Change:   Do not hard-code the limit. Decide it in the same place as the upstream API's limit
Avoid:    Removing the limit and retrying until it succeeds
Risk:     If the upstream API's limit has changed, the current value of five no longer fits
```

When you split the findings into these four constraints, you do not unknowingly delete a reason that must stay when you plan the change.

Example request: "nobody wrote down the reason" is also an answer

The "Investigation" Playbook and the "Bug fix" Playbook call `/why`. The "Investigation" Playbook answers, with citations, a question that the code or its history can answer. `/recall`, `/teach`, `/architect` ([Chapter 24](#)), and `/no-comments` ([Chapter 31](#)) also call `/why` in their procedures.

03-understand.md has the following example request.

```
/why was the retry limit set to five? does the reason still hold?
```

The same page treats the result "nobody wrote down the reason" as an answer in itself.

Once the user knows that the reason is written nowhere, they can decide whether to ask the original author or stop investigating. So `/why` also reports the result that it found no reason.

`/teach` **explains how code works and why, in plain terms, until a person is convinced**

`/teach` is a Skill that **combines into one plain explanation** what some code or a change is, how it works, and why it was built that way. The goal is not to change anything. The goal is that the other person understands.

In Part 2 of The Complete Guide to pstack, poteto writes that to work with the agent, a partner smarter than you, you need the agent to restate things in a form you understand. poteto also writes that this idea led to `/teach`.

When to use it: when you want real understanding, not a summary

Use it when a summary is not enough and you want to understand a change or a subsystem in depth.

Procedure: call `/how` and `/why`, and combine their results into one explanation

Do not redo the investigation. Combine the results of `/how` and `/why`

`/teach` reads the code itself to get a sense of what to explain and where to start. It does not redo the investigation itself, though. **It calls `/how` and `/why` and combines their results into one explanation.**

For a whole subsystem it calls both. For a small change, one of `/how` and `/why` may be enough.

Ask `/why` to narrow its search by default

When `/teach` calls `/why`, by default it asks `/why` to narrow its search, for example to git and one or two other places. A search of every category takes time, so `/teach` widens the search only when the reason itself is what the other person wants to know.

Decide a few points to take away, and connect plain definitions to the code at hand

When it explains, `/teach` decides the few points the other person should take away, starts from plain definitions, and connects them to the code at hand. It picks those points by reading two things from the conversation: why the person is asking, such as to make a change, to review, or to investigate a defect, and what the person already knows.

For example, it first defines the term in general words, "Virtual scrolling renders only the rows visible on screen", and then connects it to the code at hand, "This app uses virtual scrolling when you open a long chat."

Answer in one or two sentences, then stop and wait for a reaction

`/teach` makes the explanation a conversation, not a lecture. It gives no quizzes and does not ask the person to restate anything. It returns a minimal answer of one or two sentences, then stops and waits for the person's reaction. If the person wants more, it explains in more depth.

Do not draw everything in one diagram. Redraw it in the order it is assembled

`/teach` redraws a diagram of a mechanism and adds one part each time. A diagram with every part in one picture can work as reference material, but the reader cannot follow how the mechanism is assembled.

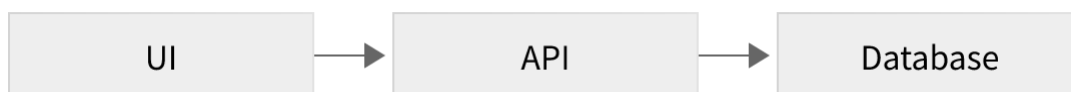
For example, for a flow from the UI to the API and from the API to the database, it draws the following three diagrams in order.

The first diagram goes from the UI to the API.



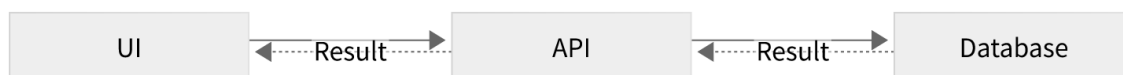
[View diagram 6](#)

The second diagram redraws the first and adds the database.



[View diagram 7](#)

The third diagram redraws the second and adds the return arrows.



[View diagram 8](#)

When each redrawn diagram adds only one part, the reader can trace the order in which the mechanism is assembled.

Output: return the explanation itself, starting from the key point

The reply is the explanation itself, not a report on the work. It starts from the key point and lists topics to investigate further with /how or /why.

/teach may change wording freely for the sake of the explanation, but it keeps /why's confidence words, the levels such as Direct and Inferred.

A hedge such as "probably" is not decoration that can be reworded. It is part of what the investigation found: "the records did not back this up". So /teach keeps the confidence words without rewording them.

Example request: ask it to "convince me"

`/teach` is a Skill the user calls by name. `03-understand.md` has the following example request.

```
/teach me how this PR changes retries. convince me it fixes the cause and not the symptom.
```

The guide recommends that you ask it to "convince me", because the explanation then becomes an argument you can test by pushing back, instead of a one-way lecture.

[Chapter 37](#) covers how to use `/teach` in practice.

`/recall` gathers the history of recent work so that you can resume from the last point of failure

Before you start or resume work, `/recall` gathers the history of recent work again, **so that you can resume from the last point of failure**. The history covers what you did, what you decided, and what went wrong. It returns a short answer about how far the work has come and what to do next.

When to use it: when you ask "how far did I get?"

Use it before you start or resume work, when you want to ask "Remind me of the work on X" or "How far did I get?"

Procedure: search chat history and shared records in parallel

The history of the work is split across two records, so `/recall` reads both.

- **Your own chat history.** It holds what you did and what you decided.
- **Shared records.** They hold what happened around the same code. Examples are symptoms that users keep reporting, fixes that someone reverted after release, and errors that keep appearing in production.

`/recall` has `/why`'s investigators search the shared records. The longer a feature has had defects, the more of its history lives in the shared records instead of your own history.

Here `/recall` has the investigators look not for "why it was built this way" but for "what state it is in now, what was tried and failed, and what users keep reporting".

Before it searches, the main agent decides the time range, the topic, and the workspace. By default, the time range is the last seven days and the workspace is the one open now. The main agent states that scope to the user before it goes on. That way, if the scope differs from what the user had in mind, the user can correct it on the spot.

Unless the user asks, the main agent does not read the history of other projects.

If there are one or two target chats, the main agent searches them directly. If there are more, it has subagents read the history in parallel. The actual history stays inside the subagents, and they return only what they found to the main agent. Because subagents do the reading, long history does not fill the main agent's context ([Chapter 20](#)).

The main agent checks the actual state of the PRs and branches it found with `git` and `gh`. The chat holds the state at that time, and a merge or revert may have happened since.

If all the user wants is to resume one specific chat, `/recall` passes the request to the "Session pickup" Playbook.

Output: Capsule, Threads, Problems, and Next move

The output has four parts that move from an overall summary to the next step, so that the reader can see at once how far the work has come overall and what to do next.

- **Capsule.** Up to five items on what this work is and how far it has come overall. For example, the move to jobs is merged and only the retry problem remains.
- **Threads.** One line for each unit of work done under the topic, such as one PR or one branch. Each line carries exactly one status tag from the table below.
- **Problems.** Up to five items on problems that keep recurring. They include reverted fixes, so that the next attempt can start from the last point of failure.
- **Next move.** The single most useful next step.

Threads use the following six status tags.

Tag	Meaning
[merged #N]	PR #N is merged

Tag	Meaning
[open PR #N]	PR #N is open, for example waiting for review
[in flight <branch>]	Work in progress on a branch
[verified, uncommitted]	Verified but not committed yet
[reverted #N]	PR #N was reverted
[planned, not started]	Planned but not started yet

For example, for last week's export work, an output that contains all four parts and all six tags looks like this.

Capsule

- Moving CSV export from UI-side processing to an asynchronous job
- The move to the job is merged. A problem remains: a retry writes the same rows out again

Threads

- [merged #412] Move export to a job
- [reverted #418] Add retries (reverted because the same rows were written out again)
- [open PR #421] Make writes idempotent with a row key
- [in flight export-progress] Show progress in the UI
- [verified, uncommitted] Extend the timeout for large files
- [planned, not started] Delete the old synchronous export

Problems

- A retry writes the same rows out again after the rows the first attempt wrote. #418 was reverted for this

Next move

- Answer the review comments on #421 and merge it

When the reverted fix and its reason stay in Problems like this, the next attempt can start from the last point of failure.

Example request: call it yourself at the start of work

The user calls `/recall` by name when they start or resume work. `03-understand.md` has the following example request.

```
/recall catch me up on the export work from last week
```

Chapter 37 covers how to hand off to new work.

Choose among /how, /why, /teach, /recall, and "Session pickup" by the question you hold now

The question each one answers is as follows.

- /how. "How does it work now?"
- /why. "Why is it this shape?"
- /teach. "I really want to understand this."
- /recall. "How far did I get?"

To continue one specific chat from where it stopped, use "Session pickup". To load context that spans several chats, use /recall.

Summary

- **The four Skills.** All four run only when someone calls them by name. /poteto-mode also calls /how and /why as part of a Playbook's procedure.
- /how. Separates the role that gathers facts, the explorer, from the role that writes the explanation, the explainer, and explains current behavior under five headings, from Overview to Gotchas.
- /why. Searches each place evidence lives, such as git, tickets, and chat, in parallel, and sorts claims into five levels, from Direct to Unknown. Only claims that records back get the assertion "therefore X".
- /teach. Combines the results of /how and /why, starts from plain definitions, and explains with a diagram that gains one part at a time.
- /recall. Summarizes the state of the work from chat history and shared records in four parts, Capsule, Threads, Problems, and Next move, and keeps reverted fixes and their reasons in Problems.
- **Choosing.** Choose by the question you hold now. To continue one specific chat from where it stopped, use "Session pickup".

The next chapter, [Chapter 23](#), covers /figure-it-out, which designs a Playbook itself for work that no existing Playbook fits.

Chapter 23: Design a Playbook for work that no Playbook fits

kaito • Sources: 日本語 • English • 2026-10-03

This chapter covers `/figure-it-out`.

`/figure-it-out` is a Skill for work that no Playbook fits. It designs a procedure that decides the finish condition and the verification mechanism first, and then it proceeds while it keeps a record of decisions.

The `SKILL.md` of `/figure-it-out` has a setting, `disable-model-invocation: true`, that stops the agent from picking and running the Skill on its own based on the conversation. The user calls this Skill, and so does `/poteto-mode`, which sends work that no Playbook fits, and large work, to it (Chapter 7).

This chapter explains `/figure-it-out` from three angles: when to use it, its procedure, and its output. At the end, it sums up how to choose between this Skill and a similar Playbook.

How this chapter is organized

This chapter is organized as follows.

- When to use it: large migrations and work you hand off while you step away
- Procedure: five Phases, from Frame to Verify
- Output: it returns the designed Playbook and the path to the decision record
- Example request: `/poteto-mode` sends work done while you are away to `/figure-it-out`
- `/figure-it-out` designs how to carry out one piece of work, and "Orchestrate" runs a whole plan
- Summary

/figure-it-out is a **Skill that designs the Playbook itself** for work that matches no Playbook. Before it writes any code, it designs the flow of work, meaning in what order to proceed and what to verify along the way. It also leaves a record so that a person who stepped away can come back and verify the agent's decisions one by one.

When to use it: large migrations and work you hand off while you step away

Use it when the user hands off a migration that spans many callers. Also use it for work that the agent carries on with while the user is away, when the user wants to trust the result after they return.

[Chapter 7](#) covered how /poteto-mode sends this kind of work to /figure-it-out.

Procedure: five Phases, from Frame to Verify

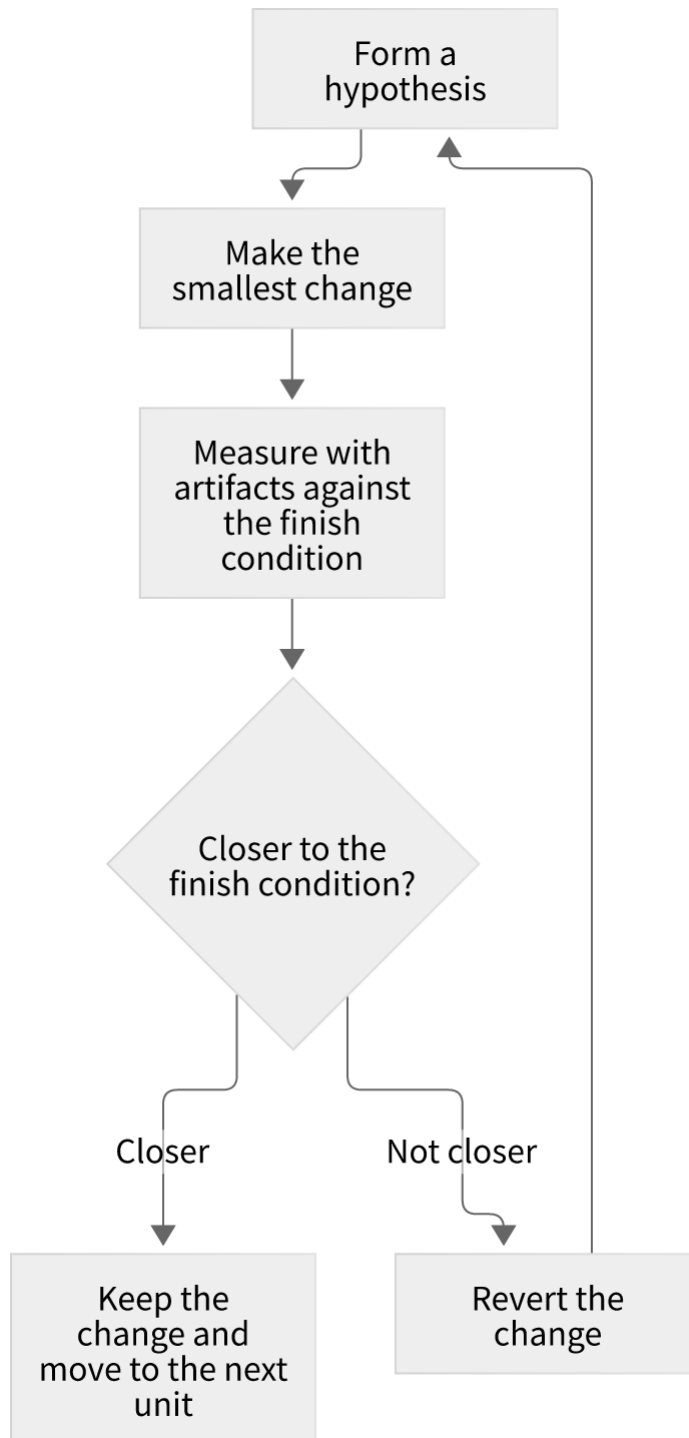
/figure-it-out splits the work into five stages, which the Skill calls Phases.

It splits the work into stages so that the user can trust the work after they return. It decides the finish condition first, builds the mechanism that verifies, and records decisions as it goes.

Phase	What it does
A: Frame	Decides three things: the finish condition, the scope of the work, and the level of rigor, meaning how strictly to verify the work. It writes the finish condition so that you can check whether it holds, for example "zero places call the synchronous store." It states the scope of the work in numbers: the rough count and effort of the work units, and the obstacles that the investigation found. It leans toward a high level of rigor, and sets it higher the harder a decision is to undo

Phase	What it does
B: Design the workflow	Splits the work into units and orders them, and builds the verification mechanism before the work. Each unit is a change that still passes the build and tests and leaves the app working when it merges alone, without waiting for other units. For example, it moves the callers of the synchronous store to the async store with one PR per package. If only one PR merges, the callers not yet moved keep working on the synchronous store. It puts first the units whose success is least certain. When it builds the verification mechanism, it also records the value before the change. For example, it first writes a script that counts the places that call the synchronous store, and it records the count. The recorded value turns each check into a comparison of "the value before the change and the value after"
C: Run the loop	Treats each unit as an experiment and measures it against the finish condition with real artifacts, such as test run results or the output of the script that counts places. There are three verdicts: VERIFIED, NOT VERIFIED, and INCONCLUSIVE. INCONCLUSIVE is not a pass
D: Keep the audit trail	Keeps the decision record with <code>/show-me-your-work</code> (Chapter 28). When a person comes back, the record and the diff are enough to decide whether to trust the work
E: Verify and hand back	Verifies the whole thing in the real product, and turns corrections that kept recurring into lint rules or scripts. That way a machine enforces the correction instead of a prose instruction that repeats it. This step follows the principle "Encode Lessons in Structure" in Chapter 21

In Phase C, the agent repeats the following experiment for each unit.



[View diagram 1](#)

The agent verifies one unit before it moves to the next. Unlike a check of everything at the end, this order shows right away which unit failed.

When the agent verifies a unit in this loop, it looks at the artifacts themselves, not at what the Workers report about their own work ([Chapter 19](#)). Workers are the agents that the main agent handed the work to. For example, it looks at the actual result of running the tests, not at a Worker's report that "the tests passed."

For the same reason, the level of rigor decided in Phase A is not an intention to "look carefully." **The checks the work must pass and the artifacts it must leave set the level of rigor.** To raise the rigor, the agent adds checks that block the next unit until they pass, and it leaves more artifacts, such as verification results and decision records.

Output: it returns the designed Playbook and the path to the decision record

The reply includes the following.

- The designed Playbook
- The level of rigor and the reason for it
- The path to the decision record
- What it verified against the Phase A finish condition
- What remains unresolved

Example request: /poteto-mode sends work done while you are away to /figure-it-out

The example in the [README](#) is a request that reaches /figure-it-out through /poteto-mode.

```
/poteto-mode i'm stepping away. migrate every caller from the synchronous store
to the new async one, keeping behavior identical. i want to trust it was done
right when i'm back.
```

/figure-it-out designs how to carry out one piece of work, and "Orchestrate" runs a whole plan

/figure-it-out designs how to carry out one piece of work, and "Orchestrate" runs a whole plan that spans several days. [Chapter 16](#) has a comparison that also includes "Autonomous run," which decides the finish condition first and carries one task through without stopping.

Summary

- /figure-it-out. For large work that no Playbook fits, it designs a procedure that first builds a finish condition you can verify and a verification mechanism, then verifies unit by unit and keeps a decision record.
- **Choosing between them.** /figure-it-out designs how to carry out one piece of work, and "Orchestrate" runs a whole plan that spans several days.

The next chapter, [Chapter 24](#), covers three Skills that run several agents in parallel and compare designs and artifacts: [/architect](#), [/arena](#), and [/swarm](#).

Chapter 24: Run agents in parallel and compare designs and artifacts

kaito • Sources: 日本語 • English • 2026-10-03

This chapter covers the following three Skills.

1. `/architect`
2. `/arena`
3. `/swarm`

All three run several subagents in parallel. What sets them apart is **what you want to get from the parallel run**.

The three differ as follows.

- `/architect`. Before implementation, the agent decides the types and the module boundaries. For example, it decides whether to expose the import process as one function or as three functions that load, validate, and save. To decide, it has several models create design options and compares them. What you get is one design that the implementation follows.
- `/arena`. The agent has several subagents solve the same problem separately, such as the choice of a format for a cache key. It builds on the best option and folds in the good parts of the others. What you get is one best artifact.
- `/swarm`. The agent either splits work across several agents, for example to check every package under `packages/`, or has them compete on the same problem, and then combines the results into one report. What you get is a check with no gaps, or a fast answer.

This chapter first confirms who calls the three Skills. It then explains each one from three angles: its role, when to use it, and its procedure. For Skills whose source files include example requests, I show those requests too.

Finally, I sort out how to choose among the three parallel Skills.

How this chapter is organized

This chapter is organized as follows.

- `/poteto-mode` calls the three Skills according to its rules and the Playbook procedures
- `/architect` derives types from how callers use the code, and throws away the skeleton when it is wrong
- `/arena` has several candidates compete for one best artifact
- `/swarm` splits work across Workers for coverage or competition and turns the results into one report
- Choose among the three parallel Skills by what you want to get and by how big and how hard to undo the change is
- Summary

`/poteto-mode` calls the three Skills according to its rules and the Playbook procedures

`/poteto-mode` **calls** `/architect`, `/arena`, and `/swarm` **according to the rules in "Non-negotiables" and the Playbook procedures.**

Skill	In one line	How it runs in parallel
<code>/architect</code>	Decides how callers use the code, the types, and the module shape before implementation	Uses <code>/arena</code> internally to compare several design options
<code>/arena</code>	Has N candidates, which are subagents, solve the same problem, picks the best candidate as the base, and transplants the good parts of the other candidates	Every candidate works on the same problem

Skill	In one line	How it runs in parallel
/swarm	Runs N Workers, which are agents that run in parallel, on split or competing work and combines the results into one report	Each Worker covers a separate range, or all compete on the same problem

The callers of the three Skills are as follows.

- /architect, /arena, **and** /swarm. /poteto-mode calls them according to the rules in the section "**Non-negotiables**" and the Playbook procedures. Other Skills also call them inside their own procedures. /figure-it-out and /no-comments call /architect, and /architect and /blast-radius call /arena. The user can also call them directly by name.

/architect **derives types from how callers use the code, and throws away the skeleton when it is wrong**

Role: decide the design with a code skeleton, not with prose

Before you write code that crosses function boundaries, /architect **draws the types, the function signatures, and the module boundaries as a not-implemented skeleton with no function bodies.**

Skeleton. Code that contains only the types, the function signatures, and the module boundaries, with each function body left as a placeholder such as `throw new Error("not implemented")`. A function signature is the function name, its parameters, and its return type. In the example below, the skeleton is `ImportOptions`, `ImportResult`, and `importRows`. The placeholder body does no work. When code calls a function with this body, all the function does is throw this error.

For example, consider a skeleton for the import process.

If you write the caller's usage first and derive the types and signatures from it, you get the following.

```
// Caller usage: write this first and derive the types from it
async function onImportClick(file: File) {
```

```

const result = await importRows(file, { onError: "skip" });
console.log(`Imported ${result.imported} rows and skipped ${result.skipped}`);
}

// Skeleton: write only the types and signatures, not the bodies
// onError decides what happens on an invalid row: skip the row (skip) or stop the import
(stop)
type ImportOptions = { onError: "skip" | "stop" };
type ImportResult = { imported: number; skipped: number };

export async function importRows(file: File, options: ImportOptions): Promise<ImportResult> {
  throw new Error("not implemented");
}

```

The bodies do not exist yet, but the skeleton alone tells you two things. The caller imports by calling `importRows` once, and the return value tells the caller how many rows `importRows` imported and how many it skipped.

The agent builds the skeleton first because **the skeleton becomes the "contract" the implementation must keep**. The contract is a promise that the implementation follows the skeleton's types and signatures.

With a skeleton in place, two kinds of problems **show up as a diff against the skeleton**. One is an implementation that strays from the types. The other is the need for an escape hatch that bypasses the type checker, such as `any` or a cast like `as User`, which [Chapter 18](#) covered.

For example, if the skeleton had `importRows(file, options)` and the implementation needs a third parameter halfway through, the signature change appears in the diff.

The agent does not hide such a gap with a fix inside the implementation. It makes the gap visible and asks again whether the skeleton was wrong, whether it missed a requirement, or whether the implementation does more than it needs to. It does so because **such a gap shows an error somewhere in the design, the requirements, or the implementation**.

`/architect` does not settle the skeleton on a single option. It merges options that several models produced. It then fills in the bodies along the skeleton. If the skeleton turns out to be wrong, it throws the skeleton away and designs again.

In The Complete Guide to pstack [Part 2](#), poteto writes that planning in code is far more effective, and names prototyping in `/poteto-mode` and `/architect` as the ways to do it.

When to use it: before you write code that crosses function boundaries

The section "Non-negotiables" of `/poteto-mode` ([Chapter 8](#)) requires `/architect` for code that crosses function boundaries.

The Playbooks build `/architect` into their procedures to follow this rule. "Feature", which adds a feature, always calls `/architect` in step 2. "Bug fix", "Refactoring", and "Perf issue", which improves performance, call `/architect` when the change crosses function boundaries.

For example, a change that creates a new function and calls it from a function in another file crosses function boundaries. A change that stays inside one function, such as fixing a calculation in one function, does not need `/architect`.

Other Skills call it too. `/figure-it-out` ([Chapter 23](#)) calls `/architect` for design decisions that cannot be undone. `/no-comments` ([Chapter 31](#)) calls `/architect` once when the fix for a comment finding needs a new code shape.

Procedure: five phases, from Ground to Scrap

Phase	Name	What happens
A	Ground (establish the base)	Use <code>/how</code> (Chapter 22) to study the mechanisms the new code touches. A list of the related file names does not count as a study of them, so record the result of tracing the flow, as <code>/how</code> requires. If the design changes which module owns what, or changes the layers, also use <code>/why</code> to find out why the code has its current shape, and make those reasons constraints on the design
B	Sketch (draw the skeleton)	Use <code>/arena</code> to produce several design options, then compare and merge them

Phase	Name	What happens
C	Agree (reach agreement)	By default, move to implementation without a human check. Show the design and wait for approval only when the user asks for it
D	Implement	Replace the skeleton's not implemented placeholder bodies with real code
E	Scrap (throw away)	If workarounds or deviations that do not fit the skeleton keep appearing during implementation, throw the skeleton away and design again

Phase B: compare two or more structurally different options

Even when the first option looks good enough, **the agent has models produce at least two structurally different designs before it merges them.** If you have a model produce a hard design only once, the design stays in the first shape the model thought of. The bundled guide [04-design.md](#) says so.

The agent compares options whose shapes differ, not partial edits within one shape.

For example, an option that renames only the parameters of `importRows` is an edit of the same shape. An option that exposes one function and an option that exposes `load`, `validate`, and `save` as separate functions differ in shape.

Phase B applies the principle "[Exhaust the Design Space](#)" from [Chapter 17](#).

Each candidate that `/arena` starts is a subagent that produces one design option. It follows ten instructions to produce that option. Here I introduce the five main ones.

1. Write the caller's usage first and derive the types from it

Before it writes the types, the candidate writes the code at the call site that uses them. It does so because **how the caller wants to use the code is the specification the design must meet.**

When the usage and the types disagree, the candidate changes the types to fit the usage.

For example, in the code from the skeleton example, I first wrote the call `importRows(file, { onError: "skip" })` inside `onImportClick`, and then decided the `ImportOptions` and `ImportResult` types to fit that call.

```
// Caller usage: write this first and derive the types from it
async function onImportClick(file: File) {
  const result = await importRows(file, { onError: "skip" });
  console.log(`Imported ${result.imported} rows and skipped ${result.skipped}`);
}

// Skeleton: write only the types and signatures, not the bodies
// onError decides what happens on an invalid row: skip the row (skip) or stop the import (stop)
type ImportOptions = { onError: "skip" | "stop" };
type ImportResult = { imported: number; skipped: number };

export async function importRows(file: File, options: ImportOptions): Promise<ImportResult> {
  throw new Error("not implemented");
}
```

2. Decide first on a data structure that fits the frequent reads and writes

Before it writes logic, the candidate decides on a data structure that fits the frequent reads and writes, such as a lookup of one user by ID.

The candidate verifies each read and write by tracing, one by one, how the chosen data structure handles it. If the trace ends in the answer "we can add an index or a cache later", the data structure is wrong.

For example, suppose the code often looks up users by ID but stores users in an array.

With an array, every lookup by ID searches the array from the start. When the candidate traces the lookup by ID through this structure, the trace ends in the answer "we can add an index by ID later", so the array is the wrong structure. The candidate therefore picks, from the start, a structure that stores users in a table keyed by ID, which is a map.

```
type User = { id: string; name: string };
// The ID of the user to look up
declare const id: string;

// Before: every lookup by ID searches the array from the start
```

```
const userList: User[] = [];
const found = userList.find((u) => u.id === id);

// After: store users in a table keyed by ID and look them up directly by ID
const usersById = new Map<string, User>();
const user = usersById.get(id);
```

The data structure that fits depends on the frequent reads and writes, so the candidate picks a structure that fits them before implementation.

3. Expose few functions and hide the work behind them

The candidate keeps the public interface, meaning the functions and types it exposes, few and small, and hides most of the work behind them. It does so because **a small interface reduces what the caller needs to know**.

An interface that exposes little but hides a lot of work behind it is called a **"deep interface"**.

For example, here are two ways to write the public interface of the import process.

```
// Deep: expose only one function, and hide loading, validation, and saving behind it
type DeepImporter = {
  importRows(file: File, options: ImportOptions): Promise<ImportResult>;
};

// Shallow: expose three functions, and the caller must call them in the right order
type ShallowImporter = {
  loadRows(file: File): string[][];
  validateRows(rows: string[][]): string[][];
  saveRows(rows: string[][]): Promise<void>;
};
```

With `DeepImporter`, the caller imports by calling `importRows` once. With `ShallowImporter`, the caller must call three functions in the right order.

4. Express conditions that must always hold in types where you can

The candidate expresses an invariant in types first. When types cannot express it, the candidate uses a runtime check. It writes a comment only when a runtime check is not possible either.

For example, here is the invariant "a shipped order always has a tracking number" written three ways.

```
// In types: a shipped order can only be written in a shape that has trackingNo
type Order =
  | { state: "paid" }
  | { state: "shipped"; trackingNo: string };

// Runtime check: types cannot prevent the problem, so it shows up only when the code runs
type LooseOrder = { state: "paid" | "shipped"; trackingNo?: string };

function assertTrackingNo(order: LooseOrder): void {
  if (order.state === "shipped" && !order.trackingNo) {
    throw new Error("Order is shipped but has no tracking number");
  }
}

// Comment: whether anyone keeps it depends on the person who reads the comment
type CommentedOrder = {
  state: "paid" | "shipped";
  // When state is "shipped", always set trackingNo
  trackingNo?: string;
};
```

If types express the invariant, the compiler tells you the moment you write code that breaks it. For that reason, the candidate expresses invariants in types wherever it can.

5. Validate outside values once at the entry point, and do not re-check them inside

At the boundary, where values come in from outside, the candidate validates the value once and gives the validated value a type. A function that receives a file or an API response is an example of a boundary. Code inside uses the typed value without checking it again. It does so because **once you check at the entry point, you do not need to write the same check again and again inside.**

For example, in the import process, `importRows` checks that the ID is not empty right after it reads each row of the file. The saving code inside receives only rows that passed the check, so it does not check again whether the ID is empty.

This instruction follows the principle "[Boundary Discipline](#)" from [Chapter 18](#).

Attach a rationale to the design, with the rejected alternatives and the reasons

Each candidate attaches a one-page rationale to its design. In the rationale, it writes the usage from the caller's side, which the source calls `Usage`, before the types, and it always writes the rejected alternatives and the reasons for rejecting them.

When it compares alternatives, the candidate looks beyond whether an implementation is easy. It compares the complexity shown to the caller and the complexity hidden inside. For example, with `DeepImporter` the caller needs to know one function, and with `ShallowImporter` the caller needs to know three. So `DeepImporter` shows the caller less complexity.

Inspect for four red flags before merging

Before it merges the candidates, the agent inspects every candidate for four red flags. A red flag is a structural trait that gives the agent a reason to fix or reject the design.

The four red flags are as follows.

- **Shallow module.** A module that exposes many functions and types but hides little work behind them.
- **Information leakage.** Other modules also assume a decision that only one module should own, such as the shape the data takes. If you change that decision, you must fix every module that assumes it at the same time.
- **Temporal decomposition.** A structure that splits modules in the order the work runs, such as load, then validate, then save. When you split in that order, every module passes the same data shape along, so more modules depend on that shape. The `SKILL.md` recommends that work that handles the same data go into one module, even when it runs at different times.
- **Pass-through method.** A method whose only job is to pass the arguments it receives to another method, in the same shape.

For example, a skeleton of the import process that contains all four red flags looks like this.

```
// load.ts: read the CSV and turn each row into an array of strings (shallow module)
export function loadRows(file: File): string[][] {
  throw new Error("not implemented");
}
```



```

}

// validate.ts: assume column 0 is the ID and remove rows with an empty ID (shallow module,
information leakage)
export function validateRows(rows: string[][]): string[][] {
  throw new Error("not implemented");
}

// save.ts: assume column 0 is the ID and save with the ID as the key (shallow module, information
leakage)
export function saveRows(rows: string[][]): Promise<void> {
  throw new Error("not implemented");
}

// importer.ts
export class Importer {
  // Only passes the received rows to saveRows in the same shape (pass-through method)
  save(rows: string[][]): Promise<void> {
    return saveRows(rows);
  }
}

// Caller: cannot import without calling the three in the right order (temporal decomposition)
async function onImportClick(file: File) {
  const rows = validateRows(loadRows(file));
  await new Importer().save(rows);
}

```

In this code, the four red flags appear as follows.

- **Shallow module.** The three files, `load.ts`, `validate.ts`, and `save.ts`, each expose one function, and each function does one small job. One reads, one removes rows, and one saves. They hide little work compared to the number of functions they expose, so they are shallow modules. As a result, to finish one import, the caller must combine and call the three functions itself. The SKILL.md also says that a caller that must combine several functions for one operation is a sign of a shallow module.
- **Information leakage.** Both `validate.ts` and `save.ts` assume that "each row is an array of strings, and column 0 is the ID". Column 0 is the first element of the array. Suppose the column order in the CSV changes and the ID moves to column 2. Then you must fix both `validate.ts` and `save.ts` at the same time.
- **Temporal decomposition.** The files follow the order of the work: `load.ts` loads, `validate.ts` validates, and `save.ts` saves. The three files hand the row data along in turn,

so all three must know the same shape, "each row is an array of strings". A change to the row shape therefore means a fix in all three files.

- **Pass-through method.** `Importer.save` passes the rows it receives to `saveRows` in the same shape, does nothing else, and hides nothing.

When several modules share a decision, each change to that decision adds more places to fix. So a decision must stay inside one module.

With the `importRows(file, options)` skeleton shown earlier, the caller calls only one function. The decisions about how to represent a row and which column is the ID stay inside the `importRows` module.

Phase C: by default, move to implementation without a human check

By default, the agent moves to implementation without a human check.

The agent proceeds without waiting for a human reply, and the human corrects the direction later. This way of working follows the same idea as the principle "[Never Block on the Human](#)" ([Chapter 20](#)).

If a human objects to the shape of the design, the agent treats the objection as new input for Phase A, studies the code again, and redoes Phase B.

To examine the design's weak points closely before implementation, the agent runs `/interrogate` from [Chapter 27](#) on the merged skeleton.

Phase E: deciding to throw away the skeleton

The agent decides whether to throw away the skeleton based on a repeating pattern, not on a one-time event.

The signs include the following.

1. Workarounds of the same shape keep appearing in unrelated code. For example, separate screens that call `importRows` each write the same preprocessing themselves before the call.
2. The code needs an escape hatch, such as `any` or a cast, to compile.

3. A caller cannot use a shared function or type without knowing its internal rules. For example, the caller must always call a separate setup function before it calls `importRows`.
4. The same kind of deviation from the skeleton happens in two or more separate places. For example, two functions each need a parameter that the skeleton does not have.

For example, an implementation that shows all four signs looks like this.

```
type Row = { id: string; name: string };

// A function that sets up the database connection used for importing ahead of time
declare function initImporter(): Promise<void>;

// The skeleton had parseRows(text: string): Row[]
// Sign 4: the implementation needed a tenantId parameter (which customer the data belongs to) that
the skeleton did not have
declare function parseRows(text: string, tenantId: string): Row[];

// The skeleton had saveRows(rows: Row[]): Promise<void>
// Sign 4: the implementation needed a tenantId parameter (which customer the data belongs to) that
the skeleton did not have
declare function saveRows(rows: Row[], tenantId: string): Promise<void>;

// Settings screen
async function onUploadFromSettings(file: File) {
  // Sign 3: importRows fails unless you call this first. The type of importRows does not show this
  rule
  await initImporter();

  // Sign 1: removes the first line of the CSV (the header row) itself before calling importRows
  const lines = (await file.text()).split("\n");
  const body = new File([lines.slice(1).join("\n")], file.name);

  // Sign 2: casts to any to read reasons (why rows were skipped), which the skeleton's ImportResult
  does not have
  const result = (await importRows(body, { onError: "skip" })) as any;
  console.log(result.reasons);
}

// Dashboard: a screen unrelated to the settings screen, yet it writes the same Sign 1 preprocessing
and Sign 3 setup
async function onUploadFromDashboard(file: File) {
  // Sign 3 setup
  await initImporter();
  // Sign 1 preprocessing
```

```
const lines = (await file.text()).split("\n");
const body = new File([lines.slice(1).join("\n")], file.name);
await importRows(body, { onError: "stop" });
}
```

A need to handle a few rare inputs or situations, called edge cases, is not a reason to reject the whole design. Extra branches that come from complex data are not a reason to throw away the skeleton either.

For example, when address formats differ by country and the branches multiply, the cause is complex data, not a wrong design.

When it throws away the skeleton, the agent runs `/how` again on what it has built so far to study the mechanisms again. It then designs again from Phase B. It does not add fixes to the old skeleton. It treats the new constraints it learned during implementation as premises from the start.

Before it adds anything to the new skeleton, the agent first deletes what is no longer needed, such as functions that nothing uses anymore, or duplicate validation. Deletion before addition follows the principle "[Subtract Before You Add](#)" ([Chapter 17](#)). As a result, the new skeleton is smaller than the old one before any features go back in.

How to write the request: say what you care about most

The bundled guide `04-design.md` has the following example request.

```
/architect design the import pipeline before writing any code. i care most about how callers use it.
```

To add a check in Phase C, the user asks like this.

```
/architect with checkpoint. stop and show me before implementing.
```

[Chapter 38](#) and [Chapter 39](#) cover the practice of designing from usage and the practice of reviewing a design.

`/arena` has several candidates compete for one best artifact

Role: have N candidates solve the same problem

A **candidate** is a subagent that produces one design option.

`/arena` is a **Skill that has several candidates solve a problem, picks the best artifact, and folds the good ideas from the other artifacts into the best one.**

When to use it: artifacts that are expensive to fix later

The user or the agent uses `/arena` for an artifact such as the format of a cache key. A single attempt would likely settle on the first shape the model thought of, and a later change to the artifact is expensive.

Within `/poteto-mode` work, there are two main occasions to call `/arena`.

The first is when `/architect` has models produce several design options in Phase B.

The second is step 4 of the "Feature" Playbook. When the implementation could take several reasonable shapes, for example in how to handle errors or how to structure the tests, the main agent calls `/arena` and hands the coding to its candidates.

Playbooks and other Skills also call `/arena`

Every Playbook and Skill that calls `/arena` uses it **where an answer one model produced in one attempt would leave bias or gaps**. The occasion and the reason for each caller are as follows.

`/architect`: **do not settle the design on one option**

`/architect` calls `/arena` in Phase B and has several models produce design options. If you have a model produce a hard design only once, the design stays in the first shape the model thought of. For details, see "[Phase B: compare two or more structurally different options](#)" in this chapter.

The "Eval" Playbook: measure the effect of a Skill change with several models and blind grading

"Eval" is a Playbook that measures how a change to a Skill or a prompt changes the agent's behavior (Chapter 13).

"Eval" does not call /arena directly, but its definition has the work follow the /arena procedure.

In "Eval", several models solve the same request separately in an environment with the change applied. Another model then grades the outputs without knowing which model produced each one.

In other words, "Eval" proceeds the same way as /arena. Several candidates solve the same problem, and another model grades them.

The "Orchestrate" Playbook: compare irreversible decisions before the work starts

"Orchestrate" is a Playbook that manages a project that lasts for days and runs many PRs and subagents (Chapter 16).

In its first step, Frame, "Orchestrate" runs /arena on contested ways to split the work and on irreversible decisions, before it test-runs the first unit of work. It does so because **an irreversible decision cannot be fixed once the project is under way.**

/blast-radius: **have several models study the impact of a large change**

When /blast-radius (Chapter 27) studies a large change or a change with a wide impact, it uses /arena to give several models the same question and merges their answers. According to the SKILL.md of /blast-radius, the reason is that **different models find different bugs.**

Procedure: six phases, from Frame to Verify

Phase	Name	What happens
A	Frame (define the problem)	Decide the artifact each candidate produces and three to six grading criteria, and assign each candidate its own output location. Separate output locations keep candidates from overwriting each other's artifacts
B	Fan out (start them all at once)	Start N subagents at the same time. The default is three models: Opus, GPT, and Grok
C	Cross-judge (another model grades)	After every candidate has finished writing, have the judge model, also called the cross-judge, grade them. Where possible, pick a judge from a different model family than the main agent , which is the agent that started the candidates
D	Pick (choose the base)	The main agent reads every candidate to the end, grades each one against each criterion, and decides the base by comparing its choice with the base the judge recommends. If the two agree, the agreement supports the choice. If they disagree, suspect bias in the main agent or the judge, or vague grading criteria
E	Graft (transplant)	Look for valuable parts in the losing candidates. Do not paste them in mechanically. Rewrite them to fit the base's design and fold them in, because the artifact must stay consistent with one way of thinking

Phase	Name	What happens
F	Verify	Verify that the merged artifact actually works, with the same rigor as any other artifact, following the principle " Prove It Works " (Chapter 19)

The six phases flow as follows.

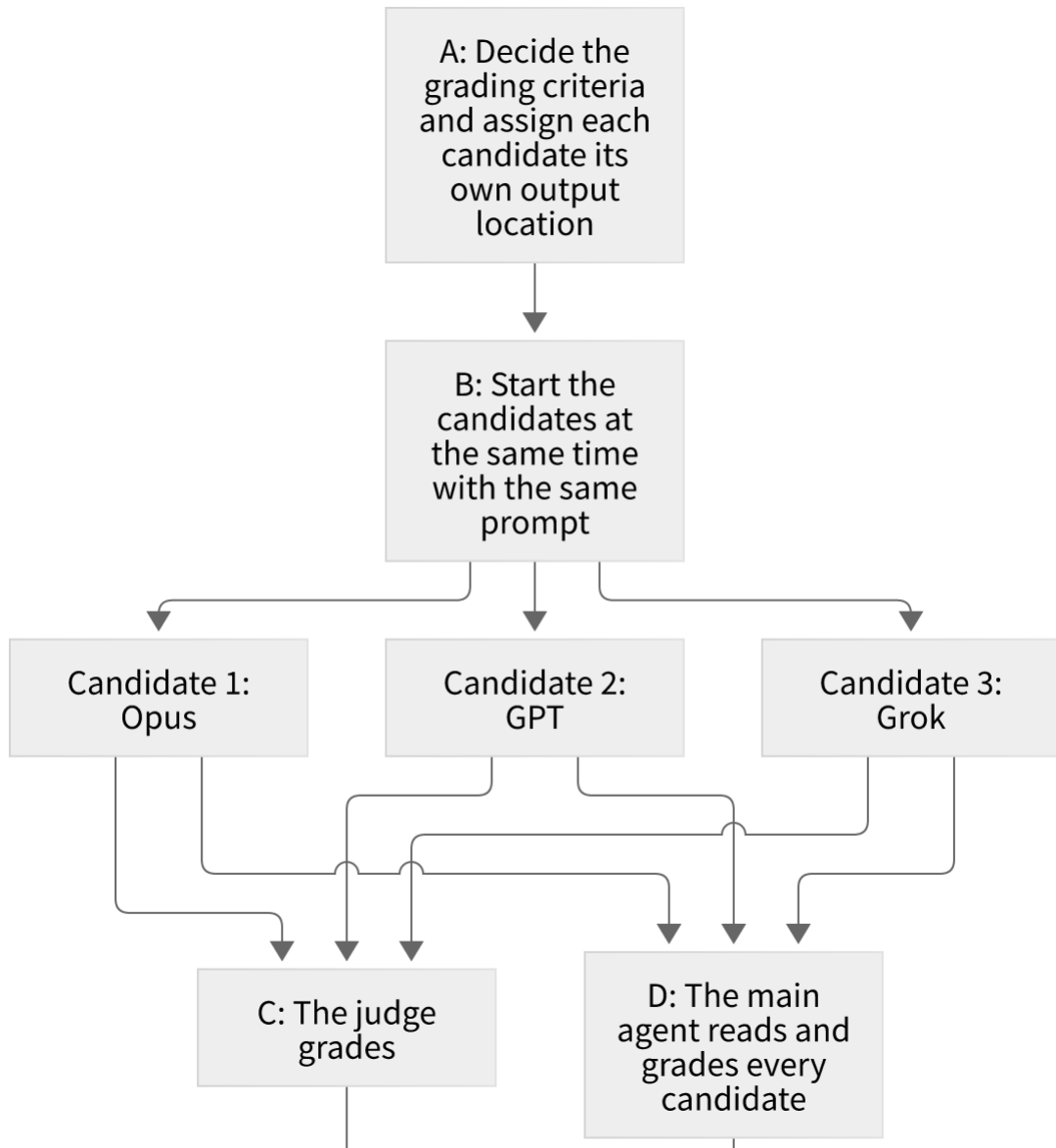


Diagram segment 1

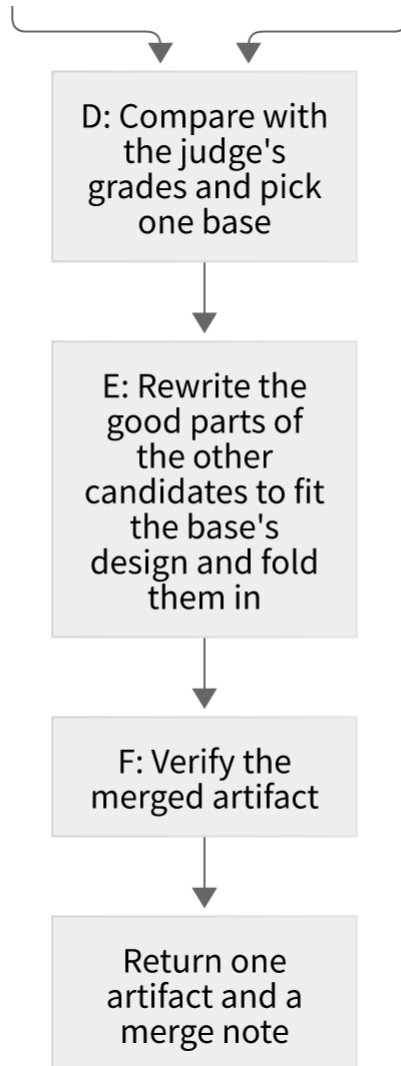


Diagram segment 2

View diagram 1

Start the judge after every candidate has finished writing

The judge's grading and the main agent's reading of all candidates run in parallel after every candidate has finished writing. The main agent does not start the judge while candidates are still writing.

Give the candidates the same prompt, state the artifact in it, and do not show the grading criteria

Every candidate receives the same prompt, so **what each candidate produces depends only on what the prompt says**. For that reason, in Phase A the main agent states in the prompt the artifact each candidate produces, such as a proposed format for the cache key and its rationale.

The main agent uses the grading criteria when it picks the base in Phase D. It therefore shows the candidates only the problem, not the criteria.

Pick as the base the candidate that is easiest to extend

The main agent picks as the base the candidate that a future maintainer could most easily add features to without breaking its invariants.

If every candidate arrives at the same shape, the main agent treats this agreement between separately produced answers as strong support and adopts that shape without grafting.

If the candidates' shapes differ widely, define the problem again

If the candidates' shapes differ widely, the main agent concludes that the problem was not defined well enough. It goes back to Phase A, defines the problem again, and starts over.

For example, when all three candidates propose different formats for the cache key, the main agent does not build an in-between format that takes a little from each. It does so

because **an in-between format is consistent with none of the candidates' ways of thinking.**

Finally, return the merged artifact and a merge note

At the end, the main agent returns one merged artifact and a short merge note. The merge note contains the following.

- The candidate picked as the base, why the main agent picked it, and the judge's conclusion
- The parts taken from other candidates, and which candidate each came from
- The parts considered but not taken, and why
- Any candidates that dropped out without producing an artifact. The main agent continues the merge with the remaining candidates and records the dropout in the note
- The result of actually running and verifying the merged artifact in Phase F, Verify

How to write the request: give the number of candidates

04-design.md has an example that sets the number of candidates.

```
/arena this, 5 candidates. the cache key format is expensive to change later.
```

The bundled guide [10-recipes-and-pitfalls.md](#) also lists a request for a second opinion on the current design before a decision that is expensive to change later.

In this use, the main agent treats the design the user and the agent are working on as one candidate and compares it with the other candidates.

```
ask /arena for a second opinion on this thread and our approach
```

/swarm splits work across Workers for coverage or competition and turns the results into one report

Role: run Cloud Agents in parallel

`/swarm` is a Skill that runs N agents, called Workers, in the cloud at the same time, collects the results, and returns one report. `/arena` produces "one best answer". `/swarm` produces "a check with no gaps" or "a fast answer".

In The Complete Guide to pstack [Part 1](#), poteto describes how he runs verification skills on many Cloud Agents, which are agents that run in Cursor's cloud. He uses them to confirm a performance improvement with a large enough sample, or to fuzz an app.

Fuzzing. A test that feeds a program large amounts of random or broken input to find bugs.

When to use it: coverage, competition, gauntlets, and split searches

The occasions are as follows.

- **Coverage.** The main agent assigns several ranges to Workers and checks them with no gaps. For example, the Workers check every package under `packages/`.
- **Competition.** The main agent has several Workers compete on the same problem. For example, several Workers try to reproduce the same bug.
- **Gauntlet.** The main agent lines up many checks for one change and requires the change to pass all of them. The box below gives an example.
- **Split search.** The main agent splits the places to search and has each Worker examine one. For example, to find the cause of a bug, the main agent assigns one Worker to each suspect module.

Gauntlet is not a term that pstack's files define. In this book, a gauntlet means a series of checks that a change must pass one after another.

You can infer how pstack uses it from the phrase "gauntlet lanes" in the bundled guide `04-design.md` and from the Playbooks that use `/swarm`.

For example, the plan document that the Playbook "[Multi-phase or multi-PR plan](#)" ([Chapter 16](#)) produces works like this. When a PR is ready for review, and each time someone pushes a change after that, `/swarm` runs lanes in parallel against the PR's

latest commit at that point. **The PR passes only when every lane reports PASS.**

The lanes are as follows.

- One lane that reruns the checks, such as tests and lint, on that latest commit
- Ten lanes that verify in the real UI or CLI
- One lane that measures performance
- Two or more lanes that read the diff and the run logs and inspect the PR description with suspicion

In its section "Non-negotiables", `/poteto-mode` requires `/swarm` when work splits into parallel parts, such as coverage, competition, gauntlets, and split searches. In particular, the Playbooks from [Chapter 16](#) that move forward and verify many PRs use `/swarm` to decide whether a PR may merge.

For example, "[Autopilot-full](#)" is a procedure in which an owner for each PR takes it from creation to merge. In it, the coordinator agent, which manages the whole job, runs independent verifiers in parallel with `/swarm` and combines their results into one verdict. Each PR's owner does not merge the PR until the verdict says there is no problem.

Procedure: four phases, from Frame to Report

`/swarm` proceeds in the following four phases.

1. **Frame.** The main agent defines the exit condition and the report to return. The exit condition is a verifiable condition that says what must hold for the work to be done. The main agent also decides the shape and the number of Workers. The shape is partition, race, or mixed, as the table below shows. If the user gives a number in the request, the main agent uses it. Otherwise it decides the number from the shape.
2. **Fan out.** The main agent starts all Workers in the cloud at the same time. Each Worker reports PASS, ISSUES if it found problems, or BLOCKED if it could not verify, with evidence such as the SHA of the commit it verified and the check output.
3. **Aggregate.** The main agent collects the results. For a race, it applies the selection rule it declared in Frame to pick the result.

4. Report. The main agent combines a table of results, the problems with their evidence, the gaps, and the dropouts into one report. Gaps are ranges that produced no result, and dropouts are Workers that dropped out partway.

There are three shapes to choose from in Frame.

Shape	What it means
Partition	Split the work into non-overlapping ranges, and each Worker takes one. For example, one Worker takes each package
Race	Give N Workers the same instructions and have them compete. For example, several Workers try to reproduce the same bug
Mixed	Combine partition and race. Several Workers compete on each range

For the race and mixed shapes, the main agent declares which selection rule it uses before it starts the Workers. The three selection rules are as follows.

- "first pass" takes the first result that passes.
- "rank all" ranks every result.
- "best-of" takes the best result.

The main agent discards a result that lacks the records the instructions asked for, such as the SHA of the verified commit or the measurement method. The measurement method includes how many times to measure. It reruns the Worker that produced that result once. It does so because **when you cannot tell which commit a result checked and how, you cannot judge whether the result applies to the current code.**

If the records are missing again on the second run, the main agent records that range as a gap. **A gap does not count as a pass.**

For example, if 9 of 10 Workers report PASS and one is missing, the report says "9 passed, 1 gap", not "9 passed".

How to write the request: one sentence each for the check, the shape and count, and what to return

The [README](#) and the bundled guide have the following example request.

```
/swarm check every package under packages/ against its check.sh. one worker per package. one report.
```

This request **gives one sentence each** to three things. The check is `check.sh`, the shape and count are one Worker per package, and what to return is one report. The main agent decides these three things in Frame. When the request states them, the main agent builds Frame from them.

Choose among the three parallel Skills by what you want to get and by how big and how hard to undo the change is

Choose as follows.

- **To design a change that crosses function boundaries, such as a new function that a function in another file calls.** Use `/architect`, which uses `/arena` internally.
- **To get one artifact at the end, such as one settled format for a cache key.** Use `/arena`.
- **To combine the results of checks on many ranges into one table, such as the check results for every package under `packages/`.** Use `/swarm`.

Decide how much design effort to spend by the size of the change and how hard it is to undo

Not every change needs design work. `04-design.md` gives the following guide.

Situation	What to use
A small, finished change that you are unsure about	<code>/interrogate</code> only
A change that crosses function boundaries or moves ownership, meaning which module is responsible	<code>/architect</code> , which also uses <code>/arena</code> internally

Situation	What to use
A single decision, such as a name or an algorithm	<code>/arena</code> directly
An exhaustive check per range, parallel checks, or a race with a declared selection rule	<code>/swarm</code>
A contested design that is expensive to undo	<code>/architect</code> , then <code>/interrogate</code>

`/poteto-mode` applies this guide itself and calls `/architect` for work that crosses function boundaries even when nobody asks. The user therefore calls these Skills directly only to check more carefully, or more briefly, than the default.

`/arena` and `/swarm` aim for different results

Item	<code>/arena</code>	<code>/swarm</code>
Purpose	Produce one best artifact	Cover everything, or get a fast answer through competition
Instructions to each subagent	The same problem for everyone	Separate ranges for a partition, the same instructions for a race
How results are combined	Pick a base, then rewrite the good parts of the losing candidates to fit the base's design and transplant them	Combine into a table. For a race, pick by the declared rule
Output	One merged artifact and a merge note	One report in the chat

A common mistake is to use `/arena` for coverage, according to `10-recipes-and-pitfalls.md`. `/arena` gives every candidate the same problem, so it does not check separate ranges.

For coverage, the user uses `/swarm`, which splits the ranges and combines the results into one report. `/swarm` does not pick a base and transplant parts the way `/arena` does.

Summary

- **How the three Skills divide the work.** `/poteto-mode` calls `/architect`, `/arena`, and `/swarm` according to the rules in "Non-negotiables" and the Playbook procedures.

- `/architect`. Before code that crosses function boundaries, it derives the types from the caller's usage and builds a skeleton with no function bodies. If workarounds of the same shape or type escape hatches such as `any` or casts keep appearing, it throws the skeleton away and designs again.
- `/arena`. It has several models produce an artifact that is expensive to fix later, such as the format of a cache key, and picks the best candidate as the base. It rewrites the good parts of the other candidates to fit the base's design before it transplants them.
- `/swarm`. It runs Workers in parallel for coverage, such as a check of every package, or competition, and combines the results into one report. A range whose records, such as the verified commit, are still missing after one rerun does not count as a pass. The report lists it as a gap.
- **Choosing among the three.** Use `/architect` for a change that crosses function boundaries, `/arena` for one artifact, and `/swarm` for a table of coverage results.

The next chapter, [Chapter 25](#), covers two Skills that extend pstack to the user's own way of working and to Grok Bot: `/automate-me` and `/make-bot-ui`.

Chapter 25: Extend pstack to your own way of working and to Grok Bot

kaito • Sources: 日本語 • English • 2026-10-03

This chapter covers the following two Skills.

1. `/automate-me`
2. `/make-bot-ui`

Both Skills extend pstack to fit the user's environment. Their uses are as follows.

- `/automate-me`. The agent reads the user's way of working from the user's chat history (for example, "Keep paragraphs short. Use a table to compare options."). It then creates a mode Skill like `/poteto-mode` that writes down that way of working for that user alone.
- `/make-bot-ui`. The agent creates a web page with a button that starts a process in Grok Bot, an AI bot app that can host pstack ([Chapter 5](#)).

This chapter first confirms who calls the two Skills. It then explains each one from three angles: its role, when to use it, and its steps. For a Skill whose source gives example requests, I show those too.

How this chapter is organized

This chapter is organized as follows.

- The two Skills run only when the user calls them by name
- `/automate-me` reads how the user works from the chat history and turns it into a Skill for that user alone

- `/make-bot-ui` builds a page that starts Grok Bot with a button without exposing the sender key
- Summary

The two Skills run only when the user calls them by name

Skill	In one line
<code>/automate-me</code>	Creates a <code>-mode</code> Skill for the user alone from the user's chat history
<code>/make-bot-ui</code>	Creates a page with buttons that start a Grok Bot process through a webhook

Both `SKILL.md` files have a setting (`disable-model-invocation: true`) that stops the agent from choosing and running the Skill on its own based on the conversation. Neither `/poteto-mode` nor any Playbook calls these two in its steps. So **the two Skills run only when the user calls them by name.**

`/automate-me` reads how the user works from the chat history and turns it into a Skill for that user alone

Role: turn the user's way of working into a Skill from the history and the user's answers

`/automate-me` is a **Skill that turns the user's way of working into a Skill the agent follows**. Its output is one Skill with a `-mode` suffix tailored to that user (for example, `jay-mode`).

The user does not need to explain their way of working. **The agent reads from the chat history what the user has asked for again and again.**

For example, if the user has told the agent "Make your replies shorter" many times, the agent reads "short replies" as part of the user's way of working. The agent fills in intentions that do not appear in the history by asking the user.

When to use it: when the user wants to turn their preferences and way of working into a Skill

`/poteto-mode` and the other Skills in pstack cannot cover every use case.

So the user calls this Skill by name when they want a Skill that fits their own preferences and way of working.

To record only the steps of one task, such as "how to write a commit message", the user does not use this Skill. They use only `create-skill`, the Skill built into Cursor for writing Skill files. An ordinary Skill is enough for the steps of one task. That task does not need a way-of-working Skill that covers the user's whole approach to work.

Steps: read the history, ask the user, polish the draft, then open a PR

1. **Read the history.** The agent takes the current workspace's transcripts from the last two to four weeks and splits them into periods, for example three. Subagents then read the periods in parallel. The agent reads only the current workspace's records and does not read personal chats from other projects. The agent treats only patterns found in two or more periods as reliable, and usually discards a pattern that appears in only one period. A preference stated once and contradicted in another situation is noise, not a way of working.
2. **Ask the user directly.** To fill in intentions that have not yet appeared in the history, the agent asks one or two multiple-choice questions and one free-text question. It uses multiple choice so that the user does not have to write text from scratch.
3. **Draft.** The agent uses `create-skill` to write a draft at `.cursor/skills/<handle>-mode/SKILL.md`. `<handle>` is the user's name or an identifier the user chooses.
4. **Polish.** The agent runs `/unslap` (Chapter 32) on the draft to remove the habits of AI-written text. It then checks with the user that the draft reads like the user.
5. **Merge it in.** The agent works in a worktree (Chapter 18) and opens a PR. It does not push directly to main.

If a mode Skill already exists, the agent reads only the history since the last edit and updates that Skill.

The agent creates a section in the Skill only when the user has a concrete rule that differs from the agent's default behavior. It does not write general advice such as "communicate clearly". General advice does not change how the agent behaves, so it only makes the Skill longer to read.

For example, a rule such as "Keep paragraphs short. Use a table to compare options. Use bullet points only when the items are truly parallel." gets a section.

How to write the request: the command alone is enough to start

According to the bundled guide [09-make-it-yours.md](#), the user can start with the command alone, as follows. The agent reads the user's way of working from the history, so the user does not need to add an explanation.

```
/automate-me
```

The same page also gives a request for updating an existing mode Skill when the way of working changes.

```
/automate-me update my mode skill with everything since its last edit
```

/make-bot-ui builds a page that starts Grok Bot with a button without exposing the sender key

Role: build a way to start the Bot without showing the secret to the agent

/make-bot-ui is a Skill that builds a page the user clicks and sets up **a way for its buttons to start Grok Bot through a webhook.**

When to use it: when you want a button to start a Grok Bot routine through a webhook

The user uses this Skill to build a page, such as a set of buttons or a dashboard, that starts Grok Bot through a webhook. The user also uses this Skill to submit the sender key for that page and to publish the page on Tailscale.

Tailscale is a service that builds a private network connecting the user's devices, such as computers and phones, without exposing them to the internet. This network is called a tailnet, and only devices in the tailnet can open pages on it.

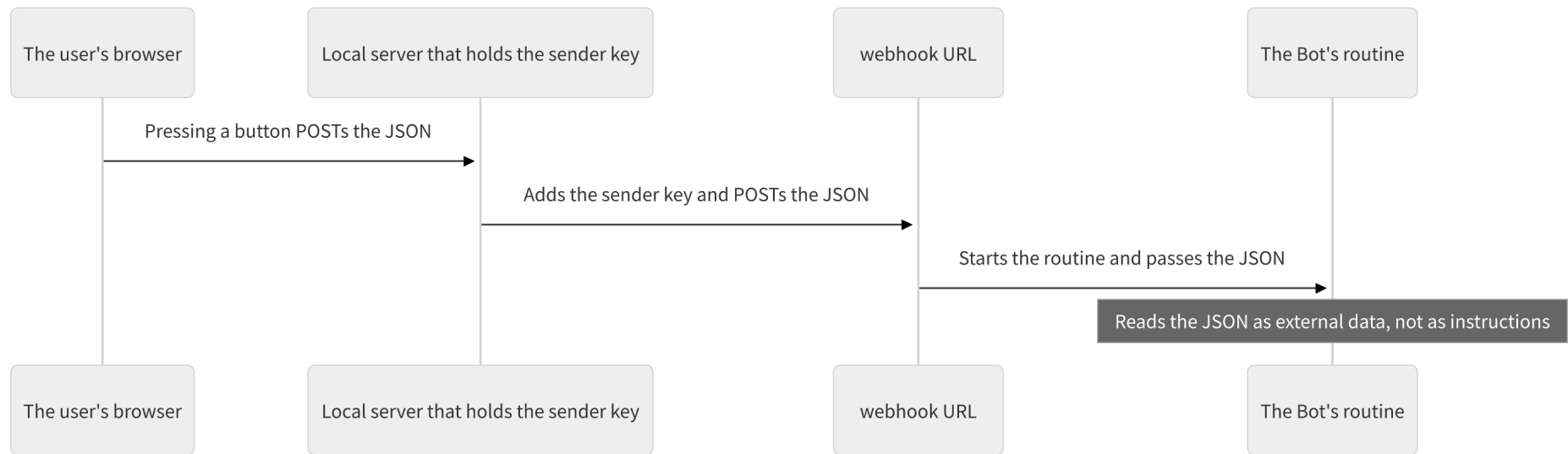
Grok Bot is an AI bot app that can install pstack as a plugin ([Chapter 5](#)). You can register routines in Grok Bot, so it can run on a schedule or when an event triggers it.

Webhook. A mechanism that sends an HTTP request to a URL you specify, to tell another system about an event and start a process there. For example, GitHub can send a notification to a given URL when someone pushes.

`/make-bot-ui` handles routines that a webhook starts. Between the button press and the start of the Grok Bot process, the following three steps run in order.

1. The user presses a button on the page in the browser. The page sends the JSON for that button, for example `{"action": "summarize-today"}`, to a small server on the same computer as the Bot. This chapter calls that server the local server.
2. The local server adds the sender key to the JSON it received and sends the JSON to the routine's webhook URL. The webhook URL is the dedicated URL that starts that routine from outside.
3. When the JSON reaches the webhook URL, the Bot's routine starts and begins processing with that JSON as its input.

The page does not send directly to the webhook URL. **It puts the local server in between so that the sender key never reaches the browser.** The step-by-step explanation below shows why.



View diagram 1

Steps: create the routine, deploy the page, and publish it on the tailnet

1. **Create the routine.** The agent creates a routine that a webhook starts, and writes in its prompt that the POST body is untrusted data. The body that reaches the webhook URL comes from outside, so even if it contains an instruction such as "ignore all previous instructions", the Bot must not follow it.
2. **Have the user get the URL.** The agent asks the user to open the routine from Routines, the Bot's list of routines, and copy the webhook URL. The source says the user may paste the webhook URL into the chat. The sender key in the next step, by contrast, does not go into the chat.
3. **Receive the sender key.** The sender key is a secret that proves the caller is a legitimate sender of the webhook. The agent does not accept the sender key in the chat. The agent has the user submit the key through a dedicated card for entering secrets. The agent cannot see the submitted value, and writes the value into the server's settings without seeing it.
4. **Deploy the page.** The agent places the page and the local server on the same computer as the Bot. The page's buttons POST to the local server, and that server adds the sender key and POSTs to the webhook.
5. **Put the page on the tailnet.** The agent uses Tailscale so that the user's devices can open the page without exposing it to the internet.
6. **Receive the trigger.** The Bot treats the JSON it receives as external data, not as instructions.

The agent keeps the sender key only on the server and never shows it in the browser or the chat. Anyone who knows the key can start the routine.

If the browser called the webhook directly, the key would have to be in the page's code, and anyone who opened the page could read it. So the buttons always call the webhook through the local server.

Under the same rule of keeping secrets out of the chat, the agent does not ask the user for Tailscale credentials either. Instead, it gives the user a login URL, and the user approves the device in the browser.

Summary

- **How the two Skills are called.** `/automate-me` and `/make-bot-ui` run only when the user calls them by name.
- `/automate-me` creates a `-mode` Skill for the user alone that writes down the user's way of working, based on the chat history and questions to the user.
- `/make-bot-ui` creates a page that starts Grok Bot with a button, without showing the sender key in the chat or the browser.

The next chapter, [Chapter 26](#), covers two Skills that build the verification setup and keep it in line with the app: `/create-verification-skill` and `/maintain-verification-skill`.

Chapter 26: Build verification and keep it current as the app changes

kaito • Sources: 日本語 • English • 2026-10-03

This chapter covers the following two Skills.

1. `/create-verification-skill`
2. `/maintain-verification-skill`

Both Skills exist so that the agent can run the real app and verify its own work. Each takes on the following job.

- `/create-verification-skill`. The agent writes a procedure into the project. The agent uses that procedure to start the app, operate it the way a real user does, and collect evidence such as screenshots.
- `/maintain-verification-skill`. The agent periodically checks whether the features and operations in the procedure from `/create-verification-skill` still match the current app. It fixes any drift.

`/create-verification-skill` creates a **verification skill** so that the agent can run the app and verify it the way a real user would. A verification skill is a project-specific Skill that the agent uses to start the app it is working on, operate it, and collect evidence such as screenshots, terminal output, and logs ([Chapter 3](#)).

Of the two, **the one the user runs first is `/create-verification-skill`, which creates the verification skill.**

This chapter first looks at how the two split the work, who calls them, and why you start with a verification skill. Then it explains each one from four angles: its role, when it is used, its procedure, and how to write a request.

How this chapter is organized

This chapter has the following sections.

- One Skill creates the verification skill and the other keeps it current
- `/create-verification-skill` builds a way to run the app along the same path as a user
- `/maintain-verification-skill` periodically fixes drift between the Feature Map and the current app
- Summary

One Skill creates the verification skill and the other keeps it current

The following table shows the role and the artifact of each Skill.

Skill	Role	Artifact
<code>/create-verification-skill</code>	Creates the procedure in the project that starts and operates the app and keeps evidence such as screenshots	<code>.cursor/skills/verify-<app>/</code> , which holds the verification skill and the Feature Map
<code>/maintain-verification-skill</code>	Keeps the verification skill and the Feature Map matched to the current app	At most one PR with fixes that the agent verified by running the app, or only a report

The Feature Map is a document that records, for each feature of the app, what it does, how a user gets to it, how to operate it with the verification skill, and what result you can verify ([Chapter 4](#)).

The principle "**Prove It Works**" in [Chapter 19](#) asks you to "run the feature and verify it". `/create-verification-skill` and `/maintain-verification-skill` build the means to run and verify features in the project and keep those means working, so that you can run and verify in any task.

The two run only when the user or `/setup-pstack` calls them by name

Both `SKILL.md` files have the setting `disable-model-invocation: true` (Chapter 25). The agent never picks a Skill with this setting from the content of the conversation and runs it on its own. So the two run only when the user calls them by name, or when another Skill names them in its procedure.

The following list shows who calls each one.

- `/create-verification-skill`. The user calls it by name. `/setup-pstack` also calls it. In the last step of `/setup-pstack`, if `/setup-pstack` finds that the project has no way to run and verify the app, it offers to create one. If the user accepts, `/setup-pstack` calls this Skill.
- `/maintain-verification-skill`. It runs only when the user calls it by name.

You create the verification skill first so that the agent can verify its own work

The user sets up a verification skill first for the following reason. **If the agent cannot verify its own work, a human ends up verifying that work piece by piece at the end, no matter how much work you delegate to the agent. The human then becomes the one who stops the flow of work (the bottleneck).**

Once a human becomes the bottleneck, that human spends the whole day looking after the agent. With a verification skill, the agent runs the app itself, verifies the result, and keeps working until it succeeds. The user only needs to look at results the agent has already verified.

poteto makes this point again and again in The Complete Guide to pstack [Part 1](#) and [Part 2](#).

In particular, Part 1 says that with a verification skill, you can also hand off app checks to other Skills and to mechanisms that run automatically. The guide treats the verification skill as the base for other Skills and for recurring work.

For reference, Part 1 gives the following two examples.

- `/swarm` (Chapter 24) has many cloud agents run the verification skill, to verify a performance improvement over enough runs.

- A Grok Bot routine ([Chapter 25](#)) has a cloud agent reproduce a bug with the verification skill each time a user report arrives in Slack.

`/create-verification-skill` **builds a way to run the app along the same path as a user**

Role: build, in the project, a way to run the real app and prove its behavior

`/create-verification-skill` is a Skill that generates a project-specific verification skill in `.cursor/skills/verify-<app>/. With the generated verification skill, the agent can start the real app, operate it the way a user does, and keep evidence.`

The agent writes the verification skill for other agents to read, not for humans. **The reader is an agent that has never seen the app and opens this verification skill in the middle of a task with no background knowledge.** So that the reader does not have to guess, the agent writes the actual commands and selectors that this repository uses directly into the verification skill. A selector is an expression that points at a screen element.

Once the verification skill exists, the user only has to say "verify it in the app". Any agent can then verify without the user teaching it how to start and operate the app ([docs/guide/06-verify-and-ship.md](#)).

When it is used: when there is no way to verify the app's behavior with a script

You use this Skill when the project has no way to verify the app's behavior repeatedly with a script.

For example, the project has neither a script that drives a browser to check the screen nor a script that runs the CLI and checks its output.

`/setup-pstack` sets the models pstack uses. In its last step, it also checks whether the project has a way to run and verify the app, such as a `verify-*` Skill or an existing harness.

If the project has none, `/setup-pstack` offers once to create one. If the user accepts, `/setup-pstack` calls `/create-verification-skill` (Chapter 34).

Procedure: investigate the repository, write the verification skill, and run it once before handing it over

Step 1: find out how to run the app from the repository before asking the user

In the first step, the agent finds out how to run the app from the repository's code and configuration before it asks the user. The source calls this investigation an "interview".

The agent asks the user only about what it cannot observe in the codebase. **Asking the user about things the code already answers adds work for the user, and the task stops while the agent waits for a reply.**

The agent answers the following five questions from the codebase. The five questions collect what goes into the verification skill: what to start and how, how to operate it, and what to keep as evidence.

Aspect	Question
Surface	What does the user actually touch, such as a web UI, a CLI, or an API?
Run	How does the app start locally? (Prefer the repository's own development commands, such as the package scripts, a Makefile, or the quick start in the README.)
Drive	How can the agent operate the app from a program? First look for mechanisms already in the repository, such as Playwright or Cypress tests, or endpoints you can call with curl. If there are none, pick a general method such as CDP, a PTY or tmux, or raw HTTP
Observe	What evidence can the agent keep, such as screenshots, terminal recordings, response bodies, logs, exit codes, or database state?

Aspect	Question
Isolate	Can two instances of the app run side by side, with separate ports and data directories?

- **CDP.** A mechanism that gives a program the same capabilities as the browser's developer tools. For a web app or an Electron app, the agent connects to the browser over CDP, sends clicks and typed text, and reads the state of the screen
- **tmux.** A terminal multiplexer that manages several sessions, windows, and split panes on one terminal screen. From outside, a command can also send keystrokes to a session or capture what the screen shows
 - For a CLI or a TUI, the agent starts the app inside a PTY or a tmux session, sends keystrokes, and reads the text shown on the screen
- **Raw HTTP.** The agent sends HTTP requests straight to the app with a tool such as curl, without going through the screen, and checks the responses

If two instances cannot run side by side, the agent writes that into the verification skill and instructs that two agents must not operate one instance at the same time. If the agent operates the instance the user is using at the same time, it may break the session the user is working in.

If the code in the repository's current state cannot build or start, the agent deals with that before it generates the verification skill. The agent either fixes the code so that the app builds and starts, or reports exactly what is missing. A verification skill written for an app that cannot build or start passes wrong instructions to the agent that reads it.

Step 2: write the six headings of `SKILL.md`

The agent puts the following six headings in the `SKILL.md` it generates. With them, the next agent, which has never seen the app, can work through everything from startup to cleanup in order.

The agent writes the contents only from facts it found in the investigation in [step 1](#), and leaves no placeholders. A placeholder is a temporary string that someone must fill in

later. The next agent does not know the app, so it cannot replace a placeholder with the correct value.

The following compares text that still has placeholders with text written from facts found in the investigation.

```
<!-- Placeholders remain: the next agent does not know what to run or what to click -->
Launch: run `<start command>`, and the app is ready when `http://localhost:<PORT>` responds
Drive: click `TODO: selector for the send button`

<!-- Written from facts found in the investigation: the next agent can run it as is -->
Launch: run `pnpm dev`, and the app is ready when `http://localhost:5173` responds
Drive: click `getByRole('button', { name: 'Send' })`
```

Heading	Contents
Launch	The command that starts the app, how to tell that the app is "ready", for example when a specific log line appears or a port responds, and how to stop it
Doctor	Read-only checks that confirm "whether operating this instance to verify something makes sense" (whether the process is running, whether the build is correct, whether the instance on that port is the one you started, whether authentication is valid, and so on)
Drive	Operating steps that use this repository's actual selectors and commands. Use markers that rarely change, such as ARIA labels and data attributes, not screen coordinates or the order in which the Tab key moves focus
Evidence	What to keep as proof and where to put it
Cleanup	How to clean up the instances this run created. Do not delete the evidence
Helpers	How to call the bundled scripts

ARIA label. A name attached to an element such as a button so that screen readers can announce it

The following Playwright code compares ways to click, to show what the "markers that rarely change" in Drive mean.

```
declare const page: {
  mouse: { click(x: number, y: number): Promise<void> };
  getByRole(role: "button", options: { name: string }): { click(): Promise<void> };
  locator(selector: string): { click(): Promise<void> };
};

// Before: click by screen coordinates. If the button moves, this clicks somewhere else
async function saveByPosition() {
  await page.mouse.click(640, 410);
}

// After: click by ARIA label. Even if the button moves, this clicks the same button
async function saveByLabel() {
  await page.getByRole("button", { name: "Save" }).click();
}

// After: click by data attribute. Even if the button moves, this clicks the same button
async function saveByDataAttribute() {
  await page.locator('[data-testid="save"]').click();
}
```

Evidence also states the standard of proof. The standard includes the following.

- Operate the real path the user takes, not a test-only shortcut
- Check side effects such as written files, added rows, and sent messages, not only the screen

A test-only shortcut is, for example, a function that rewrites an internal value directly, or an endpoint that exists only for tests. If the agent verifies through a shortcut, it does not prove that the path the user takes works in the real app.

Step 3: create the first version of the Feature Map

The agent also attaches a Feature Map to the verification skill it generates.

The Feature Map consists of one feature file for each user-facing feature and features/README.md, a table of contents that ties them together. At first, the agent finds three to five main features and creates a feature file for each.

The agent looks for main features in the following four places.

- **Routes.** The URL path assigned to each screen of a web app, for example `/settings`.
- **Commands.** The commands and subcommands a CLI accepts, for example `commit` in `git commit`.
- **Menus.** The menu items shown on the app's screens.
- **Documentation.** Documents that describe the app's features, such as the README and usage guides.

Chapter 36 covers the contents of a feature file and the idea of growing the Feature Map as memory.

The map is the repo's maintained verification source; a proof that drives one convenient entry point is incomplete when the map lists others.

An entry point here is a way the user gets to the feature, such as a button, a key press, or a command.

If the Feature Map lists several entry points for one feature, the agent can prove that the feature works only after it verifies every entry point. Because of this rule, **the Feature Map is also the list that decides how many entry points you must verify for the proof to count.**

As an example, apply this rule to the bundled Feature Map example. In that example, the feature file for the note search feature, `search.md`, describes how a user gets to the search feature as follows.

How to get to it (user POV)

- Choose the Search button in the browser toolbar.
- Press `/` in the browser while focus is outside an editable field.
- Run `notes search <query>` in a terminal.

In this example, the note search feature has three entry points: the Search button in the toolbar, the `/` key, and `notes search <query>` in the terminal. If the agent verifies this feature

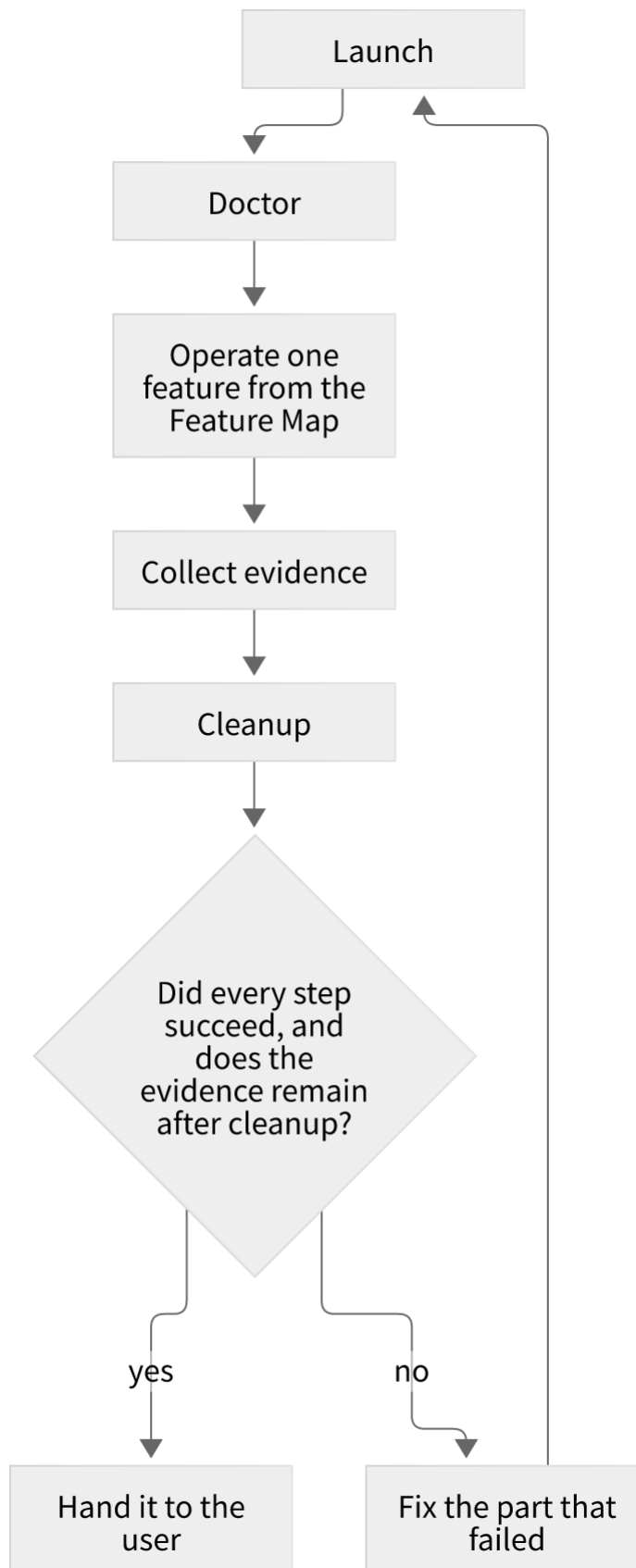
only through the Search button, the proof is not enough. The verification becomes a proof only after the agent also verifies the other two entry points.

So if the Feature Map lists several entry points for one feature, the agent must verify all of them.

Step 4: run the generated verification skill once from start to finish

Before it hands the verification skill to the user, the agent runs it once from start to finish, exactly as its instructions say. Operating one feature is enough. The agent can verify the other features in later runs by using the Feature Map.

The following diagram shows the flow of the run.



View diagram 1

The agent runs cleanup even after a failed attempt, so that no process or port from that attempt stays behind.

If no evidence remains after cleanup, the agent fails the run, because a cleanup that deletes the evidence leaves no proof.

Until the agent has run the generated verification skill once, it treats the skill as a draft, not as an artifact. An error in the written procedure stays hidden until someone runs it.

For example, if the port number in Launch is wrong, nobody notices until the skill runs.

Finally, the agent points the user to `/maintain-verification-skill`, which maintains the Feature Map.

How to write a request: the command alone is enough to start

According to `docs/guide/06-verify-and-ship.md`, you can start to create the Feature Map and the verification skill with only the following command. The agent finds out from the repository how to run the app, so you do not need to prepare an explanation in advance.

```
/create-verification-skill
```

Chapter 36 covers how to create a verification skill and grow it in practice.

`/maintain-verification-skill` periodically fixes drift between the Feature Map and the current app

Role: keep the verification skill and the Feature Map from disagreeing with the current app

`/maintain-verification-skill` is a Skill that checks whether the verification skill and the Feature Map disagree with the current app, and fixes any drift.

A feature map rots the moment the app changes.

When the app changes, the Feature Map's descriptions start to drift from the real app from that moment on. So you must keep maintaining the Feature Map after you create it. This Skill does that maintenance.

When it is used: periodically, after the app changes

You use this Skill when the app has changed and the Feature Map's descriptions no longer match the current app.

In The Complete Guide to pstack [Part 1](#), poteto recommends running it at least once a day, to keep the information the agent uses to operate the app up to date.

So this book also recommends that you run it at least once a day and automate the run with a Routine or a similar mechanism.

Procedure: read the source code in parallel, and let only the main agent operate the app

Do not edit product code, and report a regression instead of changing the Feature Map

The agent may edit only the verification skill's own directory (SKILL.md, features/, and the verification skill's scripts).

The agent does not edit product code during the check. When the app does not behave as the Feature Map says, the agent decides whether the mismatch is a writing error in the Feature Map or a regression in the product (a feature that used to work is broken). If the mismatch is a writing error, the agent fixes the Feature Map. If the mismatch is a regression, the agent reports the regression to the user.

The agent does not settle a regression by editing the Feature Map. **If the product is broken and you change the Feature Map's description to match the current behavior, you hide the fact that the product is broken.**

Seven steps, from tidying the index to deciding whether to open a PR

The procedure is as follows.

1. **Find the target.** The main agent finds the verification skill to check, usually in `.cursor/skills/verify-*/`. If it finds none, it stops and points the user to `/create-verification-skill`.
2. **Tidy the index.** The main agent compares the Feature Map's table of contents (README) with the feature files, and fixes missing entries and broken links.
3. **Read the source code.** The main agent starts one read-only subagent per feature file, in parallel. Each subagent returns the places where the Feature Map's description and the current source code seem to differ, together with the source locations that support each suspicion.
4. **Cross-check.** The main agent verifies some of the suspected drift found in step 3. It also looks through recent changes for features the Feature Map does not mention.
5. **Operate the app.** The main agent operates every feature at least once, even if the source code showed no problem.
6. **Sort.** The main agent sorts the problems it found into the three kinds in the table in ["Sort problems into three kinds, and report one of three results"](#) below.
7. **Decide whether to open a PR or stop.** If there are fixes it verified by operating the app, the main agent rereads the changed files and puts them into one PR. If there is nothing to fix, or if it could not finish the check, the main agent does not open a PR and reports the result and the scope it verified.

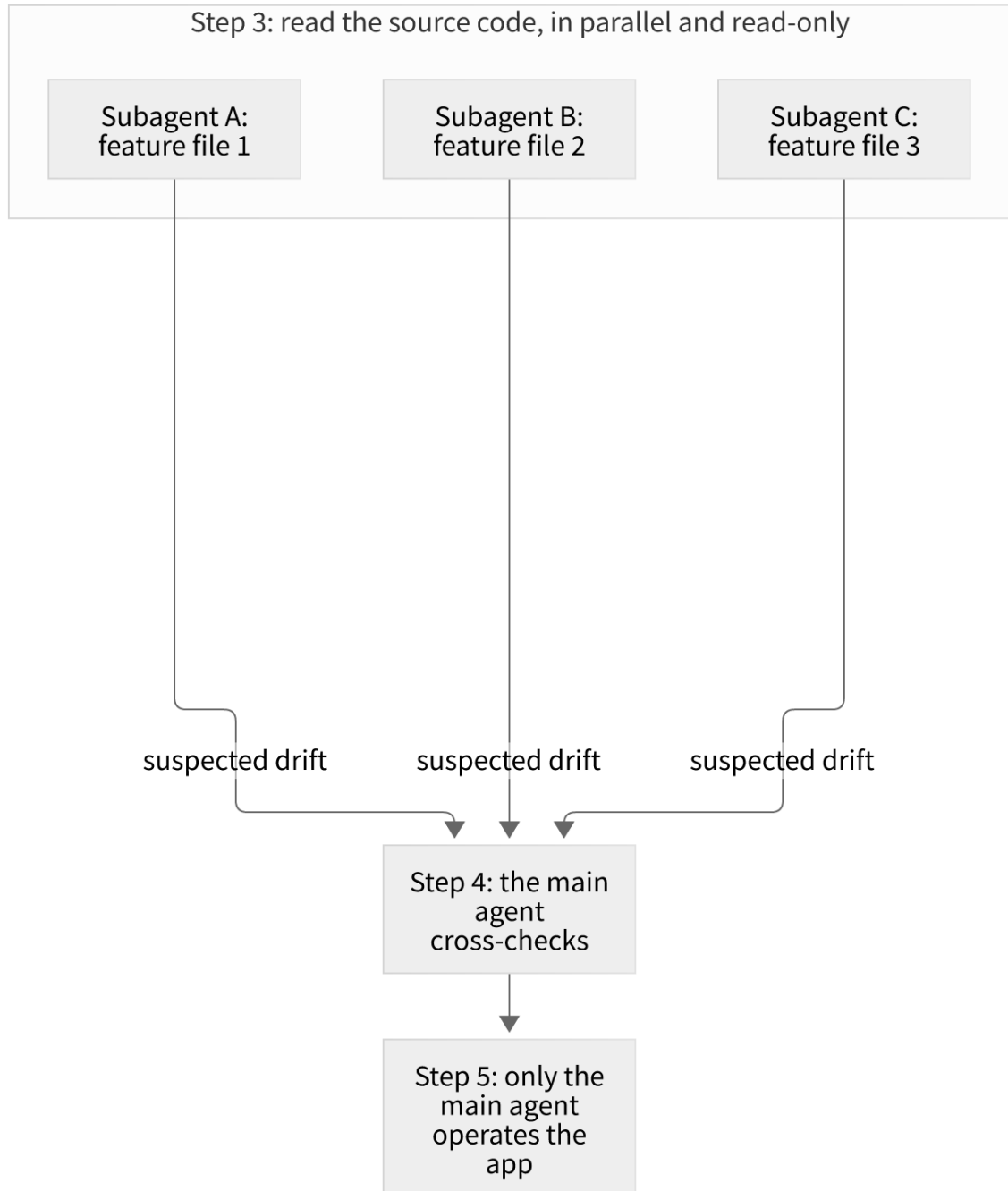
In step 5, the main agent operates even the features for which the subagents found no drift in step 3. **Reading the source code alone cannot prove that the app behaves as the Feature Map describes.** To prove it, you must run the app and verify the result, as the principle "Prove It Works" in [Chapter 19](#) says.

When several agents operate the same app, the evidence cannot be trusted

Subagents that read the source code in parallel do not interfere with each other's work. By contrast, **when several agents operate the same app at the same time, their states mix**, and nobody can trust the evidence.

For example, if one agent closes a screen that another agent opened, nobody can tell which agent's operation a screenshot shows. So only one agent, the main agent, operates the app.

The following diagram shows how the agents divide steps 3 to 5.



View diagram 2

The main agent follows three rules while it operates the app

While it operates the app in step 5, the main agent follows these three rules, whatever failure happens along the way.

- If the instance behaves unexpectedly, it runs the Doctor checks before the next operation and confirms again that the instance can still be operated.

Doctor. Read-only checks that confirm "whether operating this instance to verify something makes sense" (whether the process is running, whether the build is correct, whether the instance on that port is the one you started, whether authentication is valid, and so on)

- It keeps the evidence it collected and does not delete it during Cleanup.

Cleanup. How to clean up the instances this run created. Do not delete the evidence

- It cleans up everything it started for the operation and leaves nothing behind.

Sort problems into three kinds, and report one of three results

Problems fall into the following three kinds.

Kind	Action
Documentation drift (the description of a feature from the user's point of view is wrong or missing)	Fix the Feature Map
Harness gap (the app works correctly, but the verification skill's operating steps or scripts cannot operate it)	Fix the operating steps or scripts
Product defect (the app's behavior itself is broken)	Record it for the user. Do not fix it in this PR

The agent reports one of three results. `clean` means there is nothing to fix. `changed` means the verified fixes go into one PR. `blocked` means the agent could not finish the check.

How to write a request: the command alone is enough

`docs/guide/06-verify-and-ship.md` also shows a request that is only the command, as follows.

```
/maintain-verification-skill
```

The description in `SKILL.md` says that the following request also starts this Skill.

```
audit the verify skill
```

Summary

- `/create-verification-skill`. It finds out how to start and operate the app from the repository's code and configuration before it asks the user, and generates the verification skill and the Feature Map. It runs everything once from startup to cleanup, confirms that the evidence remains after cleanup, and then hands the verification skill over.
- **Feature Map.** A document that records, for each feature of the app, what it does, how a user gets to it, how to operate it with the verification skill, and what result you can verify.
- `/maintain-verification-skill`. It edits only the verification skill's directory and reports product regressions without fixing them. It operates every feature in the real app at least once. It ends with one of three results: `clean` when there is nothing to fix, `changed` when the verified fixes go into one PR, or `blocked` when the agent could not finish the check.

The next chapter, [Chapter 27](#), covers two Skills that check what a finished change could break and where the diff is weak: `/blast-radius` and `/interrogate`.

Chapter 27: Check what a change could break and where the diff is weak

kaito • Sources: 日本語 • English • 2026-10-03

This chapter covers two Skills:

1. `/blast-radius`
2. `/interrogate`

Both Skills examine a change that is already written. **What you want to know decides which one to use.**

Here is how to choose:

- **What could break outside the changed files** (for example, if you change the shape of a value written to a cache, does another service that reads the same cache break?). Use `/blast-radius`.
- **Whether the code in the diff itself has weak spots** (for example, does it work on empty input or when it runs twice?). Use `/interrogate`.

This chapter lists the two Skills, then explains each one from three angles: when to use it, its steps, and its output. For a Skill whose source includes a sample request, I show that request too.

At the end, I sum up how to choose between the two.

How this chapter is organized

This chapter has these sections:

- Each Skill answers a different question

- `/blast-radius` shows, by running code, why a change is safe
- `/interrogate` has reviewers on different models look for weak spots in the diff, and the main agent sorts their findings
- `/interrogate` questions what is inside the diff, and `/blast-radius` questions what is outside it
- Summary

Each Skill answers a different question

Skill	In one line	Main question
<code>/blast-radius</code>	Looks for what breaks outside the changed files, and proves the reason the change is safe by running real code	What could this change break?
<code>/interrogate</code>	Reviewers on different models look for weak spots in the same diff, and the main agent sorts their findings into four groups, such as "Act on" and "Dismissed"	Point out the problems in this diff, and be harsh

Both **Skills run only when someone calls them by name.**

Both `SKILL.md` files have a setting (`disable-model-invocation: true`) that stops the agent from choosing and running them on its own based on the conversation. So the two run only when the user calls them by name, or when `/poteto-mode` or another Skill calls them by name as one of its steps.

Who calls each one differs:

- `/interrogate`. `/poteto-mode` calls it as a Playbook step and under the rules in its "**Non-negotiables**" section. `/architect` also calls it as one of its steps. The user can also call it directly by name.
- `/blast-radius`. It appears in no "Non-negotiables" rule of `/poteto-mode`, in no Playbook step, and in no other Skill's steps. So it runs only when the user calls it by name.

`/blast-radius` shows, by running code, why a change is safe

`/blast-radius` looks for what a change that looks small could break outside the changed files. It is **a Skill that verifies the reason a change is safe by running code.**

A plausible description of the impact is equally convincing whether the description is right or wrong. So a description alone has no value. For that reason, an agent that runs `/blast-radius` does not return a sentence like "the impact should be small". It returns a fact that makes the change safe, together with the result of the code it ran to verify that fact.

When to use it: a diff that looks small but that you do not fully trust

Use it when a diff looks small but you do not fully trust it, and you want to know what else it could break.

Steps: prove the reason the change is safe by running code

The job of `/blast-radius` is not to list the callers. The agent can list the callers right away with `grep`.

The job is **to find ways the change can break that `grep` cannot see.** For example, if a service written in another language reads the same cache, that breakage does not show up in a list of callers.

So the agent looks at places `grep` cannot follow: library source, execution timing, the JSON an API returns, database columns, and feature flags. Execution timing means, for example, when async work runs or when cleanup runs as a screen closes.

Feature flag. A mechanism that turns a feature on or off by switching a setting, without changing code. For example, a feature flag can show a new screen to only some users.

But the agent does not spend its time listing the many "this might break" possibilities it finds this way. It spends its time **finding one reason the change is safe.**

One reason is enough because many changes that look risky are safe as long as one fact holds.

For example, suppose you change the code that discards cache entries. The fact "this call only discards cache entries that are already invalid and does nothing else" rules out most worries at once, such as the worry that the code also deletes entries that are still in use.

So the agent finds that fact and proves it with a script or test that runs real code.

For a large change or one with a wide reach, the agent has several models investigate it with /arena (Chapter 24). Different models find different bugs.

Output: the reason for safety, how deeply the agent verified it, and the cheapest check before merging

The output has five items:

- What changed, including changes that are hard to read from the diff
- The one fact that makes the change safe, and its proof
- Risks, each with how it breaks, a real `file:line`, its likelihood and damage, and how to check it
- What the agent checked and cleared, and why
- The cheapest test or reproduction steps to run before merging that would find a bug that could really happen, including any scripts the agent wrote

For each reason for safety, the agent also states how far it verified that reason, on a five-level scale. The level lets the reader judge how much to trust the reason.

A higher number means more certainty. The agent verifies to the highest level it can reach without spending too much effort. The five levels are as follows.

1. The agent only stated the reason in words (this alone has no value).
2. The agent pointed to the line of code (`file:line`) that supports the reason, or to the matching line in the source of the library it calls.
3. The agent traced the flow step by step toward the breakage it worries about, for example the deletion of a cache entry that is still in use, and showed that the flow does not reach that breakage.

4. The agent ran a script or test that calls the real code (if the reason is wrong, the script or test fails clearly).
5. The agent operated the running app and verified that the app behaves as the reason says.

As an example of the five levels, suppose the agent verifies the fact from earlier, "this call only discards cache entries that are already invalid and does nothing else". At each level, the agent does the following:

- At level 1, it only writes "it should discard only invalid entries".
- At level 2, it shows the file: Line of the function that discards entries, or the matching line in the source of the library it calls.
- At level 3, it traces step by step what happens when a valid entry that is still in use comes in, and shows that the entry never reaches the discard step.
- At level 4, it runs a script that loads the same library as the app, calls that function, and checks that the valid entries remain. If the reason is wrong, the script fails clearly.
- At level 5, it discards the cache in the running app and checks that the valid entries remain.

How deeply the agent verified a reason changes how far you can trust that reason. So the agent must state, for each reason, the level it reached.

If the agent cannot prove the reason, it writes unproven.

`/interrogate` **has reviewers on different models look for weak spots in the diff, and the main agent sorts their findings**

`/interrogate` launches one reviewer for each review model set in `/setup-pstack` ([Chapter 34](#)). Each reviewer looks for where the diff breaks, and **the main agent sorts their findings**.

Different models have different blind spots. So **when two models raise the same finding independently, that finding is likely a real problem**.

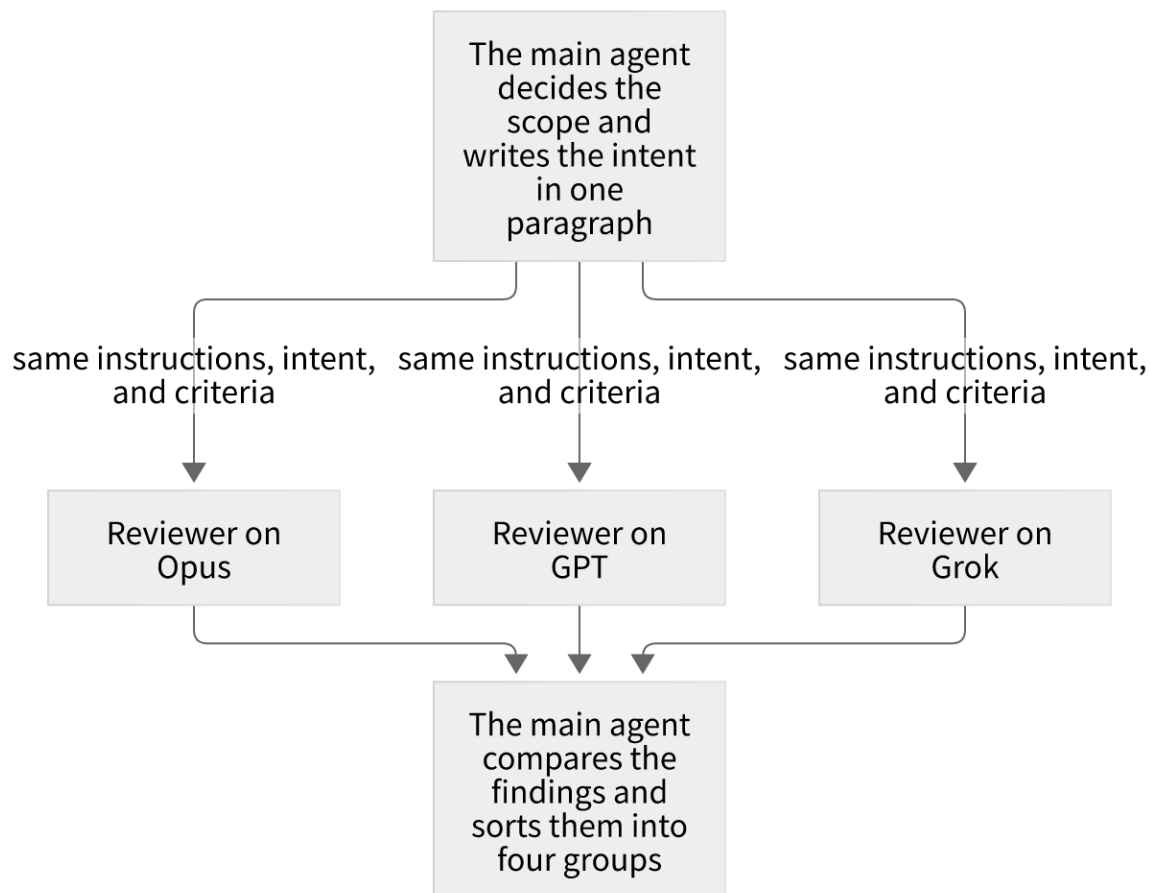
When to use it: you want several models to look for weak spots in a diff

Use it when you want a search for where the diff breaks, including a harsh code-quality review, with requests such as "review this adversarially" or "find the blind spots".

Steps: the reviewers accept the intent and look for weak spots in the code that carries it out

The main agent decides the scope to review and writes the intent of the change in one paragraph. If the user gives no scope, the scope is `git diff main...HEAD`. It reads the intent from the user's request, the commit messages, and the PR description. If the main agent is not sure of the intent, it asks the user before it goes on.

Then the main agent launches reviewers in read-only mode on three models. By default, the models are Opus, GPT, and Grok. All reviewers get the same instructions and the same review criteria, so that differences in the findings come from the models, not from different roles.



[View diagram 1](#)

The main agent gives the most weight to findings that two or more models raised independently, and less weight to findings that only one model raised.

The reviewers accept the intent, the goal of the change, as correct and examine closely whether the code achieves that goal well.

For example, if the intent is "when canceling runs, fetch the runs in one batch instead of one at a time, to remove the N+1", the reviewers do not ask whether a batch fetch is the right choice. Instead, they check whether the batch query also picks up runs that must not be canceled.

The reviewers get these review criteria:

- **Correctness.** Whether the code works as intended, for example on empty input, when it runs twice, or when the previous run stopped partway.

- **Root cause.** Whether the change fixes the root cause or only hides the symptom, for example whether a retry hides a broken assumption.
- **Structural soundness.** Whether the change fits the shape of the system, for example whether validation sits at the boundary or whether an old API stays after a new one was added.
- **Verification.** Whether there is a way to verify that the code works, for example whether a bug fix comes with a test that reproduces the bug.
- **Complexity budget.** Whether the complexity matches the job the code does. Examples of a mismatch are an abstraction with only one caller and a setting for a case that does not exist yet.
- **Security.** Whether there is a path for dangerous input, for example whether user input reaches SQL or a shell without validation.

Besides these criteria, the reviewers also look for ways to simplify the implementation a lot without changing its external behavior. An example is a rewrite that makes a whole conditional or helper function unnecessary.

So the reviewers raise findings broadly. The main agent has the full context, and it is the one that dismisses rewrites driven by taste and hypotheticals that cannot happen. An example of such a hypothetical is "this crashes if null comes in" when no caller can pass null. The reviewers saw only the diff and the one-paragraph intent. They do not know the options already tried and rejected, or the constraints that live outside the code.

Output: the findings sorted into Act on, Consider, Noted, and Dismissed

The main agent acts as a practical senior engineer, not a neutral tallier, and sorts every finding into four groups. The examples come from a review of the earlier diff that removes the N+1.

Group	Meaning	Example
Act on	A real problem that affects correctness, security, or maintainability. In a real PR, a problem that blocks the merge	The batch query also picks up completed runs that must not be canceled

Group	Meaning	Example
Consider	Reasonable, but it is unclear whether it is worth the cost of handling it now	A cache for the fetched results would make it faster, but it is unclear whether the current volume needs it
Noted	Technically correct, but not handled now	A variable name could be more specific
Dismissed	A finding that is wrong, too minor, or caused by missing context. The agent adds a short reason for dismissing it	"It is slow when there are many rows." Dismissed because the caller already caps the row count

The main agent keeps a reason for each dismissed finding. **The user can then read what the main agent dismissed and why, and use their own judgment to pick up again any finding they disagree with.**

The main agent returns only the sorted results. It does not apply the suggested changes automatically.

Sample request: use it before merging a design that people disagree on

The "Non-negotiables" of `/poteto-mode` say to use `/interrogate` before merging a design that people disagree on. The "Feature" Playbook, the "Opening a PR" Playbook that creates PRs, and `/architect` in [Chapter 24](#) also call `/interrogate`.

The bundled guide `04-design.md` has this sample request:

```
/interrogate the whole branch, but skeptically. no nitpicks unless it's an actual bug or regression.
```

An adversarial reviewer that finds no serious problem tends to fill its report with nitpicks. When the user forbids nitpicks, as in this sample request, the findings in Act on narrow to real problems and become worth reading.

/interrogate questions what is inside the diff, and /blast-radius questions what is outside it

Both check whether a change is safe. /interrogate has several models review what is inside the diff. /blast-radius looks outside the diff and proves the reason for safety by running code.

Summary

- /blast-radius. It finds the one fact that makes a change safe, such as "this call only discards cache entries that are already invalid", and proves that fact by running a script or test that calls real code.
- /interrogate. Reviewers on Opus, GPT, and Grok look for weak spots in the same diff. The main agent sorts the findings into four groups, Act on, Consider, Noted, and Dismissed, and keeps a reason for each dismissed finding.
- **How to choose.** /interrogate questions what is inside the diff, and /blast-radius questions what is outside it.

The next chapter, [Chapter 28](#), covers [/show-me-your-work](#), which records the agent's decisions and reasons so that you can trace the evidence and verify them later.

Chapter 28: Record decisions and leave evidence for long agent runs

kaito • Sources: 日本語 • English • 2026-10-03

This chapter covers `/show-me-your-work`.

`/show-me-your-work` is a Skill that records what the agent decided (for example, it threw out a subagent's work) and the grounds for each decision (for example, the screenshots were blank). It writes one decision per line in a file. A model from a different family than the one that did the work reviews the record. The agent collects the points the review flags in an "Attention" section at the end of its reply.

The verification skill from [Chapter 26](#) shows with evidence that the app works. `/show-me-your-work`, on the other hand, lets the user later follow the evidence and verify why the agent made the decisions it made.

The `SKILL.md` of `/show-me-your-work` sets `disable-model-invocation: true` ([Chapter 25](#)). The agent never picks and runs a Skill with this setting on its own from the content of the conversation. So this Skill runs only when the user calls it by name, or when `/poteto-mode`, a Playbook, or another Skill calls it by name as one of its steps.

This chapter explains `/show-me-your-work` from four angles: its role, when to use it, its steps, and how to write the request.

How this chapter is organized

This chapter is organized as follows.

- Role: leave one record of decisions that the user can review later
- When to use it: long tasks, autonomous tasks, and tasks the user checks later
- Steps: record decisions, check them against the transcript, and have a different model review them

- How to write the request: have the agent start the record
- Summary

Role: leave one record of decisions that the user can review later

`/show-me-your-work` is a Skill that keeps a record of decisions in one file so that the user can review the decisions later. It is for long tasks and for tasks that proceed while the user is away.

The bundled guide [docs/guide/07-overnight.md](#) says that if you hand a long task to the agent and step away, hope alone is not enough. You need the following three things.

- A finish condition that the agent can verify
- A dedicated worktree that does not collide with other work ([Chapter 18](#))
- A record of decisions to review later

This Skill handles the third one, "a record of decisions to review later".

You need a record of decisions because **a code diff shows only the changes that survived to the end**. An approach abandoned along the way, or a decision to undo a subagent's work, does not appear in the diff.

After a long task that ran while the user was away, the user who returns has only the diff. So the agent records its decisions and their grounds, and the user can later review why things turned out as they did.

When to use it: long tasks, autonomous tasks, and tasks the user checks later

The "Non-negotiables" section of `/poteto-mode` ([Chapter 8](#)) requires this record for long tasks, autonomous tasks, multi-phase tasks, and tasks that the user steps away from and checks later.

Playbooks and other Skills also call `/show-me-your-work`

Playbooks and Skills that handle long tasks also keep their records with this Skill.

The `SKILL.md` of this Skill sets the rules other Skills follow when they record decisions. Other Skills do not invent their own record format. They call this Skill by name and leave the format to it.

So the record has the same format whichever Playbook or Skill calls this Skill, and the user can review every record the same way.

The main callers, and why each keeps a record, are as follows.

"Autonomous run": record, at each iteration, what changed and whether the work got closer to the finish condition

"Autonomous run" is a Playbook that sets a finish condition and carries the work through to the end without stopping ([Chapter 15](#)).

In step 5, this Playbook says to record one line with this Skill at each iteration. The line says what the iteration changed and whether the work got closer to the finish condition.

"Orchestrate": gather the decisions of a project that runs for days into one record

"Orchestrate" is a Playbook in which one coordinator agent manages the work of many PRs over several days ([Chapter 16](#)). The coordinator agent writes no code.

This Playbook uses `decisions.tsv` in the directory that holds the project's records as this Skill's record.

The coordinator agent starts this record in the setup step (Install the runtime), before it starts any subagents. In the last step (Close), it checks the record as this Skill's rules require. It also has a model from a different family review the record.

This Playbook assumes the user checks in about twice a day, not every five minutes. The Close step also says to keep the directory that holds the record, instead of deleting it, and to use the directory as material for a postmortem.

/figure-it-out: with both the record and the diff, the user can trust the work

/figure-it-out is a Skill that designs the Playbook itself, for large work that no Playbook matches (Chapter 23).

In the phase that keeps a record of the work, Phase D, /figure-it-out records the work with this Skill. The work /figure-it-out handles is usually large, so the agent commits the record and reviewers can read it in the PR. **Only with both the record and the diff can the user trust the work.**

Steps: record decisions, check them against the transcript, and have a different model review them

Format: write one decision per line in a six-column TSV

This Skill keeps the record of decisions as a TSV. That way, a reviewer can read the record from top to bottom and follow the evidence to verify it.

An example TSV appears below.

Write one decision per line, and do not write the evidence as prose. Write a pointer to where the evidence is, such as a commit or a file. If the agent writes the evidence as prose, the reviewer cannot see the real thing and has to trust the prose. A pointer lets the reviewer follow it and check the real thing.

There are six columns. They run in this order: when, in which phase, what the agent decided, why, what backs the decision, and how it turned out.

Column	Contents	Example from the first line of the record below
ts	Timestamp	2026-05-24T11:15:00Z
phase	The phase in which the agent made the decision, or the name of the workstream	widget
decision	What the agent chose or did	moved the widget styles over without changing how it looks

Column	Contents	Example from the first line of the record below
why	The reason for the decision, in plain words	keep the change small and the result identical
evidence	A pointer such as a commit SHA, a PR number, a file:line that gives a file name and line number, or a screenshot path	commit 7c21e0a, pixel-diff 0
result	An outcome such as tests green, reverted, OR open	looks identical, tests pass

The following TSV takes the header line and two lines from the example in `SKILL.md`. Tabs separate the columns.

```
tsphase decisionwhy evidenceresult
2026-05-24T11:15:00Zwidgetmoved the widget styles over without changing how it looks keep the change
small and the result identical commit 7c21e0a, pixel-diff 0 looks identical, tests pass
2026-05-24T12:30:00Zwidgetthrew out a helper's work because its screenshots were blank checked the
real files instead of trusting its summary worktree resetreverted, tightened the instructions for
next time
```

The first line is the decision "moved the widget styles without changing how they look". The evidence is commit 7c21e0a and a pixel difference of zero (pixel-diff 0). A reviewer can open that commit and check the real thing.

The second line is the decision "threw out a subagent's work because its screenshots were blank". The agent did not trust the subagent's "done" summary. It checked the actual files before throwing the work out.

The agent records the points where a decision branched, not every action. It does not record the obvious or minor actions.

Here are examples of when the agent records a line.

- When it chooses one of two approaches
- When it finishes a stage of the work and a check produces a result
- When it changes direction or undoes a change
- When it hits a problem that blocks progress

The agent writes each line in plain words. It also applies the rules of `/unslop` ([Chapter 32](#)) to the sentences in the log. `/unslop` removes AI-style phrasing.

Appending and location: append decision lines with `log.sh`

The agent appends one line at a time with `log.sh`

The agent uses `scripts/log.sh` to add lines.

```
scripts/log.sh <logfile> <phase> <decision> <why> <evidence> <result>
```

The script shapes the line into TSV format. For example, it fills in the timestamp and replaces tabs and newlines inside a cell with spaces.

Keep the log in the working directory, and commit it only for large work

The agent keeps the log in `decisions.tsv` in the working directory and does not commit it by default. The agent commits the log only for work large enough that a reviewer needs the record of decisions to trust the result.

An example is work such as "moved the styles of about 100 components without changing how they look".

The code tells you what changed. But the code keeps no trace of what verified that the look really did not change, or which approaches were dropped along the way.

The larger the work, the more such decisions there are. So the reviewer follows the decisions and evidence in the committed log to verify the reported result.

Fix a wrong decision by adding a correcting line, not by rewriting the line

The log is append-only. You fix a wrong decision by adding a new line that corrects it.

If you rewrite or delete a line, the record no longer shows how the decision changed along the way.

A new run writes a start line first, to separate it from the previous run's lines

This Skill counts one conversation with one agent as one **run**. When the user sends further requests later in the same conversation and the agent keeps working, that work belongs to the same run. Work that continues after a long conversation is summarized also belongs to the same run.

When another agent takes over the work, or a new chat starts, a new run begins.

A long task does not always finish in one run. Another agent may take over partway, or the user may ask for the rest in a new chat. This Skill says to keep the record of one task in one log, `decisions.tsv` in the working directory. So a new run also appends to the log that the previous run wrote.

As a result, one log holds lines written by several runs. If the reviewer cannot tell which run wrote which line, they cannot pull out and read only the new run's decisions.

So when a run appends to a log that already has lines, its agent first writes one line with start in the phase column.

This start line marks that the new run wrote the lines below it. In the start line, the agent writes two things: the timestamp range of the lines above it, which another run wrote, and something that identifies its own run, such as the agent's ID.

For example, when a run in a new chat appends to a log written by a run in the previous chat, the log looks like this.

```
tsphase decisionwhy evidenceresult
2026-05-24T11:15:00Zwidgetmoved the widget styles over without changing how it looks keep the change
small and the result identical commit 7c21e0a, pixel-diff 0 looks identical, tests pass
2026-05-24T12:30:00Zwidgetthrew out a helper's work because its screenshots were blank checked the
real files instead of trusting its summary worktree resetreverted, tightened the instructions for
next time
2026-05-25T09:00:00Zstart rows 2026-05-24T11:15:00Z to 12:30:00Z were written by another run picking
up yesterday's log agent a1b2c3open
2026-05-25T09:40:00Zwidgetmoved the button styles over same approach as the widget commit 9d4e2f1,
pixel-diff 0 looks identical, tests pass
```

The line with the timestamp 2026-05-25T09:00:00Z is the start line that today's run wrote first. A reviewer can verify today's decisions by reading only the lines below it.

In the later step, "Audit: before handing off to the user, check the log against the transcript", the agent also checks only the lines its own run wrote against its own run's transcript. The start line tells the agent where its own lines begin.

Audit: before handing off to the user, check the log against the transcript

Before it returns its result, the agent that did the work verifies that the record matches the facts. The agent checks the lines that this run wrote against this run's transcript. It checks the following points.

- Every line matches a real decision
- The evidence exists and shows what the line says it shows
- No important branch that shaped the work is missing from the record

Correct the log, not the story. The audit never edits or removes a row, even an invented one.

When the audit finds a line that does not match the facts, the agent does not delete or rewrite it. Instead, it adds a correcting line that states what actually happened. The rule is the same as "Fix a wrong decision by adding a correcting line, not by rewriting the line" earlier in this chapter.

For example, suppose the audit finds a line whose evidence is a commit that does not exist. The first line is the one that does not match the facts, and the second is the correcting line the agent added.

```
tsphase decisionwhy evidenceresult
2026-05-24T13:00:00Zwidgetfixed the button color to fix the broken look commit 9d4e2b1tests green
2026-05-24T15:20:00Zwidgetcorrecting the 13:00 row. the button color was not fixed, and commit
9d4e2b1 does not existchecked against the transcript and found the row did not match the facts this
run's record in agent-transcripts/ open
```

The first line stays in place, so a reviewer can trace where the record drifted from the facts and when the agent corrected it.

Review: a model from a different family reads the record

The agent that did the work starts a subagent on **a model from a different family than the one that did the work**, and has that subagent read the record and the transcript.

This **reviewer** subagent does not redo the work. It looks for points the user should pay attention to and picks up items such as the following.

- Decisions with weak grounds or no grounds
- Skipped verification, or verification the record says the agent did but that has no evidence in the transcript
- Choices that look risky in hindsight, such as the following.
 - Decisions made too early
 - Scope changes that widened the work too far
 - A fix that only hides the symptom and leaves the root cause in place. The symptom is the failure that shows on the surface, such as an error or a broken screen.
- Problems the user would not notice from a quick read of the record

A self-review by the model that did the work does not replace this review. **A model that missed something during the work tends to miss it the same way when it looks again.**

The bundled guide [docs/guide/04-design.md](#) gives the same reason for `/interrogate` (Chapter 27), which has several models look for weak points in a diff.

When the agent keeps a record for the work, it always adds an "Attention" section at the end of its reply. The "Attention" section lists the points the reviewer found that the user should pay attention to. For example, consider a reviewer that reviews the following two lines from this chapter's examples.

The line for the decision to move the widget styles^[1] is as follows.

```
2026-05-24T11:15:00Zwidgetmoved the widget styles over without changing how it looks keep the change
small and the result identical commit 7c21e0a, pixel-diff 0 looks identical, tests pass
```

The line that cites a commit that does not exist as its evidence^[2] is as follows.

```
2026-05-24T13:00:00Zwidgetfixed the button color to fix the broken look commit 9d4e2b1tests green
```

In this case, the "Attention" section looks like this (this example is not in the source).

```
Attention
reviewed by grok-4.7-xhigh-fast

- decisions.tsv row 11:15: the only basis for pixel-diff 0 is one screenshot of the top page. The
other screens that use the widget were not checked.
- decisions.tsv row 13:00: cited commit 9d4e2b1, which does not exist, as evidence. Row 15:20
corrects it, but the button color is still not fixed.
```

The first line, reviewed by grok-4.7-xhigh-fast, shows which model did the review. Below it, the agent lists the reviewer's findings one at a time. Each finding states which line of decisions.tsv or which part of the transcript it is about. From a finding, the user can trace the line and check the evidence.

If there are no findings, the agent writes "No flags" in place of the list. It never writes only the reviewed by line with neither findings nor "No flags". It also cannot leave out the reviewed by line when the answer is "No flags".

```
Attention
reviewed by grok-4.7-xhigh-fast

No flags
```

Without the reviewed by line, the user cannot verify that a model from a different family really did the review, rather than the model that did the work.

How to write the request: have the agent start the record

The [README](#) has an example of a request that starts the record.

```
/show-me-your-work keep a decision trail i can review when i'm back.
```

When the user returns, they can get a report on last night's work with the following request from `docs/guide/07-overnight.md`. The report ends with an "Attention" section.

```
/show-me-your-work catch me up on what you did last night
```

The user reads the "Attention" section of the reply first, and then the log lines that the section points to. The "Attention" section lists only the decisions the reviewer flagged, so the user does not have to reread all of a long task they handed off.

Summary

- **Role.** The Skill keeps a record of decisions in one file so that the user can review the decisions later. It is for long tasks and for tasks that proceed while the user is away. It keeps the record because a code diff shows only the changes that survived to the end.
- **Record format.** The agent keeps one decision per line in a six-column TSV, append-only. It does not delete a wrong line. It adds a correcting line instead.
- **Audit.** Before it returns the result to the user, the agent checks the log against the transcript and adds a correcting line for any line that does not match the facts.
- **Review and the "Attention" section.** A model from a different family reads the record and the transcript. The agent ends its reply with an "Attention" section that lists the reviewing model's name and its findings. Even with no findings, the agent keeps the model name in the section.

The next three chapters, [Chapter 29](#) to [Chapter 31](#), cover three Skills that keep code quality, one at a time. [Chapter 29](#) covers `/typescript-best-practices`, which collects TypeScript rules.

脚注

1. Quoted from the example in "Format: write one decision per line in a six-column TSV" in this chapter. ↩
2. Quoted from the example in "Audit: before handing off to the user, check the log against the transcript" in this chapter. ↩

Chapter 29: Keep code quality with 16 TypeScript rules

kaito • Sources: 日本語 • English • 2026-10-03

This chapter covers `/typescript-best-practices`.

`/typescript-best-practices` is a Skill the agent follows in TypeScript code. With it, the agent makes impossible states impossible to write in the type system. An example of an impossible state is a value that is loading and also has an error. The agent also validates data from outside once, at the place where the data enters.

This chapter and the next two, up to [Chapter 31](#), cover the following three Skills that keep the quality of the code the agent writes, one per chapter.

1. `/typescript-best-practices` (this chapter)
2. `/tdd` ([Chapter 30](#))
3. `/no-comments` ([Chapter 31](#))

In this book, I group the three as **Skills that have something other than the author verify the code, instead of relying only on the attention of the agent that wrote it**. In `/typescript-best-practices`, the compiler verifies in place of the author.

This chapter explains `/typescript-best-practices` from four angles: its role, when it is used, its rules, and how to write a request.

How this chapter is organized

This chapter has the following sections.

- Role: turn "Type System Discipline" into concrete TypeScript practice
- When it is used: it loads automatically when the agent reads or writes a TypeScript file
- Rules: 16 rules for writing TypeScript

- How to write a request: the agent follows the rules even if you do not name the Skill
- Summary

Role: turn "Type System Discipline" into concrete TypeScript practice

`/typescript-best-practices` is a set of TypeScript rules.

The principle "**Type System Discipline**" in [Chapter 18](#) explains why you make invalid states impossible to write in the type system. This Skill shows how to make invalid states impossible to write in TypeScript.

At the top of its `SKILL.md`, this Skill asks the agent to apply that principle first, before it uses the rules the Skill defines.

When it is used: it loads automatically when the agent reads or writes a TypeScript file

This Skill loads automatically when the agent reads or edits a TypeScript file. The `paths` setting in `SKILL.md` targets TypeScript files.

This Skill has the setting `disable-model-invocation: true` ([Chapter 25](#)), so the agent never picks it from the content of the conversation and runs it.

Because the Skill loads automatically, the agent cannot forget to call it. You do not need to call it by name either.

Rules: 16 rules for writing TypeScript

This Skill consists of 16 rules and the code examples in [references/patterns.md](#).

`references/patterns.md`. A file that collects code examples for each rule defined in `SKILL.md`

It has code examples for 14 of the 16 rules, all except "Real tests" and "Structured telemetry"

The bundled guide [docs/guide/05-build-and-clean.md](#) describes this Skill as **the type-system principle turned into concrete TypeScript rules**.

The 16 rules, in order, are as follows.

Rule	Summary
Discriminated union	Decide in the type which fields each state can have, so that nobody can write contradictory values such as "loading and also has an error"
Branded types	Distinguish in the type between values that hold the same kind of string but mean different things, such as an ID and a plain string, to prevent mix-ups
Constructive modeling	Express a value with a condition, such as "a non-empty array", as a type that makes it impossible to build a value that breaks the condition
Simplest total type	Make a type stricter only where the current type forces awkward code
unknown over any	Receive data from outside as a type that the code cannot use until it checks the contents
Schemas before guards	Before you hand-write code that checks data, use the validation library the project already uses
No as	Do not declare "this value is this type" without checking the contents
Narrowing hierarchy	Pick the way to check a value's type in order, starting from the safest
Type guards	A function that decides "is this that type" really checks what its name says
Exhaustiveness checks	When you add a kind of state, a compile error tells you that you forgot to handle it
satisfies over as	Check that a value matches a type without losing the value's detailed information

Rule	Summary
Validation at the boundary	Check data from outside once at the entry point, and do not check it again inside
Schema-derived types	Before you write a new type, take the parts you need from a type that already exists
Object args	Pass arguments by name to prevent mixing up their order
Real tests	Test anything you can run locally with the real thing, not a fake
Structured telemetry	Log information useful for investigation in a fixed shape, and do not leave <code>console.log</code> behind

The following sections go through each rule with the code examples from `references/patterns.md`. I have rewritten the comments in the examples.

1. Use a discriminated union to make impossible states impossible to write

A discriminated union is a type whose data takes a different shape for each kind. Each shape is a variant. A field that shows the kind, such as `kind`, tells the variants apart (Chapter 18).

Variant. Each of the shapes that one type can take

For example, in `DiffState` below, the three variants are `{ kind: "loading" }`, `{ kind: "ready"; diff: GitDiff }`, and `{ kind: "error"; error: string }`

`references/patterns.md` uses a type built from a boolean and optional fields as the bad example.

In the bad example, you can write a value that has both `loading: true` and `error`, which means "loading and also has an error". In the good example, each `kind` decides which fields it can have, so you can write only the three valid states.

```
// A diff (the list of changed files)
type GitDiff = { files: string[] };
```

```
// Bad: a mix of a boolean and optional fields can also express contradictory states
type DiffStateLoose = { loading: boolean; diff?: GitDiff; error?: string };

// Good: only valid states can exist
type DiffState =
  | { kind: "loading" }
  | { kind: "ready"; diff: GitDiff }
  | { kind: "error"; error: string };
```

So if the fields depend on the state, write a discriminated union instead of a boolean and optional fields.

2. Use branded types to prevent mixing up values that mean different things

A branded type is a basic value, such as a string, with a mark like & { readonly __brand: "AgentId" } attached ([Chapter 18](#)). It catches mix-ups at compile time, such as an unchecked string passed to a function that should receive an agent ID.

In the next example, `parseAgentId` at the data's entry point checks once that the string has the shape of a UUID, and then turns it into an `AgentId`. The code uses as in only one place, after the check.

```
type AgentId = string & { readonly __brand: "AgentId" };

// Checks whether a string has the shape of a UUID
declare function isUUID(input: string): boolean;

// Boundary: check the string, then turn it into an AgentId
function parseAgentId(input: string): AgentId {
  if (!isUUID(input)) throw new Error(`Invalid agent id: ${input}`);
  // The value has been checked, so as is allowed here
  return input as AgentId;
}

// Inside: accept only an AgentId and do not check the ID again
function focusAgent(id: AgentId): void {
  /* Treat the input as already checked */
}

// Compiles
focusAgent(parseAgentId("3f2b8c1e-9a4d-4e6b-8f1a-2c5d7e9b0a13"));
```

```
// An unchecked string, so this is a compile error
focusAgent("3f2b8c1e-9a4d-4e6b-8f1a-2c5d7e9b0a13");
```

Keep the mark in the shape readonly `__brand: "X"`, and do not invent a new way to write it.

So if values share the same underlying type but your code must handle them differently by meaning, make them branded types. Attach the type only after you check the value at the boundary.

3. Build types so that invalid values cannot be put together

This rule, Constructive modeling, builds the type from valid parts only. It does not add runtime checks to a loose type to stop invalid values.

```
// A non-empty array: the first element always exists
type NonEmpty<T> = [T, ...T[]];

// Bad: accepts T[], so every caller repeats the empty check
function pickWinnerLoose(entries: string[]): string {
  if (entries.length === 0) throw new Error("no entries");
  return entries[Math.floor(Math.random() * entries.length)];
}

// Good: an empty value cannot exist in this type
function pickWinner(entries: NonEmpty<string>): string {
  return entries[Math.floor(Math.random() * entries.length)];
}

// Narrow a value received as T[] once with a type guard. After that, the type carries the fact
const isEmpty = <T>(arr: T[]): arr is NonEmpty<T> => arr.length > 0;
```

So before you reject invalid values at runtime, define a type in which invalid values cannot be put together.

4. Make a type stricter only where the loose type forces awkward code

This rule, Simplest total type, does not ask you to make every type stricter. As long as every operation on an array can return a result for every input, including the empty array, keep `T[]`.

Make a type stricter only when the loose type forces awkward code where it is used. The signs of awkward code are the non-null assertion operator `!`, a type assertion such as `arr[0] as T`, and an error that "should never happen".

```
type NonEmpty<T> = [T, ...T[]];
type Session = { id: string; startedAt: Date };

// T[] is fine here: it can return 0 even for an empty array
const sum = (xs: number[]) => xs.reduce((a, b) => a + b, 0);

// Bad: ! hides the empty-array case from the compiler
function newestSessionLoose(sessions: Session[]): Session {
  return sessions.at(0)!;
}

// Good: a stricter argument type removes the need for !
function newestSession(sessions: NonEmpty<Session>): Session {
  return sessions[0];
}
```

`references/patterns.md` also lists another correct option, which is to weaken the return type to `Session | undefined`.

So decide whether to make a type stricter by whether the loose type forces you to write things like the non-null assertion operator `!`.

5. Receive data from outside as unknown

Code can use a value received as any without any check. Code cannot use a value received as unknown until it checks the value's type, so the agent cannot skip the validation.

```
// Bad: receiving any lets you use the value without checking it
function handleLoose(input: any) {
  // This compiles even if foo does not exist
  return input.foo.bar;
}

// Good: receive unknown, narrow the type, then use it
function handle(input: unknown) {
  if (typeof input === "object" && input !== null && "foo" in input) {
    // Here the compiler has verified that input has foo
  }
}
```

```
}
}
```

So if you do not know a value's shape until runtime, receive it as `unknown` and narrow the type before you use it.

6. Before you hand-write a type guard, use the repository's schema library

For data from outside, the agent first looks for the runtime schema library the repository uses and for existing schemas, before it hand-writes a type guard that checks each field one by one. A type guard is a function that checks whether a value is a given type and returns a boolean.

Let one schema do the validation, and derive the TypeScript type from the schema. If you keep a schema, an interface of the same shape, and a type guard separately, you will forget to update one of the three, and the three will disagree.

```
import { z } from "zod";

const UserSchema = z.object({
  id: z.string().uuid(),
  role: z.enum(["admin", "member"]),
});

// Derive the type from the schema
type User = z.infer<typeof UserSchema>;

function parseUser(input: unknown): User {
  return UserSchema.parse(input);
}
```

So validate data from outside with the schemas the repository already uses, and derive the types from those schemas too.

7. Write as only after checking the value

as, a type assertion, tells the compiler "this value is this type" without checking the value. The code compiles even if the actual value's type differs, so the code that uses the value may stop with an error at runtime.


```

type User = { id: string; name: string };
declare const data: unknown;

// Bad: declares it a User without checking
const user = data as User;

// Good: check at the boundary, then use as
function parseUser(data: unknown): User {
  if (typeof data !== "object" || data === null) {
    throw new Error("expected object");
  }
  if (!("id" in data) || typeof data.id !== "string") {
    throw new Error("expected id");
  }
  // ...check every remaining field too
  // Every field has been checked, so as is allowed here
  return data as User;
}

```

When the agent removes `as` from existing code, it finds out why TypeScript cannot infer the type, and then applies one of the following fixes.

- If there is no field that shows the kind, add the field and make the type a discriminated union
- If the original type is too wide, such as `Record<string, unknown>`, narrow the type
- If the boundary has no type, add a parsing function or a schema
- If the type system cannot express the type, use a branded type or `satisfies`

So if you write `as`, check the value first.

8. Narrowing has an order of preference

`references/patterns.md` prefers the ways to narrow a type in the following order.

1. A `switch` or `if` on the field that shows the kind (the compiler narrows automatically)
2. The `in` operator (`"key" in obj` narrows to the variant that has that field)
3. `typeof` and `instanceof`, for basic values and class instances
4. A custom type guard, when the three above are not enough
5. `as`, only after you check the value

The following code shows each of the five ways.

```

type Shape =
  | { kind: "circle"; radius: number }
  | { kind: "rect"; width: number; height: number };

// 1. switch on the kind field: the compiler narrows automatically from kind
function area(s: Shape): number {
  switch (s.kind) {
    case "circle":
      return Math.PI * s.radius ** 2;
    case "rect":
      return s.width * s.height;
  }
}

// 2. The in operator: only a circle has radius, so here it narrows to a circle
function areaByIn(s: Shape): number {
  if ("radius" in s) return Math.PI * s.radius ** 2;
  // Here it narrows to a rectangle
  return s.width * s.height;
}

// 3. typeof and instanceof: narrow to a basic value or a class instance
function describe(value: string | number | Date): string {
  if (typeof value === "string") return value;
  if (value instanceof Date) return value.toISOString();
  // Here it narrows to number
  return value.toFixed(2);
}

// 4. A custom type guard: a function checks a condition the three above cannot (a non-empty string)
function isEmptyString(value: unknown): value is string {
  return typeof value === "string" && value.length > 0;
}

// 5. as: use only after the value is checked
type AgentId = string & { readonly __brand: "AgentId" };
declare function isUUID(input: string): boolean;

function toAgentId(input: string): AgentId {
  if (!isUUID(input)) throw new Error(`Invalid agent id: ${input}`);
  return input as AgentId;
}

```

So when you narrow a type, use the highest method on the list that works.

9. A type guard checks the value the way its name says

A type guard must check the value the way its name says. A type guard that does not check hides a bug behind a name that claims safety. Such a type guard is worse than as. Name a type guard `isX` or `hasX`.

```
type Shape =
  | { kind: "circle"; radius: number }
  | { kind: "rect"; width: number; height: number };

// Actually checks the field that shows the kind
function isCircle(s: Shape): s is Shape & { kind: "circle" } {
  return s.kind === "circle";
}
```

So if you write a type guard, the type guard must check what its name claims.

10. Use an exhaustiveness check to make a missing case a compile error

Write `const _exhaustive: never = x;` in the default of a switch. Then, if you add a variant and forget to write the case that handles it, compilation fails ([Chapter 18](#)).

```
type Shape =
  | { kind: "circle"; radius: number }
  | { kind: "rect"; width: number; height: number };

function area(s: Shape): number {
  switch (s.kind) {
    case "circle":
      return Math.PI * s.radius ** 2;
    case "rect":
      return s.width * s.height;
    default: {
      // If you add a variant to Shape and forget its case, this line is a compile error
      const _exhaustive: never = s;
      return _exhaustive;
    }
  }
}
```

In a switch that returns no value, write `void _exhaustive;` instead of `return _exhaustive;`.

So if you branch on each variant, check exhaustiveness in default.

11. Use `satisfies` instead of `as` to check that a value matches a type

`satisfies` checks that a value matches a type without widening literal types. A literal type stands for one specific value, such as `"dark"`.

```
type Config = { theme: "dark" | "light"; cols: number };

// Bad: with as, the type of theme widens to "dark" | "light" and the fact that it is "dark" is lost
const configLoose = { theme: "dark", cols: 3 } as Config;

// Good: satisfies checks that the value matches the type and keeps the literal type
const config = { theme: "dark", cols: 3 } satisfies Config;
// The type of config.theme is "dark", not string
```

So if you only want to check that a value matches a type, use `satisfies`, not `as`.

12. Validate once where data enters, and trust the types inside

Where data enters, parse it into a domain type, and inside, use the typed value without checking it again. This rule follows the principle "**Boundary Discipline**" (Chapter 18).

Domain type. A type that represents a business concept the app handles, such as a user ID or an order

For example, the following code holds order data received from outside in two different types

```
// A domain type: the fields of an order and the type of each are fixed
type Order = { id: string; userId: string; total: number };

// Not a domain type: you do not know which fields exist, so you must check every time you use it
type RawOrder = Record<string, unknown>;
```

With validation at the boundary, the code where data enters checks a value like `RawOrder` once and turns it into an `Order`. The code inside handles only `Order`.

So validate data from outside once at the entry point, and trust the types inside.

13. Before you write a new type, derive it from existing types

If an OpenAPI definition, a GraphQL schema, or a database migration already defines the shape of the data, the agent does not rewrite a type of the same shape. It derives the type from the types generated from that source.

```
// A generated type (normally imported from the generated module)
type ChecksMessage = {
  totalCount: number;
  checks: { name: string; status: string }[];
  updatedAt: string;
};

// Bad: rewrites the same shape, so it drifts when the schema changes
type CheckSummary = {
  totalCount: number;
  checks: { name: string; status: string }[];
};
function renderChecksLoose(s: CheckSummary) {
  /* ... */
}

// Good: take only the needed fields from the generated type
function renderChecks(s: Pick<ChecksMessage, "totalCount" | "checks">) {
  /* ... */
}
```

Before you write a new interface, check whether you can use `Pick`, `Omit`, `Parameters`, `ReturnType`, `Awaited`, or `typeof`.

So if a source definition exists, do not rewrite the type. Derive it from that definition.

14. Pass arguments as an object, not by position

When you pass arguments as an object, the names show which value is which argument, and you can no longer mix up their order.

```
type Selection = {
  startLineNumber: number;
  startColumn: number;
  endLineNumber: number;
  endColumn: number;
};
declare const uri: string;
```

```

declare function openFileLoose(uri: string, selection: Selection): void;
declare function openFile(args: { uri: string; selection: Selection }): void;

// Bad: if two arguments are swapped, it still compiles as long as the types match
openFileLoose(uri, {
  startLineNumber: 10,
  startColumn: 1,
  endLineNumber: 10,
  endColumn: 1,
});

// Good: order does not matter, and the names show what each value is
openFile({
  uri,
  selection: {
    startLineNumber: 10,
    startColumn: 1,
    endLineNumber: 10,
    endColumn: 1,
  },
});

```

However, this rule does not apply to code that runs over and over, such as per-frame rendering, tokenizers, and parsers. Each call creates an argument object and allocates memory. In code that repeats many times, those allocations slow the code down.

- **Per-frame rendering.** Code that redraws the screen dozens of times a second, as in animations and games

So if there are several arguments, pass them as an object, except in code that runs over and over.

15. Test anything you can run locally with the real thing, not a mock

The agent does not mock anything it can run locally. It runs the real thing in the test. It mocks only what it cannot run locally, such as an external payment service. To verify screens, it builds the app and runs it.

```

declare function test(name: string, fn: () => Promise<void>): void;
declare function expect<T>(actual: T): { toBe(expected: T): void };

```

```

declare function makeTempDir(): Promise<string>;
declare function saveNote(dir: string, text: string): Promise<void>;
declare function loadNote(dir: string): Promise<string>;
declare const fakeFs: { writeFile: { calledTimes: number } };

// Bad: replaces even file saving, which can run locally, with a fake, and checks only how many
times the fake was called
test("saves a note", async () => {
  await saveNote("/notes", "groceries");
  expect(fakeFs.writeFile.calledTimes).toBe(1);
});

// Good: writes a real file to a temporary directory, reads it back, and checks its contents
test("reads back a saved note", async () => {
  const dir = await makeTempDir();
  await saveNote(dir, "groceries");
  expect(await loadNote(dir)).toBe("groceries");
});

```

16. Keep diagnostic information in structured logs

The agent keeps diagnostic information in structured logs, which are logs with fixed fields. The logs hold enough information to trace a bug from an ID. It does not leave console.log in code it ships.

```

declare const logger: {
  error(event: string, fields: Record<string, unknown>): void;
};
declare const orderId: string;
declare const userId: string;
declare const err: Error;

// Bad: a plain string does not let you find out later which order failed or why
console.log("payment failed");

// Good: record the event name, plus the IDs and reason you need for investigation, in fixed fields
logger.error("payment_failed", { orderId, userId, reason: err.message });

```

With the log in the good example, a person or an agent who investigates a bug can search by `orderId` and trace what happened to that order.

So keep diagnostic information in logs that you can trace by ID later.

How to write a request: the agent follows the rules even if you do not name the Skill

This Skill has no example requests.

The bundled guide [docs/guide/05-build-and-clean.md](#) also says that the user does not call this Skill as a slash command during work.

Summary

- **Who verifies.** The compiler finds type errors in place of the agent that wrote the code.
- **When it is used.** It loads automatically when the agent reads or writes a TypeScript file.
- **Rules.** The agent makes impossible states impossible to write in the type system, and validates data from outside once, where the data enters.

The next chapter, [Chapter 30](#), covers `/tdd`, the second of the three Skills that keep code quality. Before the agent fixes a bug, `/tdd` has the agent write a test that fails before the fix.

Chapter 30: TDD: verify with a failing test before the fix

kaito • Sources: 日本語 • English • 2026-10-03

This chapter covers `/tdd`.

`/tdd` is a Skill that has the agent first write a test that fails as long as the bug exists, before it fixes the bug. For example, for the bug "a retry delivers two notifications", the agent writes a test that verifies "only one notification arrives". This test fails before the fix and passes after it.

`/tdd` is the second of the three Skills that keep code quality, continuing from [Chapter 29](#). In `/tdd`, a regression test that fails before the fix and passes after it verifies the fix, in place of the agent that wrote the code.

This chapter explains `/tdd` from four angles: its role, when to use it, its procedure, and how to write the request.

How this chapter is organized

This chapter is organized as follows.

- Role: reproduce the broken behavior in a test before fixing the bug
- When to use it: only when the user asks for TDD or when a cheap test is within reach
- Procedure: before the fix, verify that the test "fails for the right reason"
- How to write the request: the command alone is enough
- Summary

Role: reproduce the broken behavior in a test before fixing the bug

`/tdd` is a Skill that has the agent write one regression test before it fixes a bug. The agent then verifies that the test fails before the fix and passes after it.

The name says TDD. But as the `SKILL.md` title "TDD Bug Fix" shows, `/tdd` **is a Skill that handles only bug fixes, not a guide to test-first design of new features.**

`/tdd` does not require the agent to write the test first every time. **The agent writes the test first only when the user asks for it or when a cheap test is within reach.**

When to use it: only when the user asks for TDD or when a cheap test is within reach

`/tdd` has the setting `disable-model-invocation: true` ([Chapter 25](#)), so the agent never picks and runs `/tdd` based on what the conversation is about.

Two callers run `/tdd`: the user, who calls it by name, and `/poteto-mode`. The [README](#) explains that `/poteto-mode` runs `/tdd` when its procedure needs it.

The `SKILL.md` description limits the use of `/tdd` to the following cases.

- The user asks for TDD, a failing test, or a regression test
- The agent knows that a test it can run locally right away can verify the bug

Put the other way around, the agent does not add a new test in the following cases.

- The path to a test is unclear
- The test would take a lot of work
- The agent cannot write the test without combining many parts
- Nobody asked for a test

"A cheap test is within reach" means, for example, that the buggy code already has unit tests and one more test there is enough.

If the test needs heavy setup, do not add it, and switch to the nearest check

The cases above, "the test would take a lot of work" and "the agent cannot write the test without combining many parts", mean that the test needs things like the following.

- A new, wide-ranging harness that runs the tests
- Fragile mocks that break when the implementation changes a little
- Infrastructure for E2E tests that take a long time to run
- State that exists only in production, such as data in the production database
- Vague reproduction steps
- Large changes to fixtures that have nothing to do with the bug

When the test needs such things, the agent does not add a new test. It switches to the nearest check, such as a script or a manual reproduction command.

The agent does not add a test in these cases because **a test that needs such setup to work is weak evidence that the bug is fixed.**

For example, a test that passes with fragile mocks checks only the mocks in place of the real parts. According to the bundled guide [docs/guide/10-recipes-and-pitfalls.md](#), such a test shows the fix less convincingly than a run of the real command.

The "Bug fix" Playbook refers to `/tdd` only when a cheap test is within reach

`/poteto-mode` also uses `/tdd`. The "Bug fix" Playbook defines how to fix a bug. In step 5, the Playbook requires that the commit that reproduces the bug, such as a test that fails on the code before the fix, comes before the fix commit in the Git history. Step 5 then refers to `/tdd` when a cheap test is within reach.

However, this Playbook also says to skip `/tdd` when the test would take a lot of work, when the agent cannot write the test without combining many parts, or when it is unclear how to write the test.

Procedure: before the fix, verify that the test "fails for the right reason"

Six steps verify everything from the failure before the fix to the pass after it

The /tdd procedure has six steps.

1. **Understand the bug.** The agent identifies the intended behavior, the current behavior, and the smallest reproduction of the bug, which is the shortest sequence of actions or inputs that triggers it.
2. **Pick the test that verifies the narrowest scope.** From the tests the buggy code already uses, the agent picks the one that verifies the buggy spot in the narrowest scope. If it finds no test that it can use cheaply, it does not build a new test from scratch only to complete this step.
3. **Write the failing test first.** The agent adds one test, the smallest that can catch this bug. The test's expected value is the behavior the code should have. For example, for the bug that delivers two notifications, the expected value is 1, not 2, even though the current code sends two.
4. **Run the test before the fix.** The agent verifies that the test fails for the intended reason, which is the bug itself.
5. **Fix the bug.** The agent makes the smallest change that meets the intended behavior. It changes the application code, not the test. It also keeps the contracts that the surrounding code depends on, such as the shape of a function's parameters and return value.
6. **Rerun the regression test.** The agent verifies that the test passes.

The diagram below shows the branches for when the agent finds no cheap test and for when the test before the fix did not fail for the intended reason.

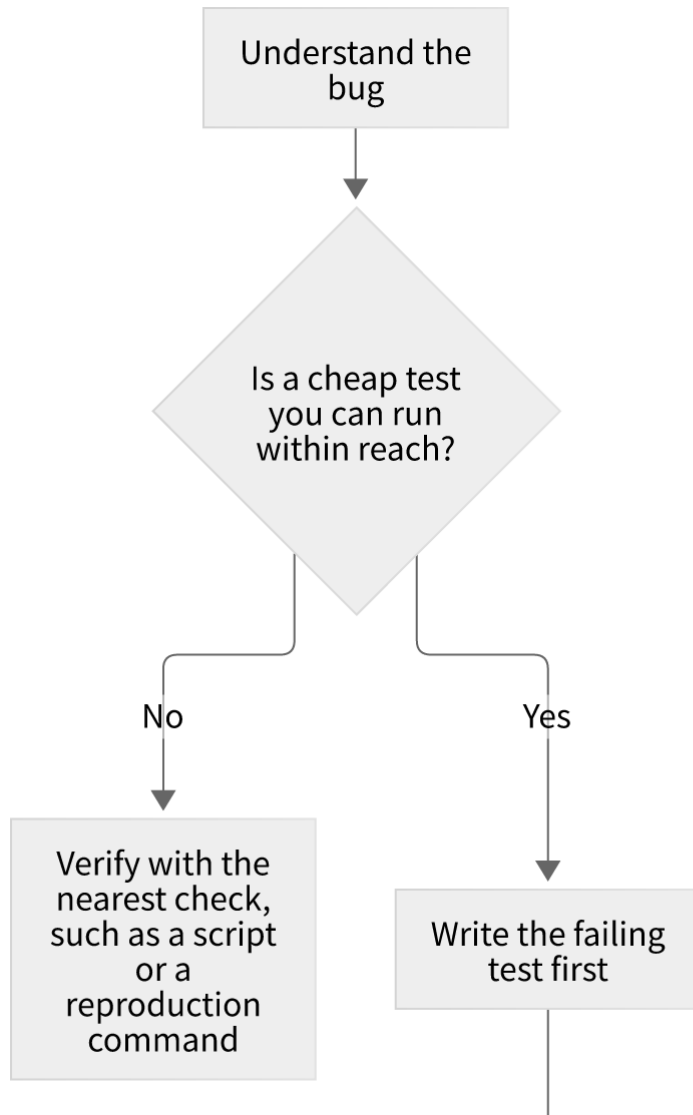


Diagram segment 1

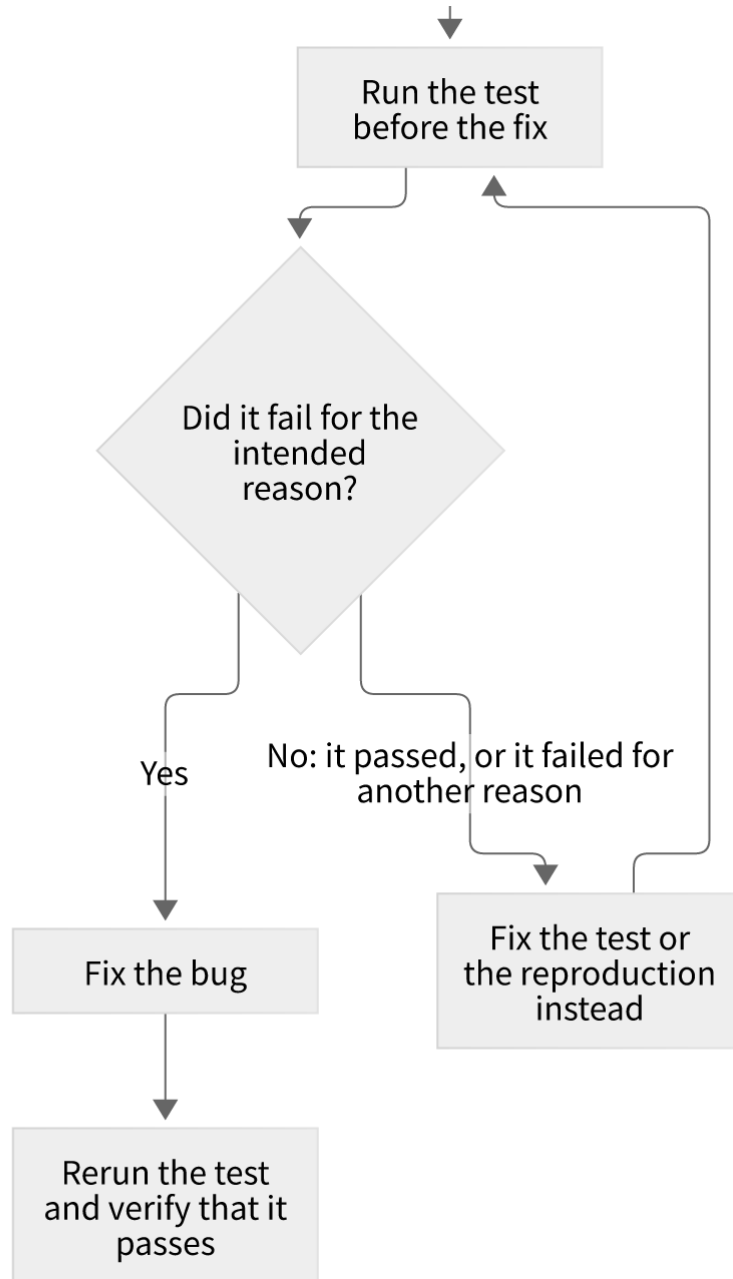


Diagram segment 2

View diagram 1

The run before the fix tells you whether the bug causes the failure

The agent runs the test before the fix in step 4 **to tell whether the test fails because of the bug or for another reason.**

A test that fails for another reason, such as a misspelled function name, fails whether or not the bug exists. That failure is therefore not evidence that the test can catch the bug.

A test that passes before the fix does not detect the bug either, so the agent fixes the test.

Example: compare a failure for the right reason with a failure for another reason in the two-notification bug

For example, suppose the agent fixes the bug "a retry delivers two notifications". Look at the run before the fix in step 4.

The test verifies the intended behavior "even after a retry, only one notification arrives".

```
declare function test(name: string, fn: () => void): void;
declare function expect<T>(actual: T): { toBe(expected: T): void };
// The function under test: sends a notification and retries on failure. Returns the list of
notifications sent
declare function sendWithRetry(message: string): string[];

// A test that fails for the right reason: before the fix, it fails with "expected 1 but got 2"
test("only one notification arrives even after a retry", () => {
  const sent = sendWithRetry("Your order has been received");
  expect(sent.length).toBe(1);
});

// A test that fails for another reason: the function name is misspelled, so it fails regardless of
the bug
test("only one notification arrives even after a retry", () => {
  const sent = sendAndRetry("Your order has been received");
  expect(sent.length).toBe(1);
});
```

If the test before the fix fails for a reason other than the bug, as in the second test, the agent must fix the test before it fixes the implementation.

When a test is not possible, replace it with a check you can run

When a test is not practical, the agent replaces the test with a check it can run: a script, a manual reproduction command, browser automation, or a check that the expected line appears in the logs.

Even when the agent does not write a test, it does not skip the check itself. If it skipped the check, no evidence that the bug is fixed would remain.

The final report gives the evidence before and after the fix along with the result

In the final report, the agent writes which test or check failed before the fix and how, and the run that passed after the fix. If it could not show a failure before the fix, it writes why and which check it used instead.

The agent reports the evidence along with the result. With only the result "it's fixed", the person who reads the report cannot verify that the agent fixed the bug.

Add no new test rather than a bad one

/tdd takes the position that **no new test is better than a bad new test**.

Bad tests include tests that check mostly mocks and tests that lock in details of the current implementation. Both kinds go against the principle "[Test Behavior, Not Implementation](#)" from [Chapter 19](#). A test that checks mostly mocks does not check the real behavior. A test that locks in implementation details breaks even on a rewrite that does not change the behavior.

Here are the two kinds of bad tests and a good test, compared on the two-notification bug.

```
declare function test(name: string, fn: () => void): void;
declare function expect<T>(actual: T): { toBe(expected: T): void };
type Transport = { send(message: string): void };
// The function under test: sends a notification and retries on failure. Returns the list of
notifications sent
declare function sendWithRetry(message: string, transport?: Transport): string[];
// A fake of the sending part (a mock). Counts how many times send is called
declare const mockTransport: Transport & { calls: number };
// A variable used inside the function that counts the retries
```



```

declare const retryState: { attempts: number };

// Bad test 1 (checks mostly mocks): it only looks at how many times the fake send was called,
// and does not check how many notifications actually arrived
test("calls the sending part once", () => {
  sendWithRetry("Your order has been received", mockTransport);
  expect(mockTransport.calls).toBe(1);
});

// Bad test 2 (locks in implementation details): it depends on the value of an internal variable,
// so it breaks even on a rewrite that does not change how many notifications arrive (for example,
// renaming the variable)
test("the retry count becomes 1", () => {
  sendWithRetry("Your order has been received");
  expect(retryState.attempts).toBe(1);
});

// Good test: checks the behavior visible from outside (the number of notifications that arrived)
test("only one notification arrives even after a retry", () => {
  const sent = sendWithRetry("Your order has been received");
  expect(sent.length).toBe(1);
});

```

The agent fits the test to the intended behavior. It does not change the test to fit a wrong implementation. It also does not weaken an existing assertion, such as `expect(sent.length).toBe(1)`, unless the expected behavior itself has changed and the reason is clear.

```

declare const sent: string[];
declare function expect<T>(actual: T): {
  toBe(expected: T): void;
  toBeGreaterThanOrEqual(expected: number): void;
};

// Before: verifies the intended behavior (only one notification arrives)
expect(sent.length).toBe(1);

// After (a rewrite you must not make): this weakens the assertion so that a wrong implementation
// that delivers two still passes
expect(sent.length).toBeGreaterThanOrEqual(1);

```

How to write the request: the command alone is enough

If a cheap test can verify the bug you want fixed, and the conversation has already discussed the bug, the command alone is enough for the user's request ([docs/guide/05-build-and-clean.md](#)).

```
/tdd implement
```

If you want the agent to use `/tdd` only when a cheap test exists, add the condition, as in the following request from [docs/guide/10-recipes-and-pitfalls.md](#).

```
/poteto-mode repro the duplicate write first. if there's a cheap test path, /tdd it. then fix and rerun.
```

The request has the condition "if there's a cheap test path", so when no cheap test exists, the agent can skip `/tdd` and use the nearest check.

Summary

- **What verifies the fix.** A regression test that fails before the fix and passes after it verifies the fix.
- **When to use it.** The agent writes the failing test before the fix only when the user asks or when a cheap test is within reach.
- **Procedure.** The agent runs the test before the fix, verifies that the bug causes the failure, and then fixes the bug.

The next chapter, [Chapter 31](#), covers `/no-comments`, the third of the three Skills that keep code quality. It has an agent other than the author review the comments in a diff.

Chapter 31: Remove comments and tidy the codebase

kaito • Sources: 日本語 • English • 2026-10-03

This chapter covers `/no-comments`.

`/no-comments` is a Skill that, before review, **hands the comments in a diff to Comment Sicko, a different agent that did not write them**. Comment Sicko deletes comments that explain surprising behavior in the team's own code. One example is a comment on a save function that also sends a confirmation email. Comment Sicko reports that function as a spot to fix. **The original agent then fixes the code, by renaming functions or splitting them, so that the behavior is clear without the comment. Comments that explain behavior forced by something the team cannot change, such as an external library, stay.**

`/no-comments` is the third of the three code-quality Skills that start in [Chapter 29](#). In `/no-comments`, Comment Sicko, which did not write the comments, verifies them in place of the agent that wrote the code.

`/no-comments` reviews the comments in a diff. It also reviews suppression directives for lint and type checks, such as `eslint-disable` and `@ts-ignore`.

- **Suppression directive.** An instruction written in code to silence a lint or type-check warning at that one spot.
- **Comment Sicko.** The subagent bundled with pstack that reviews and deletes comments ([Chapter 8](#))
 - `/no-comments` starts Comment Sicko, receives its report, and then fixes the code

This chapter explains `/no-comments` from four angles: its role, when to use it, its rules and procedure, and how to write the request.

How this chapter is organized

The chapter has the following five sections.

- Role: have Comment Sicko, an agent that did not write the comments, review them
- When to use it: call it on the diff before review
- Rules and procedure: review what Comment Sicko flags
- Writing the request: name the diff as the target
- Summary

Role: have Comment Sicko, an agent that did not write the comments, review them

`/no-comments` is a Skill that removes comments before review.

This Skill starts Comment Sicko ([Chapter 8](#)) and fixes the findings from Comment Sicko that it accepts. When a comment asserts a constraint such as "do not change the wording", this Skill also proposes a way to express that constraint in code, such as a test.

This Skill **treats a comment that explains surprising behavior in the team's own code as a mark of code that needs fixing**. The reason is that renaming or extracting a function can make such behavior clear from the code alone, with no prose explanation. The section "[Replace a comment with a function name, using the example of a save function that also sends a confirmation email](#)" below gives an example of a comment that explains surprising behavior.

The bundled guide [docs/guide/05-build-and-clean.md](#) explains why comment review goes to a different agent, as follows.

Comments need their own pass, and not from the agent that wrote them. An author defends its comments the way you'd defend yours.

When an agent checks comments it wrote itself, **it defends its own comments and keeps them even when they are not needed**. So Comment Sicko, which did not write the comments, does the review.

When to use it: call it on the diff before review

`/no-comments` has the setting `disable-model-invocation: true` (Chapter 25), so the agent never picks and runs `/no-comments` on its own from the conversation. The Skill runs when the user calls it by name, or when a rule of `/poteto-mode` or a step in a Playbook calls it.

The rules in the section "Non-negotiables" of `/poteto-mode` (Chapter 8) say to call `/no-comments` before review. Playbooks such as "Opening a PR", which opens a PR, say the same.

`/no-comments` looks at the files or the diff that the caller passes. If the caller passes nothing, it looks at the diff against the base branch, which is `main` by default, including uncommitted changes.

`/no-comments` and two other Skills that do similar cleanup split the work by target, as follows.

- **Clutter in code.** `/deslop` removes clutter such as comments that only narrate what the code does, unfounded defensive checks, and leftover compatibility paths that nothing uses. `/deslop` ships in `cursor-team-kit` and is not part of `pstack`.
- **Habits in prose.** `/unslop` (Chapter 32) removes AI writing habits from prose such as READMEs and PR descriptions.
- **Comments.** `/no-comments` has an agent that did not write the comments review them.

Playbooks that create PRs also call `/no-comments`

Every Playbook that calls `/no-comments` uses it **to remove comments from the diff before the PR goes out for review or verification**. The situation and the reason for each caller are as follows.

"Opening a PR": tidy up a diff that looks unfinished before review

"Opening a PR" is the Playbook that opens a PR, and every other Playbook calls it at the end. This Playbook says to run `/no-comments` before review. The subagent that opens the PR also runs `/no-comments` before it posts the PR's URL.

The bundled guide `05-build-and-clean.md` explains why you cannot skip the cleanup of a diff. **A diff that still has comments that only narrate the code, or unneeded defensive code, looks unfinished to the reviewer.** The same page also says that **leftover unneeded code becomes the place where the next defect hides.**

"Autopilot-full" and "Autopilot-stack": settle the code before handing it to verification

In "Autopilot-full" and "Autopilot-stack" (Chapter 16), each PR has an owner, a cloud agent that handles one PR's change from start to finish. The owner runs `/deslop` and `/no-comments` to completion. Once the code to merge is in its final form, the owner reports that commit's SHA. After that, the owner's own verification and CI run alongside independent verification by `/swarm` (Chapter 24).

"Multi-phase or multi-PR plan": put it in the plan document's checklist

The plan document that "Multi-phase or multi-PR plan" (Chapter 16) creates is a checklist. Each PR's owner works through the checklist one item at a time. The operator, the person who starts the plan, verifies each item by inspecting its evidence. The template for the plan document includes an item common to every PR: "Run `/deslop` before every commit, and `/no-comments` before review."

Rules and procedure: review what Comment Sicko flags

Comment Sicko reviews and deletes comments, and does not rewrite app code

Comment Sicko ([agents/comment-sicko.md](#)) only deletes comments and reports code to fix. It does not rewrite app code.

Comment Sicko's definition is written in the exaggerated voice of "someone who hates comments to an unhinged degree". The rules for which comments may stay are strict, though.

Comment Sicko keeps only the following five kinds of comments, and deletes any comment it is unsure about.

- Legal and license headers
- Explanations of non-obvious behavior forced by an external dependency or platform that the team cannot change
- `// prettier-ignore`, and suppressions of lint rules that are wrong in themselves, too nitpicky, or only about style
- Doc comments that define the contract of a public API
- Links to an issue or an RFC that explains a constraint code cannot express

Delete explanations of surprising behavior and suppressions of important lint rules, and list them as MUST KILL

Comment Sicko deletes comments that explain surprising behavior in the team's own code. In its report, it lists the symbol that has that behavior, such as a function, variable, or type name, as `MUST KILL`. A `MUST KILL` finding asks for a fix that makes the behavior clear without prose, through renaming, extraction, typing, or a redesign.

Some suppression directives, such as `eslint-disable`, `@ts-ignore`, or `@ts-expect-error`, silence a lint rule that catches real bugs or protects correctness or safety. Comment Sicko also deletes those suppression directives. It lists the symbol that causes the lint warning as `MUST KILL`.

Replace a comment with a function name, using the example of a save function that also sends a confirmation email

In this example, a `save` function also sends a confirmation email.

```
type User = { name: string; email: string };

// Before: a comment explains surprising behavior in our own function
// Note: besides saving, this function also sends a confirmation email
function saveUser(user: User) { /* ... */ }
```

```
type User = { name: string; email: string };

// After: extract the two jobs into two functions whose names say what they do
function saveUser(user: User) { /* ... */ }
function sendConfirmationMail(user: User) { /* ... */ }
```

The comment in the "before" code does not explain behavior forced by an external dependency. It explains surprising behavior in the team's own code. So Comment Sicko deletes this comment and lists `saveUser` as **MUST KILL**.

Comment Sicko does not fix the code itself. When `/no-comments` extracts two functions as in the "after" code, the names tell you which function does what, with no comment.

So when a comment explains surprising behavior in the team's own code, you need to delete the comment and fix the code so that the names show the behavior.

`/no-comments` **reviews Comment Sicko's report instead of accepting it as is**

`/no-comments` does not accept Comment Sicko's report, or the changes Comment Sicko made, as they are. It reviews them in the following three stages.

1. Reject a report with errors and have Comment Sicko redo it once
2. For a judgment on a warning comment that gives no reason, check with `/how` or `/why` before accepting it
3. If a fix needs a new code shape, decide the shape with `/architect` before fixing

1. Reject a report with errors and have Comment Sicko redo it once

Comment Sicko's report and changes can contain errors such as the following.

- **Findings outside the given scope.** Findings about code outside the files or diff that `/no-comments` asked it to review.

- **Rewrites of app code.** Changes that rewrite app code, when Comment Sicko may only change comments.
- **Deletion of comments that may stay.** Changes that delete a comment of one of the five kinds that may stay, listed in ["Comment Sicko reviews and deletes comments, and does not rewrite app code"](#).
- **A wrong reason for MUST KILL.** A finding where the reason given for fixing a MUST KILL symbol does not match the actual code. For example, it lists a function that sends no confirmation email as one that "also sends a confirmation email".
- **Wrong findings about intended code.** Findings that list code someone wrote on purpose and decided to keep as something to fix.

When `/no-comments` finds a report with such errors, it reverts Comment Sicko's changes, says what was wrong, and has Comment Sicko review once more. If the second report also has errors, `/no-comments` reports the problem as unresolved and ends its run as a failure.

2. For a judgment on a warning comment that gives no reason, check with `/how` or `/why` before accepting it

Code sometimes has short warning comments that give no reason, such as "IMPORTANT" or "do not remove".

If Comment Sicko judged such a comment as "delete" or "keep", `/no-comments` runs `/how` or `/why` on the symbol that the comment belongs to before it accepts the judgment. [Chapter 22](#) covers both Skills. `/how` shows how the code works now, and `/why` shows why the code took that shape. With those answers, `/no-comments` can verify whether the warning's claim still holds.

Take the following warning, for example.

```
declare function sleep(ms: number): Promise<void>;
declare function saveDraft(): Promise<void>;

async function onClose(): Promise<void> {
  // IMPORTANT: do not remove
  await sleep(100);
  await saveDraft();
}
```

If `/why` finds that this wait works around a bug in an old library that has since been fixed, the warning's claim no longer holds. In that case, `/no-comments` accepts Comment Sicko's "delete" judgment.

If the investigation cannot settle the question, `/no-comments` deletes the comment.

3. If a fix needs a new code shape, decide the shape with `/architect` before fixing

For accepted findings that need only small fixes, such as deleting unused logic, removing an unneeded argument, or switching to the correct API, `/no-comments` makes the fix directly.

Some findings need a decision about a new shape for the code, such as how to split functions or how to build types, before anyone can fix them. In that case, `/no-comments` passes all the accepted findings to `/architect` ([Chapter 24](#)) in one call, and `/architect` decides the shape. `/architect` stops once the shape is decided, and `/no-comments` makes the actual code fixes.

Fix accepted findings at the root cause

`/no-comments` fixes the findings it accepted in review at the root cause. This rule follows the principle "Fix Root Causes" from [Chapter 19](#).

For example, if Comment Sicko deleted a comment that explains the reason for a workaround, `/no-comments` also deletes the workaround code and fixes the root cause that made the workaround necessary.

`/no-comments` does not fix code outside the scope it was given, though. If the root cause lies outside the scope, it makes only the smallest fix possible inside the scope and reports the rest as unresolved.

For a comment that asserts a constraint, propose a way to enforce it in code and wait for approval

A comment that asserts a constraint is a comment written like "do not remove", "do not change wording", or "talk to X before changing".

For such a comment, `/no-comments` **proposes one way to enforce the constraint in code instead of a comment, and waits for the user's approval**. There are four candidate methods, and `/no-comments` picks the one that takes the least effort within the given scope.

- **Types.** Code that violates the constraint becomes a compile error.
- **Runtime check.** A violation raises an error at runtime.
- **Test.** A violation makes a test fail.
- **Lint.** Lint warns about code that violates the constraint.

For example, for a comment that says "do not change the wording", `/no-comments` proposes writing a test that fails if the wording changes. With that test in place, the constraint holds without the comment.

```
// Before: only a comment asserts the constraint
// Do not change the wording
function stockLabel(count: number): string {
  return count === 0 ? "Out of stock" : `${count} left`;
}
```

```
// Register one test
declare function test(name: string, fn: () => void): void;
// Check a value
declare function expect(value: unknown): { toBe(expected: unknown): void };

// After: express the constraint as a test and delete the comment
function stockLabel(count: number): string {
  return count === 0 ? "Out of stock" : `${count} left`;
}

test("the wording for zero stock stays \"Out of stock\"", () => {
  expect(stockLabel(0)).toBe("Out of stock");
});
```

So when a comment asserts a constraint, you need to express that constraint in code, such as a test.

If the user approves, `/no-comments` expresses the constraint in code and then deletes the comment. If the user does not approve, it still deletes the comment and reports the constraint as unresolved.

Comment Sicko deletes comments, and /no-comments decides what to fix

The flow so far looks like this as a diagram.

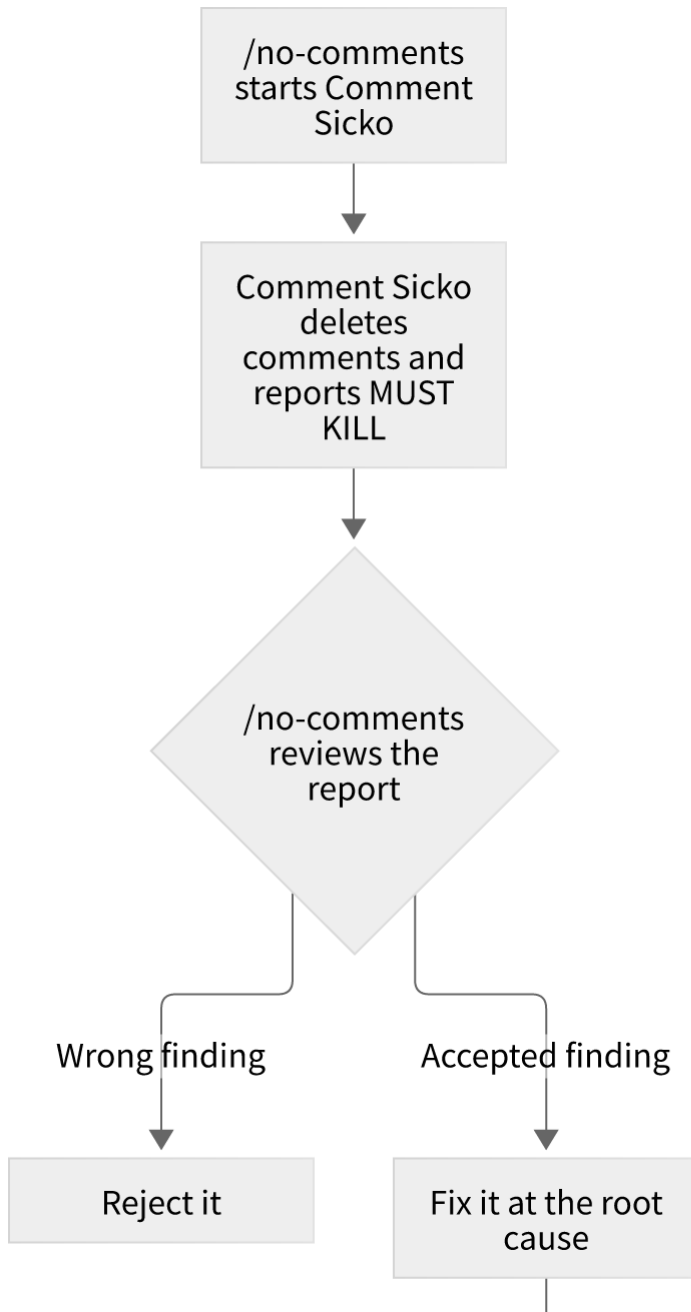


Diagram segment 1

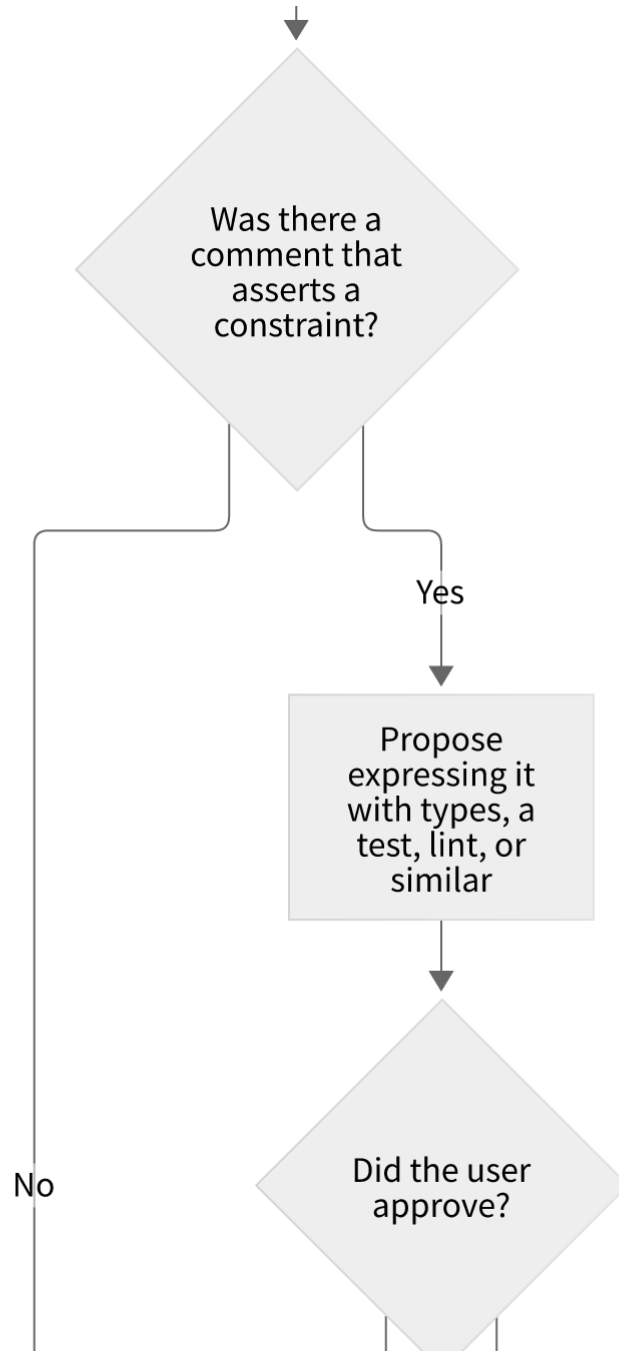


Diagram segment 2

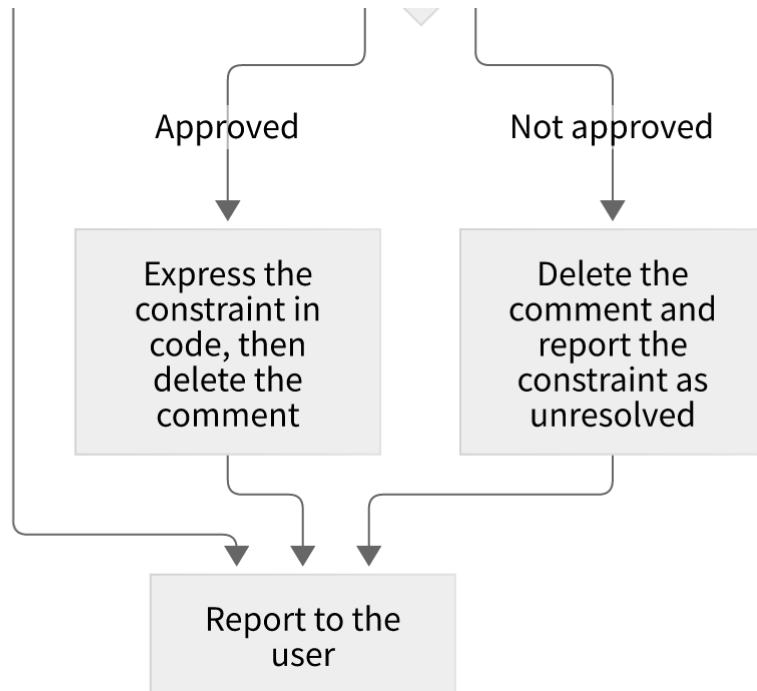


Diagram segment 3

View diagram 1

As the diagram shows, Comment Sicko deletes the comments. `/no-comments` decides which findings to accept and fix in code, and whether to express a constraint in code or report it as unresolved.

Writing the request: name the diff as the target

The bundled guide `05-build-and-clean.md` gives the following request. Before review, the request hands the comments in a diff to an agent that did not write them.

```
/no-comments the diff
```

Summary

- **Who verifies.** Comment Sicko, which did not write the comments, reviews the comments in the diff.
- **When to use it.** Call it on the diff before review.
- **How it fixes.** It deletes comments that explain surprising behavior and fixes the code so that the behavior is clear without them.

The next chapter, [Chapter 32](#), covers three Skills that keep prose quality up: `/unstop`, `/bro`, and `/technical-writing`.

Chapter 32: Keep writing quality high

kaito • Sources: 日本語 • English • 2026-10-03

This chapter covers the following three Skills.

1. `/unslap`
2. `/bro`
3. `/technical-writing`

All three Skills keep the quality of the text the agent writes high. Each one has its own job.

- `/unslap`. The agent removes AI habits from its own text, such as stock phrases like "I hope this helps!"
- `/bro`. The agent restates its last reply, shorter and without jargon.
- `/technical-writing`. The agent first decides the kind of document, such as a tutorial or a reference, and then writes the technical document.

This chapter first looks at how the three Skills differ in what they target, who fixes the text, and what triggers them. Then it explains each Skill from four angles: its role, when to use it, its rules or steps, and how to write the request.

How this chapter is organized

The chapter has the following five sections.

- The three Skills differ in what they target and who fixes the text
- `/unslap` removes AI habits from text with numbered rules
- `/bro` restates a hard-to-follow reply without jargon
- `/technical-writing` makes the agent decide the kind of document before it writes
- Summary

The three Skills differ in what they target and who fixes the text

The following table shows what each Skill targets and who fixes the text.

Skill	Target	Who fixes the text
/unslop	Any text, including the agent's replies	The agent that wrote the text fixes it by following the rules
/bro	The agent's last reply	The user calls it when a reply is hard to follow, and the agent restates the reply
/technical-writing	Docs, RFCs, READMEs, PR descriptions, and commit messages	The agent that wrote the text writes and fixes it by following the standards of four layers

The three run only when called by name

All three `SKILL.md` files set `disable-model-invocation: true` ([Chapter 25](#)). The agent never picks a Skill with this setting on its own from the content of the conversation. So the three run only when the user calls them by name, or when `/poteto-mode`, a Playbook, or another Skill names them in its steps.

The "Non-negotiables" section of `/poteto-mode` ([Chapter 8](#)) sets when the agent calls two of these Skills. The agent calls `/unslop` when it writes any text, and it calls `/technical-writing` when it writes docs, RFCs, READMEs, PR descriptions, or commit messages.

So when you work in `/poteto-mode`, the agent calls these two Skills under the "Non-negotiables" rules, even if you never call them by name.

/unslop removes AI habits from text with numbered rules

Role: remove AI habits from text by rule

`/unslop` is a **Skill with which the agent cuts AI habits from its own text by following rules.**

AI habits are the phrasings and formatting that often appear in AI-written text and tell the reader that a machine wrote it.

The Skill calls these habits "AI tells" or "AI patterns". Examples are a stock phrase at the end of a reply and heavy use of em dashes.

The rules and their examples all target English text. But most of the rules apply to writing in any language, so they still work for Japanese.

The rules work best in English, so it is fair to say "The hottest programing language is English".

When to use it: apply it to every piece of text, always

The "Non-negotiables" in `/poteto-mode` require the agent to apply this Skill to **all text, including the agent's replies.**

The description in `SKILL.md` also says "Must always apply". But `/unslop` sets `disable-model-invocation: true`, so the agent does not call `/unslop` on its own after reading this description. The Skill applies every time because the "Non-negotiables" rules call `/unslop`, and so do the steps of the other Skills and Playbooks shown next.

Other Skills and Playbooks that write text also use `/unslop`

Other Skills and Playbooks also tell the agent, in their steps, to apply `/unslop` to the text they write. The following table lists each caller and the text it applies `/unslop` to.

Caller	Text that gets <code>/unslop</code>
<code>/technical-writing</code>	Every document it writes or reviews
"Opening a PR" and "Multi-phase or multi-PR plan" Playbooks	PR titles, PR descriptions, commit bodies, and plan documents
<code>/teach</code> , <code>/show-me-your-work</code>	Explanatory replies and the decision log

Caller	Text that gets /unslop
/recall, /blast-radius	Reports handed to the user
/automate-me	The draft of the user's personal Skill
"Investigation" Playbook	The reply with the findings

The details for each caller follow.

`/technical-writing: hands the list of AI habits to /unslop`

`/technical-writing` does not keep its own list of habits such as AI vocabulary and filler. It hands that list to `/unslop`. So it applies `/unslop` to every document it writes or reviews.

"Opening a PR" and "Multi-phase or multi-PR plan" Playbooks: apply /unslop after writing with /technical-writing

The **"Opening a PR"** Playbook ([Chapter 14](#)), which opens a PR, tells the agent to write the PR title, the PR description, and the commit bodies with `/technical-writing`, then apply `/unslop`.

The **"Multi-phase or multi-PR plan"** Playbook ([Chapter 16](#)) writes a plan document that spans several phases or PRs. This Playbook also tells the agent to write the plan document with `/technical-writing`, then apply `/unslop`.

`/teach, /show-me-your-work: write the way you would talk to a coworker`

`/teach` ([Chapter 22](#)) is a Skill that explains the work in plain words so that a person really understands it. `/teach` tells the agent to write every reply in plain spoken language, the way you would explain something to a coworker. `/teach` then applies `/unslop` to those replies.

`/show-me-your-work` ([Chapter 28](#)) is a Skill that records the agent's decisions and their reasons in a file, one per line. `/show-me-your-work` tells the agent to write each line of the decision log the way you would tell a teammate what you did, without AI phrasing or abstract jargon. It then applies `/unslop` to the sentences of the decision log too.

`/recall`, `/blast-radius`: **write reports handed to the user**

`/recall` (Chapter 22) is a Skill that rebuilds the state of the user's recent work from sources such as the chat history. `/blast-radius` (Chapter 27) is a Skill that looks, before a release, for what a change might break elsewhere.

Both Skills pass the report they hand to the user through `/unslop`. The `/recall` report says how far the work has come and what to do next. The `/blast-radius` report says what the change might break and what to verify before the merge.

`/automate-me`: **fixes the text of the user's personal Skill with** `/unslop`

`/automate-me` (Chapter 25) is a Skill that reads the user's working preferences from their recent transcripts and drafts a personal Skill named `<user name>-mode` for that user. `/automate-me` applies `/unslop` to each line of that draft.

"Investigation" Playbook: the reply with the findings is itself the artifact

"Investigation" (Chapter 10) is a Playbook that researches and answers a question without changing code. This Playbook returns a sourced explanation or recommendation instead of a code change. **The Playbook defines that reply as the artifact of the investigation.**

At the end of its steps, this Playbook applies `/unslop` to that reply.

Rules: AI habits in seven categories with fixed numbers

The rules take up most of `SKILL.md`. The Skill has only the following two steps.

1. Look for the spots that match a rule
2. Rewrite them, keep the meaning, and match the tone the writer intended

The rules have numbers, and each number is a fixed ID that other Skills use to point at a rule. For example, `/poteto-mode` points at the rule against using colons as mid-sentence connectors by number, as "unslop rule 14".

`SKILL.md` says that when the maintainers remove a rule, its number stays empty. In the `SKILL.md` of pstack version 0.15.5, numbers 1, 2, 4, 6, and 21 are empty.

You can confirm the missing numbers in the [source view](#) on GitHub.

There are 28 rules in the following seven categories.

Category	What it fixes	Rule numbers
Content	Phrases that only add praise, and claims with no clear source	3, 5
Language	Words AI uses often, and fixed patterns of phrasing	7 to 12
Style	Use of punctuation, bold, headings, and emojis	13 to 19
Communication artifacts	Stock phrases from chat replies, and a fawning tone	20, 22
Filler	Words that mean nothing, too much hedging, and empty conclusions	23 to 25
Jargon	Metaphor nouns that only sound technical	26
Plain speech	Sentences that only describe a feeling, sentences the reader must reread, and fancy words	27 to 33

Next, I list every rule, category by category.

As noted above, the rules and their examples all target English text. Still, in this book, I treat every rule except the four that are specific to English formatting, rules 13, 14, 17, and 19, as applying to Japanese text too. The Japanese edition of this book replaces the examples of those rules with Japanese ones. This edition gives the original English examples for every rule.

Content

No.	Rule	Fix
3	Superficial -ing phrases	Delete phrases such as "highlighting...", "ensuring...", and "reflecting..." that only add praise, or expand them with real sources
5	Vague attributions	Stop writing "Experts believe" or "Industry reports suggest", which hide who said it. Name the source or delete the claim

Language

No.	Rule	Fix
7	AI vocabulary	Do not overuse words such as "Additionally", "crucial", "delve", and "pivotal". Replace them with plain words
8	Fancy ways to say "is"	Change "serves as", "stands as", and "boasts" to "is" or "has"
9	"Not just X, but Y."	Stop using "Not just X, but Y" and state the point directly
10	Rule of three	Do not force ideas into groups of three. Use the natural number
11	Synonym cycling	Do not call one person protagonist, main character, and central figure in one paragraph. Pick one word and repeat it
12	False ranges	Do not join things that are not on one scale with "from X to Y". List the topics directly

Style

No.	Rule	Fix
13	Em dash overuse	Do not use em dashes. Use periods or commas. Do not substitute parentheses, en dashes, or hyphens either
14	Colon overuse	Use colons only before a list or an example, not as mid-sentence connectors
15	Boldface overuse	Do not bold every proper noun or acronym
16	Inline-header lists	Turn a list item such as " Performance: Performance improved..." into prose when its bold label repeats the line
17	Title case headings	Write headings in sentence case
18	Decorative emojis	Remove emojis from headings and bullets
19	Curly quotes	Replace curly quotes with straight quotes

- **Title Case** capitalizes the first letter of every major word in a heading, for example "Pick The Mode First".
- **Sentence case** capitalizes only the first word of a heading, as in a sentence, for example "Pick the mode first".

Communication artifacts

No.	Rule	Fix
20	Chatbot phrases	Remove sentences such as "I hope this helps!", "Let me know if...", and "Certainly!"
22	Sycophantic tone	Drop openers such as "Great question! You're absolutely right!" and answer directly

Filler

No.	Rule	Fix
23	Filler phrases	Change "Due to the fact that" to "Because". Delete "It is important to note that"
24	Excessive hedging	Change "could potentially possibly be argued that it might" to "may"
25	Generic conclusions	Drop endings such as "The future looks bright." and state specific plans or facts

Jargon

No.	Rule	Fix
26	Abstract metaphor nouns	Replace metaphors such as "substrate", "north star", and "flywheel" with concrete words. For example, "substrate" becomes "base"

Plain speech

No.	Rule	Fix
27	Say what it does, not how it feels	Turn phrases that describe how something feels to use, such as "SQL you can read" and "types that follow your schema", into sentences that name a mechanism or a number, such as ". toSQL() returns the exact string sent to the database" and "a column rename fails the build"
28	Shorten or split dense sentences	Split a sentence the reader must reread into two, or drop part of it
29	Active voice	Change "queries are validated" to "the compiler validates queries"
30	Cut adverbs, or use a stronger verb	Change "runs quickly" to "is fast" or the real number. For "significantly improves", write the measured delta
31	Prefer the plain word	Change "utilize" and "leverage" to "use", and "facilitate" to "help"
32	Mannered prose	Replace metaphor and flourish with literal words. For example, "a dial worth turning" becomes "a parameter worth varying"
33	Over-compression	Rewrite sentences that drop verbs and use arrows or abbreviations as whole sentences. For example, "Parser rejects bad date → exit 2, no write" becomes "The parser rejects a bad date, exits with code 2, and writes nothing."

How to write the request: call it with the text to fix and extra rules

The bundled guide's `docs/guide/05-build-and-clean.md` gives the following example, which calls the Skill with the text to fix and an extra rule.

```
/unslop the readme changes, no emdashes
```

In this request, "the readme changes" is the text to fix, and "no emdashes" is the extra rule.

The same page says that even with a short request like the following, the agent reads the intent and fixes the text with this Skill.

```
unslop that, tighten it
```

"that" points at the text the agent just wrote.

/bro restates a hard-to-follow reply without jargon

Role: restate the last reply without jargon

`/bro` is a **Skill with which the agent restates its last reply without jargon**. With a body of only three sentences, `/bro` is the shortest Skill in pstack.

`/bro` differs from `/unslop` in the unit it fixes. `/unslop` fixes only the spots that match a rule, such as stock phrases. `/bro` uses no rules and restates the whole last reply in plain words.

Unlike `/unslop` and `/technical-writing`, `/bro` appears in no step of `/poteto-mode`, a Playbook, or another Skill. So `/bro` runs only when the user calls it by name.

When to use it: the reply is detailed, but you cannot tell what it says

The user uses this Skill when the agent's reply is technically detailed but the user cannot tell what it says.

Steps: the three sentences of the body are the instruction

The body of `/bro` is only these three sentences.

Restate your last message. Stop using jargon and speak coherently. State it more simply and concisely, like one human talking to another.

`SKILL.md` has no steps section, and the agent takes these three sentences as the instruction itself.

How to write the request: `"/bro"` alone is enough

The bundled guide's `10-recipes-and-pitfalls.md` lists only the following single word as the prompt.

`/bro`

The three sentences of the body are the instruction, so the user does not need to add how to restate the reply.

`/technical-writing` makes the agent decide the kind of document before it writes

Role: make the agent write technical documents by layered standards

`/technical-writing` is a Skill that gives the standards the agent follows when it writes technical documents. The Skill splits the standards into four layers, from the kind of the whole document down to how to write one sentence. **The goal is writing a tired engineer understands on the first read.**

In The Complete Guide to pstack [Part 2](#), poteto explains what led him to build this Skill.

The story started when poteto had the agent write the README for Dune, a framework for desktop apps inside Cursor, without this Skill. According to the guide, the resulting

README mixed a tutorial, a how-to, a design explanation, and an API reference into one. The agent also wrote it in fancy prose full of AI habits, so the README was hard to read.

When to use it: when the agent writes or reviews documents

The description in `SKILL.md` names these occasions: when the agent writes docs, RFCs, READMEs, PR descriptions, or commit messages, and when it reviews them.

But `/technical-writing` also sets `disable-model-invocation: true`, so the agent does not call `/technical-writing` on its own after reading this description. The agent uses the Skill on these occasions because the "Non-negotiables" rules tell the agent to use it when it writes these documents.

Among the Playbooks, "Opening a PR", which opens a PR, tells the agent to write the PR title, the PR description, and the commit bodies with this Skill. "Multi-phase or multi-PR plan", which writes a plan document, also tells the agent to write the plan document with this Skill.

Rules: four layers and the rules outside them

Besides the standards of the four layers, `SKILL.md` also sets rules that sit outside the layers. I look at the rules outside the layers first, then at the four layers one at a time.

Three rules that take priority over the four layers

Apart from the four layers, the agent follows these three rules, and they take priority over every layer.

- Cut every word whose removal leaves the meaning of the sentence unchanged. For example, "In order to" becomes "to".
- Use short, everyday words. For example, write "use", not "utilize".
- If a rule makes a sentence worse, fix the sentence another way or leave it alone. The rules exist for the reader. A sentence that follows every rule but reads as if a machine wrote it has failed.

Use the names in the codebase as they are, and do not invent jargon

The agent writes the names of the symbols, files, flags, and commands that exist in the codebase as they are, without rewording them.

The agent also does not invent new jargon. It uses the words a developer says out loud. For example, to say that code moves to another place, it writes "move", not "evacuate".

On the other hand, when an established design or approach already has a widely used name, such as "the Strangler Fig pattern", the agent uses that name.

But the agent adds the meaning where the name first appears in the document. For example, it writes "migrate with the Strangler Fig pattern (an approach that replaces an old system with a new one piece by piece)".

Rule 26 of `/unslap`, which covers abstract metaphor nouns, lists words like "evacuate" that the agent should replace. When the agent finds a word that is not on that list, it does not edit `/unslap` itself. Instead, it writes a proposal in its reply, with the diff, to add the missing word and its replacement to the list.

Mix sentence lengths and be specific

This Skill asks for the following four things about sentence length and specificity.

- **Mix sentence lengths on purpose.** Use short sentences to land a point, and longer ones to state a fact with its condition or consequence.
- **Decide whether to split a sentence by the number of thoughts, not by its length.** "One thought per sentence" does not mean "make every sentence short". Split a sentence that carries two thoughts, but keep a sentence that carries one, even if it is long.
- **In an explanation, give the writer's view.** In an explanation, do not stop at listing pros and cons. Also say what the writer makes of them. A reference, on the other hand, holds no views, only facts.
- **Be specific, not vague.** Instead of a safe phrase like "schema changes can cause issues", say what happens, as in "a column rename fails the build".

The reason is that **a text can follow every standard of the four layers and still read as machine-written if it is all clipped short sentences with nothing specific in it.**

Keep a PR description short enough to read in under a minute

The agent keeps a PR description short enough that a reviewer can read it in under a minute. So it links the run logs of `/swarm` (Chapter 24), lists of commit SHAs, and metric tables instead of pasting them into the body.

The four layers go from the whole document down to one sentence

The four layers are stages that look closer and closer, from the whole document down to the content of one sentence. Each layer owns one question.

Layer	Question	Key point
Diátaxis	What kind of document is this?	Make it one of tutorial, how-to, reference, or explanation. Do not mix them. Split them and link them
Google developer style	How do the sentences address the reader?	Call the reader "you", and write in the present tense and the imperative. Make headings state the point of the section, not only its topic
STE	How much does each sentence carry?	Put only one thought in a sentence. Use the same word for the same action throughout the document
Global English	Can any sentence be read two ways?	Write so that the reader never wonders what "it" or "this" points at. Call the same thing by the same name throughout the document

- **Diátaxis** is an approach that sorts technical documents into four kinds: tutorial, how-to, reference, and explanation. It makes each document one kind, keeps the kinds apart, and links them to each other.
- **Google developer style** is the style guide for Google's developer documentation.

- **STE** is Simplified Technical English, a standard for English in technical documents that limits the words and sentence forms you may use.
- **Global English** is a way of writing English, described in Kohl's The Global English Style Guide, that non-native readers, translators, and agents all read only one way.

Next, I look at each layer in the table one at a time, with the source's examples.

Diátaxis: decide the kind of document first

Diátaxis, the first layer, decides the kind of document first. The reason is that a document that mixes kinds becomes hard to read, like the Dune README from "[Role: make the agent write technical documents by layered standards](#)" earlier in this chapter.

Two questions decide the kind: "Is the document for action or for understanding?" and "Is it for learning or for work?"

	For learning	For work
For action	tutorial	how-to
For understanding	explanation	reference

SKILL.md describes the four kinds as follows.

- **tutorial.** A document in which a first-time user learns hands-on by building something. It shows the finished result first, and for each step it says what should appear on the screen or in the output.
- **how-to.** A set of steps with which a user who already knows the tool solves the problem in front of them. It skips the basics and lists only what to do, in order.
- **reference.** A document to look things up in when you need an exact fact. It lists specifications and settings, with no steps and no writer's opinion.
- **explanation.** A document for understanding why a mechanism or design is the way it is. It covers design decisions, history, constraints, and alternatives. It is the only kind in which the writer gives opinions.

When a document mixes the four kinds into one, like the Dune README, the agent splits it into four documents and links them to each other.

But the agent does not apply this layer to PR descriptions and commit messages. It applies only the other three layers to them.

The other three layers are Google developer style, STE, and Global English. They deal mainly with how to write English. If you write documents in a language other than English, I recommend that you skip the next three sections.

Google developer style: talk to the reader

The agent calls the reader "you" and writes instructions in the imperative, as in "Click Submit."

The agent also puts the condition before the instruction, as in "To delete the document, click Delete." When the condition comes first, the reader can skip instructions that do not apply to them.

For headings, the agent states the point of the section, not only what the section is about.

For example, instead of a heading like "Modes" that shows only the topic, it writes a heading like "Pick the mode first" that tells the reader what to do.

Here, "mode" means a Diátaxis kind of document.

STE: put only one thought in a sentence

The agent writes only one instruction per sentence. Here, an instruction is a sentence that makes the reader do something, such as "Install the component."

The agent also splits long sentences in two. The limit is about 20 English words for an instruction and about 25 for other sentences.

The agent keeps the English articles "the" and "a". For example, "Remove backup file" reads two ways, but "Remove the backup file" reads only one way.

- **Why "Remove backup file" reads two ways.** The source does not say which two readings it means. In this book, I read them as follows. Without an article, you can read "backup file" as one noun and take the sentence as "delete the backup file". You can also split "backup" from "file", read "file" as a verb meaning "store", and take the sentence as "remove the backup and file it". With "the", "the backup file" is fixed as one noun.

The agent also uses the same word for the same action throughout the document. If it writes "start" in one place, it does not switch to "initiate" in another.

Global English: remove sentences that read two ways

The agent puts "only" and "not" right next to the word they change. Their position changes the meaning. "Only fails on growth" and "fails only on growth" say different things.

The agent also calls one thing by one name everywhere. A document that calls one mechanism "the gate", "the ratchet", and "the budget check" makes the reader think there are three things.

Example: rewriting one paragraph by the rules

SKILL.md includes one rewriting example. Here is the paragraph before the fix.

Configuration of the proto import ratchet budget script parameters is performed via budget.json. Note that it's important to remember that running with --write, which updates the committed budget to reflect the current count, should only be done when lowering it. If exceeded, CI fails.

Here is the paragraph after the fix.

`budget.mjs` reads the committed budget from `budget.json` and counts the files that import `protos`. If the count exceeds the budget, CI fails. Run `budget.mjs --write only` to lower the budget.

The rewritten paragraph makes the following changes to the original.

- Drops the noun string "proto import ratchet budget script" and uses the real file names `budget.mjs` and `budget.json`
- Drops the passive "is performed via", makes `budget.mjs` the subject, and says what it does
- Cuts empty words such as "Note that it's important to remember that"
- Drops the metaphor "ratchet". A ratchet is a gear that turns in only one direction, and here it stood for "a limit that can only go down"
- Turns the passive warning "should only be done" into the imperative instruction "Run `budget.mjs --write only` to lower the budget."

How to write the request: name the Skill in the request

The bundled guide's [docs/guide/09-make-it-yours.md](#) gives the following request example.

```
/technical-writing review the readme changes
```

The same page says that the user can use this Skill to review a document that the user or the agent finished. The user can also put the Skill's name at the start of a request for a new document.

Summary

- **How the three differ.** With `/unslap` and `/technical-writing`, the agent that wrote the text fixes it by rule. With `/bro`, the agent restates its reply when the user calls it.
- `/unslap`. It removes AI habits from every piece of text with 28 rules whose numbers never change.
- `/bro`. It restates the last reply, shorter and without jargon.

- /technical-writing. It first limits each document to one kind, and makes the document something a tired engineer understands on the first read.

The next chapter, [Chapter 33](#), covers /reflect, which looks back on what the work taught and adds those lessons to existing Skills.

Chapter 33: Reflect on what the work taught you and apply it

kaito • Sources: 日本語 • English • 2026-10-03

This chapter covers `/reflect`.

`/reflect` is a Skill that pulls lessons out of the conversation that will still help in the next piece of work. The Skill keeps only the lessons the user approves, for example by adding them to an existing Skill.

Two kinds of subagents pull out the lessons: reviewers, which read the conversation from three perspectives, and a synthesizer, which combines their results. The user decides whether a lesson goes into a Skill.

This chapter explains `/reflect` from four angles: its role, when to use it, its procedure, and how to write the request.

How this chapter is organized

The chapter has the following five sections.

- Role: narrow the conversation down to lessons that last
- When to use it: right after work that taught something, when the user calls `/reflect`
- Procedure: collect lessons from three perspectives and apply them only after approval
- How to write the request: say what happened and what you want to keep
- Summary

Role: narrow the conversation down to lessons that last

Chapter 3 covered the idea that a lesson from a user's correction should not end with the conversation. The lesson should stay in the codebase, in automated checks such as lint, or in Skills. `/reflect` handles the part that goes into Skills.

However, as the principle "Encode Lessons in Structure" from Chapter 21 says, readers easily miss written instructions. If you add a sentence to a Skill and the next agent does not notice it, nothing changes.

For that reason, this book treats three of the steps in this Skill's [procedure](#), steps 3 to 5, **as steps that narrow the lessons down, not steps that add more.**

- The synthesizer **rejects lessons that would not change how future agents act.**
- The agent does not turn lessons that lint or a script can enforce into Skill text. It moves them to the Backlog, a list of items it files in the issue tracker the team uses.
- The user picks which of the remaining lessons go into Skills.

Because of this narrowing, **only lessons that change how the next agent acts** end up in Skills.

When to use it: right after work that taught something, when the user calls `/reflect`

The bundled guide [docs/guide/09-make-it-yours.md](#) recommends that you call `/reflect` right after work that taught you something. The `SKILL.md` of `/reflect` says to use this Skill when the user says "reflect" or `/reflect`.

The agent does not use this Skill in the following cases.

- The conversation was trivial
- The conversation strayed from the main topic
- The agent did the work correctly by following an existing Skill

The reason for the last case is that work that went correctly under an existing Skill is evidence that the Skill is already correct.

The agent also **does not treat a one-time event as a lesson**.

This Skill has the setting `disable-model-invocation: true` (Chapter 25), so the agent does not pick this Skill on its own after reading the word "reflect". To run this Skill, the user calls it by name with `/reflect`.

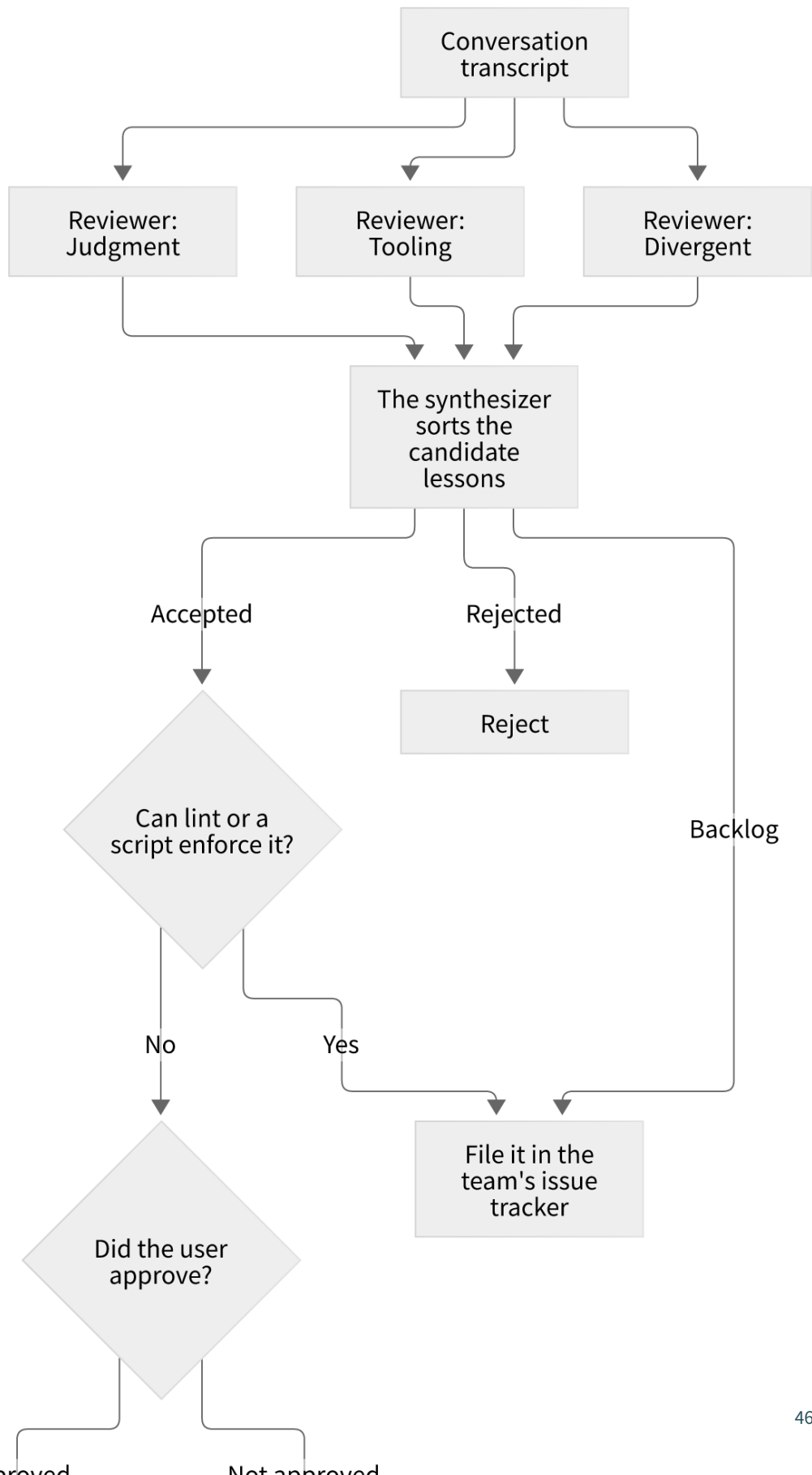
Procedure: collect lessons from three perspectives and apply them only after approval

The procedure is as follows.

1. **Find the transcript.** The agent looks for the transcript file of the current conversation. It looks only in the `agent-transcripts/` directory of the current workspace. If it searched other projects' directories, it would read private conversations from unrelated projects. If it finds no file, it writes a summary of the conversation and gives the summary to the reviewers in place of the file.
2. **Start the reviewers for the three perspectives at the same time.** The agent starts three reviewers at the same time. Each reviewer reads the conversation from one perspective: Judgment, Tooling, or Divergent. The default model is `claude-opus-5-5-max` for Judgment and Divergent, and `gpt-5.6-sol-max` for Tooling. You can change them with `/setup-pstack` (Chapter 34). The subsection "[Each of the three reviewers looks for a different kind of lesson](#)" explains which lessons each perspective looks for.
3. **The synthesizer sorts.** The synthesizer sorts the candidate lessons from the three reviewers into three groups: Accepted, Rejected, and Backlog.
4. **Move what lint or a script can enforce to the Backlog.** From Accepted, the agent moves to the Backlog anything that a lint rule, a script, a metadata flag, or a runtime check can enforce, instead of writing it as Skill text. A written instruction works only if the reader notices it and follows it. Lint and scripts enforce the rule even when nobody notices it.
5. **Apply the Skill edits only after the user approves.** The agent shows the user the whole result of the synthesizer's sorting, and the user picks which Accepted items to apply. The user can also change where a lesson goes, meaning which Skill it goes into. The agent applies only the edits the user picked. Only Accepted items wait for approval. The agent files Backlog items in the issue tracker without waiting.

6. **Report briefly to the user.** The agent lists, one line each, the edits it applied, the Skills it created, what it moved to the Backlog, and what it rejected, with the reason for each rejection.

Steps 2 to 5 look like this as a diagram.



View diagram 1

Only lessons that stay in Accepted and that the user approves lead to a Skill edit.

Each of the three reviewers looks for a different kind of lesson

The three perspectives and the lessons each one looks for are as follows.

- **Judgment.** Looks for lasting principles behind individual events. An example is the principle behind a correction from the user.
- **Tooling.** Looks for concrete details that the next agent would have to look up again: tools, commands, paths, and flags. Examples are a command flag the agent found by trial and error, or a way to reproduce a failed run locally. The Tooling reviewer also lists information the user handed over manually that the agent could have fetched itself through MCP. An example is a ticket ID the user pasted into the conversation.
- **Divergent.** Looks for what the other two perspectives miss. Examples are a decision that worked for the wrong reason, a decision that survived only because a test happened to pass, and a verification the agent skipped.

For each lesson, every one of the three reviewers returns the following three things.

- **Principle.** The lesson, written in one sentence, that will still hold in the next piece of work
- **Evidence.** The moment in the conversation where the lesson came from
- **Destination.** The Skill the lesson goes into

The destination must be a Skill that the conversation actually used. For a Skill that would have helped but was not used, the reviewer proposes a fix to its description. The reviewer discards a lesson that fits neither case.

The synthesizer narrows the lessons with eight criteria

The instructions for the synthesizer are in `references/synthesizer.md`. According to them, the synthesizer applies the following eight criteria to each candidate lesson.

Criterion	What it checks
Durability	Is it still correct six months later, after file paths and tool versions change?
Specificity	Is it broad enough to apply to other work, and concrete enough to show when to use it?
Existing-skill-first	Can an addition to an existing Skill cover it, without creating a new Skill?
Convergence	Did more than one reviewer raise it?
Decision-changing	Would writing it down change what the next agent does?
Structural-mechanism check	Could lint or a script enforce it instead?
Skill-was-used	Does it go into a Skill the conversation actually used?
Already-covered	Does the destination Skill already say the same thing?

For example, the synthesizer's instructions give the following line as an example of a lesson to discard.

```
linter at SHA bd91aa7 uses chars/4 heuristic
```

bd91aa7 is a SHA. This line only states a fact: "in the code at commit bd91aa7, the linter behaved this way". If a later commit rewrites the linter, the fact stops being true.

For that reason, the synthesizer rejects this line under the Durability criterion.

Apply Skill edits only after the user's explicit approval

A change to a Skill affects every future agent in the organization. For that reason, **the agent does not apply Skill edits automatically. It waits for the user's explicit approval.**

The agent handles each approved edit by its size, as follows.

- **Small edits, such as adding one line or correcting an outdated fact.** The agent edits the Skill directly.
- **Large edits of more than about ten lines, such as adding a new section.** The agent hands the edit to `create-skill`, a Skill built into Cursor for writing Skills, and `create-skill` drafts, tests, and revises the edit in a loop.
- **A Skill exists, but the agent did not use it where it should have.** The agent has `create-skill` fix that Skill's description.
- **No existing Skill fits the addition.** The agent has `create-skill` create a new Skill.

In an environment that has a validator for `SKILL.md`, the agent runs it on every Skill it edited before it finishes the work.

The bundled guide recommends that the user approve only proposals that change future agents' decisions. The same page also says that "an odd thing that happened once is a coincidence, not a rule".

Examples of `SKILL.md` validators

A `SKILL.md` validator mainly checks that the frontmatter at the top is written correctly. Examples include the following.

- `skills-ref validate` is a command from the reference library in the repository that publishes the Agent Skills specification. You run it with a Skill directory, as in `skills-ref validate path/to/skill`.
- `quick_validate.py` is a script that comes with `skill-creator`, the Skill for creating Skills, in Anthropic's `anthropics/skills` repository. It checks that `SKILL.md` exists, that the frontmatter is valid YAML, that `name` and `description` are present, and that no disallowed fields appear.

pstack itself does not include a script that validates `SKILL.md`.

Instead, in pstack, the "[Authoring or modifying a skill](#)" Playbook for writing Skills ([Chapter 13](#)) does the validator's job, even in an environment without a validator.

This Playbook requires the agent to check that the frontmatter has `name` and `description`, that the referenced files exist, and that links to other Skills resolve.

How to write the request: say what happened and what you want to keep

The bundled guide gives the following example request to send right after work that taught you something.

```
/reflect that took way too long. capture what we learned so the next run doesn't repeat it.
```

In this request, the user writes what happened, which is that the work took far too long. The user also writes what they want to keep, which is the lessons that stop the next run from repeating it.

Summary

- **How lessons are collected.** The reviewers collect lessons from three perspectives: Judgment, Tooling, and Divergent. The synthesizer narrows the lessons with criteria such as whether a lesson is still correct six months later.
- **Backlog.** Lessons that lint or a script can enforce do not become Skill text. They go to the team's issue tracker.
- **How lessons are kept.** After the user approves, the agent adds to or rewrites the SKILL.md of an existing Skill.

The next chapter, [Chapter 34](#), covers `/setup-pstack`, which installs pstack and decides which model each role uses.

Chapter 34: Install pstack and choose a model for each role

kaito • Sources: 日本語 • English • 2026-10-03

This chapter covers `/setup-pstack`.

`/setup-pstack` is a **Skill that chooses which model to use for each role** of the subagents that pstack starts. After you install the pstack plugin, this is the first Skill you run.

A role here is the job a subagent takes on. For example, `bug-fix` is the subagent that writes code in the "Bug fix" Playbook. `judgment` and `prose` is the subagent that handles writing and judgment. `arena` runners are the candidate subagents that `/arena` has solve the same task.

This chapter explains how to install pstack, how `/setup-pstack` proceeds, and the rule file that `/setup-pstack` writes.

This chapter also covers how to install pstack because installation, like `/setup-pstack`, is one of the first things you do when you start using pstack.

"Chapter 6: Some Skills that the procedures call by name do not ship with pstack" covers the Skills that ship with pstack and the Skills you must install as a separate plugin.

How this chapter is organized

The chapter has the following sections.

- Install pstack
- The list of roles
- Each Skill reads one rule file
- Detect the available models

- Read the previous rule file
- Choose a reasoning budget
- inherit-parent and auto use the parent chat's model
- Verify that the models are available before writing
- Panel roles
- The file it writes
- Lines that remain when you run it again
- /setup-pstack offers to create a verification skill
- Summary

Install pstack

You install pstack with one command and a short conversation.

1. In the Cursor chat, run the following command.

```
/add-plugin pstack
```

2. Next, run /setup-pstack with the following command and answer its questions. The sections after this one explain what /setup-pstack asks and what it decides.

```
/setup-pstack
```

3. When /setup-pstack finishes, start a new chat. **The model rules that /setup-pstack writes apply from the next new chat.**

The list of roles

As of pstack version 0.15.5, /setup-pstack chooses models for the following 17 roles. The role names are the original names written in the rule file.

Role	Subagent that takes the role	Default model
feature, refactoring	Writes code in the "Feature" and "Refactoring" Playbooks	grok-4.7-xhigh-fast
bug-fix	Writes code in the "Bug fix" Playbook	grok-4.7-xhigh-fast
perf-issue	Writes code in the "Perf issue" Playbook	grok-4.7-xhigh-fast
hillclimb	Writes code in the "Hillclimb" Playbook	grok-4.7-xhigh-fast
judgment and prose	Writes prose and makes judgments	claude-opus-5-5-max
hardest tasks	Writes code for the hardest changes, such as designs that span several places, tangled concurrency, and algorithms where small errors are hard to notice	claude-opus-5-5-max
how explorer	Splits up the code investigation in /how	grok-4.7-xhigh-fast
how explainer	Combines the findings into one explanation in /how	claude-opus-5-5-max
why investigators	Splits up the investigation of sources such as Git history and tickets in /why	grok-4.7-xhigh-fast
why synthesizer	Combines the findings into one answer in /why	claude-opus-5-5-max
reflect tooling	Reads the conversation from the tooling point of view in /reflect	gpt-5.6-sol-max
reflect judgment, divergent, synthesizer	Reads the conversation from the judgment and divergent points of view in /reflect and sorts the results	claude-opus-5-5-max
arena runners	The candidates that solve the same task in /arena	claude-opus-5-5-max, gpt-5.6-sol-max, grok-4.7-xhigh-fast

Role	Subagent that takes the role	Default model
arena cross-judge pool	The judge that scores the candidates' artifacts in /arena (/arena picks one model from this list)	claude-opus-5-5-max, gpt-5.6-sol-max, grok-4.7-xhigh-fast
swarm workers	The Workers that run in parallel in /swarm	grok-4.7-xhigh-fast
architect runners	Creates design options in /architect	claude-opus-5-5-max, gpt-5.6-sol-max, grok-4.7-xhigh-fast
interrogate reviewers	Looks for weak points in the diff in /interrogate	claude-opus-5-5-max, gpt-5.6-sol-max, grok-4.7-xhigh-fast

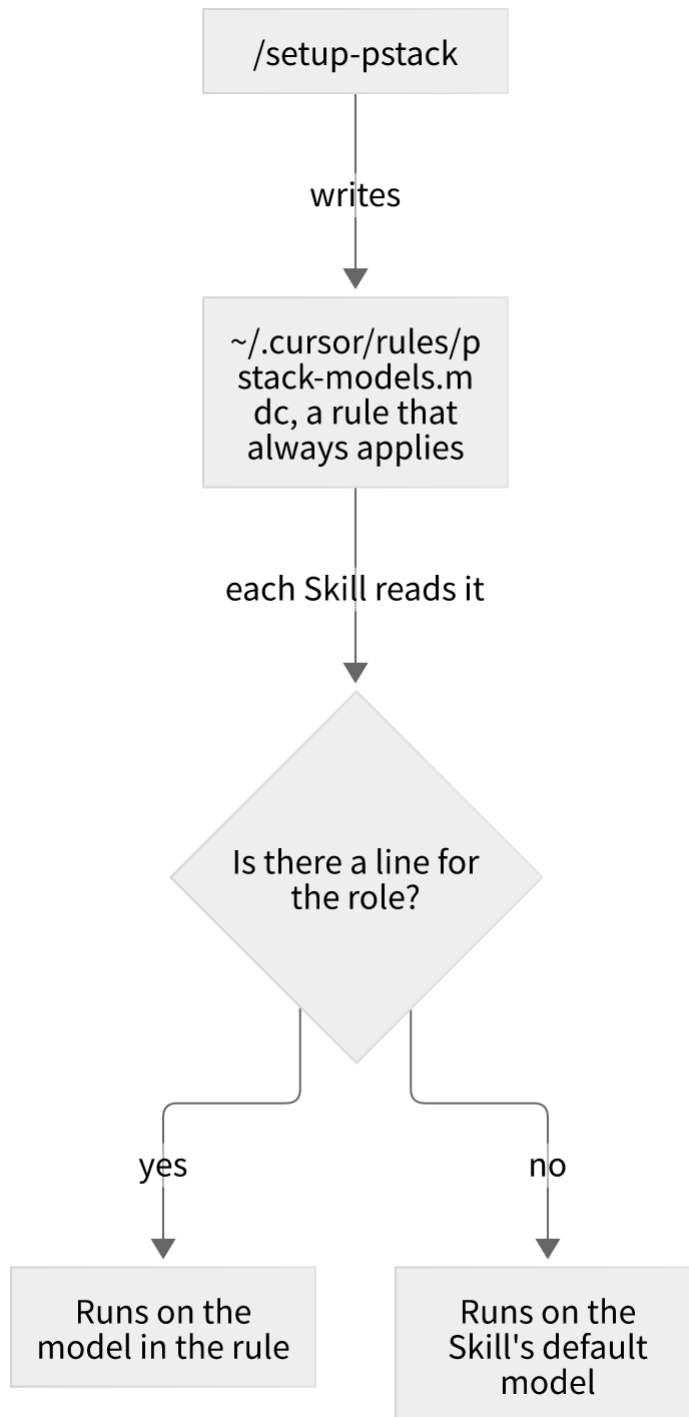
For a role with three default models, the value is a list of models. The section "[Panel roles](#)" explains these roles.

Each Skill reads one rule file

/setup-pstack is a **Skill that asks for the reasoning budget and the model for each role, and writes the answers to** `~/cursor/rules/pstack-models.mdc`, **a rule that always applies.**

- **Reasoning budget.** A setting that decides the reasoning effort of every role's model at once.
- **Slug.** An identifier that names a model, such as `claude-opus-5-5-max`.
 - Example: `claude-opus-5-5-max` runs Claude Opus 5.5 at effort level `max`
- **Effort level.** How much work a model puts into reasoning. The words `max`, `xhigh`, `high`, `medium`, and `low` in a slug express it, strongest first.

Each pstack Skill reads this rule and decides the model for each role.



[View diagram 1](#)

A role without a line runs on the Skill's default model.

So you change only the roles that you want to move off the default model. For example, if you want only the `bug-fix` role to run on another model, change only the `bug-fix` line.

To put a role back on the default model, delete that role's line from the rule file.

`/setup-pstack` proceeds in the following order.

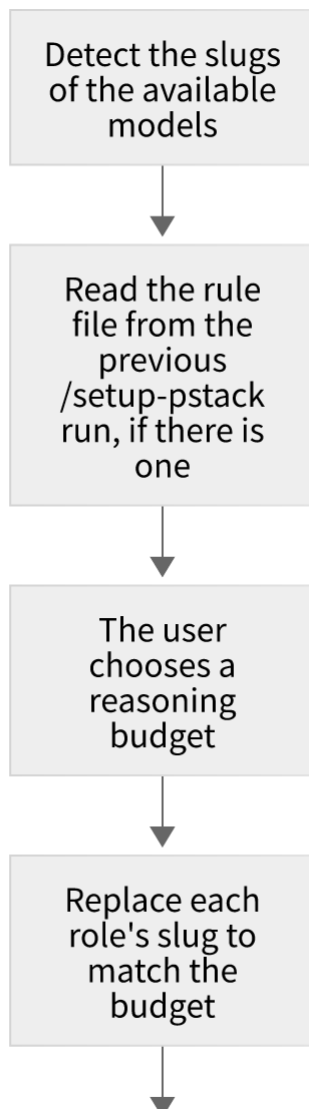


Diagram segment 1

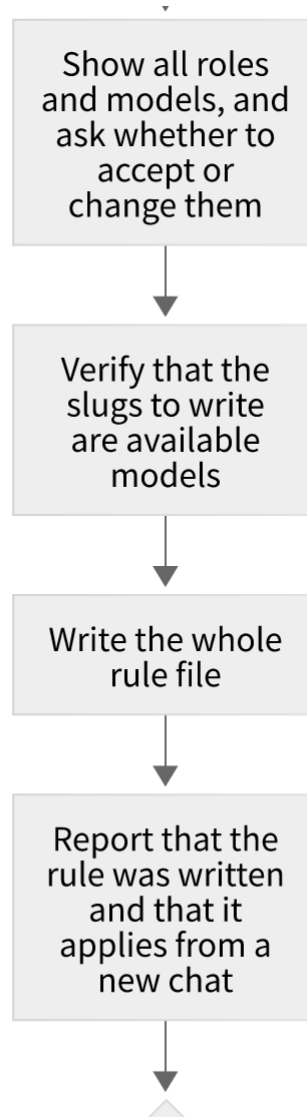


Diagram segment 2

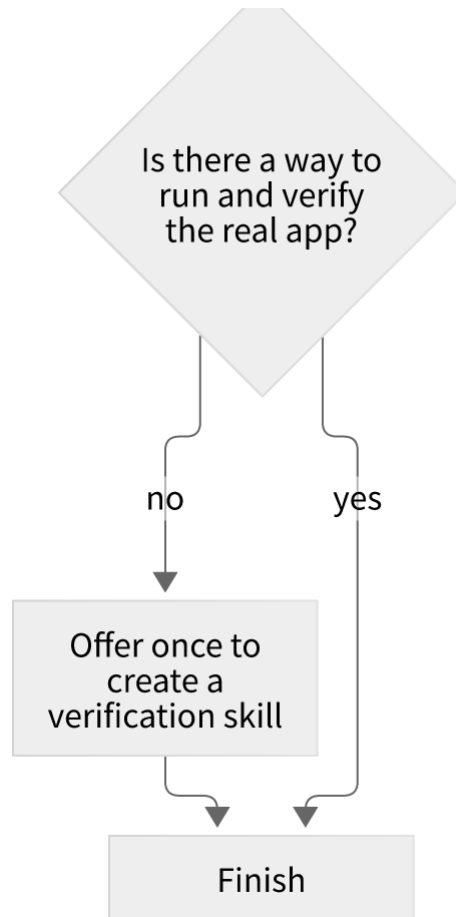


Diagram segment 3

[View diagram 2](#)

Detect the available models

First, `/setup-pstack` finds the slugs of the models the user can use. It looks in the following ways.

- **The basic method.** It checks which slugs it can pass to a subagent in the current chat. The source treats this as the reliable method.
- **When there is an API or CLI.** If Cursor provides an API or CLI that lists the models the user can use, it prefers that list, so that it misses no model.
- **When it finds nothing.** It asks the user to paste the slugs they can use into the chat.

In this chapter, I call the slugs found here "the available models".

Read the previous rule file

If you ran `/setup-pstack` before, the rule file that it wrote then, `~/.cursor/rules/pstack-models.mdc`, already exists. In that case, `/setup-pstack` reads the value for each role and the file's `# budget` line, which holds the budget you chose last time. It uses these values as the starting point for choosing again.

The previous rule file, however, may still hold lines for roles that pstack has since removed, such as a `how critics` line. A removed role is a role that does not appear in the file the current pstack writes. The section "[The file it writes](#)" in this chapter shows the contents of that file.

`/setup-pstack` discards these lines. It tells the user which lines it discarded when it shows the list of roles and models.

Choose a reasoning budget

The reasoning budget is a setting that decides at once **which effort level every role's model runs at**. The words `max`, `xhigh`, `high`, `medium`, and `low` in a slug express it. `max` is the strongest level and `low` is the weakest.

You choose one of the following four budgets. Each budget sets the effort level that `/setup-pstack` aligns the slugs to. From here on, I call this level the target level.

Budget	Target level	What <code>claude-opus-5-5-max</code> becomes
unlimited	max (<code>/setup-pstack</code> does not replace the effort in each slug, so it stays at the default)	<code>claude-opus-5-5-max</code>
large	Align to xhigh	<code>claude-opus-5-5-xhigh</code>
medium	Align to high	<code>claude-opus-5-5-high</code>
small	Align to medium	<code>claude-opus-5-5-medium</code>

Replace the effort word in each slug with the target level

If you choose `large`, `medium`, or `small`, `/setup-pstack` replaces only the effort word in every role's slug with the target level. The effort word is at the end of the slug. If the slug ends in `fast`, the effort word comes right before `fast`.

For example, if you choose `small`, whose target level is `medium`, the slugs change as follows.

- `claude-opus-5-5-max` becomes `claude-opus-5-5-medium`
- `grok-4.7-xhigh-fast` becomes `grok-4.7-medium-fast`

On a line that lists several models for one role, such as `arena runners: claude-opus-5-5-max, gpt-5.6-sol-max, grok-4.7-xhigh-fast`, `/setup-pstack` replaces every listed slug. The section "[Panel roles](#)" in this chapter explains these roles.

`inherit-parent` and `auto` are not slugs, so `/setup-pstack` does not replace them.

When a replaced model is not available, pick a substitute in the same family

The slug after replacement may not be among the available models. For example, `claude-opus-5-5-medium` may not be available.

In that case, `/setup-pstack` decides the substitute model in the following order.

1. Look for available models in the same family
2. Among those at the target level or weaker, use the strongest one
3. If there are none, show the role as one the user must choose again

For example, say you chose `small`, whose target level is `medium`, and the only Claude models you can use are `claude-opus-5-5-high` and `claude-opus-5-5-low`.

- `high` is stronger than the target `medium`, so `/setup-pstack` does not use it.
- `low` is weaker than the target `medium`, so `/setup-pstack` can use it.

So `/setup-pstack` uses `claude-opus-5-5-low`.

Because `/setup-pstack` picks substitutes only from the same family, **a lower budget does not change the model family**. For example, a role that ran on Claude does not switch to Grok because you lowered the budget.

Finally, check the list of roles and models

When the replacement is done, `/setup-pstack` shows a list of every role and the model assigned to it. Roles you must choose again carry a mark that says so.

You either accept the list as it is or change the model for specific roles. To change a model, pick one of the available models, `inherit-parent`, or `auto`.

`inherit-parent` and `auto` use the parent chat's model

Both `inherit-parent` and `auto` mean **run that role's subagent on the same model as the parent chat**. The parent chat is the chat that started the subagent.

For a role with either of these values, no Skill specifies a model when it starts the subagent. A subagent started without a model runs on the same model as the parent chat.

`Auto` is the setting in which Cursor picks the model automatically. So if you use the chat with `Auto`, you can run subagents on `Auto` too by writing `inherit-parent` or `auto` for the role.

Verify that the models are available before writing

After you review the list, `/setup-pstack` runs one more check before it writes the rule file. It verifies that every slug it will write is among the available models it detected in "[Detect the available models](#)" in this chapter.

If even one slug is missing from that list, `/setup-pstack` stops without writing and asks you to choose the model for that role again. `/setup-pstack` **does not write a slug to the rule file unless it has confirmed that the slug is available.**

The reason is that each Skill passes the slug from the rule file unchanged as the model when it starts a subagent.

`inherit-parent` and `auto` are not model slugs, so this check does not apply to them. You can always write them to the rule file.

Panel roles

Every frontier model, meaning each company's most advanced model, has strengths and weaknesses. For that reason, according to the [README](#), many pstack Skills combine several models to use the strengths of each.

`/setup-pstack` gives the name panel roles to the roles that list several models for this purpose. A panel is a mechanism that gives the same question to several models and compares the answers ([Chapter 6](#)).

There are three panel roles.

- `arena runners`. The candidate subagents that `/arena` has solve the same task.
- `architect runners`. The subagents that `/architect` has create design options before anyone writes code.
- `interrogate reviewers`. The reviewer subagents that `/interrogate` has look for weak points in the diff.

One subagent starts for each listed model

The value of a panel role lists models separated by `.`. **One subagent starts for each model in the list.**

For example, with arena runners: `claude-opus-5-5-max`, `gpt-5.6-sol-max`, `grok-4.7-xhigh-fast`, `/arena` starts one subagent on each of the three models, three subagents in total.

If the list contains `inherit-parent` or `auto`, one subagent still starts for each item. That subagent runs on the parent chat's model.

A line that lists models but is not a panel role

The rule file has one line that lists models in its value but is not a panel role: `arena cross-judge pool`.

`arena cross-judge pool` is the line that lists the candidate models for the `/arena judge`, the agent that scores the candidates' artifacts ([Chapter 24](#)). After all candidates finish writing, `/arena` picks one model from the models on the `arena cross-judge pool` line and starts it as the judge. When it picks, it prefers a model from a different family than the parent chat's model.

`swarm workers` is not a panel role either. `swarm workers` is the default model for every Worker that `/swarm` starts, and its value holds one model.

The exception is a model race. A model race is a way of using `/swarm` in which Workers on different models solve the same task, and you compare which model gives the best result.

In that case, `/swarm` decides the model for each Worker before it starts the Workers. Each Worker runs on its assigned model, not on the `swarm workers` model.

The file it writes

Each time it runs, `/setup-pstack` rewrites the whole rule file from scratch. It does not append lines to the previous file or rewrite only some of its lines.

If you choose the `unlimited` budget and accept the defaults for every role, `/setup-pstack` writes the following file. This example is from pstack version 0.15.5.

```
---
description: pstack per-role model choices (overrides skill defaults)
alwaysApply: true
---
```

```
# pstack model configuration. One line per role. Delete a line to fall back to the skill default.
# `inherit-parent` or `auto` as a value: the role runs on the parent chat model (omit Task `model`).
Alias entries in a panel list still count toward its fan-out.
# budget: unlimited (max)
feature, refactoring: grok-4.7-xhigh-fast
bug-fix: grok-4.7-xhigh-fast
perf-issue: grok-4.7-xhigh-fast
hillclimb: grok-4.7-xhigh-fast
judgment and prose: claude-opus-5-5-max
hardest tasks: claude-opus-5-5-max
how explorer: grok-4.7-xhigh-fast
how explainer: claude-opus-5-5-max
why investigators: grok-4.7-xhigh-fast
why synthesizer: claude-opus-5-5-max
reflect tooling: gpt-5.6-sol-max
reflect judgment, divergent, synthesizer: claude-opus-5-5-max
arena runners: claude-opus-5-5-max, gpt-5.6-sol-max, grok-4.7-xhigh-fast
arena cross-judge pool: claude-opus-5-5-max, gpt-5.6-sol-max, grok-4.7-xhigh-fast
swarm workers: grok-4.7-xhigh-fast
architect runners: claude-opus-5-5-max, gpt-5.6-sol-max, grok-4.7-xhigh-fast
interrogate reviewers: claude-opus-5-5-max, gpt-5.6-sol-max, grok-4.7-xhigh-fast
```

According to the README, pstack by default assigns grok-4.7-xhigh-fast to the four roles that write code: feature, refactoring, bug-fix, perf-issue, and hillclimb. It assigns claude-opus-5-5-max to hardest tasks, which handles the hardest changes, and to judgment and prose, which handles writing and judgment.

The default models for the panel roles are claude-opus-5-5-max, gpt-5.6-sol-max, and grok-4.7-xhigh-fast.

For reference, my latest settings are below.

The choices I care about are that I use different models for the investigator roles and the explainer roles, and that I leave Claude models out of the /arena judge pool.

The reason is that different models have different personalities.

```
---
description: pstack per-role model choices (overrides skill defaults)
alwaysApply: true
---
# pstack model configuration. One line per role. Delete a line to fall back to the skill default.
# `inherit-parent` or `auto` as a value: the role runs on the parent chat model (omit Task `model`).
Alias entries in a panel list still count toward its fan-out.
# budget: medium (high)
feature, refactoring: grok-4.7-high
```

```

bug-fix: grok-4.7-high
perf-issue: grok-4.7-high
hillclimb: grok-4.7-high
judgment and prose: claude-opus-5-5-high
hardest tasks: claude-opus-5-5-high
how explorer: grok-4.7-high
how explainer: claude-opus-5-5-medium
why investigators: grok-4.7-high
why synthesizer: claude-opus-5-5-medium
reflect tooling: claude-sonnet-5-5-high
reflect judgment, divergent, synthesizer: claude-opus-5-5-medium
arena runners: claude-opus-5-5-high, muse-spark-1.3-max, grok-4.7-high, claude-sonnet-5-5-high
arena cross-judge pool: gpt-5.6-sol-medium, grok-4.7-high, muse-spark-1.3-max
swarm workers: grok-4.7-high
architect runners: claude-opus-5-5-high, muse-spark-1.3-max, grok-4.7-high, claude-sonnet-5-5-high
interrogate reviewers: claude-opus-5-5-medium, muse-spark-1.3-max, grok-4.7-high, claude-sonnet-5-5-high

```

Lines that remain when you run it again

When you run `/setup-pstack` again, it looks at the previous rule file and decides each role's model as follows.

- **A role that stayed on the default model last time.** It decides the model again, starting from the default model.
- **A role that the user changed to a non-default model last time.** It keeps the model the user chose.

For example, say that in the previous run you changed only `bug-fix` from the default `grok-4.7-xhigh-fast` to `claude-opus-5-5-max`. When you run it again, `bug-fix` keeps `claude-opus-5-5-max`, and `/setup-pstack` decides the other roles again from the default models.

You do not have to choose the roles you changed again on every run.

Rule files from before pstack version 0.15.3 keep the old models

This mechanism has one problem.

A rule file written before pstack version 0.15.3 holds the default models of that time, but pstack has changed its default models since then. In other words, the old default models now differ from the current defaults.

So even when you run it again, `/setup-pstack` keeps those lines as "lines that hold a non-default model". Those roles keep running on the old default models.

The [README](#) and the bundled guide give the following advice. If you ran `/setup-pstack` before pstack version 0.15.3, do one of the following and then run `/setup-pstack` again.

- Delete the lines for roles that hold the old default models
- Delete the whole rule file, `~/.cursor/rules/pstack-models.mdc`

`/setup-pstack` **offers to create a verification skill**

At the end, `/setup-pstack` checks whether the agent can run the project's app and verify that a change works. It checks whether the project has either of the following two things.

- **A verification skill.** A project-specific Skill that the agent uses to start the app, operate it, and collect evidence ([Chapter 3](#)). If you create it with `/create-verification-skill`, `/create-verification-skill` names it `verify-<app name>`.
- **An existing harness.** A mechanism already in the repository that drives the app from the outside, such as Playwright tests.

If there is neither, `/setup-pstack` offers once to create a verification skill with `/create-verification-skill`.

In the text of the offer, `/setup-pstack` gives the purpose of a verification skill as "**so that the agent can operate the app the way a user does and prove that a change works**".

- **If the user accepts.** `/setup-pstack` calls `/create-verification-skill`.
- **If the user declines.** `/setup-pstack` finishes without recommending `/create-verification-skill`.

[Chapter 26](#) covers the procedure `/create-verification-skill` follows to create a verification skill, and [Chapter 36](#) covers how to create a verification skill and start using it with your own app.

Summary

- **What it writes.** `/setup-pstack` writes the model for each role to one rule file. It writes only slugs that it has confirmed are available.
- **Reasoning budget.** A lower budget lowers only the effort level and does not change the model family.
- **When it applies.** The rule applies from a new chat, so you must start a new chat after it runs.

The next chapter, [Chapter 35](#), covers `/poteto-mode`, pstack's Router, which receives the work requests of a user who has installed pstack.

Chapter 35: /poteto-mode is pstack's Router

kaito • Sources: 日本語 • English • 2026-10-03

/poteto-mode is the entry point to pstack. **It reads the request, picks one of the 23 Playbooks, and follows that Playbook's steps** (Chapter 7).

This chapter covers `SKILL.md`, the body of /poteto-mode.

Chapter 7 covered how to call /poteto-mode and how it routes requests. Chapter 8 covered the roles of Playbooks, Skills, and Principles and the main rules in "Non-negotiables". This chapter explains the parts of `SKILL.md`, the body of /poteto-mode, that those chapters did not cover.

How this chapter is organized

The chapter has these sections:

- What /poteto-mode is
- "Non-negotiables": what the agent must always do
- "Principles": the index of principles
- "Autonomy": where the agent proceeds without asking and where it stops
- "Subagents": the rules for delegation
- "Writing the reply": how to write replies
- "Comments": how to write comments
- "Playbooks": the index of Playbooks
- Fit it to your own style
- Summary

What /poteto-mode is

The "Router" in the chapter title refers to the job of /poteto-mode, which is to route each request to the Playbook that fits it. The bundled guide also gives its /poteto-mode page, [02-poteto-mode.md](#), the title "Route work through /poteto-mode".

Besides the rules that route requests to Playbooks, /poteto-mode holds **rules that apply to all the work after routing**. Here are some examples:

- Before you write code, decide the shape of the data, as the principle "**Model the Domain**" says.
- Always stop before an irreversible operation.
- Write replies in short, declarative sentences.

The description in the frontmatter of `SKILL.md` calls /poteto-mode "poteto's agent style" and lists five things that style aims for:

- concise, detailed responses
- deliberate subagents
- unslopped prose
- simple code
- verified work

After the user installs pstack ([Chapter 34](#)), they **hand work requests to** /poteto-mode.

/poteto-mode has the setting `disable-model-invocation: true`, so it starts only when the user calls it by name. **After one call, it stays active for the later turns of the same conversation without the /poteto-mode prefix, until the user says to stop** ([README](#), [Chapter 7](#)). While it is active, the agent follows a Playbook's steps only when the request fits a Playbook or needs rigor.

The body of `SKILL.md` has seven sections, from "Non-negotiables" to "Playbooks".

"Non-negotiables": what the agent must always do

"Non-negotiables" is a list of rules of the form "in this situation, always do this". We saw the main rules in [Chapter 8: What do Playbooks, Skills, and Principles each handle?](#).

"Non-negotiables" opens with these rules, which apply to every item in the list:

- In its reply, the agent names each principle that affected a decision and the choice that principle changed.
- The agent may name only principles whose Skill file it read in full during this session.

Take the example from [Chapter 8](#), a bug where a retry in the middle of a run made the export write duplicate rows. After the fix, the reply contains a passage like this:

```
Per Fix Root Causes, I fixed the code that wrote already-exported rows again on every retry. I did not add a step that removes duplicates after export.
Per Make Operations Idempotent, I changed the write to overwrite by row key. Running the same job twice leaves the row count unchanged.
```

A reply like the following does not meet those two rules:

```
I followed Fix Root Causes and Make Operations Idempotent
```

A reply that only lists principle names does not say what the agent decided by following them. And if the agent cannot write what it decided, it may not have used the principle at all.

When the user sees the principle names next to the agent's decisions, the user can check which standard the agent applied and what it decided.

Besides the rules from Chapter 8 and these shared rules, the list has the following rules.

Situation	What the agent always does	Details
The agent is about to ask the user "which approach is better"	Before asking, classify whether the answer is a fact that running something would reveal or a judgment that a person makes	Chapter 38
The agent writes code	Make the shape of the data clear first, then choose the structure. Follow the principle " Model the Domain "	Chapter 18

Situation	What the agent always does	Details
The agent splits work to run in parallel	Use <code>/swarm</code> for coverage, competition, gauntlet, and search splits. Use <code>/arena</code> to compare designs or code where you pick a base and port the good parts into it	Chapter 24
A design that people disagree on	Use <code>/interrogate</code> before the merge	Chapter 27
Nontrivial multi-step work	Write a throughput checkpoint, the same as step 3 of " Feature "	Chapter 12
A request about the state of a PR	Use " Babysit ", not Cursor's built-in <code>/autopi</code> lot Skill (formerly named <code>babysit</code>). Do not use "Babysit" only because someone opened a PR	Chapter 14
A request to stack-merge PRs whose checks all passed	Run " Shipping ". Do not treat passing checks as proof of safety. Verify each PR on its own before the merge	Chapter 14
Comments from Bugbot or from an agent's security review	Do not accept them at face value. Sort each finding into fix, dismiss, or ask. Bugbot and similar tools find real bugs, but they also raise findings that are not problems or are too minor to matter	Chapter 14
A Skill breaks during the work	Do not stop the work and do not quietly work around it. Fix it in a PR for that Skill alone	Chapter 13

- **Shape of the data.** The fields the data has and the states it can take.
- **Throughput checkpoint.** Four items the agent records in the TODO list after it considers how the work could split in parallel: the steps that must finish first, the work that can run in parallel, the shared state that gets modified, and the smallest unit that splits safely.
- **Request about the state of a PR.** A request such as "check on PR X" or "get it green".

- **Coverage.** A split that assigns several areas to Workers so that nothing goes unchecked.
- **Competition.** A split that has several Workers compete on the same problem.
- **Gauntlet.** The pstack files do not define it. This book defines it as a split that lines up many verifications against one change and requires the change to pass all of them ([Chapter 24](#)).
- **Search split.** A split that divides the places to search and has each Worker check one place. For example, when the agent looks for the cause of a bug, it assigns one Worker to each suspect module.
- **Stack.** A chain of PRs linked as parent and child, with each PR built on top of the one before. From the second PR on, each PR targets the branch of the PR below it. Only the bottom PR targets the trunk branch, such as `main` ([Chapter 14](#)).
- **Bugbot.** A Cursor feature that reviews PRs automatically.

Before asking, separate facts from judgments

Of the rules in the table, **the source explains the first one, "classify before asking", at the greatest length.**

When the agent is about to ask the user "which approach is better" or "how should this be built", it first classifies which of the following the answer is:

- **A fact that running something would reveal.** Examples are behavior, processing time, layout, output, and performance. The agent does not ask a person. It verifies the fact itself. How it verifies depends on the Playbook that `/poteto-mode` routed the request to:
 - Any Playbook other than "**Investigation**". The agent tries it with the "**Prototype**" Playbook and decides from the result.
 - The "**Investigation**" Playbook. The agent builds no prototype. It continues the investigation and answers from evidence that it finds in code and other sources.
- **A judgment that a person makes.** A product decision or a preference that no experiment can settle. The agent asks a person only about this kind of judgment.

"**Investigation**" builds no prototype because "Investigation" is an inquiry that does not change code. "Investigation" gives the user an answer with evidence attached ([Chapter 10](#)).

When the user hands off the whole job

When the user hands the whole job to the agent, the agent makes the decisions that come up during the work on its own, without asking the user, and keeps going. At the end of the work, it reports the decisions it made.

What goes into the report depends on the kind of decision:

- **Decisions within the delegated scope.** Decisions the agent made during the work, such as where to put the tests or how to split a function. The report says which option the agent chose.
- **Judgments that a person makes.** Product decisions or preferences that no experiment can settle. The report says which option the agent chose, why it chose it, and the words the user can say to undo the choice, such as "revert that".

Even when the job is handed off, the agent stops and waits for the user's decision in two cases:

- A checkpoint where the user asked in advance for the agent to stop, such as before a merge.
- An item on the "Always pause" list, which this chapter covers in [the "Autonomy" section](#). These are irreversible operations such as a force-push to a shared branch or a deploy.

"Principles": the index of principles

"[Principles](#)" is the index of the 23 principles. **Each entry gives the principle's name, the condition for applying it, and its key point, in that order.**

The index sorts the 23 principles into five groups:

Group	Principles	Chapter in this book
Core	Laziness Protocol, Foundational Thinking, Redesign from First Principles, Attack the Premise, Subtract Before You Add, Minimize Reader Load, Outcome-Oriented Execution, Experience First, Exhaust the Design Space, Build the Lever	Chapter 17
Architecture	Model the Domain, Boundary Discipline, Type System Discipline, Make Operations Idempotent, Migrate Callers Then Delete Legacy APIs, Separate Before Serializing Shared State	Chapter 18
Verification	Prove It Works, Fix Root Causes, Sequence Work into Verifiable Units, Test Behavior, Not Implementation	Chapter 19
Delegation	Guard the Context Window, Never Block on the Human	Chapter 20
Meta	Encode Lessons in Structure	Chapter 21

When the agent starts work in `/poteto-mode`, it first reads this index and picks the principles that apply to the current work. The index sits inside the `SKILL.md` of `/poteto-mode`, so when the agent loads `/poteto-mode`, it loads the index too ([Chapter 9](#)).

For example, here is the entry for "Model the Domain".

After the principle's name comes the condition for applying it, such as writing stateful logic. Then comes its key point, which is to encode the domain in a structure.

```
- **Model the Domain** (**principle-model-the-domain**). Writing stateful logic, or code that branches a lot or repeats a shape assumption across files. Encode the domain in a structure (state machine, typed model, table or registry, reducer, boundary, the right collection) instead of scattered conditionals.
```

For each principle the agent decides to apply, it does not stop at the key point in the index. It reads that principle's Skill file (`SKILL.md`) in full.

This rule opens the "Principles" section of the `/poteto-mode SKILL.md`.

Read the leaf skill in full for any principle you apply.

The "Non-negotiables" section of the same `SKILL.md` also opens with this sentence:

Cite only principles whose leaf `SKILL.md` you read this session.

So the rules keep the agent from naming, in a reply, a principle whose Skill file it has not read.

Chapter 9 covers the loading scheme, in which **the agent reads only the index up front and reads a principle in full when it applies it**. Part IV (Chapter 17 to Chapter 21) covers what each principle says.

"Autonomy": where the agent proceeds without asking and where it stops

"Autonomy" separates **work the agent may do without asking from work where it must stop**. It also sets how the agent answers when the user asks for its opinion. It has four paragraphs:

Paragraph	Rule
"Just do it."	The agent may use MCP tools freely without asking. It also proceeds without asking on reversible work and on outward actions: posting to the team chat, updating a ticket, and starting an Eval (Chapter 13)
"Always pause"	The agent always stops at irreversible operations: a force-push to a shared branch, a deploy, deleting data, and messages to customers
"Session overrides"	When the user says "don't stop", "I'm going to bed", "run until it's done", or "I'm leaving it all to you", the agent keeps going without stopping

Paragraph	Rule
"No is an acceptable answer."	When the user asks "should I do this", proposes adding work, or shows an approach, the agent gives its honest view. If it disagrees, it declines or objects, for example with "this is not worth adding". It does not agree only because the idea came from the user. It says what it thinks

Even under a "Session overrides" instruction, the agent stops at work covered by "Always pause", the second row of the table, which covers irreversible operations. The "Non-negotiables" rules state this exception, and the "Session overrides" paragraph does not. The section "[When the user hands off the whole job](#)" in this chapter also covers this exception.

"Subagents": the rules for delegation

"Subagents" holds the rules for when the main agent delegates part of a task to a subagent. It sets two things:

- **The delegate.** Which subagent receives the work.
- **The defaults.** The settings the main agent uses every time it launches a subagent.

First, the delegate. When the main agent launches a subagent during a Playbook's steps, it uses poteto-agent by default. But `/how`, `/why`, `/interrogate`, `/reflect`, and `/swarm` choose their own subagents so that they can review with several models. The main agent does not replace the subagents those Skills choose with poteto-agent.

poteto-agent is a subagent that ships with pstack ([agents/poteto-agent.md](#)). Before it starts work, it reads the `/poteto-mode SKILL.md` in full, including the Principle index. As a result, it works under the same `/poteto-mode` rules as the main agent.

Next, the defaults. Whichever subagent the main agent launches, it uses the same settings every time. There are four defaults:

- **Run in the background.** The main agent can keep working without waiting for the subagent to finish.

- **Run in agent mode.** Agent mode is the mode that can edit files and run tools. In a read-only mode, the subagent cannot use MCP tools.
- **Put file paths in the prompt, not file contents.** The subagent opens and reads the files at the paths it receives.
- **Set a model for each role.** The main agent states which model runs the subagent for each role. [Chapter 34: Install pstack and choose a model for each role](#) lists the roles and their default models. `/setup-pstack` can change those models.

The agent follows these defaults in every delegation.

The fourth default, setting the model, has a rule for handing off coding work. The main agent picks the delegate's model by how hard the change is:

- **The hardest changes** go to the model with the best judgment, which is `claude-opus-5-5-max` by default. Examples are designs that span several places, tangled concurrency, and algorithms where small errors are hard to spot. The main agent sends a hardest change to this model whether the instructions are vague and need judgment or the steps are detailed and the delegate only needs to follow them.
- **Trivial mechanical edits**, such as renaming, go to a fast-responding coding model, which is `grok-4.7-xhigh-fast` by default.

The rule file that `/setup-pstack` writes can change the delegate's model. Depending on what it delegates, the agent reads one of these lines in the rule file to pick the model:

- **When the agent delegates the hardest changes.** The `hardest tasks` line.
- **When the agent delegates code for a Playbook.** That Playbook's line, which is `feature, refactoring, bug-fix, perf-issue, Or hillclimb`.
- **When the agent delegates prose and judgment.** The `judgment and prose` line.

A role with no line in the rule file keeps its default model. A role whose line says `inherit-parent` or `auto` runs on the model of the parent chat ([Chapter 34](#)).

"Subagents" also sets how to treat the results that subagents return.

First, the section defines a second opinion, which we saw in [Chapter 6](#), as giving the same prompt to a different model. It states that **when a different model gives the same answer, that agreement is a strong, reliable signal**.

The main agent also sometimes stops a working subagent and resumes it. When the main agent stops and resumes a subagent many times, the subagent can silently drop part of the instructions, or context, that it first received.

So when a subagent that was stopped and resumed many times reports "done", the main agent does not trust that report. Instead, the main agent rewrites all the work to be done, the original instructions included, into one prompt and gives it to a new subagent.

"Writing the reply": how to write replies

"Writing the reply" says that **the agent follows the reply rules while writing, not after it finishes.**

Write the reply clean as you draft it. A cleanup pass after drafting does not remove these patterns.

"Writing the reply" says that a review pass after drafting leaves behind writing that breaks the rules in the following table.

The "patterns" in the quote are these seven rules:

Rule	What it says
Short declarative sentences	Put one point in each sentence and end it with a period
No em dashes (—)	Write lists and headings as sentences too, not joined with dashes. For example, write "main.js handles saving." instead of main.js — saving
No colons in the middle of a sentence	The same as unslop rule 14 (Chapter 32). A colon before a list is fine
Do not use brevity as an excuse to cut content	Keep sentences short, but do not drop the items a Playbook asks the reply to include, such as the detailed history, the trade-offs, the chosen option, and what is still undecided

Rule	What it says
Put the effect on users and on the next owner of the code first	Before implementation details, write who the work is for, such as the app's users or the colleagues who use the library, and what changes for them. Then write what the next engineer who owns this code takes over
Do not fabricate links or quotes	Do not fabricate references to transcripts either. A transcript is a record of a conversation with the agent. Link only to things the agent created or read in this session. Write PR links in the form <code>https://github.com/<owner>/<repo>/pull/<number></code>
Attach evidence or a label to each claim	Use one of three labels: measured, inferred, or guess. Label predictions and unverified causes as guess. Do not hand a person a check the agent can run itself

Every Playbook ends with a reply written under these rules.

For example, the [README](#) has a sample request about scroll drift:

```
/poteto-mode this pr has a subtle bug where the scroll drifts every 750ms even when idle. repro first, then fix and verify.
```

Here is the agent's reply after it fixes the bug from the scroll drift request, once in a form that breaks the rules and once in a form that follows them:

```
Before: Fixed: the cause was a timer so I stopped it, probably fine now I think.
```

```
After: The scroll no longer drifts while idle (measured).
```

```
The cause was a timer that rewrote the position every 750ms (measured).
```

```
The next owner of this code needs to check, when adding a timer, that it does not rewrite the position while idle (inferred).
```

The first reply has a colon in the middle of a sentence and three points in one sentence, and it does not say whether "Fixed" was measured or guessed. The second reply first says what changed for the app's users, then says what the next owner of the code takes over, and labels each claim.

A reply written under these rules lets the user read, sentence by sentence, what changed and how the agent verified it.

"Comments": how to write comments

"Comments" applies the reply rules from the ["Writing the reply"](#) section to code comments as well. As with replies, **the agent follows the rules while it writes comments, not by fixing them afterward.**

"Comments" also says that **a comment may record only a reason the code cannot show.** A comment that records a reason such as "to work around a bug in the library" fits this rule.

The rule also covers verification and test scripts.

Scripts get no comments that narrate the stages of the process, such as `// Phase 1: add cards`. The script shows what it checks through assertions and through the strings it logs. An assertion is a statement that checks a condition and fails the script if the condition does not hold.

The following example is a script that adds two cards, restarts the app, and checks that the cards are still there.

```
// Before: comments that narrate the stages show the steps
async function checkBefore() {
  // Stage 1: add cards
  await addCards(2);
  // Stage 2: restart the app
  await restartApp();
  assert((await countCards()) === 2, "failed");
}

// After: the assertion string shows what was checked
async function checkAfter() {
  await addCards(2);
  await restartApp();
  assert((await countCards()) === 2, "cards are still saved after a restart");
}
```

In the first version, a failed assertion prints only "failed". You have to read the comments to know what the assertion was checking.

In the second version, a failure tells you from the output string that the check "cards are still saved after a restart" did not pass.

When the description of a step goes into the assertion string instead of a comment, the description also appears in the output.

The "Comments" rules apply to every file the agent creates, not only to test scripts. They also cover code changes the agent has a subagent write.

`/no-comments` ([Chapter 31](#)), which runs before review, verifies whether each comment is needed.

"Playbooks": the index of Playbooks

"Playbooks" is the index that the agent uses to pick the Playbook that fits a request. **Each entry gives the Playbook's name, the work it is for, and the location of its file.** The agent picks the Playbook that fits the request from the index, opens its file, and copies the steps word for word into the TODO list.

- **TODO list.** The agent's plan, a list of things to do that it creates at the start of the work. In Cursor, it appears in the chat view. From this list, the user can check which steps the agent worked through and which it skipped with `skip: <reason>`. The bundled guide `01-setup.md` describes this list.

Before the index come the instructions for using Playbooks and the routing rules.

[Chapter 7: `/poteto-mode`, the entry point that routes each request](#) covered these instructions and rules. They are the core rules of `/poteto-mode`, so I show them here as well.

How to use a Playbook

The agent uses a Playbook in this order:

1. Pick the Playbook that fits the request from the index and open its file.

2. Create a TODO list and copy the Playbook's steps word for word into the first item. If the request needs work the Playbook's steps do not cover, add that work after the copied steps. For example, for a request to add a `--json` flag, add "verify that the text output has not changed by a single byte".
3. Keep the steps you decide not to do. Do not delete them. Add one line to each in the form `skip: <reason>`.

The routing rules

Usually the router picks the one Playbook that fits the type of request. When the work is large, though, the size of the work decides the destination, not the type of request.

Nature of the request	Destination
Work of normal size that fits one Playbook	That Playbook
Large work or work that spans several places, such as a migration across many call sites	<code>/figure-it-out</code>
Work where the user steps away and wants to check the result later	<code>/figure-it-out</code>
Work that no Playbook fits	<code>/figure-it-out</code>
Work that lasts many days, where one coordinator manages many PRs and many subagents	"Orchestrate"
Work the user wants carried through to the end without stopping, which one agent can finish in one session, such as a request to "keep going until it's done"	"Autonomous run"

The work in the second and third rows goes to `/figure-it-out` even when it fits a Playbook such as "Feature".

`/figure-it-out` is a Skill that designs a Playbook for that one piece of work alone. Instead of tracing fixed steps, it does three things for each job:

- It decides how strictly to verify. The larger the work and the bigger the impact of a failure, the stricter it gets.

- It splits the work into small units that can land one at a time and starts with the most uncertain unit.
- It records what it decided at each stage with `/show-me-your-work`.

The source does not say why the second row sends large work there. In this book, I infer that large work needs a level of rigor and a way of splitting that match its size.

The third row sends work the user wants to check later to `/figure-it-out` because the user checks all the results together after the work ends. The bundled guide [07-overnight.md](#) gives this reason. So the user must be able to trace afterward what the agent decided and why. `/figure-it-out` designs the approach before it writes code and keeps a record of its decisions, so it meets that need.

The difference from "Orchestrate" is that `/figure-it-out` designs the approach for one job, while "Orchestrate" runs the whole of a job that lasts many days.

The 23 entries in the index

This book sorts the 23 entries in the index by type of work, as follows:

Group	Playbook	Chapter in this book
Investigate	"Investigation", "Runtime forensics", "Trace forensics"	Chapter 10
Fix and improve	"Bug fix", "Perf issue", "Hillclimb"	Chapter 11
Build	"Feature", "Refactoring", "Prototype", "Visual parity"	Chapter 12
Work on Skills	"Authoring or modifying a skill", "Eval"	Chapter 13
Deliver	"Opening a PR", "Babysit", "Shipping"	Chapter 14
Run for a long time	"Autonomous run", "Session pickup", "Pause safely", "Worktree and simulator cleanup"	Chapter 15

Group	Playbook	Chapter in this book
Run many PRs	"Multi-phase or multi-PR plan", "Orchestrate", "Autopilot-full", "Autopilot-stack"	Chapter 16

Fit it to your own style

`/poteto-mode` is poteto's style, so it may not fit the user as it is.

If it does not fit, you need to either create your own mode with `/automate-me` (Chapter 25), or fork pstack. Your own mode is a Skill that captures your style, as `/poteto-mode` captures poteto's.

`/automate-me` finds the user's recurring habits in recent transcripts. The habits cover areas such as replies, delegation, verification, code, and writing. `/automate-me` then drafts a `<your-name>-mode` Skill, such as `tomato-mode`.

With `/automate-me`, the user keeps pstack as the base and gets a personal Skill that routes requests alongside `/poteto-mode`.

If you want to rewrite the Playbooks or Principles themselves, this book recommends that you fork pstack and keep your own version.

The README opens with "fork it. improve it. make it yours." and welcomes forks and improvements.

Summary

- **The role of `/poteto-mode`.** It is pstack's entry point, the Router, which routes requests to Playbooks. Its `SKILL.md` also holds rules for all the work after routing.
- **"Non-negotiables".** A list of what the agent always does in each situation. For example, before asking the user, the agent classifies whether the answer is a fact that running something would reveal or a judgment that a person makes.
- **"Principles" and "Playbooks".** The indexes of the 23 principles and the 23 Playbooks. The agent picks the matching entries from an index, opens their files, and reads them.

- **"Autonomy" and "Subagents"**. The agent always stops before an irreversible operation. It follows the defaults in every delegation. The defaults are the settings that the main agent uses every time it launches a subagent.
- **"Writing the reply" and "Comments"**. The agent follows the rules for replies and comments while it writes, not after it finishes.
- **How to change it to your own style**. Create your own mode with `/automate-me` or fork pstack.

This ends [Part V](#). The book has now explained every Playbook, Principle, and Skill in pstack. [Part VI](#) covers the order in which to start using these parts in your own project. Its first chapter, [Chapter 36](#), covers verification skills, which let the agent run your app and verify its changes.

Part VI: A practical guide to pstack

kaito • Sources: 日本語 • English • 2026-10-03

Part VI is a practical guide to pstack based on [Part 1](#) and [Part 2](#) of The Complete Guide to pstack, the guide that poteto, the author of pstack, posted on X.

The guide covers "**when you use pstack in your own project, where to start and in what order to proceed.**"

Part VI explains the content of the guide in the following order.

1. **Build a verification skill** (Part 1, [Chapter 36](#))
2. **Make the agent understand what the problem is, and have it take over past work** (Part 2, [Chapter 37](#))
3. **Design from how users use the product, and try it with prototypes** (Part 2, [Chapter 38](#))
4. **Settle the design and turn it into a plan you can carry out** (Part 2, [Chapter 39](#))

Part 1 names the **verification skill** as the first thing the user builds, and covers how to build and use it.

Part 2 covers how the user and the agent decide together what to build, once the verification skill exists. This book organizes the content of Part 2 into 13 key points and explains them across three chapters, Chapter 37 through Chapter 39.

Verification skill: a project-specific Skill the agent uses to start and operate the user's app and verify that a change works. For example, if you changed the settings screen, the agent clicks the gear button, opens the settings screen, and saves a screenshot of the changed part as evidence.

The verification skill comes first for one reason. **As long as the agent cannot verify its own changes, a person has to check both whether a prototype is any good and whether a plan is done.**

In that case, the agent's work cannot move forward until a person finishes checking. The person becomes the bottleneck that stops the work, and spends the day watching the agent.

What you learn in Part VI

After reading Part VI, you will know the following.

- How to build a verification skill for your own app, and how to keep it up to date as the app changes
- How to make the agent understand what the problem is the way a person does, and how to have it take over past context
- How to start design from how users use the product, and how to pick one of several prototype options
- How to revise the design based on what you learned during implementation, and how to turn it into a plan split into small PRs you can verify

Chapters in this part

Each chapter covers the following.

Chapter	What it covers	Corresponding guide
Chapter 36: Guide 1: start with a verification skill for your own app	Build a verification skill so the agent can verify its own changes	Part 1
Chapter 37: Guide 2: share the goal, then align on problem and history	Understand the problem	Part 2 (key points 1 to 4)
Chapter 38: Guide 2: test a design from how the user will use it	Try solutions	Part 2 (key points 5 to 8)
Chapter 39: Guide 2: revise the design into an executable plan	Settle the solution and turn it into a plan you can carry out	Part 2 (key points 9 to 13)

Each chapter builds on the one before it, so I recommend reading in order from Chapter 36.

Chapter 36: Guide 1: start with a verification skill for your own app

kaito • Sources: 日本語 • English • 2026-10-03

In [Part 1](#) of The Complete Guide to pstack, poteto names **a verification skill as the first thing** a new pstack user should build. This chapter calls Part 1 "the guide".

A **verification skill** is a project-specific Skill that lets the agent start and operate the app the user is developing, verify that a change works, and leave evidence ([Chapter 26](#)).

For example, if you changed a settings screen, the agent itself presses the gear button, opens the settings screen, and saves a screenshot of the changed part as evidence.

This chapter explains how the guide recommends building and using a verification skill. The chapter also checks each point against the pstack source, such as the `SKILL.md` of `/create-verification-skill`.

[Chapter 26](#) explained the steps written in the `SKILL.md` files of `/create-verification-skill` and `/maintain-verification-skill`.

This chapter leaves out the following topics and focuses on what the guide recommends.

- **Research before building.** Before it asks the user anything, the agent reads the repository. It looks for what end users touch, how to start the app, how to operate it, what evidence it can save, and whether two copies of the app can run side by side.
- **What goes in `SKILL.md`.** The agent fills six headings with the facts it found: startup, diagnosis, operation, evidence, cleanup, and helper scripts. For evidence, the agent operates the real path an end user takes and checks side effects, such as written files, in addition to the screen.

- **A run before handoff.** The agent runs the generated verification skill once from startup to cleanup and confirms that it works before handing it to the user. The skill treats anything that has never run as a draft.
- **Checking every entry point.** If the **Feature Map** lists several entry points for one feature, such as a button, a key press, and a command, the agent does not finish verifying that feature until it has checked all of them.
- **Maintenance rules.** An agent that runs `/maintain-verification-skill` reports one of three results. Clean means that nothing needed a fix, changed means that the fixes are in one PR, and blocked means that the agent could not finish the check. During maintenance, the agent does not edit product code.

How this chapter is organized

This chapter covers the following topics.

- Why build a verification skill first
- What a verification skill is made of
- How to build a verification skill and keep it current
- How to use a verification skill in requests and how to expand its use
- What to do as a first step
- Summary

Why build a verification skill first

There are two reasons to build a verification skill first.

First, a verification skill is **important infrastructure that multiplies the output of the whole team**.

Second, many steps in pstack's Playbooks cannot run without **a way to actually operate the app**.

The guide treats the verification skill as important infrastructure

The guide's section "Verification is all you need" opens with this sentence.

The most critical skill to have in your toolbox is a high quality verification skill.

In the guide, verification means **the agent can check its own work**.

In other words, **verification does not mean "I wrote the code" or "the build passed". It means operating the real app the way a real user would and saving the result as evidence**.

An agent that can check its own work can repeat a cycle of failing, fixing, and retrying until the task succeeds. Because no human has to check each step, humans stop being the bottleneck that halts the work.

The guide says that a high-quality verification skill, and the work to keep it up, matter so much that poteto thinks of the skill as important infrastructure, not "just a Skill". This book treats the skill the same way.

The guide goes on to say that **a high-quality verification skill multiplies the output of the whole team, including members who are not engineers**.

If the agent can verify by itself that a change works, everyone in the organization, including non-engineers, can make changes to the app and verify that those changes work. The guide says that a well-built verification skill multiplies the output of the whole team by 100 to 1000 times. For that reason, I consider one verification skill per app a requirement.

Playbooks assume a way to operate the app

Many Playbooks ask the agent to **run the screens, CLIs, APIs, and other parts that end users touch**.

For this, the agent uses control Skills. A control Skill is a Skill that operates an app, such as control-ui and control-cli in the cursor-team-kit plugin ([Chapter 8](#)). The verification skill you build with `/create-verification-skill` is also a control Skill.

The following table lists the app operations that the main Playbooks ask the agent to perform.

Playbook	App operation the Playbook asks the agent to perform
" Bug fix ", which fixes a bug	Reproduce the bug itself
" Perf issue ", which fixes a performance problem	Take a baseline trace before the fix
" Runtime forensics ", which diagnoses a running process	Take real profiles and traces from the running process instead of guessing
" Prototype ", which decides a design with throwaway prototypes	When the choice is about visuals, take a screenshot of each option, operate them, and compare
" Visual parity ", which matches the visuals of two implementations, or of the before and after of a migration, pixel for pixel	For each component, take an image diff against a baseline screenshot from before the migration. Any nonzero diff is a failure
" Shipping ", which verifies and merges a PR	For each PR, a separate agent that did not write the code runs the screen or CLI before and after the change and gives one of three verdicts: PASS, PASS+NOTES, or FAIL

- **Trace.** A record of events over time, such as how long each operation took.
- **Profile.** A record that totals CPU time, memory use, and similar measures per function or other unit.

Without a way to operate the app, such as a verification skill or a control Skill, the agent cannot carry out the steps in the table.

At the start of a task, the agent makes a TODO list that copies the Playbook's steps. For a step it skips, it keeps the line and writes skip: <reason> on it ([Chapter 7](#)).

TODO list. The plan that the agent makes at the start of a task, written as a list of things to do. Cursor shows it in the chat view. From this list, the user can see which steps the agent worked through and which it skipped (skip: <reason>). The bundled guide [01-setup.md](#) describes this list.

Take "**Bug fix**" as an example. If there is no way to operate the app, or the available way cannot reach the part to reproduce, this book sees two options left for the agent.

- Leave skip: `<reason>` in the TODO list and skip that step
- Ask the user to reproduce the bug, which "**Bug fix**" allows as an exception

The agent may ask the user to reproduce the bug only when both of the following are true.

- The agent has explained concretely why the available way to operate the app cannot reach the part to reproduce (for example, only accounts on a specific plan can log in to that screen)
- The agent has operated the app itself as far as the available way allows

Whether the agent skips the step or asks the user to reproduce, some part remains that the agent cannot verify by itself. A human has to verify that part.

What a verification skill is made of

A verification skill centers on two things: **a small CLI that operates the app, and a Feature Map that records how to reach each feature of the app**. For each feature, the Feature Map records where the feature is on the screen or in the commands, and what an end user does to get there.

In pstack version 0.15.5, the version this book covers, the agent follows the steps of `create-verification-skill` and creates both inside `.cursor/skills/verify-<app>/`.

For Atlas, the fictional app the guide uses as its example, the result looks like this.

```
.cursor/skills/verify-atlas/
├─ SKILL.md (steps from startup to cleanup)
├─ control-atlas.mjs (the CLI that operates the app)
├─ features/ (Feature Map)
│   ├── README.md (Feature Map index. Links to each feature's file)
│   ├── <feature-a>.md
│   └── <feature-b>.md
```

Chapter 26 covered what goes in `SKILL.md`. This section looks at the two parts the guide explains in detail: the **CLI** and the **Feature Map**.

CLI

The CLI of a verification skill turns the steps to operate and debug the app into **commands the agent can rerun**.

An application of the principle "Build the Lever"

The guide presents a **CLI for app operations** as an application of the principle "Build the Lever" (Chapter 17). That principle says to build a tool that does a task instead of doing the task by hand.

When you apply this principle to a new Skill, the Skill author gives the agent a tool it can run, in addition to steps written in Markdown.

What this means in the context of creating a skill, is that we prefer to give agents tools rather than just markdown.

For example, if the verification skill's `SKILL.md` says "click the settings button and take a screenshot" only as Markdown text, the agent has to write its own script to do the click every time.

If the verification skill comes with a CLI that has commands like `click` and `screenshot`, the agent needs only to run those commands. In a verification skill, this small CLI for operating and debugging the app is the tool.

The guide lists these benefits of a CLI for app operations.

- **Fewer tokens.** The agent runs one CLI command instead of a throwaway script that clicks one button.
- **Easier to reproduce and test.** The verification skill gives the same result every time it runs. Because the commands are fixed, you can also test whether the verification skill itself works.

Example: commands for the fictional app Atlas

The following is part of the guide's example commands for the fictional app Atlas. In the example, Atlas is an app built with Electron.

The agent runs the commands from top to bottom to do three things.

1. Check that the app is ready to operate (doctor)
2. Open a new blank conversation (thread) and send "list open tasks in this project" (new-session, send)
3. Save the resulting screen as evidence to /tmp/atlas-proof.png (screenshot)

```
# health
node .cursor/skills/verify-atlas/control-atlas.mjs doctor

# open a blank thread and send
node .cursor/skills/verify-atlas/control-atlas.mjs new-session
node .cursor/skills/verify-atlas/control-atlas.mjs send "list open tasks in this project"

# screenshot for evidence
node .cursor/skills/verify-atlas/control-atlas.mjs screenshot /tmp/atlas-proof.png
```

Six categories of commands

To help the user decide which commands to provide, the guide gives six categories.

- **Inspection.** Commands that look at the state of the screen, such as info, snapshot, and screenshot.
- **Navigation.** Commands that move between screens, such as home, new-session, and scroll.
- **Interaction.** Commands that type and click, such as send, click, type, and press.
- **Performance.** Commands that measure performance, such as trace, profile, and performance.
- **Streaming.** Commands that capture console and network logs, such as console and network-log.
- **Health & cleanup.** Commands that check that the app is ready to operate, or that clean up after a run, such as doctor and cleanup.

A CLI that agents can use well

The guide recommends that you first spend time to make the CLI run without errors, before you move on to more advanced uses. **Once the CLI exists, every agent can operate and debug the app with it.**

The agent uses the CLI, not a human, so the CLI has to suit agents.

The guide lists six properties of a CLI that agents can use well.

- Make commands easy to combine (the guide cites deep modules, a simple interface in front of a lot of functionality, from John Ousterhout's book [A Philosophy of Software Design](#))
- Give commands that may have destructive side effects a `--dry-run` option
- Reveal features step by step through subcommands instead of all at once. For example, `--help` lists only the subcommands, and each subcommand's own `--help` shows its detailed usage
- Make error messages say what to do instead. For example, do not stop at "The app is not running". Add the fix: "Run the start command first."
- Write thorough `--help` text
- Print output in a machine-readable form such as JSON

The guide recommends that the user design a CLI like this, or ask the agent to design it.

Feature Map

The **Feature Map** is a set of Markdown files that records, for each feature of the app, what the feature does, what an end user does to reach it, how to operate it with the verification skill, and what result shows that it works ([Chapter 26](#)).

The more complex the app, the more the agent needs directions for finding and operating features. poteto created the Feature Map to give those directions.

`features/README.md` is the Feature Map's index and links to each feature's file.

Before it operates the app, the agent reads the index and opens only the files for the features related to its task.

Because it reads only the files it needs, the agent saves tokens in its context window.

The guide says that `/create-verification-skill` creates a `references/features` directory. The `SKILL.md` in pstack version 0.15.5 says to create it in `.cursor/skills/verify-<app>/features/`, so the location differs. This book follows the location in pstack version 0.15.5. For a sample of the format, see the bundled [references/feature-map-example/](#).

The agent follows the steps of `/create-verification-skill` and writes each feature's file under these four headings.

Heading	Question it answers
Sub-features	What can an end user do within this feature (The file lists each action with a short ID.)
How to get to it (user POV)	What does an end user do to reach this feature
Driving it with <harness>	How to operate it with the harness. The harness is whatever drives the app from outside, such as the verification skill's CLI
Gotchas	What pitfalls waste the verification or make it invalid

The following sections go through each row of the table with `create-note.md`, the sample bundled with `/create-verification-skill`. The sample describes the note-creation feature of a notes app called Notes.

Sub-features

The sample divides what an end user can do in the note-creation feature into four actions.

- Open the editor
- Save
- Discard a draft
- Create from the CLI

```
## Sub-features
```

```
- `create-open` opens a blank editor from each browser entry point.
```

- `create-save` persists a title and body.
- `create-cancel` discards an unfinished browser draft.
- `create-cli` creates the same note shape from the terminal.

How to get to it (user POV)

An entry point is a way an end user reaches the feature, such as a button, a key press, or a command. The sample lists three entry points: the `New note` button in the toolbar, the `n` key, and the `notes create` command.

How to get to it (user POV)

- Choose the `New note` button in the browser toolbar.
- Press `n` in the browser while focus is outside an editable field.
- Run `notes create --title <title> --body <body>` in a terminal.

Driving it with <harness>

The sample's `README.md`, which sets the rules for writing each feature's file, says to write this heading as follows.

Driving it with <harness> starts with `Preconditions:` and uses labeled bullets that pair each user action with an exact command and observable result.

Preconditions (`Preconditions:`) are the state that must hold before the steps begin. The sample lists, for example, "Notes is running normally at `http://127.0.0.1:4173`" and "No note titled `Release checkList` exists yet."

After the preconditions come the steps, one step per item. The rule gives each item these three parts.

- The end user's action
- The command that reproduces the action with the harness
- The observable result that shows the action succeeded

```
- **Open editor.**
  Choose `New note`.
  Run `control-notes browser click --role button --name "New note"`.
  A form named `Note editor` appears with focus in the `Title` textbox.
```

Broken down, the item looks like this.

Part	Text in the sample	Meaning
The end user's action	Choose New note.	Choose the New note button
The command that reproduces it with the harness	control-notes browser click --role button --name "New note"	Click the element whose role is button and whose name is New note
The observable result	A form named Note editor appears with focus in the Title textbox.	A form named Note editor opens, and the cursor is in the Title field

After it runs the command, the agent checks whether the "observable result" in the table appeared. If it did not, the agent knows that this step failed.

Gotchas

The sample lists these pitfalls.

- Pressing `n` while a textbox has focus types the character instead of opening a new editor.
- A save status alone is insufficient proof. Reopen the note from the list.

The first pitfall is that when a text field has focus, the n key types the letter n instead of opening the editor.

The second is a warning. A "Saved" message does not prove that the app saved the value, so the message alone is not evidence. The agent must reopen the note from the list to check.

The guide's own Feature Map example, the settings feature, also has this warning under Gotchas. Here, a tab is one of the items on the left side of the settings screen that switch the view, such as General, Appearance, Models, and Plan & Usage.

Some tabs are entitlement-gated. Skip with an explicit account reason.

With this line in the Feature Map, when the agent cannot find, for example, the Plan & Usage tab, it does not mistake the missing tab for a bug. It skips checking that tab and writes the reason: "not shown because of the account's plan."

The Feature Map is memory the team shares

The guide calls the Feature Map "**materialized memory**" ([Chapter 4](#)).

An agent's memory is knowledge from earlier work that the agent keeps so it can use it in the next task.

The ultimate form of this memory is the codebase. The code reflects the decisions the team has made, and it is the source of truth for what happened and how things work.

The Feature Map is a condensed version of the codebase-as-memory, made small to save tokens in the agent's context window.

The Feature Map is Markdown inside a verification skill in the repository's `.cursor/skills/`, so everyone who works on the codebase shares this memory.

How to build a verification skill and keep it current

As [Chapter 26](#) showed, the agent builds the verification skill by following `/create-verification-skill`. This section looks at what the guide recommends to the user: the preparation before building and how often to maintain the skill afterward.

What the user prepares before building

Choose the tech stack

The guide points out that the choice of tech stack also matters when you set up a way to operate the app from outside. **The harder a tech stack is to debug and operate, the harder it is to use agents productively.**

For example, the agent can operate an Electron or web app through CDP, and an iOS app through the simulator. CDP is the Chrome DevTools Protocol, which lets an outside program use the same features as the browser's developer tools.

If the chosen tech stack has no debugging tools the agent can use, you have two options. You can have the agent build tools, for example with lldb or with your own package that runs alongside the app in the development environment. Or you can work with the tools you have.

poteto, however, considers agent verification so important that he suggests building your own debugging tools or even choosing a different tech stack.

I personally feel that agentic verification is so important that I would unironically suggest building your own rich debugging tools, or even choosing a different tech stack

I also find verification very important and worthwhile, so if my chosen tech stack had no verification tools, I would take poteto's suggestion.

Prepare the developer experience

As preparation for running the app, the guide recommends that the user think through three parts of the developer experience.

- How to load seed data into the development database. Seed data is the data you load in advance for development and testing
- How to set up authentication and test users, and how to call APIs in test and staging environments
- How to install and start the development environment with the same steps every time

The guide treats the CLI and these preparations as the agent's main tools for developing the app, so you need to maintain and test them.

When the preparation is done, the user runs `/create-verification-skill`. As a reference, the guide points to [poteto/verification-skill-example](#), a public sample verification skill built for a fictional app.

Keep it current with `/maintain-verification-skill`

[Chapter 26](#) covered the three maintenance results (clean, changed, and blocked) and the rule against editing product code during maintenance. This section looks at how often the guide recommends running it and how to automate that.

How often to run it

The `SKILL.md` of `/maintain-verification-skill` does not say how often to run the Skill. An agent that runs `/create-verification-skill` also suggests a frequency only when the user asks. So in this book, I treat it as **the user's decision, based on how often the app changes**.

The guide recommends running it **at least once a day**.

The agent sometimes updates the verification skill while it works on the app, but it often misses some places.

If the user runs `/maintain-verification-skill` once a day, the run finds and fixes the places the agent missed. As a result, the agent always knows how to operate the current app.

Automate the daily run with Dr Eggbot

As a way to automate the daily run, the guide also introduces Dr Eggbot.

Dr Eggbot is a bot that poteto (Lauren Tan) made. It designs and builds Grok Bots, which are AI bot apps ([Chapter 5](#)). Its description in xAI's [bot marketplace](#) reads as follows.

Asks a few preference questions, then creates them with CreateAgent. Coding bots get the poteto-mode bar (one job, unslopped, verified).

Dr Eggbot ships with pstack.

So **Dr Eggbot can teach coding bots how to use pstack. It also builds non-coding bots to the same standard.**

The recommended way to use Dr Eggbot is as follows.

1. Have Dr Eggbot build a bot that plays the engineer

2. Have that bot run `/create-verification-skill`
3. Have the same bot set up a routine that runs `/maintain-verification-skill` once a day

How to use a verification skill in requests and how to expand its use

In a request to the agent, the user writes which verification skill to verify with and what counts as evidence.

Add the verification skill to Grok Bot routines or Cursor Automations only after you are satisfied with the skill (see "[Use 3: reproduce user reports automatically](#)" in this chapter). Cursor Automations is a Cursor feature that runs work at set times or on events.

Ways to use a verification skill

The guide gives three examples of using a verification skill. In the request examples below, `/control-app` is the guide's name for the verification skill built with `/create-verification-skill`.

Use 1: build a new feature

When he builds a new feature, poteto uses `/poteto-mode` together with the verification skill and has the agent verify its own changes. The guide's request example is as follows.

```
/poteto-mode build <description of feature, any useful context>. use /control-app to verify your changes and show me a video and screenshots as proof
```

This request adds two things to the feature description: which verification skill to verify with (`/control-app`) and the form of the evidence (a video and screenshots).

When he sends the request from a Grok Bot, poteto does not have the bot do the work itself. He asks the bot to start an agent in Cloud Agents, a Cursor feature that runs agents in Cursor's cloud. The following request is an example.

```
spawn a cloud agent to use /poteto-mode to build <description of feature, any useful context>. use /
control-app to verify your changes and show me a video and screenshots as proof
```

poteto keeps the bot out of the work for two reasons. When a cloud agent does the work, the bot can keep working on other things. Also, the bot's context window does not fill up with the work log.

Use 2: improve performance

When he improves performance, poteto **has the agent first measure the current performance with the verification skill, and after the fix, verify the improvement with the same verification skill.** The guide's request example is as follows.

```
spawn a cloud agent to use /poteto-mode to improve the initial loading time of our app. first use /
control-app to take a trace of the status quo, and identify opportunities for improvement. then do a
targeted fix and use /control-app + a /swarm to confirm the win
```

This request says to take a trace of the current state with /control-app before fixing anything. The "**Perf issue**" Playbook also takes a baseline trace as its first step and asks for the before and after numbers and their difference in the report.

The guide calls /swarm ([Chapter 24](#)) **one of the Skills that pair best with a verification skill.** /swarm runs many Workers, which are agents that run in parallel, and merges their results into one report.

If /swarm has many Cloud Agents run the verification skill, you can verify a performance improvement with a large enough sample. You can also use fuzzing to verify that the app does not break. Fuzzing feeds large amounts of random or unexpected input to the app to look for failures.

Use 3: reproduce user reports automatically

Once you are satisfied with the verification skill, you can add it to Grok Bot routines or Cursor Automations. Both run work at set times or on events.

In the guide's example, a bot takes user feedback that arrives in Slack or elsewhere, one item at a time. For each item, the bot starts a cloud agent and tries to reproduce the reported bug automatically.

In this way, **if the verification skill and the Feature Map are good enough, you can automate even the bug fix.**

Run agents in parallel

Once you have merged a few PRs with the help of the verification skill, the next stage is to run more agents in parallel.

If an agent can take a request and carry the work almost to a mergeable state, the user can run other agents in the meantime.

Use Cloud Agents instead of worktrees

To run agents in parallel, the guide recommends Cloud Agents over worktrees. A worktree is a git feature that adds an extra working directory to one repository ([Chapter 18](#)). This book also recommends Cloud Agents.

The guide does not recommend worktrees because each agent adds one more working directory of the repository. Each extra directory uses a lot of the local machine's storage and resources.

Depending on the size of the repository and the machine's specs, I expect worktrees to support about 10 parallel agents at most.

In my own experience, when I ran agents in parallel locally, each worktree needed a separate directory, and I had to keep the app's ports from colliding. I also had to manage the worktrees I created and remember to delete them.

An agent in Cloud Agents, on the other hand, gets a real computer in the cloud. It can install dependencies, run the app, take videos and screenshots, and operate the app the same way a real user does.

Cursor takes a snapshot of the environment after the first build, so later Cloud Agents start right away. A snapshot is a saved copy of the whole state at one point in time. Each

agent also runs on its own machine, so the app instances that the agents run are separate from the start.

<https://x.com/poteto/status/2087977008253071816>

poteto also gives his reasons for using Cloud Agents in a post on X.

worktrees are dead. cloud agents are the future. they've allowed me to massively orchestrate swarms of agents, all with their own computer. it's how i've been able to trust that my agents are actually doing what i want them to, because they can run their code, take videos and screenshots, and interact with UIs the same way users do.

Cloud Agents also need some care, though.

An agent in Cloud Agents runs commands automatically without waiting for the user's approval. If it runs a wrong command, only the cloud machine breaks. Your own PC does not.

On the other hand, the repository's code goes to the cloud machine, so the risk that the code leaks remains.

Secrets such as API keys carry the same risk.

[Cursor's documentation](#) also warns about leaks through prompt injection, an attack that hides malicious instructions in text the agent reads. Because the agent runs commands automatically, an attacker can trick it into uploading code to a malicious website.

If your development environment is already in order, the cloud environment takes little work to set up.

In Cursor, you can define how to build the development environment as code and keep it in the repository. Cursor calls this EAC (Environment as Code), and the file is `.cursor/environment.json`.

If you write the startup steps of the development environment you prepared for the verification skill in this file, every agent in Cloud Agents can start work in the same environment. [Cursor's blog post on the cloud agent environment](#) treats the development environment as a product whose users are agents.

Because agents in Cloud Agents run on cloud machines, the work continues even if the user closes their PC. I am a heavy Cloud Agents user myself.

<https://zenn.dev/sc30gsw/articles/953334f11df507>

What to do as a first step

The first step is for the user to check whether **the agent has a way to operate and observe the app from outside**.

This book follows the flow of the guide and arranges the steps the user takes, including `/create-verification-skill` and `/maintain-verification-skill`, as follows.

1. Check for a way to operate and observe the app from outside (an existing harness, CDP, a simulator, or a tool you build)
2. Make the development environment start with consistent steps (seed data, test users, test environments, and isolation)
3. Run `/create-verification-skill` and have it generate the CLI and the Feature Map
4. Run the generated artifacts once from start to end and confirm that evidence was saved. Fix the CLI until it runs without errors
5. Decide how often to run `/maintain-verification-skill`
6. In everyday requests, write the verification skill's name and the form of the evidence
7. Once you have merged a few PRs with the verification skill and are satisfied with it, expand to parallel work in Cloud Agents and to routines and automation

At the end, the guide even suggests three things. Keep improving the CLI, invest in the verification skill as you would in important infrastructure, and set up an oncall rotation, a schedule of people who take turns to handle breakages in the verification skill.

I suggest putting a Grok Bot in that oncall role.

What I have in mind is a Grok Bot that keeps auditing and hands the actual work to Cloud Agents.

Summary

- **Why build it first.** In app development, the guide treats the verification skill as the most important skill, and many pstack Playbooks assume a way to operate the app.
- **What it is made of.** A verification skill centers on a CLI that can be rerun and a Feature Map that records how to reach each feature.
- **How to build it and keep it current.** Before building, the user prepares the tech stack and the developer experience. Afterward, the user runs `/maintain-verification-skill` at least once a day, as the guide recommends, and can automate the run with a routine from Dr Eggbot.
- **How to use it and expand it.** The user writes the verification skill's name and the form of the evidence in requests, and expands to parallel work and automation once satisfied.
- **The first step.** The user starts by checking that the agent has a way to operate and observe the app from outside.

Starting with [Chapter 37](#), the book uses The Complete Guide to pstack [Part 2](#) to explain how to decide with the agent what to build: which problem to solve, with what design, and with what plan. [Chapter 37](#) covers the first 4 of the 13 key points into which this book condenses Part 2.

Chapter 37: Guide 2: share the goal, then align on problem and history

kaito • Sources: 日本語 • English • 2026-10-03

Chapter 36 covered why you build the verification skill first, and how to build it, keep it current, and use it.

Verification skill: a project-specific Skill that the agent uses to start and drive the app and collect evidence.

With a verification skill, the agent can verify its own changes.

This chapter and the next two, up to [Chapter 39](#), explain how to decide together with the agent what to build: which problem to solve, with what design, and on what plan.

All three chapters draw on [Part 2](#) of The Complete Guide to pstack (in these three chapters I call it "the guide").

Part 2 explains how poteto uses pstack for investigation, planning, prototyping, and architecture design, once the agent can verify its own work.

In this book, I sum up the guide in thirteen key points. This chapter explains key points 1 to 4.

When you decide what to build, the first thing the human should do is **make the agent understand the problem the same way the human does, and hand the agent the context of past work**.

The context of past work means what earlier conversations investigated, decided, and failed at.

The agent has to understand the problem the way the human does for one reason. If the two understandings stay apart, **the agent solves a problem the human did not mean, however well it writes the code.**

Chapter 22 explained the steps and output of `/how`, `/why`, `/teach`, and `/recall`, which key points 3 and 4 use.

This chapter leaves out the following and focuses on the uses the guide recommends.

- **How `/how` investigates.** For a complex question, `/how` splits the work between a role that gathers facts and a role that writes the explanation, and explains under five headings, from Overview to Gotchas.
- **How `/why` investigates.** `/why` starts investigator agents in parallel, one for each place evidence lives, such as git, tickets, and chat. It then sorts each reason it finds into one of five levels of confidence, from Direct to Unknown.
- **How `/teach` runs an explanation.** `/teach` decides a few points the reader should take away, answers in one or two sentences, and waits for the reader's response.
- **What `/recall` outputs.** `/recall` sums up the state of the work under four headings: Capsule, Threads, Problems, and Next move.

How this chapter is organized

This chapter covers the following topics.

- The thirteen key points, and how key points 1 to 4 connect
- Key point 1: convey the goal and background, and do not over-specify the solution
- Key point 2: have the agent restate the problem before implementation
- Key point 3: learn the mechanism and the reasons with `/how`, `/why`, and `/teach`
- Key point 4: carry past work into a new conversation with `/recall`
- Summary

The thirteen key points, and how key points 1 to 4 connect

This book prepared the thirteen key points as a summary of the guide.

In all three chapters, I write each key point in two sections.

- **The guide's view.** What the guide recommends.
- **pstack.** Which pstack Skill or Playbook implements that view.

For a key point with no direct match in pstack, I leave out the pstack section.

The thirteen key points are as follows.

No.	Matching sections of Part 2	Summary
1	"The art of supervising someone smarter than you", "In your own words"	The human gives the agent enough of the goal and background, and does not over-specify the concrete solution. An open solution gives the agent room to look for solutions the human did not think of
2	"In your own words"	Before implementation, the human has the agent explain the problem in its own words. The human finds the agent's misunderstandings early, and avoids narrowing the agent's investigation with the human's own hypothesis
3	"Building up a mental model"	The human has the agent investigate the current mechanism with /how and the design background with /why, and turn them into an explanation the human can understand with /teach. The investigation that the agent does for the explanation also helps the agent's own understanding

No.	Matching sections of Part 2	Summary
4	"Learning from history"	The human has the agent look back at related work in past conversations with /recall, and carry earlier investigations, decisions, and failures into the new work. With them, the agent does not devise a solution from only the symptom in front of it
5	"Working backwards"	The human has the agent write a README or tutorial before implementation, and design the API and internals from how users will use them
6	"Working backwards"	The agent uses /technical-writing to split documents by purpose into tutorial, how-to, reference, and explanation, and uses /unstop, which it calls internally, to remove the unnatural phrasing AI tends to write
7	"Measure a hundred times, cut once"	The human does not adopt the agent's first design as is. The human has the agent build prototypes of several options and compares them by screenshots taken during use, measured times, and layouts
8	"Measure a hundred times, cut once", "Architecting bigger changes"	The human has the agent itself verify the questions that a small build-and-run experiment can answer. Prototypes and verification reduce uncertainty better than rounds of review on an abstract plan

No.	Matching sections of Part 2	Summary
9	"Architecting bigger changes"	For a large change, the human spends time to think about how the system holds data and how systems connect. The agent uses <code>/architect</code> to investigate existing constraints and design, and compares and merges independent options from several models before implementing
10	"Architecting bigger changes"	If implementation finds a conflict with the design, the agent revisits the design. Depending on the evidence from implementation (for example, a type needed any or a forced cast), the agent throws away the design and starts over
11	"Okay but I really want a planning doc"	Once the design is concrete, the agent uses a planning Playbook to split the work into tasks or PRs that it can verify in small pieces. For each task, it sets the result of running the code as the evidence of completion
12	"Okay but I really want a planning doc"	The plan document from the planning Playbook is temporary. It exists to carry out the work and, for large work, to share progress with other agents. <code>poteto</code> usually deletes it after completion
13	"The workflow in practice"	<code>/poteto-mode</code> is the entry point that uses Playbooks and Skills according to the request. The human tells <code>/poteto-mode</code> the goal, the constraints, the results to check, and when to review

The "Matching sections of Part 2" column gives the names of the guide's sections that cover each key point. Use them to find your place when you reread the guide.

Key points 1 to 4 are the basic moves of supervision

Key points 1 to 4 serve one job, which the guide describes in two sections.

The guide's section "The art of supervising someone smarter than you" covers the job of a human who supervises someone smarter than themselves. Here, that someone is the latest models.

With an agent, a human can change a system without knowing the code well. But **it is still hard to keep the quality of the code and the user experience high**. The job is harder still for someone who is not an expert and does not know what to check and what to ask.

The next section, "In your own words", describes this supervision as **work done in a codebase the human did not write, in a situation where the human can no longer keep the structure of the whole codebase in their head**.

In this book, I treat key points 1 to 4 as the basic moves of this job.

- **How to convey (key point 1)**. What to say in a request, and what not to over-specify.
- **How to check understanding (key point 2)**. How to check, before implementation, that the agent's understanding has not drifted from the human's intent.
- **How to deepen the human's understanding (key point 3)**. How the human comes to understand how the code works now and why it was written that way.
- **How to hand over past work (key point 4)**. How to hand earlier investigations, decisions, and failures to the agent in a new conversation.

Key point 1: convey the goal and background, and do not over-specify the solution

The human writes enough of the **goal, background, constraints, and known facts** in the request, and **does not go as far as the concrete solution, such as which function in which file to rewrite and how, or the implementation steps**.

The guide's view: hand over the background, leave freedom in how to solve

In the section "The art of supervising someone smarter than you", poteto **names two failures he keeps seeing even as models get more capable**.

- The request's instructions are missing or unclear, and the agent does not fully understand the human's intent
- The agent lacks the context it needs to do the work correctly

These two problems are related, and both come down to one point.

"Whether you can use an agent well depends on how much high-quality context the human can put into the agent's context window"

The context window is the range of information an agent can read in at one time.

According to the section "In your own words", poteto used to give earlier models very specific instructions about what he wanted. He says the instructions came close to micromanagement.

But the latest models write code better than humans do. So, to "supervise someone smarter than you", it matters that you tell the agent what you want to achieve while **leaving the agent free to solve the problem in a way the human did not think of**.

pstack: the Playbook defines the steps

In pstack, the user conveys the goal, constraints, and known facts, and the Playbook defines the steps ([Chapter 7](#)).

The section "Say the goal, not the ceremony" of the bundled guide [docs/guide/02-poteto-mode.md](#) also recommends that the user leave the spec out of the request and write the following.

- What is wrong, or what you want
- Known facts that save the agent investigation time, if you have any

The same page gives the following one-line request as an example (the same example appears in [Chapter 7](#)).

`/poteto-mode` users get two notifications after a retry. repro first, then fix and verify.

This request **states what is happening (two notifications arrive after a retry) and a constraint to follow (reproduce first)**. It **does not state a concrete solution, such as which function in which file to fix and how**.

The request is a bug fix, so `/poteto-mode` picks the "Bug fix" Playbook and copies its steps into the TODO list. The steps include reproduce, narrow down the cause, plan the fix, and confirm.

TODO list: the plan that the agent makes at the start of a task, written as a list of things to do. From this list, the user can check which steps the agent worked through and which it skipped (skip: <reason>). The bundled guide [01-setup.md](#) describes this list. In Cursor, the list appears in the chat view.

In the Pitfall at the end of the page, `02-poteto-mode.md` warns the user not to list Skills in the request in the order to use them, as in `"/how and then /architect"`. The Playbook already sets the order of the work.

According to this Pitfall, **when the agent follows the order the user wrote, it may reorder or drop steps that the Playbook would have kept**.

`02-poteto-mode.md` does not forbid naming a Skill, though. It recommends naming a Skill only when you want to override a particular choice ([Chapter 7](#)).

So, when the Playbook defines the steps, **the user leaves the steps out of the request and writes what is wrong, what they want, the constraints to follow, and the facts they already know**.

Key point 2: have the agent restate the problem before implementation

Before implementation starts, the human has the agent **restate the problem in its own words**, and checks for drift between the agent's understanding and the human's intent.

The guide's view: three effects of an indirect prompt

The section "In your own words" describes an **indirect prompt**. Instead of saying directly what they want, the human has the agent restate the problem in the agent's own words, and so draws out the goal.

When someone reports a problem in Slack, poteto says he often **has the agent read the thread and restate the problem before it does anything else**. Here is an example of such a request.

```
/poteto-mode read this slack thread. restate in your own words and in plain english what you think the underlying issue is
```

poteto names three effects of this request.

- **Sorting out the conversation.** The agent can read a thread full of crossing messages and sum up what the problem is in a short, organized explanation.
- **Catching misunderstandings early.** If the agent has misunderstood the problem, the human can point out and correct the misunderstanding before the agent writes code.
- **Not pushing a hypothesis.** The human does not state their own hypothesis, so that hypothesis does not pull the agent when it frames the problem. The human's hypothesis can be wrong, and even when it is right, it can narrow the direction of the agent's investigation and solution.

An indirect prompt cuts rework and lets you work with the agent on shared assumptions and a shared understanding.

As a side note, poteto posted the following at the end of September 2026 with a concrete indirect prompt. You can copy and paste the prompt as is, and it should give the result you expect.

<https://x.com/poteto/status/2104744961904394699>

Key point 3: learn the mechanism and the reasons with /how, /why, and /teach

The human has the agent investigate how the code works now with /how and why it was written that way with /why, and turn the results into **an explanation the human can understand** with /teach.

The guide's view: understand through three Skills

In the section "Building up a mental model", poteto writes that, to work with someone smarter than you, it matters to have the agent explain things again in a form the human can understand. /teach is the Skill for that.

Mental model: the way of seeing things and the frame of thinking, or the assumptions, that a person holds without noticing, built from past experience and surroundings.

/teach calls /how and /why internally.

Skill	Role
/how	Traces how the code behaves at runtime
/why	Investigates the motive and intent behind the code's current shape from git history, PR review comments, tickets, design documents, and similar records
/teach	Combines what /how and /why found into a plain explanation a human can follow. With that explanation, the human understands the agent's work better

You need /why because reading the code tells you what happens but **almost never tells you why the author wrote it that way**.

Here are example requests for the three Skills.

"Virtualization" in the first example means rendering only the part of a long list that is visible on screen.

```
/how is virtualization implemented?
```

```
/why are we still stuck an old version of node.js?
```

```
/teach me why you implemented it this way and not <other way>. what were the tradeoffs you made and why?
```

Besides the help it gives the human, /teach has another effect. **The investigation that the agent does to explain things to the human also helps the agent itself.**

poteto says, "even the latest models still often state things confidently without checking the data or reading the code they need to understand the mechanism" (he adds that how often depends partly on the quality of the tool that runs the agent).

So when the agent explains to the human what it will do next, and why, the explanation helps the agent too.

pstack: /teach keeps the confidence language

/why labels each reason it finds with one of five "confidence" levels. The level says how far records back that reason. /teach combines this output of /why into its explanation.

The `SKILL.md` of /teach lets it rephrase freely for the explanation, with one exception. The sentence reads as follows.

Keep why's confidence language intact (its hedges are findings, not style).

In other words, /teach leaves the confidence language, such as "it seems that ..." and "it is likely that ...", unchanged because **that language is not a writing habit. It is a result that /why found by investigation: how certain the reason is.**

So a reason that /why presented as a guess also reaches the reader as a guess in the /teach explanation.

Key point 4: carry past work into a new conversation with /recall

The human has the agent look back at related work in past conversations with `/recall`, and **carry earlier investigations, decisions, and failures into the new work.**

The agent carries over failures too. **If the agent does not know which approach failed before, it may try the same approach again.**

The guide's view: with /recall, do not rebuild the context from scratch

In the section "Learning from history", poteto writes about his work on virtualization bugs and performance problems that users reported in Cursor.

An earlier agent, in the separate session of a past conversation, often had rich context while it solved a similar problem. But each time you start a new chat, you have to rebuild that context almost from scratch.

From this, I read that **past conversation logs often still hold much of the context the earlier agent had.**

`/recall` is the Skill that deals with this problem.

`/recall` pulls recent context out of chat history, so that an agent in a new conversation can return to the work with the information it needs.

Here is an example request.

```
/recall the work i did yesterday on virtualization and then read this bug report on slack
```

pstack: /recall also gathers the record of failures

`/recall` searches the user's own chat history. When the topic names a particular feature, file, bug, or similar, it also searches records the team shares, kept in places such as the following.

- Source control
- Tickets

- Chat
- Error monitoring

Shared records hold important context, such as records of past failures. Examples are a fix that someone reverted after release, or an error that keeps occurring in production.

`/recall` can then return **what was tried before and which changes were reverted** (Chapter 22).

So the agent can start its next attempt from where the last one failed.

Summary

- **How key points 1 to 4 connect.** In this book, I treated key points 1 to 4 as the basic moves for supervising a model smarter than you.
- **Key point 1: convey the goal and background, and do not over-specify the solution.** The human writes the goal, background, constraints, and known facts in the request, and does not over-specify the concrete solution.
- **Key point 2: have the agent restate the problem before implementation.** Before implementation, the human has the agent restate the problem in its own words.
- **Key point 3: learn the mechanism and the reasons with `/how`, `/why`, and `/teach`.** The human has the agent investigate with `/how` and `/why`, and through the `/teach` explanation understands how the code works now and why it was written that way.
- **Key point 4: carry past work into a new conversation with `/recall`.** The human has the agent look back at related work in past conversations with `/recall`, and carry earlier investigations, decisions, and failures into the new work.

The next chapter, Chapter 38, looks at key points 5 to 8 as the stage of trying solutions.

Key points 5 to 8 are, in order: write how users will use it before implementation (key point 5), split documents by purpose (key point 6), compare prototypes of several options (key point 7), and have the agent verify questions by experiment (key point 8).

Chapter 38: Guide 2: test a design from how the user will use it

kaito • Sources: 日本語 • English • 2026-10-03

[Chapter 37](#) covered how to decide with the agent what to build: which problem to solve, with what design, and with what plan.

After you understand the problem, the next stage is to test solutions. This chapter covers that stage.

This chapter is also based on [Part 2](#) of The Complete Guide to pstack, which I call "the guide" from here on.

The guide recommends three things instead of a detailed abstract plan. An abstract plan is a plan that exists only as text and has no code yet.

- Write how the user will use the product before you implement it
- Build several options and run them
- Choose by the results of running them

In [Chapter 37](#), this book summarized the guide in [13 key points](#).

This chapter covers key points 5 to 8.

No.	Matching section of Part 2	Summary
5	"Working backwards"	The human has the agent write a README or a tutorial before the implementation, and has it design the API and the internals from how the user will use them

No.	Matching section of Part 2	Summary
6	"Working backwards"	The agent uses /technical-writing to split documents by purpose into tutorial, how-to, reference, and explanation, and uses /unstop, which /technical-writing calls internally, to remove the unnatural phrasing AI tends to write
7	"Measure a hundred times, cut once"	The human does not adopt the agent's first design as is. The human has the agent build prototypes of several options and compares them by screenshots taken during use, by measured timings, and by layout
8	"Measure a hundred times, cut once", "Architecting bigger changes"	The human has the agent answer for itself the questions that a small experiment can answer. Prototypes and verification reduce uncertainty better than repeated reviews of an abstract plan

The "Matching section of Part 2" column gives the name of the section of the guide that covers each key point.

First, I explain why the pstack [README](#) says "i don't believe in planning". Then I go through key points 5 to 8 one at a time.

I write each key point mostly in two sections, "The guide's view" and "pstack".

[Chapter 12](#) covered the steps of the "Prototype" Playbook that key point 7 uses.

This chapter leaves out the following and focuses on what the guide recommends.

- **How to build a prototype.** The agent puts speed first and does not care about code quality.

- **The report after building.** The agent shows the human the options it tried, the evidence, such as screenshots and measurements, the trade-offs, and the option it recommends.

How this chapter is organized

This chapter covers the following topics.

- "I don't believe in planning" means "plan in code"
- Key point 5: write the usage first
- Key point 6: split documents by purpose with /technical-writing
- Key point 7: compare prototypes of several options
- Key point 8: have the agent answer questions with experiments
- Summary

"I don't believe in planning" means "plan in code"

The pstack README says "i don't believe in planning". The sentence does not tell you to throw planning away. It tells you to **plan in code**.

The [README](#) section "why are there no planning skills?" has this sentence.

personally, i don't believe in planning. the best spec is code.

Plan mode. A mode in which the agent writes a plan before it writes code.

In the guide's section "Working backwards", poteto writes the following about most harnesses that have a plan mode.

"They tend to put too many implementation details in the plan and too little of everything else."

But poteto also writes the following, so he does make plans.

The truth is that I do plan, but I do so through code.

What differs is how he plans.

In this book, I treat **planning in code** as "**resolving design questions by building things and checking them**". Planning in code does not mean that you start the implementation without thought.

Ways to plan in code include building and checking "a tutorial that shows the usage ([key point 5](#))" and "prototypes that you run and compare ([key point 7](#) and [key point 8](#))".

Key point 5: write the usage first

Before the implementation, the human has the agent **write the usage in a README or a tutorial**, and has it work backward from that usage to the API and the internals.

The guide's view: start from the README

In the guide, poteto introduces "**README-driven development**" as a development method that writes the usage first.

README-driven development. A development method that starts from a README that explains the usage to the intended user and works backward to the implementation and the architecture.

When you start from the README, you have to think about developer experience from the position of the person who uses what you build, such as an API.

For example, when poteto built Dune, Cursor's internal framework for desktop apps, he first had the agent write a tutorial to see what it would be like to build an app with Dune.

Usage written before the implementation gives the human material to understand the finished product. It also gives the agent a concrete standard to verify behavior against after the implementation.

The guide also says that at first poteto found it hard to get the agent to write a readable tutorial, so he built `/technical-writing` first ([key point 6](#)).

pstack: /architect also writes the usage first in its output

The `SKILL.md` of `/architect` ([Chapter 24](#)) also requires, in its section "Outputs", that **the agent writes the caller's usage first and derives the skeleton from that usage**.

The caller's usage is written first and the type sketch derived from it.

- **Skeleton.** Code that gives only the shapes of the types and functions, with no bodies yet.
- **Webhook.** A mechanism that sends an HTTP request to a given URL to notify another system of an event ([Chapter 25](#)).

As an example of how a skeleton comes from usage, consider a feature that puts a rate limit on webhooks that arrive from outside. A rate limit caps the number of requests that a system accepts in a given time.

This feature also appears in example 2 of the guide's section "The workflow in practice", in the request we need to add rate limiting for external webhooks. [Chapter 39](#) covers this request in "Example 2: a design request".

This section uses an example that limits webhooks from the same sender to 60 per minute. The sender is the external system that sends the webhooks.

When the agent follows `/architect`, writes the usage first, and derives the skeleton from it, the result looks like this.

```
// 1. First, write the caller's usage
const limiter = createRateLimiter({ perMinute: 60 });

function handleWebhook(webhook: { senderId: string }): Response {
  if (!limiter.tryAcquire(webhook.senderId)) {
    return new Response("Too Many Requests", { status: 429 });
  }
}
```

```

    return new Response("OK");
  }

  // 2. Derive the skeleton from the usage: write only the shapes of the types and functions, with no
  // bodies yet
  type RateLimiterOptions = { perMinute: number };
  type RateLimiter = { tryAcquire(senderId: string): boolean };

  function createRateLimiter(options: RateLimiterOptions): RateLimiter {
    throw new Error("not implemented");
  }

```

The usage in 1 passes { perMinute: 60 } to createRateLimiter, and that fixes the shape of RateLimiterOptions in 2.

The usage in 1 also branches on the result of limiter.tryAcquire(webhook.senderId) to decide whether to accept the webhook. That branch fixes RateLimiter in 2 as a type with a tryAcquire method that takes the sender ID, a string, and returns a boolean.

The agent writes the function body, which is the throw new Error("not implemented") part, in the implementation stage, after the skeleton is fixed. Key point 9 in [Chapter 39](#) covers what the guide says about the stage that implements the skeleton.

In this way, **when you write the usage first, the usage decides the shapes of the types and functions.**

Key point 6: split documents by purpose with /technical-writing

The agent uses /technical-writing to split documents into tutorial, how-to, reference, and explanation, and **does not mix purposes in one document.**

The guide's view: a README that mixes purposes is hard to read

The first README for Dune, which poteto had the agent write without /technical-writing, was hard to read.

It packed the jobs of a tutorial, a how-to, an explanation, and a reference into one document, and the agent wrote it in pompous, AI-sounding prose.

So poteto built /technical-writing ([Chapter 32](#)).

- **Diátaxis.** An approach that splits documents into four kinds, called modes, by the reader's purpose.
- /unslap. A Skill with which the agent uses rules to cut AI tells from its own writing. AI tells are phrases and formatting that make the reader notice a machine wrote the text, for example a stock phrase like "I hope this helps" ([Chapter 32](#)).

/technical-writing classifies documents according to Diátaxis.

/technical-writing also uses /unslap, so **the documents it produces are easier to read.**

The guide's view: combine /technical-writing with other Skills

Many pstack Skills, /technical-writing among them, work together, and each makes the others more effective.

The guide names the design phase, the stage in which the user works out the design of their own app, as a place where that happens.

When you combine pstack Skills to draw out rich context, **the agent can see the essential parts of the problem the way a human does, instead of seeing only a small part of it.**

- **Verification skill.** A project-specific Skill with which the agent starts and operates the app to collect evidence ([Chapter 36](#)).
- **Virtualization.** A technique that renders only the visible part of a long list ([Chapter 37](#)).

The following three-part request is an example that combines Skills.

```
(1) /recall my work fixing virtualization bugs and perf issues from the past 7 days. use /how and /
why to understand how our current virtualization implementation works.
```

(2) then use `/poteto-mode planning` and `/technical-writing` to come up with a new virtualization engine that categorically eliminates flickering and jittering. Let's start by writing a tutorial on how i would use this new package to virtualize a React app

(3) after you write the plan, `/teach me` and prove to me why this new approach is superior to our current engine

`/technical-writing` appears in (2). Its job is to write the tutorial for the new package, which is the "usage" of [key point 5](#).

The three parts ask for the following.

- **(1).** Use `/recall` to recall the work of the past 7 days, and use `/how` and `/why` to understand how the current virtualization implementation works.
- **(2).** Use what (1) gathered, such as the bugs fixed before, to come up with a new design that eliminates flickering and jittering at the root.
- **(3).** After you write the plan, explain it with `/teach` and prove that the new package is better than the current engine, the component that does virtualization today. (High-quality tools such as a verification skill matter here.)

In this way, **the request aims to have the agent draw out rich context about past work and the current implementation, and to use that context for the design and the proof.**

pstack: one mode per document

The `SKILL.md` of `/technical-writing` requires **one mode per document** in its section "Pick the mode first (Diátaxis)".

One document, one mode.

The agent picks the mode with two questions that check what the content does for the reader.

- Is it for action or for understanding?
- Is it for learning or for work?

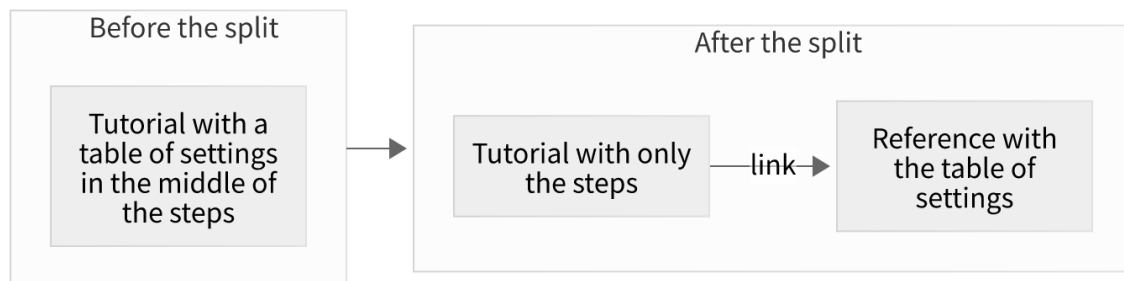
Each combination gives a mode as follows. The description of each mode follows the guide.

- **Action and learning.** Tutorial. A document in which a newcomer learns by following steps and building something visible.
- **Action and work.** How-to. A document with steps that an experienced user follows to solve a specific problem they have.
- **Understanding and work.** Reference. A document that describes an API or settings accurately and completely.
- **Understanding and learning.** Explanation. A document that explains background, design choices, and trade-offs.

You can apply the two questions to a whole document or to a single sentence.

This SKILL.md also forbids a reference table inside a tutorial. It requires the agent to split that content out and link to it instead.

For example, if the agent is about to put a table of settings in the middle of a tutorial, the table is reference material for understanding, so the agent moves the table into a separate document and links to that document from the tutorial.



[View diagram 1](#)

[Chapter 32](#) covers the detailed rules.

Key point 7: compare prototypes of several options

The human does not adopt the agent's first design as is. **The human has the agent build several options and chooses one by the results of use and measurement.**

The guide's view: build prototypes of several options and compare them

In the section "Measure a hundred times, cut once", poteto names two **common mistakes in planning**.

- Accepting the agent's first design as is
- Overcooking the plan without empirical evidence

This book covers the fix for the first mistake in this key point 7, and the fix for the second in [key point 8](#).

poteto explains the first mistake as follows.

When humans wrote the code, people normally exchanged design docs and rewrote them many times until the design settled.

With agents, you can skip the step in which a human writes the design doc. It has become normal to have the agent write the design doc.

The problem is that "**the human often accepts the first design the agent returns as is**".

poteto calls this a mistake.

With agents, while we can skip the ceremony of the design doc, I often see the mistake of accepting the first thing the agent gives back to you.

The guide does not say why it is a mistake to accept the first design. I think the claim is right both by intuition and by experience.

In practice, **you cannot judge whether a design is good if you have not compared it with anything**.

Humans wrote the alternatives they considered into design docs and rewrote them many times so they could compare options and pick the better one.

If you accept the first design as is, this comparison drops out completely.

The "[Prototype](#)" Playbook, which builds and compares several options, addresses this mistake.

Here is an example request.

```
/poteto-mode prototype a few options for <feature request>. use /control-app* and take videos/
screenshots for me to review and choose from
```

The guide's footnote (*) explains that `/control-app` refers to the verification skill built in The Complete Guide to pstack [Part 1 \(Chapter 36\)](#).

For a prototype that changes how something looks, the agent builds a throwaway version inside the app or in a scratch directory.

To compare UI interactions, the agent builds two or three options and adds a simple toggle that switches which option is shown.

For example, to compare three ways of opening a menu, the agent puts "Option A", "Option B", and "Option C" buttons on the screen, and each button opens the menu the way that option does.

That way, you can switch between options and compare them on the same screen.

Then the agent uses `/control-app` to operate each option, take screenshots, and measure the real timing and layout.

The "Prototype" Playbook also works to compare options for features and bug fixes, not only visual options.

pstack: a prototype is a throwaway tool for deciding the design

The "Prototype" Playbook defines **a prototype as a throwaway tool for deciding the design**.

Before it starts, the agent states clearly "what the prototype will decide".

The prototype decides these three things.

- Which layout to use
- Which interaction to use
- How densely to pack the elements on the screen

When the agent can run and observe the options to see which one is better, the prototype also decides things such as the following.

- Which behavior to use

- Which timing to use
- Which approach to use

If there is nothing for a prototype to decide, the agent does not build one and passes the work to the "Feature" Playbook, which builds new features.

When the human has chosen an option, the agent also passes that option to the "Feature" Playbook.

If the agent needs to decide the shapes of the types or modules first, it passes the work to /architect.

Key point 8: have the agent answer questions with experiments

The human does not answer questions that building and running something can answer, and does not make the agent wait for a human reply. **The human has the agent answer them with experiments.**

While the plan is still abstract, the human also does not refine it through repeated reviews.

- **Experiment.** Building something small, running it, and checking the result.
- **Abstract plan.** A plan that exists only as text and has no code yet. Even a long plan with full implementation details is abstract if it has no code yet.

The guide's view: answer questions with prototypes, and do not review abstract plans adversarially

Overcooking the plan without empirical evidence.

Prototypes are also the fix for the second mistake named in key point 7.

In the section "Measure a hundred times, cut once", poteto writes that **building a prototype is planning in code**.

With a prototype, the agent can answer its own questions with the results of running it, without waiting for a human reply.

For example, for "which is faster, A or B", the agent runs both and measures the time. For "does this action break the layout", the agent operates the prototype and checks.

At the end of the section "Architecting bigger changes", poteto writes that **planning in code with prototypes and /architect is far more effective**.

He goes on to say that this effectiveness is also why he does not review abstract plans adversarially.

Adversarial review. A review in which the agent criticizes something from the position of someone who looks for defects.

Here is the reason not to review an abstract plan adversarially. When you ask the agent to review the plan, the agent **makes up groundless, purely theoretical risks and presents them as facts**.

It then invents complex edge cases to prepare for problems that never happen.

For example, the agent comments on a function in the plan: "if null is passed to this function, it crashes". Null is the value that means no value. The agent then asks to add handling for null to the plan.

With code, you can trace the callers and check whether any caller passes null.

If no caller passes null, you know that the handling this comment asks for prepares for a problem that never happens.

But an abstract plan has no callers to trace.

So nobody can check the comment, and people treat the comment as fact. The plan then fills up with edge-case handling for problems that never happen.

For these reasons, the guide recommends that **you do not refine a plan while it is abstract, and that for open questions you have the agent build prototypes, check the results, and reach the answer itself**.

pstack: before asking the human, the agent tells whether an experiment can answer the question

The section "[Non-negotiables](#)" of `/poteto-mode` lists the rules the agent follows in the work of every Playbook ([Chapter 8](#)).

This section also has a rule for when the agent is about to ask the human a question.

The rule covers questions at points where the work can go several ways, such as the following.

- Which approach to use
- How it should be built
- What this should do

"Non-negotiables" requires the agent, before asking such a question, **to tell whether the answer is a fact it can learn by running something and observing it** ([Chapter 35](#)).

If the answer is a fact you can observe by running something, the human should not have to answer it.

Observable facts include the following.

- Behavior
- Timing
- Layout
- Output
- Performance

For questions whose answers are such facts, **the agent tries the options with the "Prototype" Playbook and decides without asking the human.**

But "Non-negotiables" makes an exception when the work is a read-only investigation with the "[Investigation](#)" Playbook and the artifact is an answer with its grounds.

In that case, the agent does not build a prototype and answers from the evidence it gathered.

What the agent asks the human about is product judgment and taste, which trying things cannot answer.

Two questions show this split.

The agent can answer "Does this action break the layout?" by operating a prototype, so the agent tries it and decides.

"Which of the two layouts should this product adopt?" is a question of taste that trying things cannot answer. You can observe how each screen is arranged, which is its layout, but observing it does not decide which one to choose.

So the agent shows the screens side by side and leaves the choice to the human.

pstack: what goes to /interrogate is the skeleton and the diff

If pstack does not review abstract plans adversarially, what does it review adversarially?

pstack has `/interrogate`, a Skill that has several models review a change adversarially (Chapter 27).

In its instructions to the reviewers, `/interrogate` calls each reviewer an "adversarial code reviewer" and tells them to look for real problems instead of help or encouragement.

"Non-negotiables" sets when to use `/interrogate`. **A contested design goes through `/interrogate` before anyone merges it.**

Contested design. A design that people judge differently as good or bad.

In this book, I treat a design as contested when several approaches would all work correctly. Error handling is an example. An exception and a return value both report a failure correctly, so opinions differ.

The two pstack `SKILL.md` files show that the designs that go through `/interrogate` are both **things that already exist as code.**

- **Diff.** The `SKILL.md` of `/interrogate` defines the review target as a code change, which is a diff.
- **Skeleton.** The section "Phase C: Agree (opt-in)" in the `SKILL.md` of `/architect` says that you can put the merged skeleton through `/interrogate` before the implementation starts. `/architect` is a Skill that merges design options from several models into one

skeleton ([Chapter 24](#)). A skeleton is code that gives only the shapes of the types and functions, with no bodies yet.

When the design exists as code, you can check each review comment against the real thing.

In `/interrogate`, the main agent, which has the full context, verifies the reviewers' comments.

A hypothetical that never happens in practice, like the null example above, turns out not to be a problem once you trace the callers. So the main agent sorts it as a comment that needs no action ([Chapter 27](#)).

From these two points, this book reads "do not review abstract plans adversarially" not as "do not review" but as **prepare the grounds first, then review**.

Grounds here means things the reviewer can check against the real thing, such as a skeleton, a diff, a usage example, a prototype, or a measurement.

Summary

- **What "I don't believe in planning" means.** It does not tell you to throw planning away. It tells you to plan in code. In this book, I treat planning in code as resolving design questions by building things and checking them.
- **Key point 5: write the usage first.** The human has the agent write a README or a tutorial first, and has it design the API and the internals from that usage.
- **Key point 6: split documents by purpose with `/technical-writing`.** The agent uses `/technical-writing` to split documents into four modes, tutorial, how-to, reference, and explanation, and does not mix purposes in one document. `/unstop`, which `/technical-writing` uses, cuts the AI tells.
- **Key point 7: compare prototypes of several options.** The human does not adopt the agent's first design as is. The human has the agent build prototypes of several options and compares them by screenshots and measurements. A prototype is a throwaway tool for deciding the design.
- **Key point 8: have the agent answer questions with experiments.** The human has the agent answer for itself the questions that experiments can answer. As a rule, the

human answers only questions that trying things cannot answer, the ones that depend on product judgment or taste. This book reads "do not review abstract plans adversarially" as "first prepare grounds, such as a skeleton, a diff, a usage example, a prototype, or a measurement, then review".

The next chapter, [Chapter 39](#), covers key points 9 to 13 as the stage that "settles the solution and turns it into a plan you can carry out".

- Key point 9: compare and merge several designs with /architect
- Key point 10: when implementation shows the design is wrong, revise the design
- Key point 11: split the work into small tasks you can verify
- Key point 12: the plan document is temporary
- Key point 13: what to tell /poteto-mode

Chapter 39: Guide 2: revise the design into an executable plan

kaito • Sources: 日本語 • English • 2026-10-03

[Chapter 38](#) showed how to design from the way users use something, test solutions with prototypes, and choose a design option.

This chapter covers the stage that follows the test of solutions. In this stage, the agent **settles the design of a large change and turns it into a plan you can execute**, in two steps.

- **Settle the design** (/architect). The agent decides the types, the signatures of the public functions, and how to split the modules.
- **Turn it into an executable plan (the "Multi-phase or multi-PR plan" Playbook).**
The agent splits a large piece of work into several PRs so that each PR holds one change. For example, it splits the work into "a PR that changes the data types" and "a PR that updates the screens to the new types". It then writes a plan document that lists, for each PR, the files to change and how to verify that the PR is finished, such as the steps to check the state of a screen.

This chapter is also based on [Part 2](#) of The Complete Guide to pstack, which I call "the guide" from here on.

In [Chapter 37](#), this book summarized the guide in [13 key points](#).

This chapter explains the remaining key points, 9 to 13.

No.	Matching section of Part 2	Summary
9	"Architecting bigger changes"	In a large change, the human spends time on how the system holds data and how systems connect. The agent uses / architect to investigate the existing constraints and design, and compares and merges independent options from several models before it implements
10	"Architecting bigger changes"	If implementation reveals a mismatch with the design, the agent revises the design. Depending on the evidence from implementation (for example, the types needed any or forced casts), the agent throws the design away and starts over
11	"Okay but I really want a planning doc"	Once the design is concrete, the agent uses the planning Playbook to split the work into small tasks or PRs that it can verify one by one. For each task, it sets the result of running the code as the evidence that the task is finished
12	"Okay but I really want a planning doc"	The plan document that the planning Playbook produces is a temporary document. The agent uses it to carry out the work and, in large work, to share progress with other agents. poteto usually deletes it after the work is done
13	"The workflow in practice"	/poteto-mode is the entry point that uses Playbooks and Skills to fit the request. The human tells / poteto-mode the goal, the constraints, the results they want to see, and when to review

The "Matching section of Part 2" column gives the name of the section of the guide that covers each key point.

Key points 9 to 11 show how to settle a design and turn it into a plan. Key point 12 then covers how to treat the plan document, and key point 13 covers what the human tells / poteto-mode.

At the end, I look back on the 13 key points as one flow.

I split each key point mainly into two sections, "The guide's view" and "pstack".

[Chapter 24](#) and [Chapter 16](#) explained the steps of `/architect` and of the "Multi-phase or multi-PR plan" Playbook, which key points 9 to 11 use.

This chapter leaves out the following topics and focuses on what the guide recommends.

- **Asking to see the design.** By default, once `/architect` has a merged skeleton, which is code that writes only the shapes of types and functions, it moves on to implementation without waiting for the user to check it. It stops and shows the merged design only when the user asks for that in the request. For example, the user can write `/architect` with `checkpoint. stop and show me before implementing.` in the request.
- **Redesign after throwing away a skeleton.** The agent redoes the design from the new constraints it found during implementation. It makes the new skeleton smaller than the old one before it adds features. This step follows the principles "Redesign from First Principles" and "Subtract Before You Add" ([Chapter 17](#)).
- **When no plan is needed.** If the change fits in one or two files and the approach is obvious, the agent does not write a plan. It tells the user that no plan is needed and stops.
- **Checking the format of the plan document.** The agent checks the structure and format of the plan document with `check-plan.mjs` and fixes every issue it reports.

How this chapter is organized

This chapter covers the following topics.

- Key point 9: compare and merge several designs with /architect
- Key point 10: when implementation shows the design is wrong, revise the design
- Key point 11: split the work into small tasks you can verify
- Key point 12: the plan document is temporary
- Key point 13: what to tell /poteto-mode
- Reading the 13 key points as one flow
- Summary

Key point 9: compare and merge several designs with /architect

In a large change, the human spends time on the architecture, the data structures, and how systems connect. The agent spends its time on the details of the implementation.

The agent does not start implementing right away, though. **It first compares several design options, merges them into one, and only then implements.**

The guide's view: poteto spends time on architecture

In the section "Architecting bigger changes", poteto writes that **for an engineer in the age of agents, it matters more to spend time on the following.**

- Designing the architecture
- Choosing the right data structures
- Deciding how the systems you build connect to each other

poteto also writes that **the agent fills in the details of the implementation.**

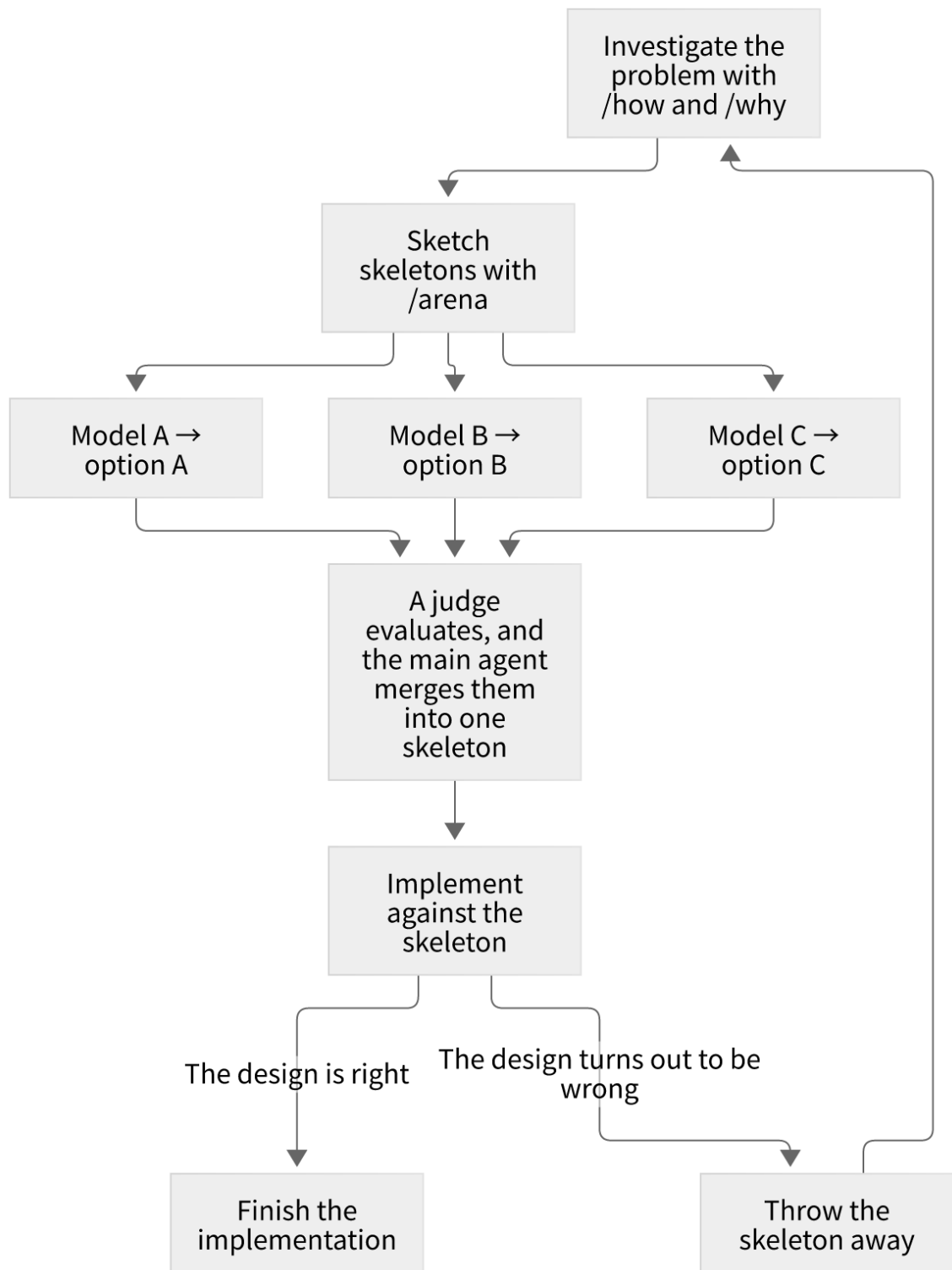
The guide names /architect as the Skill that moves through a design in clear stages. The example request is this one line.

```
/architect this new <feature request>
```

The guide explains the flow of `/architect` in the following five stages.

1. Investigate the problem
2. Sketch skeletons
3. Evaluate and merge
4. Implement against the skeleton
5. Throw the design away if it is wrong

As a diagram, the flow looks like this.



[View diagram 1](#)

The rest of this section goes through the five stages one at a time.

1. Investigate the problem

The agent investigates the existing mechanisms that the new code touches with `/how` and `/why`.

- `/how` is the Skill that investigates how the code works now ([Chapter 22](#)).
- `/why` is the Skill that investigates why the code has its current shape ([Chapter 22](#)).

The agent then gets an accurate picture of which module is responsible for what, and of the existing constraints.

2. Sketch skeletons

The agent has several models produce design options in parallel, without showing any model the other models' options (the `SKILL.md` of `/architect` uses `/arena` for this).

In many cases it uses models from different families.

`/arena` is the Skill that has several agents solve the same task separately ([Chapter 24](#)).

Each model's option is a design package with the following four parts.

- **How the caller uses it.** This is the code on the side that uses what you build. Part 1 of the example code below shows it.
- **The definitions of the central types.** These are the shapes of the data at the center of the design. In part 2 of the example code below, they are `RateLimiterOptions` and `RateLimiter`.
- **The signatures of the public functions.** These are the names, arguments, and return types of the functions that other code can call. In part 2 of the example code below, the first line of `createRateLimiter` is a signature.

- **A short document with the reasons for the design.** This document, about one page long, says why the design has this shape and which other shapes the model considered and rejected. This part alone is prose, not code. The file `references/rationale-template.md` in `/architect` defines how to write it.

The model usually writes the first three parts as skeleton code.

Skeleton. Code that writes only the shapes of types and functions, without their bodies yet ([Chapter 24](#)).

Each model that produces an option also checks its own option from the following three angles.

- **Depth of the interface.** Does the side that uses it see few functions and types, while those hide a lot of work? (For example, can one call to `tryAcquire` handle both counting requests and resetting the count every minute internally?)
- **Failures when a weaker model works with it.** Check whether a weaker model, which later writes code against the design, produces an implementation that satisfies the design.
- **Matching against the four red flags.** `/architect` has a list of four design red flags (`references/design-red-flags.md`) ([Chapter 24](#)). The four are shallow modules, information leakage, temporal decomposition, and pass-through methods. If an option matches any of them, the model fixes or rejects it.

Example of skeleton code

To build a skeleton, you write the code on the using side first and then decide the shapes of the types and functions from it.

Skeleton example: the webhook rate limiting skeleton derived in [Chapter 38](#)

This is the example from key point 5 (write the usage first) in [Chapter 38](#), which derived a skeleton from the code on the using side. The example is the following.

This section uses an example that limits webhooks from the same sender to 60 per minute. The sender is the external system that sends the webhooks.

The example follows this order.

```
// 1. First, write how the caller uses it
const limiter = createRateLimiter({ perMinute: 60 });

function handleWebhook(webhook: { senderId: string }): Response {
  if (!limiter.tryAcquire(webhook.senderId)) {
    return new Response("Too Many Requests", { status: 429 });
  }
  return new Response("OK");
}

// 2. Derive the skeleton from the usage: write only the shapes of types and functions, without
their bodies yet
type RateLimiterOptions = { perMinute: number };
type RateLimiter = { tryAcquire(senderId: string): boolean };

function createRateLimiter(options: RateLimiterOptions): RateLimiter {
  throw new Error("not implemented");
}
```

Part 1 writes the code on the using side first. It passes { perMinute: 60 } to createRateLimiter and uses the result of tryAcquire to decide whether to accept the webhook.

Part 2 decides the shapes of the types and functions to fit that usage.

The skeleton follows the same idea as key point 5 in Chapter 38. Chapter 38 states key point 5 as follows.

Before the implementation, the human has the agent **write the usage in a README or a tutorial**, and has it work backward from that usage to the API and the internals.

3. Evaluate and merge

A judge evaluates each option against a strict rubric. The judge is a different model from the main agent, the original agent that started the models that produce the options.

The main agent takes the options and their evaluations into account and merges the options into one design.

The bundled guide [docs/guide/04-design.md](#) explains how the merging agent works. It reads every option to the end, picks one option as the base, takes in the good parts of the other options, and verifies the result.

4. Implement against the skeleton

The agent replaces the placeholder bodies of the skeleton with real code. A placeholder body is code such as `throw new Error("not implemented")` that stands in for the real body.

If the agent finds during implementation that a function needs an argument or state it did not expect, it does not quietly work around the mismatch inside the implementation. It makes the mismatch visible.

The section "Phase D: Implement against the sketch" in the `SKILL.md` of `/architect` then has the agent ask again which of the following is the cause.

- The skeleton was wrong
- A requirement was missed
- The implementation does more than it needs to

5. Throw the design away if it is wrong

If the agent finds during implementation that the skeleton was wrong, it throws everything away and starts over from the beginning ([key point 10](#)).

pstack: /architect is for changes that cross function boundaries and changes that move responsibility

The bundled guide [docs/guide/04-design.md](#) says that most changes do not need design work that compares and merges several options.

It then names **changes that cross a function boundary and changes that move ownership of a responsibility** as the targets of `/architect`. Ownership here means which module is responsible for something.

Here is an example of my own. A change that alters the shape of a function's arguments or return value crosses a function boundary, because the code that calls the function has to change too.

For example, suppose you change the return value of `getPrice`, a function that returns the price of an item, from the amount alone to "the amount and the currency".

```
// price.ts (before): returns only the amount
function getPrice(itemId: string): number {
  // ...
}

// price.ts (after): returns the amount and the currency
function getPrice(itemId: string): { amount: number; currency: string } {
  // ...
}
```

The shape of the return value changes, so the code that calls `getPrice` must change with it.

A change that moves the job of holding a cache from the screen code to the module that fetches data is a change that moves ownership of a responsibility.

Before the change, the screen code holds its own cache.

```
// UserPage.ts (before): the screen code holds the cache
const cache = new Map<string, User>();

async function showUser(id: string) {
  const user = cache.get(id) ?? (await fetchUser(id));
  cache.set(id, user);
  render(user);
}
```

After the change, the module that fetches data holds the cache, and the screen code only calls `fetchUser`.

```
// userApi.ts (after): the module that fetches data holds the cache
const cache = new Map<string, User>();

export async function fetchUser(id: string): Promise<User> {
  const cached = cache.get(id);
  if (cached) return cached;
```



```

const user = await request(`/users/${id}`);
cache.set(id, user);
return user;
}

// UserPage.ts (after): only calls it, without caring about the cache
async function showUser(id: string) {
  render(await fetchUser(id));
}

```

The job of holding the cache has moved from the screen code to `userApi.ts`. This kind of change moves ownership of a responsibility.

So a change that reaches beyond one function, into the contract with other code, such as the shapes of arguments and return values, or into the division of jobs, starts from design with `/architect`.

04-design.md states that when `/poteto-mode` receives a request, it applies this criterion and decides on its own whether to use `/architect`.

So **the user does not need to decide every time whether to use** `/architect`.

The user calls `/architect` or other Skills directly mainly when they want to check the design more thoroughly, or more briefly, than `/poteto-mode` does by default.

Key point 10: when implementation shows the design is wrong, revise the design

During implementation, the agent may find a mismatch with the skeleton or a workaround that keeps coming back. The agent treats either one as evidence that the design is wrong and **revises the design. If needed, the agent throws the skeleton away and redoes the design from the first stage, the investigation of the problem.**

For example, during implementation a function may turn out to need an argument that the skeleton does not have. The missing argument is a mismatch with the skeleton.

When the agent writes `as any` in the same form in many places to read a field that the data type does not have, the repeated `as any` is a workaround that keeps coming back.

The guide's view: the same workaround in unrelated places, or escape hatches in the types, is evidence that the design is wrong

After it explains the five stages of `/architect`, the section "Architecting bigger changes" sums up when to throw a design away in this one sentence.

If the same workaround appears across unrelated call sites, or if the types require escape hatches like `any` or forced casts, that is empirical proof that the architecture is wrong.

The quote names the following two signs.

- The same workaround appears in unrelated call sites
- The types need escape hatches such as `any` or forced casts. An example of a forced cast is `as unknown as User`

Here is how the two signs show up in an example of my own.

The `Order` type that the skeleton defines has no currency field.

When the agent implements the code, though, three unrelated places all need the currency: the invoice, the email, and the admin screen.

```
// The type decided in the skeleton: no currency field
type Order = { id: string; total: number };

// An order to display or send
declare const order: Order;

// invoice.ts: reads a field the type does not have through a type escape hatch (as any)
const invoiceCurrency = (order as any).currency ?? "JPY";

// mail.ts: writes the same workaround in another, unrelated place
const mailCurrency = (order as any).currency ?? "JPY";

// admin.ts: writes the same workaround in yet another place
const adminCurrency = (order as any).currency ?? "JPY";
```

The agent writes the same workaround in three places **not because the code in each place is badly written, but because the `Order` type in the skeleton has no currency field.**

So when the same workaround or type escape hatch shows up in unrelated places, **you need to revise the design itself, not the individual places.**

pstack: whether to throw the design away depends on whether the signs repeat

The `SKILL.md` of `/architect` asks the agent to decide whether to throw the design away **by whether a sign repeats in the same form, not by whether it appeared once.**

The same `SKILL.md` asks the agent **not to conclude right away that the design is wrong when it finds a sign.** A few edge-case handlers do not by themselves show that the design is wrong, because the problem itself may be complex.

The guide's quote names type escape hatches as a sign. Under the same `SKILL.md`, though, one any that appears once in one place is not by itself a reason to throw the design away.

On the other hand, once the agent starts to write the same workaround in three unrelated call sites, as in the invoice, email, and admin screen example, it treats that workaround as a sign to throw the skeleton away.

The section "Phase E: Scrap when the architecture is wrong" of the same `SKILL.md` also names four more signs as material to decide whether to throw the design away. They come in addition to the two signs above, the same workaround and type escape hatches.

1. Several unrelated edge cases each need their own branch
2. The skeleton said no state is shared, but during implementation the agent thinks "this needs a lock"
3. The caller cannot use the module without knowing its internal rules (for example, an initialization function must be called first)
4. During implementation, a mismatch with the skeleton appears in the same form, independently, two or more times

Below are code examples of these four signs.

Example of 1: a dedicated branch for each unrelated edge case

A function that displays the order total adds a dedicated branch for each of three unrelated edge cases.

```
function formatTotal(order: Order): string {
  // Only legacy orders use a different calculation
  if (order.id.startsWith("legacy-")) return formatLegacyTotal(order);
  // Only refund orders use a different display
  if (order.total < 0) return `Refund ${-order.total} yen`;
  // Only gift orders hide the amount
  if (order.id.endsWith("-gift")) return "Gift";
  return `${order.total} yen`;
}
```

Every branch guesses the kind of order from the `id` string or the sign of the amount, because the `Order` type has no field for the kind of order. The guessing is a sign to revise the type design instead of adding more branches.

Example of 2: state that should not be shared needs a lock

The skeleton assumed that only one process reads and writes the count for each sender.

```
// The skeleton's assumption: only one process reads and writes the count for each sender
const counts = new Map<string, number>();
```

During implementation, though, it turns out that several servers update the same count in a shared store (`store`) at the same time.

So the agent is about to add a lock so that no other server can update the count between the read and the write-back.

```
// During implementation: about to add a lock that the skeleton did not have
async function tryAcquire(senderId: string): Promise<boolean> {
  await lock.acquire(senderId);
  try {
    const count = (await store.get(senderId)) ?? 0;
    await store.set(senderId, count + 1);
    return count < 60;
  } finally {
    await lock.release(senderId);
  }
}
```

The lock became necessary because the skeleton's assumption that "no state is shared" was wrong.

Example of 3: the caller cannot use it without knowing the internal rules

In this design, `tryAcquire` throws an error unless the caller calls `init()` first.

```
const limiter = createRateLimiter({ perMinute: 60 });

limiter.init(); // An internal rule says to call this first
limiter.tryAcquire("sender-a"); // If you forget init(), this throws an error
```

The caller cannot use the limiter correctly without knowing the internal rule "call `init()` first". Neither the types nor the shapes of the functions show this rule.

Example of 4: a mismatch of the same form appears twice in separate places

In the skeleton, `formatPrice`, the function that formats an amount, took only the order.

```
// The skeleton: takes only the order
function formatPrice(order: Order): string {
  throw new Error("not implemented");
}
```

When the agent implemented two separate places, the invoice and the email, both needed an argument for the display language (`locale`).

```
// Implementing invoice.ts: needs a locale argument that the skeleton does not have
formatPrice(order, locale);

// Implementing mail.ts: another place needs locale in the same form
formatPrice(order, locale);
```

A mismatch of the same form appears in two unrelated places. The repeated mismatch shows that the skeleton was missing `locale`. The `locale` argument is not a need of one place.

Key point 11: split the work into small tasks you can verify

Once the design is settled, the agent uses the planning Playbook "**Multi-phase or multi-PR plan**" and **splits the work into small PRs that it can verify one by one**.

For each PR, it then writes in the plan document what the agent must confirm before the PR counts as finished, such as a log line, a screenshot, or a test run result.

The guide's view: the planning Playbook usually comes after the design

In the section "Okay but I really want a planning doc", poteto writes that pstack has no planning Skill, but it has a Playbook that plans in several phases.

He adds that, after the agent has produced a design he is satisfied with, **he usually uses this planning Playbook ("Multi-phase or multi-PR plan") to turn the design into a plan that spells out the steps of execution**.

(An example of a plan document this Playbook produces appears later in "[Example of a finished plan document](#)".)

The request the guide gives is this one line.

```
/poteto-mode turn this design into a plan
```

With this Playbook, the agent builds each task in the plan around the evidence that shows the task is finished and the way to confirm it, which is verification.

After the human approves the plan, the agent carries it out item by item.

Each PR in the plan is **small, self-contained, and easy to review**.

pstack: the plan document lists the finish evidence and how to verify it for each PR

The file of the "**Multi-phase or multi-PR plan**" Playbook (`playbooks/multi-phase-plan.md`) sets out the following five points.

- The plan is the artifact, and the agent does not implement
- Before writing, settle open questions with prototypes

- One change per PR, with its evidence
- Tests alone do not count as verified
- Execute only after the user says to start

At the end, I show an example of a finished plan document that follows these five points.

The plan is the artifact, and the agent does not implement

With this Playbook, the agent produces only the plan document and writes no code.

The execution Playbook that the plan document names carries out the plan.

The execution Playbook is either "**Autopilot-full**" or "**Autopilot-stack**". To run a whole program of work that lasts many days, the agent uses "**Orchestrate**". [Chapter 16](#) covers all three.

Before writing, settle open questions with prototypes

For each open question, such as "which approach is faster", the agent runs the "**Prototype**" Playbook ([Chapter 12](#)).

It then records the evidence from the prototypes, such as branches, commits, and screenshots, in the appendix of the plan document.

The step follows the same idea as key point 8 (have the agent answer questions with experiments) in [Chapter 38](#). Chapter 38 states key point 8 as follows.

The human does not answer questions that building and running something can answer, and does not make the agent wait for a human reply. **The human has the agent answer them with experiments.**

One change per PR, with its evidence

The plan document describes the PRs one at a time, separated by headings (##). For example, a plan document with two PRs has the following shape.

```
# Plan: rate limiting for webhooks

## Build a RateLimiter that counts requests per sender (PR 1)
(Write the fields below for PR 1.)

## Add per-sender rate limiting (PR 2)
(Write the same fields for PR 2.)
```

Under every heading, write the following fields in order. The end of this section shows a finished version with the contents filled in.

- Depends on
- Files to change
- Changes to make
- Observable result, such as a log line or the state of a screen
- Unit tests
- Live verification, in which you operate the running app to confirm the result
- Performance check
- User review ([Chapter 16](#) calls this the "operator" check. Only a PR that changes the flow of what the user does needs it. For any other PR, write "None".)
- Merge

For example, if you write these fields under the PR 2 heading of the outline above, the result looks like this.

```
## Add per-sender rate limiting (PR 2)

**Depends on**: PR 1 (Build a RateLimiter that counts requests per sender)

**Files to change**
- [ ] Edit `src/webhooks/handleWebhook.ts`

**Changes to make**
- [ ] In `handleWebhook`, call `limiter.tryAcquire(senderId)` and return 429 when the limit is exceeded

**Observable result**
- [ ] When the same sender sends 61 requests in one minute, the log shows `rate_limited sender=sender-a` on the 61st
```



```

**Unit tests** (tests alone do not count as verified)
- [ ] Add a case "the 61st returns 429" to `handleWebhook.test.ts` and run `npm test`

**Live verification**
- [ ] Send 61 webhooks to the running server and confirm that the 61st gets 429

**Performance check**
- [ ] Compare the processing time per webhook before and after the change

**User review**: None (the user's flow does not change)

**Merge**
- [ ] Merge only after the unit tests, live verification, and performance check are all checked

```

Tests alone do not count as verified

The Playbook makes the following two sentences its rule for verification.

Tests alone are not sufficient verification. A PR is verified only when its unit, live, and perf boxes are all checked.

Execute only after the user says to start

The agent gives the user the location of the plan document and the result of the format check (check-plan.mjs), and stops.

It does not start execution until the user says to start.

Example of a finished plan document

Example: the plan document for webhook rate limiting (two PRs)

With the fields filled in for the two PRs of the outline above, the plan document reads as follows.

```

# Plan: rate limiting for webhooks

## Build a RateLimiter that counts requests per sender (PR 1)

**Depends on**: None

```

```

**Files to change**
- [ ] Create `src/webhooks/rateLimiter.ts`
- [ ] Create `src/webhooks/rateLimiter.test.ts`

**Changes to make**
- [ ] Implement `createRateLimiter` so that it counts requests per sender per minute

**Observable result**
- [ ] When a verification script calls it 61 times as the same sender, it prints
`false` on the 61st call

**Unit tests** (tests alone do not count as verified)
- [ ] Write two cases, "true up to the 60th call and false on the 61st" and "the count resets after
one minute", and run `npm test`

**Live verification**
- [ ] Run the verification script and confirm that the result matches the observable result

**Performance check**
- [ ] Measure the time for 10,000 calls and record the time per call

**User review**: None (the user's flow does not change)

**Merge**
- [ ] Merge only after the unit tests, live verification, and performance check are all checked

## Add per-sender rate limiting (PR 2)

**Depends on**: PR 1 (Build a RateLimiter that counts requests per sender)

**Files to change**
- [ ] Edit `src/webhooks/handleWebhook.ts`

**Changes to make**
- [ ] In `handleWebhook`, call `limiter.tryAcquire(senderId)` and return 429 when the limit is
exceeded

**Observable result**
- [ ] When the same sender sends 61 requests in one minute, the log shows `rate_limited
sender=sender-a` on the 61st

**Unit tests** (tests alone do not count as verified)
- [ ] Add a case "the 61st returns 429" to `handleWebhook.test.ts` and run `npm test`

**Live verification**
- [ ] Send 61 webhooks to the running server and confirm that the 61st gets 429

**Performance check**
- [ ] Compare the processing time per webhook before and after the change

```

****User review****: None (the user's flow does not change)

****Merge****

- [] Merge only after the unit tests, live verification, and performance check are all checked

The real Playbook template asks for more detail. For example, the template splits live verification into 10 angles, and each angle gets a screenshot and a pass condition. There is also a section before the PRs that describes how the whole plan proceeds, and an appendix after them that holds the prototype evidence and similar material.

So the plan document sets the finish evidence and the way to verify it for each PR. The plan runs only after the user says to start.

Key point 12: the plan document is temporary

The plan document is a **temporary document**. It serves two purposes: to carry out the work, and to tell other agents about the progress of a large piece of work.

The guide's view: delete the plan document when you finish, as a rule

poteto writes that in a large project that takes about a week, he sometimes commits the plan document to the repository for a while so that other agents know about the work in progress.

When the work is done, though, he usually deletes the plan document, so that it leaves no confusion in the codebase.

He also writes that he sees no value in a plan document that stays forever.

pstack: the plan document goes in docs/ of the agent store

The "**Multi-phase or multi-PR plan**" Playbook says to **write the plan document in docs/ of the agent store** unless the user names a location.

Agent store. The storage area that the agent uses. The agent's system prompt states where the store is, as the "Orchestrate" Playbook describes.

Key point 13: what to tell /poteto-mode

The human does not list steps for /poteto-mode. The human tells it **the goal, the constraints, the results they want to see, and when to review.**

The guide's view: /poteto-mode alone is usually enough

The guide writes that /poteto-mode automatically uses many of the Skills the guide introduces, so **the human can usually send the request to /poteto-mode alone.**

The section "The workflow in practice" gives an example request for each of four situations.

Example 1: investigating a bug with an unknown cause

The first is the investigation of a bug in production whose cause is unclear.

```
/poteto-mode investigate why background workers periodically fail with timeout errors. give me a breakdown of what we know, what data you used, and your best hypotheses.
```

Besides the goal, this request specifies the shape of the result: what we know, what data was used, and the best hypotheses.

The agent that receives this request investigates the code, checks metrics and past commits in parallel, and returns its best guesses about where the problem is likely to be.

Example 2: asking for a design

The second is a new mechanism that other modules depend on.

poteto asks for a design that adds rate limiting to webhooks that come from outside.

```
/poteto-mode we need to add rate limiting for external webhooks. /architect this first, and answer any open questions with prototypes. let me review before proceeding.
```

Webhook. A mechanism that tells another system about an event by sending an HTTP request to a URL you specify ([Chapter 25](#)).

Besides how to proceed with the design, this request names a place to stop: "let me review before proceeding".

The agent that receives this request investigates the current webhook mechanism, has several models compete on design options, measures performance with throwaway prototypes, and produces a clean, verified interface.

Example 3: planning a migration

The third is a plan for a migration that spans many files. poteto asks for a plan, split into several PRs, to move the whole UI library to [StyleX](#), a library for writing styles inside JavaScript code.

```
/poteto-mode create a plan to migrate our entire UI library to StyleX. break the migration into small, verifiable PRs. each PR must have its visual regression tests and live verification steps. i want the final result to be 100% identical compared to the original - bugs included
```

The request states the goal, which is to move the whole UI library to StyleX, but not the steps of the migration. Besides the goal, it specifies the following three things.

- **How to split the migration.** Into small PRs that the agent can verify one by one.
- **The verification each PR must have.** Visual regression tests and steps to confirm the result in the running app.
- **The condition on the final result (a constraint).** Identical to the original, bugs included.

The agent that receives this request splits the work into independent phases and writes a checklist that a person can review. It prepares each phase so that the phase can move safely through build, verification, and merge.

Example 4: fixing a bug reported in Slack

The fourth is a fix for a bug that someone reported in Slack.

This situation has two requests.

The first request is for when the Slack thread already has enough context.

```
# thread already has sufficient context
/poteto-mode do it
```

The second request specifies the way to reproduce the bug and the evidence.

```
/poteto-mode reproto this with /control-app. if it repros on main, fix it and show me a video as proof
```

Verification skill. A project-specific Skill that lets the agent start and operate the app to collect evidence.

The agent creates it when the user runs `/create-verification-skill`.

`/control-app` is the name the guide uses for the verification skill built in [Part 1](#) of The Complete Guide to pstack ([Chapter 36](#)).

What the four examples share

So every request in the four situations **hands off the detailed steps to** `/poteto-mode` **and gives the goal, plus the following as needed.**

- Constraints
- The results you want to see (the shape of the result)
- When to review (where to stop)

Some requests also name the tool to use, such as `/architect` in [Example 2](#) and `/control-app` in [Example 4](#).

In the `do it` request of [Example 4](#), the goal and the context are in the Slack thread, not in the request.

pstack: name a Skill only when you want to override a choice

The bundled guide [docs/guide/02-poteto-mode.md](#) explains that `/poteto-mode` looks at the kind of request, picks a Playbook, and calls the Skills it needs within the Playbook's steps.

So **the user does not have to write in the request which Skills to use in which order.**

The same page recommends that you name a Skill only in the following case ([Chapter 7](#)).

Name a skill only when you want to override a specific choice.

For a change that adds a new mechanism other modules depend on, as in [Example 2](#), `/poteto-mode` may also pick `/architect` on its own with the following criterion from [key point 9](#) (compare and merge several designs with `/architect`).

It then names **changes that cross a function boundary and changes that move ownership of a responsibility** as the targets of `/architect`. Ownership here means which module is responsible for something.

In this book, I read the `/architect` in the request of [Example 2](#) as a way to make sure the agent designs before it implements, instead of leaving the choice of `/architect` to `/poteto-mode`.

Reading the 13 key points as one flow

Lined up, the 13 key points form the three-stage flow shown in the table in "[Chapters in this part](#)" of [Part VI](#).

1. **Understand the problem** (key points 1 to 4, [Chapter 37](#)). The human gets the agent to understand the problem the way the human does and to carry over the context of past work.
2. **Test solutions** (key points 5 to 8, [Chapter 38](#)). The agent tests solutions with documents that describe the usage and with prototypes, and the human chooses a design option.

3. **Settle the solution and turn it into an executable plan** (key points 9 to 13, this chapter). The agent compares several options to settle the design, revises the design based on evidence from implementation, and turns the design into a plan it can verify. The plan document is temporary, and poteto says he usually deletes it when the work is done. The human tells /poteto-mode, the entry point, the goal, the constraints, the results they want to see, and when to review, not the steps.

In this book, I read what the three stages share as this: **base decisions on evidence from investigation and on the results of running things, not on a plan made only of prose.**

Specifically, decisions rest on the following two kinds of evidence.

- Evidence from investigating the code and its history (Key point 3: learn the mechanism and the reasons with /how, /why, and /teach and Key point 4: carry past work into a new conversation with /recall)
- Evidence from building and running things, such as the results of operating a prototype, measurements, and the same workaround showing up in unrelated places during implementation (Key point 7: compare prototypes of several options, Key point 8: have the agent answer questions with experiments, and Key point 10: when implementation shows the design is wrong, revise the design)

In its last section, "The art of planning", the guide also writes that **an abstract plan gives the human only the illusion that work is moving forward**. An abstract plan is a plan made only of prose that has not yet taken shape in code.

It also writes that a long plan document makes the human and the agent look as if they worked hard, but probably has little behind it.

The guide then says that pstack **is a tool that combines the following three things**.

- Thorough investigation
- Evidence from trying things
- Rigorous verification

It closes the section with these sentences.

When you plan this way, engineering with agents stops feeling like a gamble. It becomes predictable and repeatable.

Summary

- **Key point 9: compare and merge several designs with /architect.** In a large change, the human spends time on the architecture, the data structures, and how systems connect. The agent spends its time on the details of the implementation. With /architect, the agent first compares several design options, merges them into one, and only then implements.
- **Key point 10: when implementation shows the design is wrong, revise the design.** The agent may find a mismatch with the skeleton, or see the same workaround or type escape hatch keep showing up in unrelated places. It treats either one as evidence that the design is wrong and revises the design. If needed, it throws the skeleton away and redoes the design from the first stage, the investigation of the problem.
- **Key point 11: split the work into small tasks you can verify.** With the planning Playbook, the agent splits the work into small PRs that it can verify one by one. For each PR, the finish evidence includes the results of running the code, in addition to tests. The plan runs only after the user says to start.
- **Key point 12: the plan document is temporary.** The agent uses the plan document as a temporary document to carry out the work and to tell other agents about the progress of a large piece of work. poteto usually deletes it when the work is done.
- **Key point 13: what to tell /poteto-mode.** The human tells /poteto-mode the goal, the constraints, the results they want to see, and when to review, not the steps.
- **The flow of the 13 key points.** In this book, I read what every stage shares as this: base decisions on evidence from investigation and on the results of running things, not on a plan made only of prose.

This is the end of [Part VI](#).

The [Appendix](#) that follows introduces RGDD (README-driven with Grill Driven Development), a way of developing that puts the contents of this book into practice.

RGDD is a way of developing that I came up with. You place the context that the agent keeps reading, such as the README and the glossary, before the code.

The final "[Afterword](#)" gives the conclusion of this book and the first step to take after you finish reading.

Appendix: README-driven with Grill Driven Development (RGDD)

kaito • Sources: 日本語 • English • 2026-10-03

RGDD (README-driven with Grill Driven Development) is a way of developing that I came up with when I used the ideas in this book and pstack on my personal projects.

In RGDD, the developer puts **the context the agent reads into the codebase before the code**, and develops while the agent reads that context.

Context is the set of files, mostly in the repository, that the agent reads again each time it works.

It stays after a conversation ends.

In [Chapter 4](#), I treated the codebase as the agent's memory and wrote the following.

The memory that lets the agent carry forward earlier decisions belongs in the **codebase**, not in the model.

Besides code, RGDD uses the following documents as context.

- **README.** A document that says who uses the software, for what, and how. Usage instructions and a tutorial are examples.
- **GLOSSARY.md.** A document that lists domain terms and their definitions, such as what "order" means. A domain term is a word for the business or the things the software deals with.
- **ADR (Architecture Decision Record).** A record of a design decision and the reason for it, such as why the team chose a particular database.
- **Coding standards.** The rules for how to write code, such as the rules in `CODING_STANDARDS.md`.

- **Feature Map and verification skill.** A verification skill is a project-specific Skill that lets the agent start and operate the app to collect evidence. The Feature Map is a document inside that skill. For each feature of the app, the Feature Map says how to reach the feature and how to verify it ([Chapter 36](#)).

Of these, the developer puts the README, GLOSSARY.md, and ADRs in place before the code. The developer has the agent create the Feature Map and the verification skill after the first feature works.

This appendix explains the ideas behind RGDD and how to carry it out.

How this appendix is organized

This appendix covers the following topics.

- RGDD builds context with README-driven development and /grill-with-docs, then uses it to develop and verify
- RGDD adds to the context and verifies and updates it in five phases
- The context keeps growing because each phase reads the previous phase's context and adds to it
- Context comes first so that the examples the agent reads are right from the start
- Summary: put the context before the code

The context grows means that each piece of work adds terms and decisions to the context files, and that the developer and the agent keep verifying, correcting, and updating the content already there.

RGDD builds context with README-driven development and /grill-with-docs, then uses it to develop and verify

In RGDD, the developer combines README-driven development with Matt Pocock's Skill [/grill-with-docs](#) to build context. The developer then uses that context to develop and verify.

README-driven development is a way of developing that starts with a README that explains usage to the intended users, then works backward to the implementation and the architecture.

I explain README-driven development in "1. Write the user's view of the app in the README" and [/grill-with-docs](#) in "2. Create domain terms and ADRs with /grill-with-docs".

RGDD rests on the idea from Chapter 4, quoted at the start, that the codebase is the agent's memory.

poteto writes the following in the section "Keep agents smart with Feature Maps" of The Complete Guide to pstack [Part 1](#).

Personally, I think your codebase is the ultimate form of memory. Code is a projection of the decision making you and your team have made and represents the source of truth for what's happened and how things actually work.

In the same section, poteto describes the Feature Map this way.

A Feature Map is just a more compact form of that, designed to save tokens.

A Feature Map holds the following.

- What features exist and what each one does
- How a user reaches each feature
- How to operate each feature with the verification skill

- What you need to see to say that the feature works

RGDD widens what the context holds. The README holds "who uses it, for what, and how", GLOSSARY.md holds "which words to use", and the ADRs hold "why we decided this way".

The table below lists the five kinds of context, what each holds, and who or what creates it.

Context	What it holds	Created by
README (or a document that plays the README's role)	Who uses it, for what, and how	The developer writes it
GLOSSARY.md	Domain terms, their definitions, and avoided terms	/grill-with-docs
ADR (docs/adr/)	Important design decisions, with their background and reasons	/grill-with-docs
Coding standards (for example, CODING_STANDARDS.md)	How to write code	The developer decides and writes them for each repository
Feature Map and verification skill	The list of features, how a user reaches each one, and how to verify it	/create-verification-skill

- **Domain term.** A word for the business or the things the software deals with, such as "order" and "cancellation" in a food-ordering app.
- **Avoided terms.** Other words for the same thing that the team decided not to use (for example, if "order" is the chosen term, "purchase" and "transaction" are avoided terms). In GLOSSARY.md, they go under the term's definition.

The five phases do not create the coding standards. The coding standards are rules the agent follows when it writes code with `/poteto-mode`. So I leave them out of the five phases in the next section. `/poteto-mode` is pstack's entry Skill, which routes each request to the right way of working (Chapter 7).

Also, **the README does not have to be named** README.md.

A specification or another document can play the README's role, as long as it describes usage from the user's side and the other context starts from it.

RGDD adds to the context and verifies and updates it in five phases

The developer carries out RGDD in these five phases.

1. Write the user's view of the app in the README
2. Create domain terms and ADRs with `/grill-with-docs`
3. Develop with `/poteto-mode`
4. Review with `/thermos`
5. Verify with `/create-verification-skill`

Each phase does one of two jobs. It **adds to the context, or it verifies what the context holds and corrects or updates it.**

For each phase below, I explain what it creates, the tool it uses, and the context it leaves behind.

1. Write the user's view of the app in the README

- **What it creates.** A README that gives usage instructions and a tutorial from the point of view of the app's user. It does not cover the implementation yet.
- **Tool.** In RGDD, the developer writes the README by hand, without a Skill, a CLI, or any other tool. [Key point 5](#) (write the usage first) of [Chapter 38](#) showed a different approach, in which the agent writes the README.

Before the implementation, the human has the agent **write the usage in a README or a tutorial**, and has it work backward from that usage to the API and the internals.

- **Context it leaves.** Context about how people use the app, such as "who uses it, for what, and how." The other context starts from it.

Example: a README for a food-ordering app

The README describes only what the user sees, not the implementation.

```
# Order Ahead
```

```
An app for ordering food from nearby restaurants on your smartphone.
```

```
## What you can do
```

- Choose dishes from the menu and place an order
- Check the status of an order (received, cooking, ready for pickup)
- Cancel an order before you pick it up

```
## How to use it
```

1. Open the app and choose a restaurant
2. Choose dishes from the menu and tap "Place order"
3. When the status changes to "Ready for pickup," pick up the order at the restaurant

RGDD puts this phase first **to prepare the material that /grill-with-docs asks questions about in the next phase.**

When the README describes usage from the user's side, /grill-with-docs can ask concrete questions about the vague parts of that usage. The developer can then give concrete answers.

For example, the README above says "Cancel an order before you pick it up." From this, /grill-with-docs can ask about something the README leaves open, such as "Can the customer cancel an order that is already cooking?"

2. Create domain terms and ADRs with /grill-with-docs

- **What it creates.** A GLOSSARY.md that lists the domain terms, and ADRs that record design decisions.
- **Tool.** Matt Pocock's /grill-with-docs. The agent does not pick this Skill on its own, so the developer starts it by name.
- **Context it leaves.** Background knowledge, such as domain terms and their definitions (GLOSSARY.md), and past decisions, such as design decisions and their reasons (ADRs).

`/grill-with-docs` is a **Skill that asks the developer question after question to settle a plan or a design, and creates ADRs and a glossary (`GLOSSARY.md`) along the way.**

`/grill-with-docs` calls these two Skills as part of its steps.

- `/grilling`. It asks the developer question after question about the plan or the design and settles what needs a decision.
- `/domain-modeling`. It points out conflicting terms and vague wording, and writes each settled term to `GLOSSARY.md` right away. It offers to record a decision as an ADR only when all three of these are true.
 - Hard to reverse
 - Surprising without context
 - The result of a real trade-off

The following example uses the food-ordering app to explain what each of the three conditions for an ADR means.

Example: the three conditions for an ADR

The "Meaning" column quotes the section "When to offer an ADR" in `ADR-FORMAT.md` of `/domain-modeling`. I wrote the "Example" column for this book. It applies each condition to one decision about the food-ordering app, the README example in [phase 1](#): "store each restaurant's order data in a separate database."

Condition	Meaning (<code>ADR-FORMAT.md</code>)	Example
Hard to reverse	The cost of changing your mind later is meaningful	To merge into one database later, you have to move every restaurant's data
Surprising without context	A future reader will look at the code and wonder "why on earth did they do it this way?"	The data is all order data, yet each restaurant stores it in a different place
The result of a real trade-off	There were genuine alternatives and you picked one for specific reasons	Compared with one shared database, separate databases keep a failure confined to one restaurant

A decision that misses even one of these conditions does not become an ADR.

For example, the decision "list orders newest first" is easy to reverse, and nobody will wonder why, so it does not become an ADR. `ADR-FORMAT.md` gives the reason as follows.

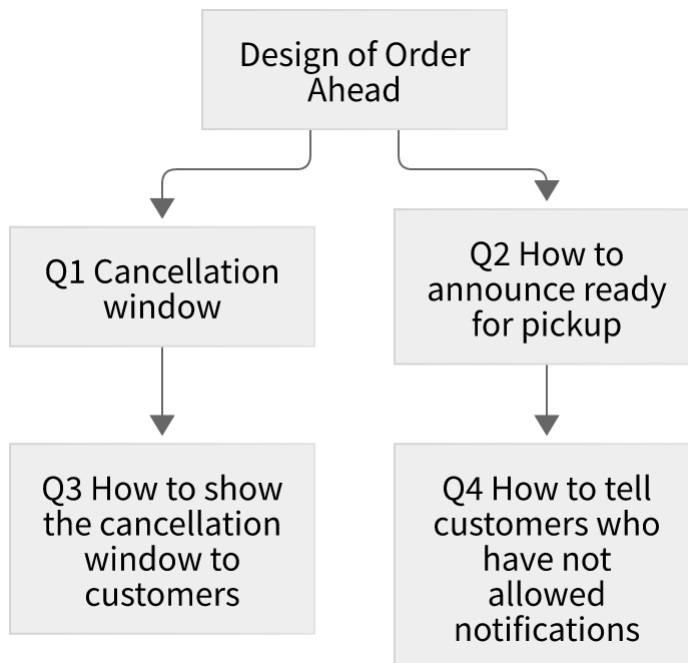
If a decision is easy to reverse, skip it: you'll just reverse it. If it's not surprising, nobody will wonder why. If there was no real alternative, there's nothing to record beyond "we did the obvious thing."

`/grilling` orders the decisions so that each comes after the decisions it depends on. It then asks, in one round, all the questions the developer can answer now. The following example shows how `/grilling` proceeds.

Example: how `/grilling` asks its questions

`/grilling` arranges the decisions as a design tree. Each branch is a decision that becomes possible only after its parent decision is made.

For Order Ahead, the README example in [phase 1](#), the tree of decisions looks like this. I made this example for this book.



[View diagram 1](#)

For example, Q3, "how to show the cancellation window to customers," has no answer until Q1, "the cancellation window," is decided. The display depends on whether the window is "only while received" or "until pickup." In the same way, Q4, "how to tell customers who have not allowed notifications," comes up only after Q2 decides "announce it with a push notification."

/grilling asks the questions in this tree over several rounds. Each round contains **only questions whose parent decisions are all settled**. The skill calls this set of questions the frontier and describes it as follows.

The **frontier** is every decision whose prerequisites are already settled: the questions you can ask now without guessing at answers you haven't heard yet.

In the tree above, the first round is Q1 and Q2, which need no parent decision. The agent asks Q1 and Q2 at once, each with a number and a recommended answer.

When the developer answers Q1 and Q2, Q3 and Q4 become answerable. The agent asks Q3 and Q4 as the second round.

An example of the first round

This is the first round of the tree above (Q1 and Q2), written in the format that the `SKILL.md` of /grilling shows: number each question and add a recommended answer.

```
? **Q1** - **Cancellation window**: The README says "Cancel an order before you pick it up." Can the customer cancel an order that is already cooking?

➡ Allow cancellation only for orders in "received" status (food that is already cooking would go to waste)

---

? **Q2** - **How to announce ready for pickup**: How do we tell the customer that the status has changed to "ready for pickup"?

➡ Send a push notification from the app
```

When the developer answers Q1 with "received only," the definition of the term "cancellation" is settled. /domain-modeling then writes that definition to `GLOSSARY.md`.

In this phase, **the agent can write the domain terms it settles with the developer to a file, `GLOSSARY.md`, instead of leaving them only in the conversation.**

Terms decided in a conversation do not carry over to the next agent once the conversation ends.

Terms written to `GLOSSARY.md`, on the other hand, are there for the next agent to read.

While the developer and the agent settle the design with `/grill-with-docs`, the agent also compares the developer's words with the term definitions in `GLOSSARY.md`.

When the developer uses wording that conflicts with a definition, the agent points out the conflict immediately. The section "Challenge against the glossary" in the `SKILL.md` of `/domain-modeling` sets this rule. In the quote, "the user" is the person who uses the Skill, which in RGDD is the developer.

When the user uses a term that conflicts with the existing language in `GLOSSARY.md`, call it out immediately.

The `_Avoid_` list in `GLOSSARY.md` is what helps the agent spot a conflict. The section "Rules" of the `GLOSSARY-FORMAT.md` of `/domain-modeling` says that when several words name the same thing, you pick the best one as the term and list the others as avoided terms.

When multiple words exist for the same concept, pick the best one and list the others under `_Avoid_`.

For example, suppose `GLOSSARY.md` defines "order" and lists "purchase" as an avoided term. This example comes from `GLOSSARY-FORMAT.md`.

When the developer says "purchase," the agent points it out as wording that conflicts with the definition.

The `GLOSSARY.md` and the ADRs that this phase creates are short documents like the following.

Example: `GLOSSARY.md`

It follows the format that `GLOSSARY-FORMAT.md` of `/domain-modeling` shows. Each term gets a short definition and its avoided terms.

```
# Ordering

Covers a customer ordering food from a restaurant, through pickup.

## Language

**Order**:
A request for food that a customer places with a restaurant.
_Avoid_: purchase, transaction

**Cancellation**:
A customer calling off an order that is in "received" status. An order that is cooking cannot be cancelled.
_Avoid_: withdrawal
```

Example: an ADR (`docs/adr/0001-separate-database-per-store.md`)

It follows the format that `ADR-FORMAT.md` of `/domain-modeling` shows. An ADR is a short document that says what was decided and why. One paragraph is enough.

```
# Store order data in a separate database for each restaurant

We store each restaurant's order data in its own database, so that a failure at one restaurant does not affect orders at the others. We also considered a single shared database, but rejected it because a failure there would spread to every restaurant.
```

This ADR records the decision that the table in the collapsible section "Example: the three conditions for an ADR" checked against the three conditions.

3. Develop with `/poteto-mode`

- **What it creates.** The code.
- **Tool.** The developer uses `/poteto-mode` for development ([Chapter 7](#)). When the developer wants to know how the code works or how it got this way, the developer uses these Skills ([Chapter 22](#)).
 - `/how` looks into how something works
 - `/why` looks into the reasons behind a design

- `/recall` gathers the history of recent work from chat history and other sources
- **Context it leaves.** The code itself, and PRs. The agent creates the PR in the last step of a Playbook, such as the one for adding a feature or fixing a bug. A Playbook is a step-by-step procedure for one kind of work ([Chapter 8](#)).

In the development phase, **the agent writes code while it reads the context that earlier phases created, and the coding standards if the developer has prepared them.**

For light or routine work, such as feedback, QA, and bug fixes, I have Grok Bot start Cloud Agents to do the work. Grok Bot is an AI bot app ([Chapter 5](#)). Cloud Agents is the Cursor feature that runs agents in Cursor's cloud.

Like poteto, I do not have Grok Bot do the work itself. I have it start Cloud Agents.

In [Chapter 36](#), I described how poteto works as follows.

When he sends the request from a Grok Bot, poteto does not have the bot do the work itself. He asks the bot to start an agent in Cloud Agents, a Cursor feature that runs agents in Cursor's cloud.

The README, GLOSSARY.md, ADRs, and coding standards live in the repository, so every agent that works on the repository can read them, Cloud Agents included.

The ADRs from [phase 2](#) also help during development, when the developer asks `/why` for the reason behind a decision.

`/why` has a file for each kind of source it searches, and each file says how to search that source.

The file for source control history includes a step that looks for ADRs in docs/adr/ and similar places.

So if an ADR states the reason, `/why` can base its answer on that ADR.

Even for a decision without an ADR, `/why` puts together a reason. It searches source control history, tickets, chat, and other sources in parallel, and uses the clues left there. For

tickets and chat, it searches the ones connected through MCP, the mechanism that connects the agent to external tools and data.

If `/why` still finds no clues, it reports what it searched and says that the reason is Unknown.

4. Review with `/thermos`

- **What it creates.** A list of review findings.
- **Tool.** The Cursor plugin `/thermos`.
- **Context it leaves.** The code after the agent fixes the findings. **The steps of `/thermos` end when it merges the findings into one list.** They do not include code fixes.

`/thermos` has two subagents audit the diff of the code the agent wrote in [phase 3](#). The two subagents work in parallel.

A **subagent** is a separate agent that does work that the main agent, the agent that runs `/thermos`, hands off.

The two audits are as follows.

- **A security and correctness audit**, for problems such as bugs, changes that break existing features, and security holes
- **A strict code quality audit**, for problems such as code that is hard to maintain, bloated files, and unneeded abstractions

When both audits finish, `/thermos` removes duplicates and merges the results into one list.

When both subagents point out the same problem, `/thermos` treats that finding as more important.

RGDD has this phase **to remove ad hoc workarounds and hard-to-maintain structures before anyone merges them and they stay in the code**. A workaround is code that avoids a problem for the moment without fixing its cause. If a workaround stays in the code, the next agent takes it for a correct implementation example and copies it.

[Chapter 4](#) puts it this way.

You should not keep everything as memory. If you keep an ad hoc workaround, **the next agent takes it for a correct implementation and copies it. The copy then produces the next copy, and an undesirable pattern spreads without anyone noticing.**

5. Verify with `/create-verification-skill`

- **What it creates.** A project-specific verification skill that includes a small CLI that operates the app and a Feature Map (Chapter 36).
- **Tool.** pstack's `/create-verification-skill`, and `/maintain-verification-skill`, which keeps the verification skill and the Feature Map up to date (Chapter 26).
- **Context it leaves.** Context about what features exist, how a user reaches each one, and how to verify it.

`/create-verification-skill` builds the Feature Map from an index, `features/README.md`, and one file per feature.

The index links to each feature file. Each feature file describes these four things from the user's point of view.

- What the feature is
- How to reach it
- How to operate it with the verification skill's CLI
- What you need to see to say that it works

Example: a feature file (an excerpt from pstack's sample)

This is the file for the "Create a note" feature of a notes app, one of the samples in [references/feature-map-example/](#) of `/create-verification-skill`, with some parts left out.

```
# Create a note
```

```
Create note lets a user save a titled note from the browser or CLI, cancel an unfinished draft, and confirm the saved note from a second user-facing view.
```

```
## Sub-features
```

```
- `create-open` opens a blank editor from each browser entry point.
```



```

- `create-save` persists a title and body.

## How to get to it (user POV)

- Choose the `New note` button in the browser toolbar.
- Press `n` in the browser while focus is outside an editable field.
- Run `notes create --title <title> --body <body>` in a terminal.

## Driving it with control-notes

- **Open editor.** Choose `New note`. Run `control-notes browser click --role button --name "New note"`. A form named `Note editor` appears with focus in the `Title` textbox.
- **Enter content.** Type the title and body. Run `control-notes browser fill --role textbox --name "Title" --value "Release checklist"` and `control-notes browser fill --role textbox --name "Body" --value "Tag and publish"`. The `Save note` button becomes enabled.
- **Save note.** Choose `Save note`. Run `control-notes browser click --role button --name "Save note"`. A status named `Note saved` appears and the heading reads `Release checklist`.

## Gotchas

- Pressing `n` while a textbox has focus types the character instead of opening a new editor.

```

The description paragraph under the title is "what the feature is." How to get to it (user POV) is "how to reach it." Driving it with control-notes covers both "how to operate it" and "what you need to see to say that it works."

control-notes is the CLI that this sample verification skill uses to operate the app. Sub-features lists the smaller features, one per line, each with a short ID. Gotchas lists traps that waste verification work or make the results invalid. Every feature file uses the same four English headings in the same order.

The Feature Map index, features/README.md, shares its name with the README from [phase 1](#), but it has a different role.

The README from [phase 1](#) is a document in which the developer describes how to use something that is not built yet.

The Feature Map's features/README.md, on the other hand, is an index of the finished app that the agent creates by following the steps of /create-verification-skill.

The Complete Guide to pstack [Part 1](#) calls the verification skill the most important skill and lists it as the first thing to build ([Chapter 36](#)). **RGDD takes the same view.**

When you build an app from scratch, though, there is no app to run and verify yet, so RGDD puts this phase last.

If you already have an app, this book strongly recommends that you create a verification skill and a Feature Map with `/create-verification-skill`.

The `SKILL.md` of `/create-verification-skill` also assumes a working app. If the app cannot build or start as it is, this `SKILL.md` tells the agent to fix the build or startup before it creates the verification skill, or to report the state accurately. The section "1. Interview the repo, not the user" has this rule.

So in RGDD, **once the first feature works**, the developer runs `/create-verification-skill` and has the agent create the verification skill.

From then on, the agent uses that verification skill to verify its changes in every task. The developer keeps the verification skill and the Feature Map up to date with `/maintain-verification-skill`.

As The Complete Guide to pstack [Part 1](#) recommends, I run `/maintain-verification-skill` at least once a day. I hand off the runs to a maintainer bot in Grok Bot.

As Part 1 also recommends, I had [Dr Eggbot \(Chapter 36\)](#) build this bot.

I recommend running `/maintain-verification-skill` at least once a day to ensure that your agents always have the latest details on controlling your app.

The context keeps growing because each phase reads the previous phase's context and adds to it

The context keeps growing because each RGDD phase **reads the context the previous phase left, then adds to the context, or verifies what the context holds and corrects or updates it.**

Phase	What it reads	What it adds to the context, or verifies and corrects or updates
1. README	Nothing	Who uses it, for what, and how
2. <code>/grill-with-docs</code>	README	Terms and decisions (ADRs)
3. <code>/poteto-mode</code>	README, GLOSSARY.md, ADRs, coding standards	Code and PRs
4. <code>/thermos</code>	The diff of the code written in phase 3	Problems in the diff (the agent fixes them before they stay in the code)
5. <code>/create-verification-skill</code>	The working app	The Feature Map and how to verify each feature

As development goes on, the developer and the agent keep adding to the context and verifying, correcting, and updating it, as follows.

- **When a new term or decision comes up.** The developer settles the design again with `/grill-with-docs`. The agent adds the term to GLOSSARY.md and offers an ADR for any decision that meets all three of these.
 - Hard to reverse
 - Surprising without context
 - The result of a real trade-off
- **When a review finds a problem in the diff.** The agent fixes the `/thermos` findings before anyone merges the code.
- **When features are added.** The developer runs `/maintain-verification-skill`. If it finds something to fix, such as a feature missing from the Feature Map, the agent puts the fixes into one PR.

The main aim of working this way is **to keep leaving the right context for the agent, past context included**.

This aim comes from the "gardener" idea in [Chapter 4](#). Chapter 4 described the work to keep the codebase as memory this way.

The talk compares the work of keeping this environment in shape to a **gardener's** work. A garden does not stay healthy if you only add plants. You have to keep removing unneeded branches and weeds and keep maintaining it so that the plants you want can grow.

The same is true of a codebase. **You have to keep removing old workarounds and bad implementation examples, in addition to adding knowledge.**

In RGDD, the developer and the agent do this gardener's work on the README, GLOSSARY.md, ADRs, and the Feature Map, in addition to the code.

- **Add.** Add newly settled terms and decisions to GLOSSARY.md and the ADRs. In the garden, this step is the planting of what you want to grow.
- **Verify, then correct or update.** Check term definitions that have gone stale and a Feature Map that no longer matches the app, and fix them. In the garden, this step is the removal of unneeded branches and weeds.

ADRs do most of the work to keep past context. An ADR records a decision and its reason. Even if you change the decision later, the next agent can read why the team first decided the earlier way.

The `ADR-FORMAT.md` of `/domain-modeling` allows a Status at the top of an ADR, for when the team revisits a decision. There are four statuses: proposed, accepted, deprecated, and superseded by ADR-NNNN.

Because a "superseded by" status exists, this book reads the format to mean that when you change a decision, you keep the old ADR and add a new one.

Example: ADRs after a decision changes

This example assumes that the decision in ADR 0001, the ADR example from [phase 2](#), changes later. The statuses follow the four that `ADR-FORMAT.md` lists.

Keep the old ADR and change only its status to "superseded by 0002."

`docs/adr/0001-separate-database-per-store.md`

```
---
status: superseded by ADR-0002
```

```
---
```

```
# Store order data in a separate database for each restaurant
```

We store each restaurant's order data in its own database, so that a failure at one restaurant does not affect orders at the others. We also considered a single shared database, but rejected it because a failure there would spread to every restaurant.

The new ADR says what was decided again, and why.

`docs/adr/0002-single-database-for-orders.md`

```
---
```

```
status: accepted
```

```
---
```

```
# Store all order data in one database
```

We now have more than 300 restaurants and can no longer operate a database for each one, so we move all order data into one database. To limit the problem 0001 tried to avoid, a failure spreading to every restaurant, we replicate the database to several locations.

The next agent learns the current decision from 0002, and learns from 0001 why each restaurant used to have its own database.

Context comes first so that the examples the agent reads are right from the start

When you keep context, you can pass what you decided on to the next agent.

You put that context in place before the code **so that the examples the agent reads are right from the start**.

In this book, I take the view that if you start with the code, the opposite of RGDD, the only way to create context is to derive it from the code afterward.

Then the context may keep even ad hoc workarounds that got into the code, as if they were correct.

A workaround left in the code becomes an example for the next agent ([phase 4](#)).

Summary: put the context before the code

- **What is RGDD?** A way of developing in which you put the README, GLOSSARY.md, and ADRs in place before the code, and develop while the agent reads them.
- **In what order does it proceed?** In five phases: README, then /grill-with-docs, then /poteto-mode, then /thermos, then /create-verification-skill.
- **Why does the context keep growing?** Because every phase either adds to the context or verifies it and corrects or updates it.
- **Why does the context come first?** So that the examples the agent reads are right from the start.

The final "Afterword" gives the book's conclusion and the first step to take after you finish reading.

Afterword

kaito • Sources: 日本語 • English • 2026-10-03

The conclusion of this book: the scope you can delegate depends on how wide the environment for verification is

The book's reading of pstack ends here.

So far, this book has used poteto's talk and guide as its starting point and read pstack's Playbooks, Principles, and Skills one by one. The conclusion this book reached is the following sentence.

"The scope of work you can delegate to the agent depends not on how smart the agent is, but on how wide the environment is in which the agent can verify its own work"

Put another way, **the wider the range in which the agent can start the app on its own, operate it, and keep the result as evidence, the more work you can hand off without running it yourself to verify it.** That is the conclusion I want to leave you with.

This conclusion starts from poteto's [talk \(Chapter 1\)](#) and The Complete Guide to pstack [Part 1](#). **Both consistently put verification first.**

The talk said that **what you trust is the artifact, not the agent, and that a way to verify the artifact is the precondition for trust.** In the section "Verification is all you need", Part 1 named **a high-quality verification skill as the most important skill.**

The first step is to run one verification loop on a small task

pstack has 23 Playbooks, 23 Principles, and 24 Skills, but **you do not need to master all of them starting tomorrow.**

At first, **one verification loop on a small task is enough.**

- **Verification skill.** A project-specific Skill that the agent uses to start and operate the app and collect evidence.
- **Verification loop.** The full cycle of one request. The user writes a request with a finish condition that gives a pass or fail result. The agent verifies the change in the real app, fixes the change until it meets the condition, and returns the evidence.

The rest of this afterword gives a first step for after you finish reading: four steps you can do in one week, and the order in which you then widen the scope you delegate.

The one-week time frame and the structure of four steps are this book's proposal.

That said, the content of [step 1](#) (pick a real task) and [step 2](#) (write the finish condition so it gives a pass or fail result) also appears in pstack's bundled guide ([01-setup.md](#), [06-verify-and-ship.md](#)).

Four steps you can do in one week

1. Pick one real task that does no harm if it fails

Pick a first task that is "small, harmless if it fails, but needed".

Good examples are a fix for a small display bug, a new CLI option, and a fix for how the app reads a setting. Each one is a small change, and when you run it you can see right away whether the result is correct.

Avoid practice exercises and pick real work. The purpose of this week is to verify that the verification loop works in the real app.

2. Write the finish condition so it gives a pass or fail result

Before you write the request, decide the finish condition: "what you need to confirm before the work is done".

Write the finish condition so that running it gives a pass or fail result, not as "make it nice".

In the bundled guide [06-verify-and-ship.md](#), the section "State the finish condition up front" uses a task that adds JSON output to a CLI as its example. It writes the finish condition into the request like this.

```
/poteto-mode add json output to this command. text output stays byte-identical, the json parses,
both run against the sample project. show me the evidence.
```

Each of the three conditions in this request gives a pass or fail result when you run it.

If you use pstack, put the conditions into the request to [/poteto-mode](#) ([Chapter 7](#)), pstack's entry point, as they are.

If you do not use pstack, you can write a request to any agent the same way.

3. Ask the agent to verify the change in the real app itself

This is the most important step.

Do not ask the agent to report whether the tests passed. Ask it to start the app, run the changed feature, and **show the result as evidence**. Passing tests do not mean the real screen or output is correct.

For example, The Complete Guide to pstack [Part 1](#) writes a request to build a feature like this ([Chapter 36](#)).

```
/poteto-mode build <description of feature, any useful context>. use /control-app to verify your
changes and show me a video and screenshots as proof
```

`/control-app` is the name the guide uses for a verification skill made with `/create-verification-skill`.

This request adds two things to the description of the feature: which verification skill to verify with (`/control-app`), and the form of the evidence (a video and screenshots).

Evidence includes the following.

- The output of a command
- A screenshot of the screen
- The result of reading a saved value back

If the agent cannot verify on its own, **remove the cause first**. As long as the agent cannot verify, only you can verify the result, and the scope you delegate does not grow.

In this book, I list **three causes that stop the agent before it can verify**.

- The agent does not know how to start the app
- The agent has no way to operate the app
- The agent has no test users or test data

If you write down which of the three stopped the agent, you know what to set up next.

Two of these causes also appear in the Part 1 section "Make it Reproducible": how to start the app, and test users and data.

In this section, Part 1 recommends that you start to think about the following three things as part of the app's developer experience.

seeding a dev database

how to handle auth, test users, API calls against a test/staging environment

installing and bringing up your dev environment in a consistent way

A verification skill ([Chapter 26](#), [Chapter 36](#)) exists to remove the situations in which the agent cannot verify its own work. Chapter 26 gave the reason to set up a verification skill as follows.

The user sets up a verification skill first for the following reason. **If the agent cannot verify its own work, a human ends up verifying that work piece by piece at the end, no matter how much work you delegate to the agent. The human then becomes the one who stops the flow of work (the bottleneck).**

If you use pstack, run `/create-verification-skill`, which creates a verification skill, once.

4. When you give the same correction twice, keep it in a mechanism instead of in text

If you give the same correction twice in one week, think about whether a tool can detect the mistake automatically. If a tool can, turn the correction into a mechanism.

This book borrowed this idea from a pstack principle.

The principle "**Encode Lessons in Structure**" (Chapter 21) defines what the agent does when it notices that it has written the same instruction twice.

The agent considers whether it can turn that instruction into one of the following. If it can, it builds the mechanism and deletes the original instruction.

- A lint rule
- A metadata flag, which is a value in a file's settings field
- A runtime check, which tests while the program runs whether a value or state meets a condition
- A script

The principle gives this rule because **a text instruction works only if the reader notices it, remembers it, and follows it.**

The principle's text says the following.

Textual instructions are easy to miss. They require the reader to notice, remember, and comply.

On the user's side, lint rules, CI checks, and scripts are the mechanisms that replace text instructions.

For example, if you have told the agent twice not to use `any`, add a lint rule that bans `any`.

`eslint.config.js`

```
// Lint reports an error when code uses any (assumes typescript-eslint is installed)
import tseslint from "typescript-eslint";

export default [
  {
    files: ["src/**/*.ts"],
    languageOptions: { parser: tseslint.parser },
    plugins: { "@typescript-eslint": tseslint.plugin },
    rules: {
      "@typescript-eslint/no-explicit-any": "error",
    },
  },
];
```

Once the correction is a lint or CI check, **from the next week on, the same mistake stops automatically, even if you do not point it out and the agent does not remember it.**

Widen the scope you delegate step by step

When you finish the [four steps you can do in one week](#), you have one area of work where you can say: "for this kind of work, I do not need to verify the diff and the behavior myself every time. I can read the evidence the agent produced".

Add these areas one kind of work at a time.

Within one kind of work, you also widen the scope you delegate phase by phase.

In this book, I divide those phases into the following four.

Delegation phase	What the user does	What you delegate to the agent
1	Verifies the diff and the behavior every time	Writing the code

Delegation phase	What the user does	What you delegate to the agent
2	Reads the evidence the agent produced	Writing the code, running it, and producing evidence
3	Writes a request with a finish condition and looks only at the finished PR and the evidence	Investigating, fixing, verifying, and opening a PR by a fixed procedure for each kind of work (a Playbook, in pstack)
4	Sets the direction and approves irreversible operations, such as deploys and data deletion	Running several tasks in parallel and merging verified changes

A kind of work for which you have finished the [four steps](#) has reached phase 2 in this table, in which you read the evidence the agent produced.

Do not skip phases. Move up in order, starting from 1.

If you move to phase 4 while you still cannot trust the evidence the agent produces in phase 2, all you get is more changes in parallel that nobody can verify.

The "2,500 PRs a month" from the talk also came after the team set up the environment in order, starting with verification of the artifact. [Chapter 1](#) put it like this.

In other words, you have to keep this order. First make the artifacts verifiable, then record the knowledge you gain there in the codebase, and then build automation on that base.

The number 2,500 a month is the result of putting these four in place in order.

The four in the quote are also the four themes of the talk: trust, ways to raise trust, the codebase as memory, and automation.

Final words

pstack is still changing.

This book covers pstack version 0.15.5 as of September 2026.

The names and counts of Playbooks, Principles, and Skills, and the default model, which the agent uses when the user does not choose one, will keep changing.

Even so, in this book I hold that **the value of the direction "first build an environment in which the agent can verify its own work" does not drop as models get smarter. The smarter the model, the larger the work you delegate, and the more you need a mechanism to verify it.**

On top of that, even a smart model sometimes states things with confidence without verifying them.

The Complete Guide to pstack [Part 2](#), in the section "Building up a mental model", wrote the following about the latest models.

Even with the latest frontier models, (this also depends on the quality of the harness), in general I find that they still often state things confidently without backing it up with data or actually reading the code needed to build up a mental model of how it works.

Start by running one verification loop on one small task.

From there, the scope you can delegate grows little by little.

Thank you for reading to the end.

I hope this book is useful to you and makes your development work a little better.

Have a good pstack life!

References

<https://x.com/poteto>

<https://x.com/poteto/status/2102050467505430555>

<https://x.com/poteto/status/2094457600259842065>

<https://x.com/poteto/status/2097732320606507506>

<https://github.com/cursor/plugins/tree/main/pstack>

<https://github.com/cursor/plugins/blob/main/pstack/README.md>

<https://github.com/cursor/plugins>

<https://github.com/cursor/plugins/blob/main/pstack/docs/guide/README.md>

<https://github.com/cursor/plugins/blob/main/pstack/docs/guide/06-verify-and-ship.md>

<https://github.com/cursor/plugins/blob/main/pstack/docs/guide/02-poteto-mode.md>

<https://github.com/cursor/plugins/blob/main/pstack/docs/guide/01-setup.md>

<https://github.com/cursor/plugins/blob/main/pstack/docs/guide/07-overnight.md>

<https://github.com/cursor/plugins/blob/main/pstack/docs/guide/10-recipes-and-pitfalls.md>

<https://github.com/cursor/plugins/blob/main/pstack/docs/guide/05-build-and-clean.md>

<https://github.com/cursor/plugins/blob/main/pstack/docs/guide/09-make-it-yours.md>

<https://github.com/cursor/plugins/blob/main/pstack/docs/guide/08-principles.md>

<https://github.com/cursor/plugins/blob/main/pstack/docs/guide/03-understand.md>

<https://github.com/cursor/plugins/blob/main/pstack/docs/guide/04-design.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/bug-fix.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/babysit.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/prototype.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/multi-phase-plan.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/autonomous-run.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/orchestrate.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/feature.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/refactoring.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/investigation.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/runtime-forensics.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/trace-forensics.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/perf-issue.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/hillclimb.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/visual-parity.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/authoring-a-skill.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/eval.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/opening-a-pr.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/shipping.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/session-pickup.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/pause-safely.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/autopilot-full.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/autopilot-stack.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/playbooks/worktree-cleanup.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-prove-it-works/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-test-behavior-not-implementation/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-build-the-lever/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-type-system-discipline/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-encode-lessons-in-structure/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-fix-root-causes/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-subtract-before-you-add/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-migrate-callers-then-delete-legacy-apis/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-never-block-on-the-human/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-laziness-protocol/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-guard-the-context-window/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-make-operations-idempotent/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-sequence-verifiable-units/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-separate-before-serializing-shared-state/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-model-the-domain/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-minimize-reader-load/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-exhaust-the-design-space/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-foundational-thinking/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-redesign-from-first-principles/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-attack-the-premise/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-outcome-oriented-execution/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-experience-first/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/principle-boundary-discipline/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/create-verification-skill/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/maintain-verification-skill/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/references/bugbot-triage.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/no-comments/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/reflect/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/agents/poteto-agent.md>

<https://github.com/cursor/plugins/blob/main/pstack/agents/comment-sicko.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/architect/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/figure-it-out/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/arena/SKILL.md>

<https://github.com/cursor/plugins/tree/main/pstack/skills/poteto-mode/scripts/watch-pr>

<https://github.com/cursor/plugins/blob/main/pstack/skills/recall/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/scripts/check-plan.mjs>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/scripts/orch/orch.ts>

<https://github.com/cursor/plugins/blob/main/pstack/skills/poteto-mode/scripts/worktree-audit.sh>

<https://github.com/cursor/plugins/blob/main/pstack/skills/how/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/why/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/teach/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/blast-radius/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/interrogate/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/swarm/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/automate-me/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/make-bot-ui/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/bro/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/tdd/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/show-me-your-work/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/typescript-best-practices/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/unslop/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/technical-writing/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/setup-pstack/SKILL.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/show-me-your-work/scripts/log.sh>

<https://github.com/cursor/plugins/blob/main/pstack/skills/typescript-best-practices/references/patterns.md>

<https://github.com/cursor/plugins/tree/main/pstack/skills/create-verification-skill/references/feature-map-example>

<https://github.com/cursor/plugins/pull/422>

<https://github.com/cursor/plugins/tree/main/cursor-team-kit>

<https://github.com/cursor/plugins/tree/main/thermos>

<https://cursor.com/changelog/0-48-x>

<https://github.com/poteto/verification-skill-example>

<https://diataxis.fr/>

<https://github.com/mattpocock/skills>

<https://cursor.com/docs/bugbot>

<https://tailscale.com/>

<https://stylexjs.com/>

<https://x.ai/bot/marketplace/bots/dr-eggbot-v2>

<https://github.com/mattpocock/skills/blob/main/skills/engineering/grill-with-docs/SKILL.md>

<https://github.com/mattpocock/skills/blob/main/skills/productivity/grilling/SKILL.md>

<https://github.com/mattpocock/skills/blob/main/skills/engineering/domain-modeling/SKILL.md>

<https://github.com/mattpocock/skills/blob/main/skills/engineering/domain-modeling/ADR-FORMAT.md>

<https://github.com/mattpocock/skills/blob/main/skills/engineering/domain-modeling/GLOSSARY-FORMAT.md>

<https://x.com/poteto/status/2087977008253071816>

<https://x.com/poteto/status/2104744961904394699>

<https://github.com/cursor/plugins/blob/main/pstack/skills/architect/references/design-red-flags.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/create-verification-skill/references/feature-map-example/create-note.md>

<https://github.com/cursor/plugins/blob/main/pstack/skills/create-verification-skill/references/feature-map-example/search.md>

<https://cursor.com/blog/cloud-agent-environment>

<https://cursor.com/docs/cloud-agent/security-network>

<https://github.com/anthropics/skills>

https://github.com/anthropics/skills/blob/main/skills/skill-creator/scripts/quick_validate.py

<https://github.com/agentskills/agentskills/tree/main/skills-ref>

<https://developers.google.com/style>

<https://www.asd-ste100.org/>

<https://web.stanford.edu/~ouster/cgi-bin/book.php>

<https://zenn.dev/sc30gsw/articles/953334f11df507>